

DOCKER БЕЗ СЕКРЕТОВ

РАЗРАБОТКА И РАЗВЕРТЫВАНИЕ
ПРИЛОЖЕНИЙ С ПОМОЩЬЮ DOCKER



Docker Demystified

*Learn How to Develop and
Deploy Applications Using Docker*

Saibal Ghosh



www.bpbonline.com

Сайбал Гош

DOCKER БЕЗ СЕКРЕТОВ

Санкт-Петербург
«БХВ-Петербург»
2023

УДК 004.2
ББК 32.973-018.2
Г57

Гош С.

Г57 Docker без секретов: Пер. с англ. — СПб.: БХВ-Петербург, 2023. — 224 с.: ил.
ISBN 978-5-9775-1196-4

Книга подробно рассказывает о развертывании и поддержке контейнерных приложений с использованием технологии Docker. Описан принцип работы образов, контейнеров и связанных с ними хранилищ Docker Storage, рассмотрена система контейнеризации Docker Swarm, показаны принципы сетевого взаимодействия Container Network Model. Раскрыты вопросы использования плагинов в сервисах Docker, рассмотрено развертывание служб в Swarm. Отдельная глава посвящена обеспечению безопасности в экосистеме Docker, масштабированию и поддержке контейнерных приложений.

Для разработчиков ПО и системных архитекторов

УДК 004.2
ББК 32.973-018.2

Группа подготовки издания:

Руководитель проекта	<i>Павел Шалин</i>
Зав. редакцией	<i>Людмила Гауль</i>
Перевод с английского	<i>Мария Попович</i>
Компьютерная верстка	<i>Ольги Сергиенко</i>
Дизайн обложки	<i>Зои Канторович</i>

Copyright 2021 BPB Publications, India. All rights reserved.
First published in the English language under the title
Docker Demystified. Learn How to Develop and Deploy Applications Using Docker, ISBN 9789389845877
by BPB Publications India (sales@bpbonline.com)
Russian translation rights arranged with BPB Publications, India

© 2021 BPB Publications, Индия. Все права защищены.
Впервые опубликовано на английском языке под названием
Docker Demystified. Learn How to Develop and Deploy Applications Using Docker, ISBN 9789389845877
издательством BPB Publications India (sales@bpbonline.com)
Права на перевод на русский язык предоставлены издательством BPB Publications, Индия

"БХВ-Петербург", 191036, Санкт-Петербург, Гонимая ул., 20

ISBN 978-93-89845-87-7 (англ.)
ISBN 978-5-9775-1196-4 (рус.)

© BPB Publications, India, 2021
© Перевод на русский язык, оформление. ООО "БХВ-Петербург",
ООО "БХВ", 2023

Оглавление

Об авторе.....	11
Благодарности.....	13
Предисловие	15
Глава 1. Контейнеризация и Docker	17
Введение.....	17
Жизнь до контейнеризации	18
Концепция контейнеризации.....	18
Преимущества контейнеризации	19
Docker	21
Docker Engine	21
Компоненты Docker Engine	22
Docker Hub и Docker Registry	23
Контейнеры Linux и Windows	24
Контейнеры и Windows.....	24
Микросервисы и контейнеризация	24
Безопасность в Docker.....	25
Заключение.....	27
Основные тезисы	27
Контрольный тест.....	28
Ответы	28
Вопросы.....	29
Ключевые термины	29
Глава 2. Контейнеры и образы	31
Контейнеры: общая характеристика.....	31
Запуск контейнера	32
Состояния контейнеров Docker.....	34
Процессы внутри контейнера.....	36
Исследование контейнера и получение информации	38
Журналы контейнеров	39
Базовая архитектура контейнера.....	43

Образы: общая характеристика	44
Образы: детальное изучение	45
Копирование при записи	47
Где хранятся образы?	48
Исследование образа и получение информации	49
Сохранение образа	49
Сохранение образа с помощью команды COMMIT	50
Dockerfile	52
Кеш сборки	53
Многоступенчатые сборки	54
Заключение	59
Основные тезисы	59
Контрольный тест	60
Ответы	62
Вопросы	62
Ключевые термины	62
Глава 3. Драйверы хранилища и тома	63
Драйверы хранилища Docker	63
Поддерживаемые драйверы хранилища	64
Поддержка <i>backing filesystem</i>	64
Драйверы хранилища <i>overlay</i> и <i>overlay2</i>	65
Характеристика и принцип работы драйвера хранилища <i>overlay2</i>	66
Тома Docker	68
Заклучение	76
Основные тезисы	76
Контрольный тест	77
Ответы	78
Вопросы	79
Ключевые термины	79
Глава 4. Модель <i>Container Network</i> и сетевой мост Docker	81
Модель <i>Container Network</i>	81
Интерфейсы драйвера CNM	82
<i>Libnetwork</i>	83
Драйверы Docker	83
Сетевой мост Docker	84
Концепция пространства имен Linux	84
Мост Docker	89
Заклучение	90
Основные тезисы	90
Контрольный тест	90
Ответы	91
Вопросы	91
Ключевые термины	92
Глава 5. Docker Swarm	93
Введение	93
Что такое Docker Swarm	93
Преимущества Docker Swarm	94

Создание и настройка кластера Swarm.....	95
Консенсус в Docker Swarm	98
Концепция сервиса в Swarm	99
Создание реплик	100
Масштабирование сервиса.....	101
Реплицированные и глобальные сервисы.....	103
Очистка кластера Swarm.....	104
Блокировка и разблокировка кластера Swarm	106
Сети в Docker Swarm.....	109
Создание сервиса с опубликованным портом.....	113
Обход маршрутизирующей <i>mesh</i> -сети в Swarm.....	114
Шифрование трафика в оверлейной сети.....	115
Устранение неполадок в Docker Swarm.....	115
Заключение.....	115
Основные тезисы	116
Контрольный тест.....	116
Ответы	117
Вопросы.....	118
Ключевые термины	118
Глава 6. Сети Docker.....	119
Введение.....	119
Сети в Docker	119
Сеть Bridge	120
Сеть Host	126
Сеть None	126
Использование пространства имен существующего контейнера.....	127
Сопоставление портов.....	128
Сеть MACVLAN.....	130
MACVLAN в режиме <i>bridge</i>	131
MACVLAN в режиме <i>Trunk Bridge 802.1Q</i>	136
Сеть Overlay	138
Основные тезисы	153
Контрольный тест.....	153
Ответы	155
Вопросы.....	155
Ключевые термины	156
Глава 7. Безопасность в Docker. Часть 1.....	157
Введение.....	157
Пространства имен ядра	157
Контрольные группы.....	163
Память.....	164
Центральный процессор.....	166
<i>Capabilities</i>	167
Обязательный контроль доступа.....	170
Docker и <i>AppArmor</i>	176
Docker и <i>SELinux</i>	181
<i>Seccomp</i>	185
Заключение.....	186

Основные тезисы	186
Контрольный тест	187
Ответы	188
Вопросы	188
Ключевые термины	189
Глава 8. Безопасность в Docker. Часть 2.....	191
Введение	191
Docker Enterprise Edition	191
Установка Docker Enterprise Edition.....	192
Установка Universal Control Plane	196
Установка Docker Trusted Registry	199
Загрузка и установка клиентского пакета	200
Функции безопасности Docker Enterprise Edition	203
Управление доступом на основе ролей	208
Заключение.....	215
Основные тезисы	215
Контрольный тест	216
Ответы	218
Вопросы.....	218
Ключевые термины	218
Предметный указатель.....	219

*Посвящается всем тем, кто продолжал верить в меня,
когда у них не было для этого причин*

Об авторе

Сайбал Гош работает главным архитектором в компании Ericsson India Ltd. За свою карьеру он успел примерить на себя разные роли, включая администратора баз данных, технического консультанта, технического писателя, разработчика приложений и преподавателя.

У него есть набор специфических навыков, которые Сайбал стремится оптимально сочетать: глубокое понимание технологий и острое чувство прагматизма в бизнесе. С годами он освоил различные технологии и активно использует их для решения реальных задач в своей профессии. В последние годы Сайбал Гош работает в качестве DevOps и проводит много времени, взаимодействуя с Docker и Kubernetes. Он считает, что DevOps — это будущее разработки приложений. Сайбал помешан на технической коммуникации и старается, чтобы она была правильной и точной. Он обладает более чем двадцатилетним техническим опытом в сферах инфраструктуры и безопасности, консалтинга и разработки ПО. Вне работы Сайбал любит читать, медитировать, заниматься йогой и проводить время со своей семьей.

Благодарности

Эта книга была бы невозможна без благословения моих покойных родителей и постоянной поддержки моей жены Мули и сына Арпана, которым часто приходилось мириться с моей грубой и отрывистой манерой общения, поскольку я работал над книгой поздно вечером и в выходные дни, с трудом выкраивая время для семьи.

Я хотел бы поблагодарить технических рецензентов книги: Индраджита Нанди, Дебасиса Майти и Сабинасачи Банерджи, которые выделили время в своем плотном графике, чтобы провести техническую экспертизу книги и вселить в меня уверенность в том, что книга действительно соответствует целям, ради которых она была написана. Я также благодарен многим другим людям, которые поддерживали меня во время написания этой книги, в первую очередь моему брату Прабалу, который помог мне со всеми диаграммами в книге, а также моим друзьям и коллегам, не дававшим мне расслабиться, часто спрашивая меня о ходе работы над проектом, тем самым неявно подталкивая меня.

Генезис этой книги лежит в моей борьбе за глубокое понимание Docker в процессе его изучения, а также в попытке найти ответы на вопросы и сомнения, которые приходили мне в голову, когда я пытался разобраться во всех новых концепциях и терминах, часто сталкиваясь с морем сомнений, в котором не было видно берегов. Если эта книга поможет «демистифицировать» Docker для новичков в этой технологии, я буду считать свою работу хорошо выполненной, а свои усилия не напрасными.

Наконец, я хотел бы поблагодарить издательство BPB Publications за помощь в работе над книгой.

Предисловие

До недавнего времени все приложения были монолитными, то есть представляли собой единое и неделимое целое. Если нужно было что-то изменить, приходилось останавливать весь стек приложения, вносить изменения и тестировать их. Это был трудоемкий, негибкий и ненадежный процесс. Иногда в тестовой среде все шло хорошо, а после запуска приложение работало со сбоями или не работало вовсе!

Приходилось тратить несколько дней на тщательный анализ, чтобы в итоге обнаружить, например, что некоторые библиотеки из среды разработки отсутствуют в рабочей среде.

Контейнеры устроены иначе.

Приложение помещается в один исполняемый пакет вместе со всеми своими файлами конфигурации, библиотеками и зависимостями (объектами). Операционная система хоста используется сразу несколькими контейнерными приложениями, и Docker отвечает за координацию всех действий и совместное использование ресурсов хоста. Поэтому контейнеры такие гибкие и быстрые, а пользователю не приходится поддерживать сразу несколько операционных систем.

Сегодня большинство крупных приложений разделены на небольшие компоненты — микросервисы. Такой подход дает огромные преимущества, ведь каждый компонент можно разрабатывать, тестировать, развертывать и масштабировать по отдельности. Для каждого микросервиса можно выбрать свои базы данных, язык программирования и прочие инструменты, независимо от других приложений.

Разумеется, архитектура на основе микросервисов стала очень популярна наряду с контейнерами, ведь именно с помощью контейнеров можно создавать автономные, независимые друг от друга приложения.

Каждое приложение работает самостоятельно, являясь в то же время частью целого, поскольку из микросервисов можно собрать одно большое приложение.

Docker идеально подходит для работы с микросервисами. Каждое приложение можно развернуть в отдельный контейнер или даже разбить на отдельные процессы, выполняющиеся в независимых контейнерах Docker.

Из этой книги вы узнаете, как использовать Docker для разработки, развертывания и администрирования приложений. Здесь приводятся практические инструкции со скриншотами. Все понятия рассматриваются на конкретных примерах.

Краткое описание книги:

Глава 1: введение. Вы узнаете, что такое контейнеры и в чем их преимущества, чем они отличаются от виртуальных машин, а также как развивался Docker и как он стал популярнейшим инструментом для разработки приложений. Мы поговорим также о Docker Registry, микросервисах и встроенных средствах безопасности Docker.

Глава 2: подробное описание контейнеров и их основных компонентов — образов. Вы научитесь запускать контейнеры, определять их состояния и работать с журналами; узнаете об архитектуре контейнеров и взаимосвязи образов и Dockerfile.

Глава 3: понятие о драйверах хранилища и томах в экосистеме Docker.

Глава 4: описание сетей Docker, ориентированных на приложения. Рассматривается структура сетей Docker, которая обеспечивает все необходимое для нормального функционирования и благодаря достаточному уровню абстракции не усложняет процесс разработки.

Глава 5: анализ работы Docker Swarm. Вы узнаете, что такое оркестрация контейнеров и как использовать совокупные возможности контейнеров, взаимодействующих в локальной или облачной среде.

Глава 6: углубленный обзор сетей Docker и возможностей реализации различных вариантов сетей. Вы узнаете, как устроены сети и как сетевые функции Linux помогают создать надежную и безопасную сетевую инфраструктуру в Docker.

Глава 7: описание функций безопасности, доступных в Linux, и возможностей их применения в Docker. Вы узнаете, как создать безопасную среду Docker.

Глава 8: об обеспечении безопасности контейнерных приложений с помощью соответствующих функций Linux и различных компонентов Docker Enterprise Edition.

ГЛАВА 1

Контейнеризация и Docker

Введение

Разработчики программного обеспечения все чаще используют контейнеризацию как альтернативу виртуализации или дополнение к ней. *Контейнеризация* — это метод, при котором программное обеспечение вместе со всеми своими зависимостями (объектами) и средой выполнения упаковывается в контейнер, чтобы затем его можно было запускать в разных инфраструктурах и на разных платформах. При контейнеризации обычно потребляется меньше вычислительных ресурсов, чем при использовании виртуальных машин, потому что для каждой виртуальной машины требуется отдельная копия операционной системы, а для контейнеров — нет. В ближайшем будущем сложно будет представить себе разработку программного обеспечения без применения контейнеризации в том или ином виде.

Изучив эту главу, вы получите представление о том, что такое контейнеры, каковы их преимущества и принципы использования; узнаете, как появился Docker и как разрабатываются масштабные приложения с применением архитектуры на основе микросервисов и Docker.

Для лучшего восприятия материал разбит на несколько тем.

- ◆ Жизнь до контейнеризации
- ◆ Концепция контейнеризации
- ◆ Преимущества контейнеризации
- ◆ Docker
- ◆ Docker Engine
- ◆ Docker Hub и Docker Registry
- ◆ Контейнеры Linux и Windows
- ◆ Микросервисы и контейнеризация
- ◆ Безопасность в Docker

Жизнь до контейнеризации

В начале 2000-х гг. приложения работали на серверах, и нужно было иметь достаточно вычислительной мощности, чтобы избежать проблем. Но бизнес не стоит на месте. Время от времени компании совершенствовали используемые приложения или создавали новые, при этом приходилось наращивать мощность имеющихся серверов или, что еще хуже, покупать новые. ИТ-специалисты постоянно пытались рассчитать, сколько ресурсов понадобится в ближайшем будущем, и подготовить нужный объем. Обычно на всякий случай они покупали серверы с запасом, более мощные, чем было необходимо, что приводило к нерациональному использованию ресурсов. Иногда серверы использовались лишь на 10–15 % от своего потенциала, следовательно, компании тратили ресурсы попусту.

Что было дальше? В 2005 году мир заговорил о виртуализации.

Оказалось, что теперь необязательно покупать новые физические серверы для увеличения масштаба, — с помощью технологии виртуализации можно использовать уже имеющиеся свободные мощности на серверах, чтобы запускать множество бизнес-приложений и экономить деньги благодаря оптимизации использования ресурсов. Правда, на каждой виртуальной машине нужна отдельная *операционная система (ОС)*. ОС потребляет вычислительные ресурсы, требует обслуживания, установки исправлений и т. д. Часто каждая ОС приобретается по лицензии. Но хуже всего то, что на виртуальных машинах приложения почти всегда работают медленнее, чем на «голом металле» (*bare metal*), т. е. без предустановленной ОС. И хотя виртуализация определенно стала шагом вперед, она не решала всех проблем.

Концепция контейнеризации

Основой контейнеризации стала система *Linux Container (LXC)*. LXC — это метод виртуализации на уровне операционной системы, с помощью которого можно запускать несколько изолированных экземпляров Linux (контейнеров) на одном хосте с использованием всего одного ядра Linux.

Главное отличие контейнеров от виртуальных машин (ВМ) в том, что контейнеру не нужна отдельная ОС, а потому он гораздо «легче» виртуальной машины. Все контейнеры на одном сервере или хосте используют одну ОС, что позволяет очень экономно использовать ресурсы хранилища, центрального процессора и оперативной памяти. Виртуальная машина запускается относительно медленно, а контейнеру, благодаря его легкости, требуется гораздо меньше времени — иногда меньше секунды!

Более того, компаниям не приходится покупать много лицензий на операционные системы, а затем обслуживать все эти операционные системы, а значит, удастся повысить рентабельность инвестиций и сэкономить.

Контейнер использует ядро операционной системы вместе с другими контейнерами. Отдельная операционная система ему не нужна. Контейнеры меньше виртуальных машин, поэтому на одном хосте их можно разместить гораздо больше.

Это лишь один аспект. Другой связан с разработкой программного обеспечения. Пожалуй, всем знакома ситуация, когда при разработке и тестировании программы не обнаруживается никаких проблем, а затем, при развертывании ее в рабочей среде, что-то идет не так. Например, приложение работает слишком медленно или не запускается вовсе.

Мы тратим много часов на анализ первопричины и наконец обнаруживаем, что в рабочей среде не хватает библиотеки, установлена другая версия исправлений или есть иные отличия от тестовой среды.

С контейнерами таких проблем не возникает, потому что контейнер вмещает в себя приложения, а также файлы конфигурации, библиотеки и другие зависимости (объекты), необходимые для его выполнения. Этот пакет отделен от операционной системы и других объектов. Его легко можно перенести в другую среду или на другую платформу, и там он будет работать точно так же. Мы пишем код, упаковываем его в контейнер, а потом выполняем где угодно и *сколько угодно раз*. Благодаря возможности свободного перемещения можно написать код в одной среде и переместить его в другую, где он будет выполняться точно так же. Разве это не замечательно?

Сама концепция контейнеризации и изоляции процессов не появилась в одночасье, а развивалась постепенно. В 2013 году появился *Docker Engine* — легкий и эффективный инструмент контейнеризации с открытым кодом для сборки и контейнеризации приложений, значительно повлиявший на развитие и распространение этой технологии. Исследовательская компания Gartner прогнозировала, что к 2020 году более 50 % организаций будут использовать контейнеры, а это очень большое число.

Преимущества контейнеризации

Какие преимущества имеет контейнеризация? Мы уже знаем, что *по сравнению с виртуальными машинами (ВМ)* контейнеры потребляют гораздо меньше ресурсов. Контейнеры могут совместно использовать одно ядро ОС, поэтому на одном хосте или сервере контейнеров помещается гораздо больше, чем виртуальных машин (ВМ). Очевидно, что все больше разработчиков приложений будут выбирать контейнеры, а не ВМ. Для сравнения на рис. 1.1 представлены минимальные наборы ресурсов для ВМ (слева) и контейнера (справа).

Как видим, *виртуальная машина* использует слой гипервизора и отдельные операционные системы для каждого приложения, т. е. между приложением и операционной системой хоста существует больше уровней. *Контейнеры* же предъявляют меньше требований к серверу. В этом его ключевое отличие: на одном сервере можно уместить больше контейнеров, чем виртуальных машин, без потери производительности. У контейнеров своей операционной системы — они используют ОС хоста.

Следующее преимущество, которым обладают контейнеры благодаря отсутствию выделенной операционной системы, — легкость. Запускать и останавливать их

можно гораздо проще и быстрее по сравнению с традиционными виртуальными машинами. Контейнер можно запустить меньше чем за секунду. Поскольку не нужно загружать операционную систему, приложение начинает работу моментально.

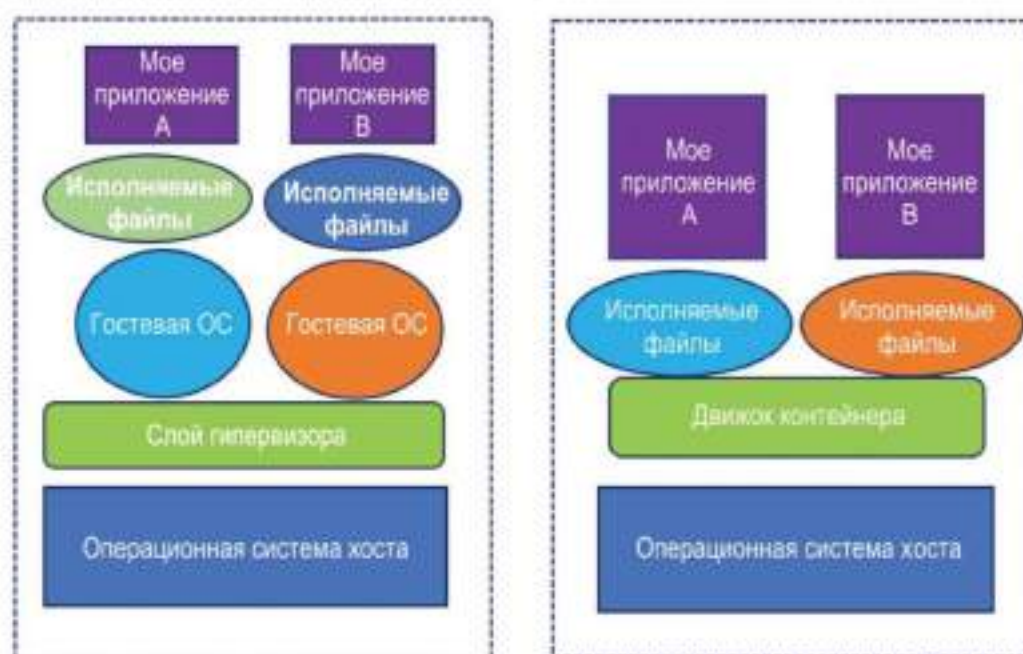


Рис. 1.1. Минимальные наборы ресурсов для VM и контейнера

Для обеспечения безопасности у контейнеров есть так называемые пространства имен, которые мы рассмотрим позже. Сейчас достаточно знать, что приложения в контейнере существуют *в изолированной среде* («песочнице») и не взаимодействуют друг с другом, если явно не настроить связь между ними. Такой подход предоставляет встроенный механизм безопасности приложений. Если вредоносному коду все же удастся проникнуть в один контейнер, он не сможет распространиться на другие.

Еще одно важное преимущество контейнеров по сравнению с VM — полная идентичность среды разработки и рабочей среды, о чем упоминалось выше. У всех разработчиков бывали ситуации, когда приложение прекрасно работает на их ноутбуке, но не на рабочем сервере, — просто потому, что среды на них различаются. С контейнерами такой проблемы не возникает.

Вместе с приложением в контейнер упакована и его среда выполнения, а значит, оно будет одинаково выполняться как в среде разработки, так и в тестовой и в рабочей среде. Более того, контейнеры можно использовать для *конвейера непрерывной интеграции и доставки* (Continuous Integration/Continuous Delivery, CI/CD), чтобы повысить продуктивность и эффективность разработки.

Наконец, преимущество контейнеров заключается в том, что обычно они выполняются как один сервис. Например, у нас может быть по отдельному сервису для базы данных, веб-приложения, аналитики о выполнении приложения и т. д. Мы уже говорили о преимуществах изолированности сервисов, но не упоминали о мас-

штабируемости. Масштаб контейнеров может увеличиваться или уменьшаться в зависимости от затрачиваемых ресурсов, — для оркестрации этих процессов в Docker есть встроенный механизм под названием *Docker Swarm*. На рынке представлены и другие популярные инструменты кластеризации и оркестрации, например Mesos и Kubernetes, но Docker Swarm, в отличие от них, представляет собой довольно изящный инструмент с внушительным количеством полезных функций.

Docker

Начнем с основ: что такое Docker?

Docker — это продукт Docker Inc., компании из Сан-Франциско; изначально разрабатывался как технология контейнеризации в рамках dotCloud.

В общих чертах Docker — программное обеспечение для создания, контроля, администрирования и оркестрации контейнеров. Он работает в Linux и Windows. Сейчас¹ это программное обеспечение разрабатывается в рамках *opencore-проекта*² Moby на GitHub. Проект создан Docker в целях развития контейнеризации приложений. Здесь любители контейнеров могут экспериментировать и обмениваться идеями. Как сказал Соломон Хайкс, основатель и бывший генеральный директор Docker Inc.: «Проект Moby для Docker — это открытая лаборатория для исследований и разработок».

Как приложение Docker состоит из нескольких компонентов: `containerd`, `runC`, `InfraKit` и т. д. Сообщество работает над отдельными инструментами и целым приложением, а затем Docker Inc. упаковывает их вместе и выпускает.

Вот что говорит об этом Соломон Хайкс:

«Наши команды должны работать не только над отдельными компонентами, но и над их сборками. Как в автомобильной промышленности, где один и тот же узел или агрегат могут использоваться в разных моделях машин».

Сейчас релизы Docker создаются при помощи Moby; существует более 80 компонентов, объединенных в сборки.

Docker Engine

Docker Engine — это клиент-серверное приложение, которое можно загрузить с Docker Hub или собрать вручную из исходного кода на GitHub. Docker Engine состоит из трех основных компонентов:

¹ Книга была написана в 2019 году. В начале сентября 2021 года компания Docker объявила о новых тарифах для бизнеса и сохранении бесплатного доступа к Docker Desktop для личного использования, а также для образовательных учреждений, некоммерческих opencore-проектов и малого бизнеса. С условиями тарификации можно ознакомиться здесь: <https://www.docker.com/pricing/>. — *Ред.*

² *Opencore-проекты* — проекты с открытым исходным кодом. — *Ред.*

1. Сервер, на котором запускается фоновый процесс демон.
2. REST API (Representational State Transfer Application Programming Interface — передача репрезентативного состояния программного интерфейса приложения), который управляет взаимодействием программ с демоном Docker и пр.
3. Простейший интерфейс командной строки Docker.

Компоненты Docker Engine

Docker REST API предоставляет *интерфейс командной строки*, или *CLI* (command-line interface), для взаимодействия с демоном Docker. Также для этой цели можно использовать скрипты. Предоставляемые API используются несколькими приложениями, и демон Docker берет на себя всю сложную работу по созданию, администрированию и контролю объектов Docker, таких как контейнеры, образы, сети, хранилища, тома и т. д.

Docker предоставляется по лицензии на свободное программное обеспечение Apache 2.0.

Docker использует простую клиент-серверную архитектуру, в которой клиент Docker взаимодействует с демоном Docker, отвечающим за сборку, выполнение, обслуживание, администрирование и распределение контейнеров. Клиент и демон Docker могут выполняться в одной системе или в рамках распределенной архитектуры. Клиент и демон Docker взаимодействуют с помощью REST API, Unix-сокета или сетевого интерфейса. На рис. 1.2 показана простая схема работы Docker Engine.

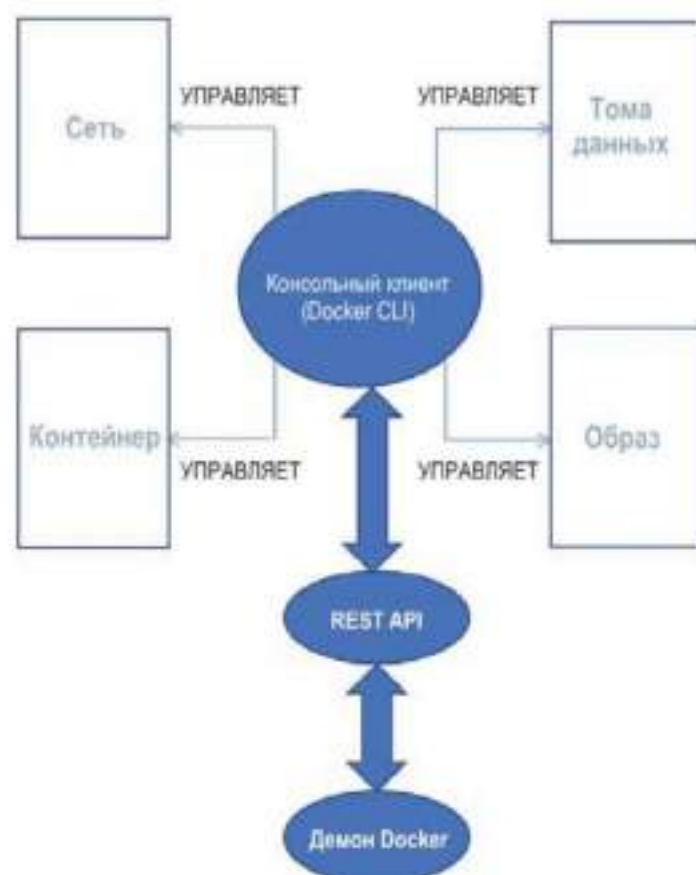


Рис. 1.2. Схема работы Docker Engine

Docker Engine поставляется в двух вариантах:

Docker Engine Community Edition — идеальная платформа для независимых разработчиков или небольших команд, которые только начинают использовать Docker для контейнерных приложений. Этот продукт имеет относительно ограниченную функциональность, но с него можно начать работу с Docker.

Docker Engine Enterprise Edition — платформа для компаний, разрабатывающих приложения с использованием технологии контейнеризации. Она содержит встроенные функции безопасности и предполагает соблюдение *соглашения об уровне обслуживания* (Service Level Agreements, SLA) корпоративного класса.

SLA — это соглашение между клиентом и поставщиком об уровне предоставляемой услуги, который выражается в измеримых показателях. Нарушение SLA влечет за собой штрафные санкции.

Docker Enterprise Edition предназначен для создания, поставки и запуска критически важных бизнес-приложений в больших масштабах.

Более подробно о версиях и их различиях см. в документации Docker: <https://docs.docker.com/install/overview/>.

Docker Hub и Docker Registry

Реестр (Registry) — это система хранения и доставки данных. В случае с Docker речь идет о хранении образов с использованием тегов. Помимо сервиса *Docker Registry* для этой цели можно использовать сторонний публичный или закрытый (private) реестр, например *Docker Hub*, AWS Container Registry, Google Container Registry и т. д.

У одного образа может быть несколько версий с разными тегами. В реестр образы загружаются с помощью команды `push`, а извлекаются из него помощью команды `pull`.

Драйвер хранилища по умолчанию — локальная файловая система, которая отлично подходит для разработки и небольших развертываний. Если нужно больше места, Docker нативно поддерживает облачные хранилища, в том числе Microsoft Azure, S3, OpenStack и т. д.

Docker Hub — это облачный репозиторий для создания, тестирования, хранения и распространения образов контейнеров, облачная версия Docker Registry.

При желании мы можем управлять реестром сами. Если для загрузки образов из реестра требуется проверка подлинности, это *закрытый (приватный) реестр*, а если не требуется — *публичный (открытый)*.

Реестр может стать отличным дополнением системы CI/CD. Обычно это происходит следующим образом: фиксация в системе управления версиями приводит к сборке в *системе непрерывной интеграции (CI)*, и если сборка выполнена успешно, в реестр отправляется новый образ. Уведомление от реестра автоматически

запускает развертывание в промежуточной среде. Кроме того, можно отправлять уведомление о новом образе в другие системы.

Контейнеры Linux и Windows

Docker совместим с Windows и Linux. Изначально Docker использовал функционал Linux, например пространство имен, контрольные группы, файловую систему UnionFS и т. д. Современные контейнеры зародились именно в Linux. Docker работает почти в любой системе Linux с ядром версии 3.10 и выше. Эта версия вышла в 2011 году, так что любой дистрибутив Linux, выпущенный позже, подходит для Docker. Кстати, контейнер можно с минимальными усилиями перенести с одного дистрибутива Linux на другой, что очень удобно для разработчиков.

Контейнеры и Windows

Инженеры Docker и Microsoft совместными усилиями реализовали стабильную поддержку Docker в Windows. Теперь Windows Server 2016 и более поздние версии поставляются с Docker Engine Enterprise. Более того, разработчики могут использовать Docker нативно в Windows 10 при помощи Docker Desktop. Между версиями Docker для Linux и Windows нет особой разницы. Они работают с одинаковыми CLI, API, форматами образов и сервисами распределения контента.

Правда, у Docker нет привычного для пользователей Windows графического интерфейса. Команды выполняются только через CLI. В Windows и Linux нет нативных (собственных) способов подключения к графическому интерфейсу приложения.

Подключение через VNC (Virtual Network Computing — система удаленного доступа) или RDP (Remote Desktop Protocol — протокол удаленного рабочего стола) требует дополнительных усилий и может увеличивать риски для безопасности.

Пользователи Linux реже используют графический интерфейс, так что этот недостаток скорее актуален для пользователей Windows. Администратор Linux привык к командной строке, а в Windows он чаще управляет системой посредством пользовательского интерфейса.

Микросервисы и контейнеризация

В модульном подходе к разработке программного обеспечения нет ничего нового. У нас уже были программные модули, подпрограммы, программные компоненты и т. п. Главное отличие *микросервисов* заключается в том, что каждый фрагмент программного обеспечения имеет четкие границы, поэтому его можно развертывать, масштабировать и обновлять независимо от других компонентов. Эти преимущества прекрасно дополняются возможностями контейнеризации, доступными в Docker.

Монолитное приложение имеет неделимую структуру, куда обычно входят клиентский интерфейс, база данных и серверное приложение, обрабатывающее бизнес-логику. Если где-то в системе произошло изменение, приходится перестраивать и

снова развертывать все серверное приложение, что довольно трудоемко, а приложение на этот период прекращает работу.

В архитектуре на базе микросервисов все устроено иначе. Если нужно изменить функцию, добавить бизнес-логику или скорректировать что-то в системе, то нет необходимости переделывать все приложение — достаточно внести изменение в отдельный компонент, поскольку микросервисы существуют независимо друг от друга. Каждый сервис выполняется независимо от других. Раз зависимостей нет, следовательно, можно изменить, улучшить или удалить один модуль приложения, не затронув другие.

Можно изменить один модуль приложения на базе микросервисов, не прерывая работу оставшейся части, если инфраструктура предоставляет необходимые вычислительные ресурсы.

Но как с этим связаны контейнеры? Мы уже знаем, что контейнеры и микросервисы прекрасно сочетаются друг с другом. Контейнеры предоставляют отдельным микросервисам изолированную среду, чтобы их можно было развертывать, изменять, обновлять и масштабировать независимо от остальных компонентов приложения. Неважно, на каком языке написан микросервис, главное, чтобы его можно было легко развертывать и масштабировать в рамках контейнера.

Поэтому сегодня разработчики всё реже используют монолитную архитектуру и всё чаще создают приложения на основе микросервисов.

Безопасность в Docker

Безопасности в Docker посвящены две отдельные главы, поэтому здесь мы рассмотрим только самое основное. Тему безопасности можно разделить на четыре аспекта.

1. Безопасность на уровне ядра и поддержка пространств имен и контрольных групп (cgroups).
2. Поверхность атаки, связанная с демоном Docker.
3. Уязвимости в профиле конфигурации контейнера.
4. Усиление имеющихся функций безопасности, в том числе с помощью сторонних инструментов.

Кратко рассмотрим каждый из этих аспектов.

Первый аспект заключается в следующем. Контейнеры Docker очень похожи на контейнеры LXC, в том числе с точки зрения безопасности. LXC — это метод виртуализации на уровне операционной системы, с помощью которого можно запускать несколько изолированных экземпляров Linux (контейнеров) на одном хосте с использованием лишь одного ядра Linux. Docker создает для контейнера пространство имен и контрольные группы.

Пространство имен защищает процессы внутри контейнера от влияния со стороны других контейнеров или даже хоста.

Контрольные группы позволяют накладывать ограничения на ресурсы. Они предоставляют метрики для справедливого распределения вычислительных ресурсов между контейнерами.

Кроме того, у каждого контейнера есть полноценная сетевая инфраструктура, так что им не нужен доступ к сети других контейнеров. Это настройка по умолчанию, но можно разрешить контейнерам взаимодействовать друг с другом в пределах хоста. Все контейнеры на одном хосте Docker сообщаются между собой с помощью «мостов» (bridge), примерно как физические компьютеры, соединенные через коммутатор Ethernet.

Таким образом, в контейнер уже встроены некоторые меры безопасности. В частности, вредоносный код не распространяется автоматически на другие контейнеры, DoS-атаки затруднены, и в целом в системе более или менее гарантирована стабильная работа.

Второй аспект безопасности — поверхность атаки демона Docker. Контейнеры запускаются с помощью демона Docker, который обычно работает в режиме суперпользователя или администратора (root). Разумеется, доступ к демону должен быть только у доверенных пользователей, которых нужно тщательно проверять.

Однако даже после тщательной проверки пользователей, работающих с демоном Docker, потенциальная угроза безопасности остается, поскольку Docker устроен таким образом, что у контейнеров и хост-системы могут быть общие папки с неограниченным доступом.

Следовательно, при желании можно легко изменить файловую систему хоста через контейнер. Примерно так же устроены системы виртуализации, где ничто не мешает предоставить виртуальной машине или даже блочному устройству доступ к корневой файловой системе.

Это небезопасно. Допустим, мы реализовали механизм подготовки контейнеров с веб-сервера через API. И нужно принять все меры предосторожности, чтобы злоумышленник не смог распространить через контейнер Docker вирус, который заразит саму хост-систему.

Конечно, можно отключить режим суперпользователя (root), но это существенно ограничит и наши возможности, и в реальном мире так почти никто не делает.

Здесь есть один интересный момент: у контейнеров Docker имеется инфраструктура, выполняющая много задач, для которых в ином случае потребовались бы права root, тогда как в большинстве случаев контейнерам настоящие права root не нужны. Поэтому контейнеры могут работать с ограниченным набором возможностей без полноценного режима root. Такой подход косвенно создает дополнительные меры по обеспечению безопасности, снижающие риск атак.

Третий аспект безопасности — профиль конфигурации. Docker позволяет вносить изменения в профиль, так что можно добавлять или удалять capabilities (букв.: возможности), соответственно повышая или понижая уровень безопасности. Лучше всего следовать принципу минимальных привилегий, т. е. оставить только те права, которые явно необходимы процессу.

Кроме того, в Docker имеется функция проверки подписи Docker Content Trust (DCT), которая разрешает Docker Engine запускать только подписанные образы. Эта функция встроена непосредственно в демон Docker.

Четвертый аспект. Для защиты в Docker доступны не только функции безопасности, предоставляемые ядром Linux, но и множество известных сторонних инструментов.

Помимо *capabilities* (подробнее это понятие будет рассмотрено в *главах 7 и 8*), предоставляемых Docker для защиты безопасности, вполне можно использовать сторонние инструменты. Им не нужна специальная конфигурация для Docker, потому что они применяются на уровне всей системы, а не отдельных контейнеров.

Политику безопасности можно определить в любом механизме контроля доступа. Суть в том, что защиту Docker можно усилить, не настраивая или модифицируя каким-либо образом сам Docker. Об этих и других функциях безопасности мы поговорим в *главах 7 и 8*.

Заключение

В этой главе мы рассмотрели принципы и преимущества контейнеризации. Вы узнали, как Docker развивался и набирал популярность. Docker предоставляет платформу контейнеризации для упаковки приложений вместе со всеми зависимостями в отдельный контейнер, который затем можно запускать в любой среде. Docker хорошо сочетается с архитектурой на основе микросервисов, и сегодня много крупных приложений разрабатывается именно с помощью этих двух технологий. Наконец, мы кратко обсудили безопасность в Docker, а также Docker Hub и Docker Registry.

Итак, теперь, когда вы понимаете основы, мы перейдем к углубленному изучению архитектуры и компонентов Docker в следующей главе.

Основные тезисы

- ◆ Контейнеры включают в себя приложение вместе со средой выполнения, чтобы приложение можно было без проблем запускать где угодно.
- ◆ Контейнеры гораздо легче виртуальных машин, поэтому их проще запускать и обслуживать.
- ◆ Компания Docker Inc. предоставила контейнерную технологию для коммерческого использования.
- ◆ Docker совместим с Windows и Linux.
- ◆ Docker отлично сочетается с микросервисами.
- ◆ В Docker встроены технологии кластеризации и оркестрации.
- ◆ В целом Docker обеспечивает адекватный уровень безопасности без дополнительных инструментов.

Контрольный тест

Выберите подходящие варианты ответов.

1. Контейнеры Docker легче виртуальных машин, потому что...
 - Контейнеры Docker обычно меньше виртуальных машин.
 - У контейнеров Docker нет гипервизора.
 - У контейнеров Docker нет гипервизора, но используется операционная система хоста.
 - У контейнеров Docker есть маленький гипервизор и используется операционная система хоста.
2. Docker Swarm — это:
 - Решение для кластеризации.
 - Решение для аварийного восстановления.
 - Решение для кластеризации и оркестрации.
 - Решение для кластеризации и резервного копирования.
3. Архитектура на основе микросервисов:
 - Имеет четкие границы.
 - Выполняется только на серверах без предустановленной ОС.
 - Выполняется только в контейнерах.
 - Является открытой и не имеет четких границ.
4. Контейнер Docker:
 - Использует сетевой стек хоста, на котором выполняется.
 - Использует сетевой стек, выделенный для каждого контейнера.
 - Требуется создание сетевого стека вручную.
 - Управляет собственным независимым сетевым стеком.
5. Контейнер Docker:
 - Может легко менять масштаб.
 - Не может легко менять масштаб.
 - Очень уязвим для злоумышленников.
 - Продолжит выполняться даже при сбое хоста.

Ответы

1	с
2	с
3	а

4	д
5	а

Вопросы

1. Что такое контейнер?
2. Чем контейнер отличается от виртуальной машины?
3. Что такое Docker Registry?
4. Что такое Docker Swarm?
5. Что такое микросервисы?
6. Какие встроенные функции безопасности имеются в Docker?

Ключевые термины

- ♦ *Контейнер*. Docker — это программное обеспечение, с помощью которого код приложения и все его переменные и зависимости в среде выполнения упаковываются в контейнер, чтобы приложение выполнялось одинаково на любой платформе.
- ♦ *Docker Swarm*. Это собственный инструмент кластеризации и оркестрации в Docker, с помощью которого можно управлять несколькими контейнерами Docker на хосте (хостах).
- ♦ *Микросервисы*. Микросервисы имеют четко определенные границы и могут разворачиваться, масштабироваться и обновляться независимо друг от друга.
- ♦ *Docker Registry*. Реестр — это система хранения и доставки контента. В случае с Docker реестр хранит образы, ориентируясь на теги. Docker Registry — это сервис хранения образов Docker.

ГЛАВА 2

Контейнеры и образы

Настоящая глава посвящена контейнерам и их основным компонентам — образам. Вы узнаете, как устроены контейнеры и образы и как с ними работать.

Темы главы следующие.

- ◆ Контейнеры: общая характеристика
- ◆ Запуск контейнера
- ◆ Состояния контейнеров Docker
- ◆ Процессы внутри контейнера
- ◆ Исследование контейнера и получение информации
- ◆ Журналы контейнеров
- ◆ Базовая архитектура контейнера
- ◆ Образы: общая характеристика
- ◆ Образы: детальное изучение
- ◆ Dockerfile

Прежде чем продолжить, нужно установить Docker. Здесь и во всей книге я привожу примеры из Docker на Linux, но с Windows все будет работать аналогично.

Контейнеры: общая характеристика

Контейнеры создаются из образов, а образы можно назвать шаблонами для создания контейнеров. *Контейнер* — динамическая реализация образа, который представляет собой статический tar-файл (tarball), содержащий файловую систему. Подробнее об образах мы поговорим чуть позже в этой главе. Сейчас достаточно понимать, что контейнеры создаются на их основе.

Итак, откуда берутся образы? Их можно создавать самостоятельно, и об этом мы также поговорим далее. Кроме того, существует очень много уже готовых образов в общедоступных реестрах, например Docker Hub, Google Container Registry,

Amazon EC2 Container Registry, Azure Container Registry и т. д. При установке Docker мы по умолчанию подключаемся к реестру Docker Hub.

Запуск контейнера

Давайте перейдем к практике и запустим наш первый контейнер. Сначала нужно убедиться, что в вашей системе установлен Docker. Самый простой способ сделать это — проверить версию Docker:

```
docker --version
```

или:

```
docker -v
```

Если Docker не установлен, вы увидите примерно следующие выходные данные (рис. 2.1).

```
root@my-docker:~# docker --version
Command 'docker' not found, but can be installed with:

snap install docker      # version 19.03.8, or
apt install docker.io

See 'snap info docker' for additional versions.
root@my-docker:~#
```

Рис. 2.1

Если же Docker установлен, выходные данные будут выглядеть так, как на рис. 2.2. Как видим, в этом примере установлен Docker version 19.03.9.

```
root@my-docker:~# docker --version
Docker version 19.03.9, build 9d988398e7
root@my-docker:~# docker -v
Docker version 19.03.9, build 9d988398e7
root@my-docker:~#
```

Рис. 2.2

Следующий шаг — запустить контейнер, т. е. создать и запустить его в своей системе. В этом примере мы создаем контейнер *busybox* и запускаем его в системе. *BusyBox* — это комплект программного обеспечения, который предоставляет несколько утилит Unix в одном исполняемом файле. Контейнер *busybox* создаем следующей командой:

```
docker run --name my_first_container busybox:latest
```

Получаем выходные данные (рис. 2.3).

В выходных данных на рис. 2.3, сказано, что найти образ локально не удалось. Это означает, что образа на компьютере не было, и он был загружен с Docker Hub (реестр по умолчанию, к которому Docker подключается автоматически при стан-

дартной установке). Далее вы можете видеть набор символов и сообщение о том, что образ загружен (Pull complete). Ниже идет еще одна строка символов — дайджест (digest), или контрольная сумма, слоя образа в *шестнадцатеричном формате*. О слоях образа мы поговорим позже. Пока достаточно знать, что образы состоят из слоев файлов и каталогов, которые предоставляют файловую систему для создания контейнеров.

```
root@my-docker:~# docker run --name my_first_container busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
d9cbbca60e5f: Pull complete
Digest: sha256:836945da1f3afe2cfff376d379852bbb82e0237cb2925d53a13f53d6e8a8c48c
Status: Downloaded newer image for busybox:latest
root@my-docker:~#
```

Рис. 2.3

Рассмотрим элементы команды для запуска нашего первого контейнера:

```
docker run --name my_first_container busybox:latest
```

- ◆ `docker`: относится к интерфейсу командной строки Docker (CLI), посредством которого мы обычно общаемся с Docker Engine.
- ◆ `run`: команда запуска контейнера. Из контекста понятно, что мы хотим запустить контейнер, но можно использовать и полный вариант: `docker container run`.
- ◆ `--name`: имя контейнера. В нашем случае это `my_first_container busybox:latest`, нужный нам образ. Тер `latest` означает, что мы хотим извлечь новейший из доступных образов BusyBox.

Давайте запустим еще один контейнер и добавим несколько флагов для конкретики. На этот раз извлечем образ `Alpine` — дистрибутива Linux на основе BusyBox. Присвоим контейнеру имя `alpine`. Используем следующую команду:

```
docker run --detach --interactive --tty --name alpine alpine:latest
```

Получим следующие выходные данные (рис. 2.4).

```
root@my-docker:~# docker run --detach --interactive --tty --name alpine alpine:latest
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
cbdbe7a5bc2a: Pull complete
Digest: sha256:9a839e63dad54c3a6d1834e29692c8492d93f90c59c978c1ed79109ea4fb9a54
Status: Downloaded newer image for alpine:latest
97d66ef6edc1724228d322c10996391d87f0d356aff404ea09617fbe2a2212e2
root@my-docker:~#
```

Рис. 2.4

Обратите внимание на несколько флагов на рис. 2.4.

`--detach` означает, что контейнер будет запущен в фоновом режиме.

`--interactive` означает, что контейнер будет принимать входные данные и предоставлять выходные данные.

--tty означает, что для выходных данных будет предоставлен терминал, когда контейнер работает в интерактивном режиме.

У этих флагов есть сокращенные варианты: -d для --detach, -i для --interactive и -t для --tty.

В команде эти флаги можно объединить следующим образом:

```
docker run -dit --name alpine alpine:latest
```

Уточним кое-что, прежде чем перейти к следующей теме. Мы создали контейнер и сразу запустили его; но можно создать контейнер и без запуска. Об этом мы поговорим в разделе о состояниях контейнера.

Состояния контейнеров Docker

В этом разделе мы поговорим о возможных состояниях контейнера Docker.

- ◆ Running (выполняется);
- ◆ Paused (приостановлен);
- ◆ Exited (завершил работу);
- ◆ Restarting (перезапуск);
- ◆ Dead (некорректно удален);
- ◆ Stopped (остановлен);
- ◆ Created (создан).

Если контейнер выполняется, его статус (Running) отображается как UP.

Давайте создадим контейнер и проверим его состояние. Команда для проверки состояния контейнера Docker — `docker ps --all` или `docker container ls -a`. Используем следующую команду:

```
docker run -dit --name my_container busybox:latest
```

Получим выходные данные (рис. 2.5).

```
root@my-docker:~# docker run -dit --name my_container busybox:latest
45066cf74cdfa1e794d92a51f1f84a9b43d4493c4b93beec21fe6700e6c4dff6
root@my-docker:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
45066cf74cdf	busybox:latest	"sh"	6 seconds ago	Up 5 seconds		my_container

Рис. 2.5

Мы видим, что контейнер выполняется. С помощью следующей команды можно приостановить выполнение:

```
docker container pause my_container
```

Получаем выходные данные (рис. 2.6).

```

root@my-docker:~# docker container pause my_container
my_container
root@my-docker:~# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
45066cf74cdf   busybox:latest   "sh"      41 seconds ago   Up 40 seconds (Paused)
root@my-docker:~#

```

Рис. 2.6

Если контейнер выполнил свою задачу и завершил работу, он имеет состояние `Exited` (завершил работу). Для дальнейшего использования нужно перезапустить его вручную. Допустим, мы создаем контейнер `busybox` для выполнения одной задачи — шесть раз отправить `ping` (пинг) к `localhost` (локальный хост). Мы не будем запускать этот контейнер в фоновом режиме, поскольку хотим посмотреть выходные данные в терминале. Запустим в хост-системе следующую команду:

```
docker run -it --name my_container1 busybox:latest ping -c 6 localhost
```

Получаем следующие выходные данные (рис. 2.7).

```

root@my-docker:~# docker container run -it --name my_container1 busybox:latest ping -c 6 localhost
PING localhost (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: seq=1 ttl=64 time=0.060 ms
64 bytes from 127.0.0.1: seq=2 ttl=64 time=0.053 ms
64 bytes from 127.0.0.1: seq=3 ttl=64 time=0.063 ms
64 bytes from 127.0.0.1: seq=4 ttl=64 time=0.054 ms
64 bytes from 127.0.0.1: seq=5 ttl=64 time=0.058 ms

--- localhost ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 0.041/0.054/0.063 ms
root@my-docker:~# docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
6529aaeb58a9   busybox:latest   "ping -c 6 localhost"   24 seconds ago   Exited (0) 18 seconds ago
45066cf74cdf   busybox:latest   "sh"      23 minutes ago   Up 23 minutes (Paused)
root@my-docker:~#

```

Рис. 2.7

Как видим, контейнер `my_container1` был создан, шесть раз отправил `ping` к `localhost`, а затем завершил работу. Теперь его статус — `Exited`. На рис. 2.7 видно, что контейнер `my_container` находится в состоянии `Paused` (приостановлен). Чтобы понять, как работает механизм приостановки и возобновления работы, возобновим работу контейнера `my_container` следующей командой:

```

docker container unpause my_container
docker container ls -a

```

Получаем соответствующие выходные данные (рис. 2.8).

Контейнер `my_container`, как и ожидалось, возобновляет работу и выполняется. Выполнение заняло 53 минуты, хотя мы точно знаем, что на некоторое время его работа была приостановлена.

```

root@my-docker:~# docker container unpause my_container
my_container
root@my-docker:~# docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
6529aaeb58a9   busybox:latest   "ping -c 6 localhost"   29 minutes ago   Exited (0) 29 minutes ago
45066cf74cdf   busybox:latest   "sh"      53 minutes ago   Up 53 minutes
root@my-docker:~#

```

Рис. 2.8

В процессе перезапуска мы можем «поймать» контейнер Docker, и тогда он будет иметь состояние *Restarting* (перезапуск).

Контейнер Docker будет иметь статус *Dead* (некорректно удален) при безуспешной попытке его удалить, когда ресурсы заняты или недоступны и контейнер был удален только частично. Эти контейнеры уже нельзя перезапустить и нужно удалить вручную, хотя демон Docker попытается удалить такие контейнеры при перезапуске.

Рассмотрим еще пару примеров: остановим контейнер командой *stop*, а затем используем команду *create*, чтобы создать, но не запускать контейнер. Команды и их выходные данные приводятся на рис. 2.9 и 2.10.

```
docker container stop my_container
```

```
root@my-docker:~# docker container stop my_container
my_container
root@my-docker:~# docker ps --all
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
6529aaeb58a9	busybox:latest	"ping -c 6 localhost"	58 minutes ago	Exited (0) 58 minutes ago		my_cont
45066cf74cdf	busybox:latest	"sh"	About an hour ago	Exited (137) About a minute ago		my_cont

```
root@my-docker:~#
```

Рис. 2.9

```
docker container create --name my_container2 alpine:latest
```

```
root@my-docker:~# docker container create --name my_container2 alpine:latest
49fb879aaae1e263849652d48815d6aa7a05e63bdfb7a272f829ece45f1dee2b
root@my-docker:~# docker ps --all
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
49fb879aaae1	alpine:latest	"/bin/sh"	9 seconds ago	Created		my_contain
6529aaeb58a9	busybox:latest	"ping -c 6 localhost"	About an hour ago	Exited (0) About an hour ago		my_contain
45066cf74cdf	busybox:latest	"sh"	2 hours ago	Exited (137) 11 minutes ago		my_contain

```
root@my-docker:~#
```

Рис. 2.10

Итак, мы остановили контейнер, и его статус изменился на *Exited* (завершил работу) с кодом 137, как показано на рис. 2.9. На рис. 2.10 контейнер со статусом *Created* (создан). Это означает, что он пока не запущен.

Процессы внутри контейнера

Давайте создадим контейнер *busybox* и зайдем в него с помощью параметра *exec*. Затем выполним несколько команд и изучим их выходные данные. Итак, запускаем интерактивный режим и указываем параметр *tty*. Запускаем команды:

```
docker run -dit --name test_cont busybox:latest
docker exec -i -t test_cont sh
```

Получаем выходные данные (рис. 2.11).

Внутри контейнера можно выполнять команды, относящиеся к данному контейнеру. Например, это контейнер *busybox*. Он содержит программу, которая предостав-

ляет все необходимые команды для управления средой Linux. Это значит, что в контейнере можно выполнять обычные команды Linux. Например, установив контейнер с базой данной MySQL, можно выполнять в нем команды базы данных. Как вы уже знаете из главы 1, среда контейнера изолирована с помощью пространства имен, поэтому команды можно выполнять безопасно.

```
root@my-docker:~#
root@my-docker:~#
root@my-docker:~# docker run -dit --name test_cont busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
d9cbbca60e5f: Pull complete
Digest: sha256:836945da1f3afe2cfff376d379852bbb82e0237cb2925d53a13f53d6e8a8c48c
Status: Downloaded newer image for busybox:latest
4fe5eea65ea913d47dc926dd3394ffacad30192855cf1fab8aade077358225f1
root@my-docker:~# docker exec -i -t test_cont sh
/ #
```

Рис. 2.11

Выполним внутри контейнера несколько простых команд Unix и посмотрим, что получится (рис. 2.12 и 2.13).

```
ps
ls
mkdir test
cd test
touch myfile
```

```
/ # ps
PID    USER    TIME    COMMAND
   1   root      0:00    sh
   7   root      0:00    sh
  12   root      0:00    ps
/ # ls
bin    dev    etc    home   proc   root   sys    tmp    usr    var
/ # mkdir test
/ # cd test
/test # touch myfile
```

Рис. 2.12

```
cd ..
ls
```

```
/test # cd ..
/ # ls
bin    dev    etc    home   proc   root   sys    test   tmp    usr    var
/ #
```

Рис. 2.13

Как видите, мы выполняем команды так, будто находимся на хосте с операционной системой на базе Unix. Это важный момент: *в контейнер можно заключить базу данных, свою операционную систему, собственный веб-сервер; практически всё,*

что можно выполнять на хосте. Нечто подобное можно делать и на виртуальных машинах, но контейнеры серьезно опережают их по производительности — они более легкие и гибкие, не используют слой гипервизора и не требуют отдельной операционной системы.

Исследование контейнера и получение информации

Информацию о контейнере можно получить при помощи команды `docker container inspect <id_контейнера/имя_контейнера>` или просто `docker inspect <id_контейнера/имя_контейнера>`. В результате мы получим объект JSON, содержащий большой объем информации, включая идентификатор контейнера (`id`), образ, использовавшийся для его создания, конфигурацию хоста, драйверы хранилища и параметры сети. Это очень ценная информация, если знать, что ищешь.

Давайте получим информацию о недавно созданном контейнере `test_cont`. Запустим команду и изучим выходные данные:

```
docker container inspect test_cont
[
  {
    "Id": "8a03514f6d3e9bb46c79f40220624ee690db23ed77fb65106a447c2d-04fafc66",
    "Created": "2020-05-28T04:49:50.071682396Z",
    "Path": "sh",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 4386,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2020-05-28T04:49:50.701489625Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:78096d0a54788961ca68393e5f8038704b97d8af374249dc5c8fae-clb8045e42",
    "ResolvConfPath": "/var/lib/docker/containers/8a03514f6d3e9bb46c79f40220624ee690db23ed77fb65106a447c2d04fafc66/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/8a03514f6d3e9bb46c79f40220624ee690db23ed77fb65106a447c2d04fafc66/hostname",
    "HostsPath": "/var/lib/docker/containers/8a03514f6d3e9bb46c79f40220624ee690db23ed77fb65106a447c2d04fafc66/hosts",
    "LogPath": "/var/lib/docker/containers/8a03514f6d3e9bb46c79f40220624ee690db23ed77fb65106a447c2d04fafc66/8a03514f6d3e9bb46c79f40220624ee690db23ed77fb65106a447c2d04fafc66-json.log",
```

```
"Name": "/test_cont",
"RestartCount": 0,
"Driver": "overlay2",
"Platform": "linux",
"MountLabel": "",
"ProcessLabel": "",
.....
.....
```

Выходные данные здесь немного сокращены, но мы вполне можем получить представление об объеме и типе предоставляемой информации. Позже мы увидим, что с помощью команды `inspect` можно получить детализированные сведения об образах и других объектах Docker.

Журналы контейнеров

В ходе выполнения контейнеров создается информация, которая регистрируется в журнале *Docker* или журнале сервиса Docker (подробнее о сервисах мы будем говорить в главе 5). Информация из журналов отображается в терминале. Linux по умолчанию выводит данные в терминал (`STDOUT`); для сообщений об ошибках используется `STDERR`.

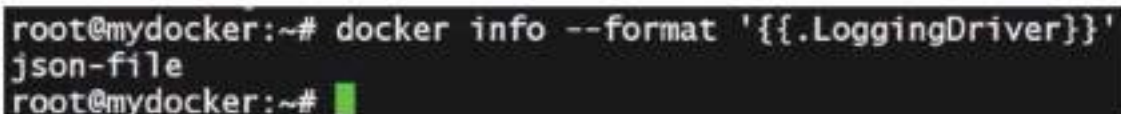
Иногда для получения полезной информации из журналов требуется дополнительная конфигурация. Например, если наш драйвер журналирования (логирования) отправляет журналы, например, в файл, а не в стандартный поток `STDOUT` или `STDERR`, то маловероятно, что в журналах Docker по умолчанию мы найдем полезную информацию.

Прежде чем двигаться дальше, нужно понять, что такое драйверы журналирования (или логирования). *Драйверы журналирования* — это программное обеспечение, которое помогает собирать информацию о запущенных контейнерах и сервисах. У демона Docker есть драйвер журналирования по умолчанию, но мы можем указать другой драйвер, и с этого момента контейнер будет использовать именно его. Драйвер журналирования по умолчанию — `json-file`.

Запустим следующую команду, чтобы узнать, какой драйвер журналирования выбран в нашей системе.

```
docker info --format '{{.LoggingDriver}}'
```

Получаем выходные данные (рис. 2.14).



```
root@mydocker:~# docker info --format '{{.LoggingDriver}}'
json-file
root@mydocker:~#
```

Рис. 2.14

Docker поддерживает разные драйверы журналирования, и мы можем указать желаемый драйвер, изменив соответствующий параметр в файле `/etc/docker/daemon.json`, а затем перезапустив демон Docker.

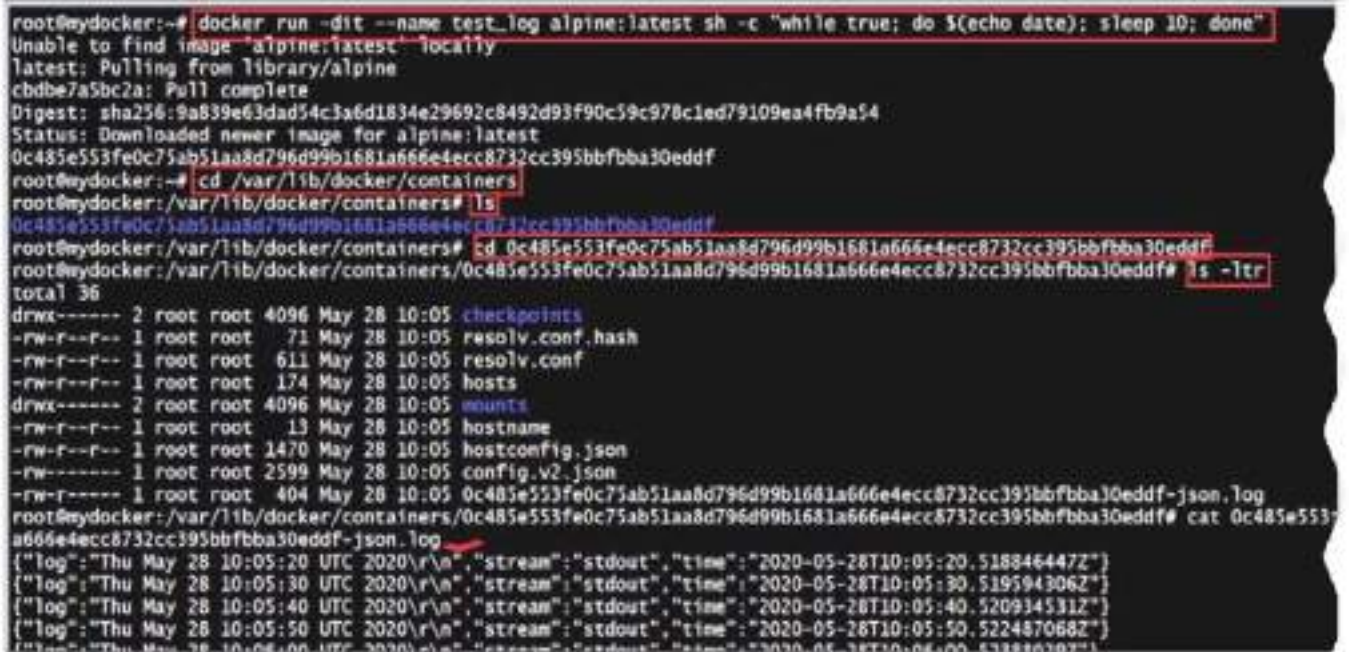
Новый драйвер будет применяться ко всем контейнерам, созданным после редактирования файла `daemon.json`. Есть еще один способ изменить драйвер журналирования по умолчанию — запустить контейнер с параметрами `--log-driver` и `--log-opt`.

Журналы контейнеров Docker хранятся в файлах JSON в каталоге `/var/lib/docker/containers/<id_контейнера>` в хост-системе. Имена журналов имеют формат `<id_контейнера>-json.log`. Для журналов указывается выходной поток (STDOUT или STDERR) и метка времени. Важно отметить, что в каждом журнале содержится информация только об одном контейнере.

Давайте в этом убедимся, создав контейнер и заглянув в его журналы с помощью следующих команд:

```
docker run --name test -dit alpine:latest sh -c "while true; do $(echo time) sleep 10; done"
cd /var/lib/docker/containers
ls
cd 0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf ls -ltr
cat 0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf-json.log
```

Получим следующий результат (рис. 2.15).



```
root@mydocker:~# docker run --name test_log alpine:latest sh -c "while true; do $(echo date); sleep 10; done"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
cbbde7a5bc2a: Pull complete
Digest: sha256:9a839e63dad54c3a6d1834e29692c8492d93f90c59c978c1ed79109ea4fb9a54
Status: Downloaded newer image for alpine:latest
0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf
root@mydocker:~# cd /var/lib/docker/containers
root@mydocker:/var/lib/docker/containers# ls
0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf
root@mydocker:/var/lib/docker/containers# cd 0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf
root@mydocker:/var/lib/docker/containers/0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf# ls -ltr
total 36
drwx----- 2 root root 4096 May 28 10:05 checkpoints
-rw-r--r-- 1 root root 71 May 28 10:05 resolv.conf.hash
-rw-r--r-- 1 root root 611 May 28 10:05 resolv.conf
-rw-r--r-- 1 root root 174 May 28 10:05 hosts
drwx----- 2 root root 4096 May 28 10:05 mounts
-rw-r--r-- 1 root root 13 May 28 10:05 hostname
-rw-r--r-- 1 root root 1470 May 28 10:05 hostconfig.json
-rw----- 1 root root 2599 May 28 10:05 config.v2.json
-rw-r--r-- 1 root root 404 May 28 10:05 0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf-json.log
root@mydocker:/var/lib/docker/containers/0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf# cat 0c485e553fe0c75ab51aa8d796d99b1681a666e4ecc8732cc395bbfba30eddf-json.log
{"log": "Thu May 28 10:05:20 UTC 2020\r\n", "stream": "stdout", "time": "2020-05-28T10:05:20.518846447Z"}
{"log": "Thu May 28 10:05:30 UTC 2020\r\n", "stream": "stdout", "time": "2020-05-28T10:05:30.519594306Z"}
{"log": "Thu May 28 10:05:40 UTC 2020\r\n", "stream": "stdout", "time": "2020-05-28T10:05:40.520934531Z"}
{"log": "Thu May 28 10:05:50 UTC 2020\r\n", "stream": "stdout", "time": "2020-05-28T10:05:50.522487068Z"}
{"log": "Thu May 28 10:06:00 UTC 2020\r\n", "stream": "stdout", "time": "2020-05-28T10:06:00.523803077Z"}
```

Рис. 2.15

Как видим, записи журнала создаются каждые 10 секунд с указанием даты и времени в системе. Это журнал контейнера.

Если мы будем хранить журналы в системе, потребуется очень много места, поэтому лучше использовать *агрегатор журналов*, который будет собирать и сохранять их в центральном расположении, освобождая в системе место для новых журналов.

Следует учитывать, что нам нужно отслеживать несколько наборов журналов, например для приложения внутри контейнера или для хост-системы, поэтому необходима хорошо продуманная стратегия управления журналами.

Рассмотрим несколько команд для управления журналами. Создадим отдельный контейнер `test_logs`, который будет каждые 10 секунд выводить дату и время. Затем с помощью различных команд просмотрим журнальные записи в зависимости от того, что мы хотим узнать. Запустим набор команд:

```
docker run --name test_logs -dit alpine:latest sh -c \
"while true; do $(echo time) sleep 10; done"
docker ps
docker logs b0c4ee4a7b9f
```

Смотрим выходные данные на рис. 2.16.

```
root@my-docker:~# docker run --name test_logs -dit alpine:latest sh -c "while true; do $(echo time) sleep 10; done"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
cbdbe7a5bc2a: Pull complete
Digest: sha256:9a839e63dad54c3a6d1834e29692c8492d93f90c59c978c1ed79109ea4fb9a54
Status: Downloaded newer image for alpine:latest
b0c4ee4a7b9fd34b2083dd250fd509448bc25a99fdfa6ea799c18896ab99d46d
root@my-docker:~# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
b0c4ee4a7b9f        alpine:latest      "sh -c 'while true; ..." 20 seconds ago     Up 18 seconds      0.0.0.0:22->22/tcp   test_logs
root@my-docker:~# docker logs b0c4ee4a7b9f
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
```

Рис. 2.16

Можно использовать флаг `-follow` (букв.: следовать) или `-f`, чтобы отслеживать все новые журнальные записи, как в следующем примере:

```
docker logs b0c4ee4a7b9f -follow
```

Выходные данные приведены на рис. 2.17.

```
root@my-docker:~# docker logs b0c4ee4a7b9f --follow
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys      0m 0.00s
```

Рис. 2.17

Чтобы получить больше деталей, можно использовать флаг `--details`. Правда, в нашем случае это ничего не даст, поскольку здесь нет никаких деталей. Запускаем команду и смотрим результаты на рис. 2.18.

```
docker logs b0c4ee4a7b9f --details
```



```

root@my-docker:~# docker logs b0c4ee4a7b9f --details
real    0m 10.00s
user    0m 0.00s
sys     0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys     0m 0.00s
real    0m 10.00s
user    0m 0.00s

```

Рис. 2.18

С помощью команды `--tail` можно указать определенное количество (*n*) строк, которые мы хотим посмотреть, например 8 последних журнальных записей, как в следующем примере (рис. 2.19).

```
docker logs b0c4ee4a7b9f --tail 8
```

```

root@my-docker:~# docker logs b0c4ee4a7b9f --tail 8
user    0m 0.00s
sys     0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys     0m 0.00s
real    0m 10.00s
user    0m 0.00s
sys     0m 0.00s
root@my-docker:~#

```

Рис. 2.19

Флаг `-t` позволяет просмотреть метку времени записей (рис. 2.20):

```
docker logs b0c4ee4a7b9f -t
```

```

root@my-docker:~# docker logs b0c4ee4a7b9f -t
2020-05-29T04:27:31.519135698Z real    0m 10.00s
2020-05-29T04:27:31.519212995Z user    0m 0.00s
2020-05-29T04:27:31.519220849Z sys     0m 0.00s
2020-05-29T04:27:41.520698477Z real    0m 10.00s
2020-05-29T04:27:41.520738781Z user    0m 0.00s
2020-05-29T04:27:41.520746441Z sys     0m 0.00s
2020-05-29T04:27:51.522308943Z real    0m 10.00s
2020-05-29T04:27:51.522360573Z user    0m 0.00s

```

Рис. 2.20

Разные флаги можно легко комбинировать. Например, если нужно просмотреть последние 10 записей с метками времени.

```
docker logs b0c4ee4a7b9f -t --tail 10
```

Получаем выходные данные (рис. 2.21).

Кроме того, можно использовать `grep`¹ для поиска по параметрам или просмотра ошибок, используя с этой целью следующую команду:

¹ *grep* — программа для поиска; утилита командной строки, которая на вводе строки, отвечающие заданному *регулярному выражению*, и выводит их, если вывод не отменен. Название происходит от английской фразы «search globally for lines matching the regular expression and print them» («искать везде строки, соответствующие регулярному выражению, и выводить их»). — *Ред.*


```

root@my-docker:~# docker logs b0c4ee4a7b9f -t --tail 10
2020-05-29T05:54:42.626012966Z sys      0m 0.00s
2020-05-29T05:54:52.627276140Z real    0m 10.00s
2020-05-29T05:54:52.627334608Z user    0m 0.00s
2020-05-29T05:54:52.627350790Z sys    0m 0.00s
2020-05-29T05:55:02.630140588Z real    0m 10.00s
2020-05-29T05:55:02.630208936Z user    0m 0.00s
2020-05-29T05:55:02.630217790Z sys    0m 0.00s
2020-05-29T05:55:12.633596791Z real    0m 10.00s
2020-05-29T05:55:12.633676941Z user    0m 0.00s
2020-05-29T05:55:12.633686853Z sys    0m 0.00s
root@my-docker:~#

```

Рис. 2.21

```

docker logs <id_контейнера> | grep pattern
docker logs <id_контейнера> | grep -i error

```

Управление журналами в Docker — весьма важная функция, потому что в системе с сотнями и даже тысячами контейнеров от качества этого процесса зависит эффективность диагностики неисправностей. Обработка информации от такого большого количества контейнеров для получения конструктивного результата — далеко не тривиальная задача.

Большинство драйверов журналирования могут отправлять журналы в центральное расположение или даже к инструментам управления, но у них нет функций анализа и нормализации данных. Для этого требуется отдельный инструмент, известный как *отправитель журналов*, или *конвейер данных*, например Logstash и Filebeat, которые считывают данные из различных источников, анализируют и нормализуют их, а затем отправляют в центральное расположение в формате, поддерживаемом такими инструментами, как Kibana или Elasticsearch.

Базовая архитектура контейнера

На рис. 2.22 изображена схема базовой архитектуры контейнера:

Docker Engine — это один из центральных компонентов Docker. На схеме видно, что Docker Engine состоит из интерфейса командной строки (CLI), REST API и сервера Docker.

Сервер Docker, ключевой компонент Docker Engine, включает демон Docker, *dockerd*, — часть ПО, отвечающая за создание образов, контейнеров, сетей и т. д.

Интерфейс REST (REST API) взаимодействует с сервером Docker, а Docker CLI позволяет пользователю взаимодействовать с Docker.

Далее схема показывает, что сервер Docker также включает *containerd* — часть ПО, выполняющая всю тяжелую работу для сервера Docker, включая создание контейнеров и управление ими.

Containerd, в свою очередь, связан с другой частью ПО — *runC*, которую можно определить как низкоуровневую реализацию среды выполнения (запуска) контейнера. Работая на базе *runC*, *containerd* добавляет такие функции, как управление образами, контейнерами, сетями и т. д.

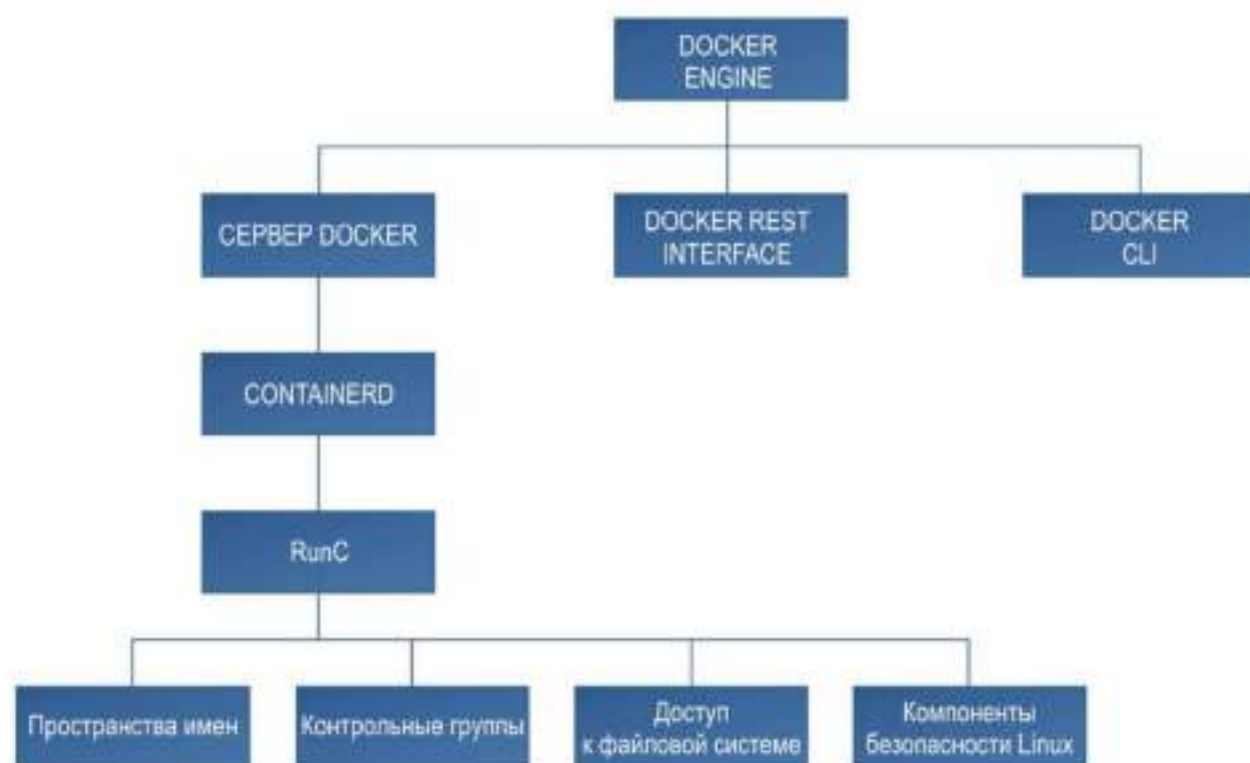


Рис. 2.22. Базовая архитектура контейнера

runC — это интерфейс командной строки по стандартам *Open Container Initiative*, *OCI* (Инициатива «Открытый контейнер»), т. е. инструмент создания и запуска контейнеров в соответствии со спецификациями *OCI*; узнать больше можно на сайте *OCI*: <https://www.opencontainers.org/>.

runC позволяет свободно применять функционал Linux, включая пространства имен и контрольные группы (*cgroups*), а также инструменты безопасности, такие как *AppArmor* и *SELinux*, и инструменты контроля доступа к файловой системе в экосистеме контейнеров.

Образы: общая характеристика

Образ Docker — это шаблон только для чтения, который предоставляет структуру файловой системы для создания контейнеров. Образ состоит из нескольких слоев файлов и каталогов, расположенных в определенном порядке и составляющих основу, на которой будет создан контейнер. Слои в составе образа являются *неизменяемыми*, т. е. после создания их нельзя модифицировать, можно только удалить. Контейнер создается путем добавления к образу тонкого слоя чтения и записи. Из одного образа можно создать *сколько угодно* контейнеров. Многократное использование образов — фундаментальный принцип Docker (об этом мы поговорим чуть позже). Это возможно благодаря *механизму копирования при записи*, с помощью которого контейнеры могут менять слои образа путем копирования их в свой тонкий слой чтения и записи и последующего внесения нужных изменений.

Образы: детальное изучение

Образы состоят из слоев, расположенных друг над другом. Иногда эти слои обозначают термином *diff*¹, поскольку каждый последующий слой *отличается* от предыдущего.

До Docker версии 1.10 каждому слою присваивался идентификатор образа (image id), состоящий из 12- или 64-символьной шестнадцатеричной строки. При создании образа каждый слой ссылался на «родительский» (parent), расположенный под ним, вплоть до самого нижнего слоя, у которого не было родителя. Эта концепция отображена на рис. 2.23.



Рис. 2.23. Образ Docker

Начиная с версии 1.10, у слоев нет идентификатора образа, вместо этого слои идентифицируются с помощью *дайджеста* (digest), или *контрольной суммы*, выражаемой в шестнадцатеричном формате (см. рис. 2.24).

```
Digest: sha256:fe8d824220415eed5477b63addf40fb06c3b049404242b31982106ac204f6700
```

Рис. 2.24

Существенное изменение заключается в том, что слой больше не является синонимом образа. До версии 1.10 каждый слой соответствовал определенному образу с идентификатором.

Поскольку раньше невозможно было определить, правомерно ли изменено содержимое слоя во время отправки в реестр или извлечения из него, безопасность слоя не гарантировалась. Теперь можно проверить целостность слоя образа с помощью контрольной суммы. Шестнадцатеричный показатель вычисляется путем применения к содержимому слоя алгоритма SHA-256 (Secure Hash Algorithm). Если содержимое изменено, изменится и контрольная сумма, поэтому нарушение безопасности легко заметить.

¹ От *difference* — отличие, разница. — Ред.

Образ Docker больше не создается на основе родительского образа. Теперь для компиляции образа Docker Engine использует список контрольных сумм слоев в определенном порядке.

Итак, образ теперь состоит из *объекта конфигурации*, представляющего собой набор дайджестов (контрольных сумм) SHA в определенном порядке, с помощью которых Docker Engine может генерировать файловую систему для контейнера, создаваемого на основе этого образа (см. рис. 2.25).

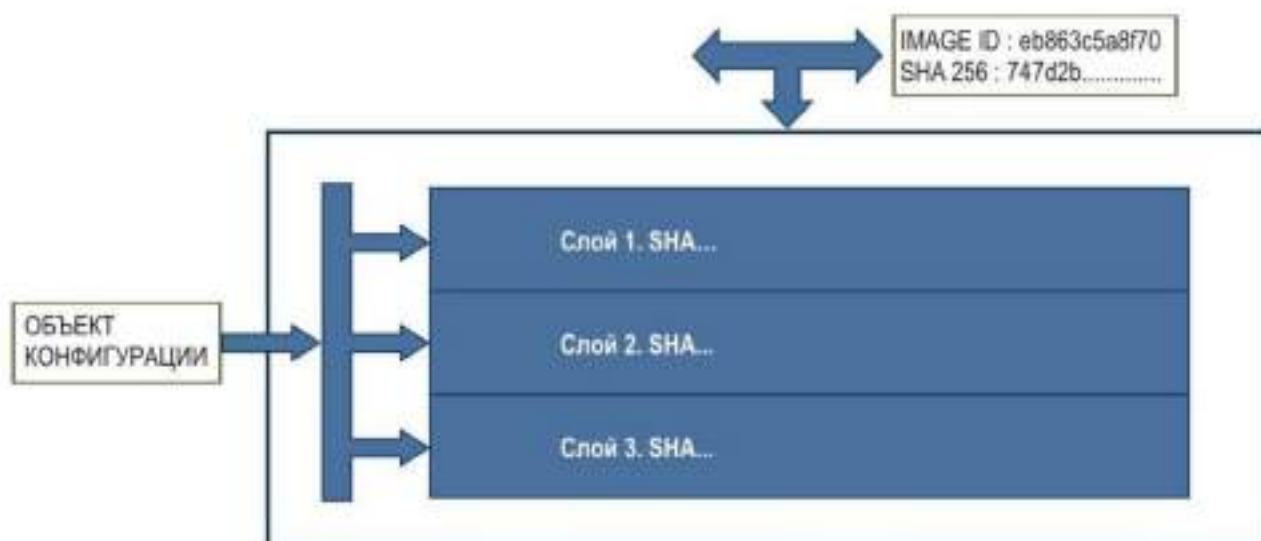


Рис. 2.25. Образ Docker

Давайте создадим образ, чтобы понять, как это работает. Запустим следующую команду, а затем изучим выходные данные на рис. 2.26:

```
docker image pull ubuntu:latest
```

```
root@my-docker:~# docker image pull ubuntu:latest
latest: Pulling from library/ubuntu
d51af753c3d3: Pull complete
fc878cd0a91c: Pull complete
6154df8ff988: Pull complete
fee5db0ff82f: Pull complete
Digest: sha256:747d2dbbaaee995098c9792d99bd333c6783ce56150d1b11e333bbceed5c54d7
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
root@my-docker:~#
```

Рис. 2.26

Мы извлекли из реестра Docker Hub образ `ubuntu` и видим, что он состоит из четырех слоев. Контрольная сумма, т. е. дайджест (digest) образа: `sha256:747d2dbbaaee995098c9792d99bd333c6783ce56150d1b11e333bbceed5c54d7` (см. рис. 2.24). У образа есть идентификатор (image id), который можно увидеть, запустив одну из следующих команд:

```
docker image ls
docker image ls --all
docker image ls --digests
```

Последняя команда нужна, чтобы получить более подробные выходные данные (рис. 2.27).

```

root@my-docker:~# docker image ls --digests
REPOSITORY          TAG             DIGEST                               IMAGE ID          CREATED
SIZE
ubuntu              latest          sha256:747d2dbbaaee995098c9792d99bd333c6783ce56150d1b11e333bbced5c54d7 1d622ef86b13     6 weeks ago
73.9MB
root@my-docker:~#

```

Рис. 2.27

На рис. 2.27 мы видим идентификатор образа (image id) и дайджест SHA (digest), соответствующий тому, что мы видели при извлечении образа из реестра.

Запустим очередную команду и посмотрим выходные данные (рис. 2.28).

```
docker image history 1d622ef86b13
```

```

root@my-docker:~# docker images
REPOSITORY          TAG             IMAGE ID          CREATED          SIZE
ubuntu              latest          1d622ef86b13     6 weeks ago     73.9MB
root@my-docker:~# docker image history 1d622ef86b13
IMAGE              CREATED          CREATED BY          SIZE            COMMENT
1d622ef86b13       6 weeks ago     /bin/sh -c #(nop)  CMD ["/bin/bash"] 0B
<missing>          6 weeks ago     /bin/sh -c mkdir -p /run/systemd && echo 'do_ 7B
<missing>          6 weeks ago     /bin/sh -c set -xe  && echo '#!/bin/sh' > /_ 811B
<missing>          6 weeks ago     /bin/sh -c [ -z "$(apt-get indextargets)" ] 1.01MB
<missing>          6 weeks ago     /bin/sh -c #(nop)  ADD file:a58c8b447951f9e30_ 72.8MB
root@my-docker:~#

```

Рис. 2.28

Вы обратили внимание на `<missing>` в столбце **IMAGE**? Хотя это немного странный способ представления выходных данных, но ошибки здесь нет: поскольку с версии Docker 1.10 у слоев нет идентификаторов, эта часть указана как `<missing>` (отсутствующая).

Идентификатор в первой строке относится не к верхнему слою образа, а ко всему образу, фактически это идентификатор образа (image id).

Что еще мы здесь видим? Мы знаем, что образ — это просто файловая система, на базе которой создается контейнер. Образ состоит из слоев, расположенных друг над другом в определенном порядке, причем последующий слой отличается от предыдущего. Разумеется, верхний слой может полностью отличаться от нижнего, в этом случае он весь был бы *diff*. Но если этот верхний слой использует компоненты, которые доступны в нижнем слое, то сам он может хранить только уникальные (или отличные, *different*) компоненты, а все остальное использовать совместно с нижним слоем.

Содержимое слоя *diff* определяется при помощи хеша SHA-256. Из одного образа можно создать множество контейнеров, потому что у контейнера есть уникальный верхний слой чтения и записи, при этом остальная часть образа остается прежней.

Копирование при записи

Копирование при записи (Copy-On-Write, COW) — это хороший пример совместного доступа к данным среди образов и контейнеров. Если какой-то процесс хочет изменить или добавить данные, операционная система делает копию этих данных.

Копия создается только для того, чтобы добавить или изменить данные в образе. У всех остальных процессов остается доступ к исходным данным.

Если контейнер хочет внести изменения в определенный слой образа, этот слой копируется на слой чтения и записи. Docker использует механизм копирования при записи для образов и для контейнеров. Изменения между образами или запущенными контейнерами отслеживаются с помощью драйверов хранилища, таких как *aufs* или *OverlayFS*, или инструментов, называемых *снэпшотами* (*snapshotters*), предназначенных для создания моментальных снимков (о сохраняемости данных мы поговорим в следующей главе).

Где хранятся образы?

Обычно в Linux образы хранятся на диске в каталоге `/var/lib/docker/<драйвер_хранилища>`. Слои хранятся как каталоги. Рассмотрим пример.

Запустим команду `docker info`, чтобы узнать, какой драйвер хранилища мы используем (рис. 2.29):

```
docker info
```

```
root@my-docker:~# docker info
Client:
 Debug Mode: false

Server:
 Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
 Images: 1
 Server Version: 19.03.11
 Storage Driver: overlay2
```

Рис. 2.29

Выходные данные здесь немного усечены для краткости. Теперь проверим, где хранится образ:

```
cd /var/lib/docker/overlay2
ls -ltr
```

Получим примерно следующие выходные данные (рис. 2.30).

```
root@my-docker:~# cd /var/lib/docker/overlay2
root@my-docker:/var/lib/docker/overlay2# ls -ltr
total 20
drwx----- 3 root root 4096 Jun  6 04:05 f6edbeabb8d61cdda13cbe68ea2d914e26bffffea13966e0d758de819a9421caf
drwx----- 4 root root 4096 Jun  6 04:05 30540544135b039636112e5e8aab4fb46933734547ddd41699b6e1ea5570ddd1
drwx----- 2 root root 4096 Jun  6 04:05 1
drwx----- 4 root root 4096 Jun  6 04:05 c5ab42b68f9cb5cde50cf06c0b039e917a0a153925b0e75b4e72b828add62424
drwx----- 4 root root 4096 Jun  6 04:05 8d09cflbde46e0516a1982e48ed47dcdcea9993fcd0105ac2d9aac0a897f4a98
root@my-docker:/var/lib/docker/overlay2#
```

Рис. 2.30

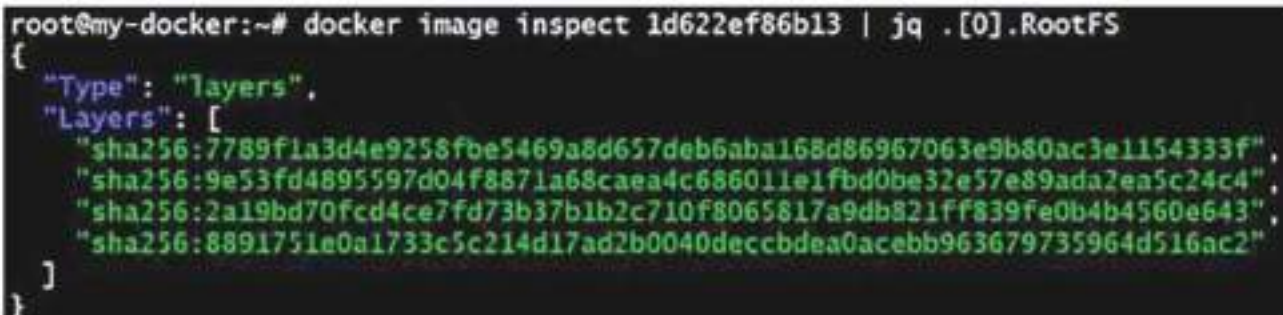
Мы видим четыре слоя и еще один слой с именем 1. Но это мы обсудим в следующей главе.

Исследование образа и получение информации

Мы можем извлечь очень много полезной информации с помощью команды `docker image inspect <id_образа>`. Правда, читать эту информацию будет непросто. Для форматирования выходных данных можно использовать `jq` — легкую и гибкую утилиту для обработки JSON в командной строке. Выходные данные мы получаем в формате JSON, поэтому можем использовать `jq` как для получения общего представления о результатах, так и для выбора наиболее интересных деталей. Загружаем инструмент командой `apt install jq`.

Если мы хотим просмотреть слои образа, запускаем следующую команду (рис. 2.31):

```
docker image inspect 1d622ef86b13 | jq .[0].RootFS
```



```
root@my-docker:~# docker image inspect 1d622ef86b13 | jq .[0].RootFS
{
  "Type": "layers",
  "Layers": [
    {
      "sha256:7789f1a3d4e9258f8e5469a8d657deb6aba168d86967063e9b80ac3e1154333f"
    },
    {
      "sha256:9e53fd4895597d04f8871a68caea4c686011e1fbd0be32e57e89ada2ea5c24c4"
    },
    {
      "sha256:2a19bd70fcd4ce7fd73b37b1b2c710f8065817a9db821ff839fe0b4b4560e643"
    },
    {
      "sha256:8891751e0a1733c5c214d17ad2b0040deccbdea0acebb963679735964d516ac2"
    }
  ]
}
```

Рис. 2.31

Мы видим все слои образа и их дайджесты SHA. И они не совпадают с именами каталогов на рис. 2.28.

Сохранение образа

Образ можно сохранить как tar-файл. Запускаем следующие команды и изучаем результат на рис. 2.32.

```
docker image pull nginx:latest
docker image ls
docker image save 4392e5dad77d > mynginx.tar
ls -ltr
ls -sh mynginx.tar
```

Можно добавить флаг `--output`, чтобы создать резервную копию и при необходимости использовать ее при загрузке Docker.

Запускаем команду и изучаем результат на рис. 2.33:

```
docker save --output mynginx.tar nginx
```

```

root@my-docker:~# docker image pull nginx:latest
latest: Pulling from library/nginx
afb6ec6fdclc: Pull complete
dd3ac8106a0b: Pull complete
8de28bda69b: Pull complete
a2c431ac2669: Pull complete
e070d03fd1b5: Pull complete
Digest: sha256:c870bf53de0357813af37b9500cblc2ff9fb4c00120d5fe1d75c21591293c34d
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
root@my-docker:~# docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx                latest             4392e5dad77d        5 days ago         132MB
root@my-docker:~# docker image save 4392e5dad77d > mynginx.tar
root@my-docker:~# ls -ltr
total 133148
-rw-r--r-- 1 root root      420 Jan  1  1970 manifest.json
-rw-r--r-- 1 root root    3408 Apr 24 01:07 1d622ef86b138c7e96d4f797bf5e4baca3249f030c575b9337638594f2b63f01.json
-rw-r--r-- 1 root root 136332800 Jun  8 05:10 mynginx.tar
root@my-docker:~# ls -sh mynginx.tar
131M mynginx.tar
root@my-docker:~#

```

Рис. 2.32

```

root@my-docker:~# docker save --output mynginx.tar nginx

```

Рис. 2.33

Сохранение образа с помощью команды COMMIT

С помощью команды `COMMIT` можно создать и сохранить образ без использования `Dockerfile` (о `Dockerfile` мы поговорим чуть позже в этой главе).

Чтобы понять этот процесс, создадим контейнер `ubuntu` и войдем в него (рис. 2.34):

```

docker run -dit --name ubuntu ubuntu:latest
docker exec -it ubuntu sh

```

```

root@my-docker:~# docker run -dit --name ubuntu ubuntu:latest
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
d51af753c3d3: Pull complete
fc878cd0a91c: Pull complete
6154df8ff988: Pull complete
fee5db0ff82f: Pull complete
Digest: sha256:747d2dbbaaee995098c9792d99bd333c6783ce56150d1b11e333bbceed5c54d7
Status: Downloaded newer image for ubuntu:latest
193254bac04efffe5d7eb50ab9bdeb3cabbe7f71ebe0112608634489aa5662f6
root@my-docker:~# docker exec -it ubuntu sh
#

```

Рис. 2.34

Находясь внутри контейнера, попробуем отправить `ping` к `google.com`. Однако команда завершается ошибкой, т. к. `ping` не установлена в ОС `Ubuntu`, которую мы использовали для создания контейнера. Команда и результат приведены ниже (рис. 2.35).

```

ping google.com

```

```
root@my-docker:~# docker exec -it ubuntu sh
# ping google.com
sh: 1: ping: not found
#
```

Рис. 2.35

Установим команду ping следующим образом:

```
apt update && apt install iputils-ping -y
```

Когда команда успешно установлена, снова пингуем google.com и смотрим, что получится:

```
ping google.com
```

Как видите, теперь все прекрасно работает (рис. 2.36).

```
# ping google.com
PING google.com (172.217.6.78) 56(84) bytes of data.
64 bytes from sfo07s17-in-f14.1e100.net (172.217.6.78): icmp_seq=1 ttl=56 time=2.16 ms
64 bytes from sfo07s17-in-f14.1e100.net (172.217.6.78): icmp_seq=2 ttl=56 time=1.57 ms
64 bytes from sfo07s17-in-f14.1e100.net (172.217.6.78): icmp_seq=3 ttl=56 time=1.61 ms
64 bytes from sfo07s17-in-f14.1e100.net (172.217.6.78): icmp_seq=4 ttl=56 time=1.57 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.572/1.729/2.162/0.250 ms
#
```

Рис. 2.36

Было бы замечательно, если бы команда ping была установлена в образе ubuntu, не так ли? Давайте сохраним этот контейнер в образ, который впоследствии можно будет использовать. Тогда нам не придется устанавливать команду ping отдельно, ведь она уже будет установлена. Выходим из контейнера в хост-систему и с помощью команды `commit` сохраняем контейнер в образ. Команды и результат ниже (рис. 2.37).

```
exit
docker commit ubuntu myubuntu
```

Здесь: `ubuntu` — имя контейнера, а `myubuntu` — имя создаваемого образа.

```
# exit
root@my-docker:~# docker commit ubuntu myubuntu
sha256:ff514dbfa904f2028cf7e5f10f9a916a0bc685bbba4ea3cea7eb3f849ed76fdc
```

Рис. 2.37

Теперь, если мы проверим свой набор образов, то обнаружим новый доступный образ `myubuntu`, который можно использовать для создания контейнеров. Попробуем сделать это следующим образом (рис. 2.38).

```
docker images
```



```
root@my-docker:~# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myubuntu	latest	ff514dbfa904	16 seconds ago	97.3MB
ubuntu	latest	1d622ef86b13	6 weeks ago	73.9MB

Рис. 2.38

Dockerfile

Dockerfile — это текстовый файл, в который помещаются команды для сборки образа.

В отличие от обычных текстовых файлов, Dockerfile не имеет расширения `.txt`, он сохраняется без расширения.

Мы уже знаем, что образы — это слои файлов и каталогов, которые хранятся в определенном порядке и на основе которых Docker Engine создает файловую систему для контейнеров. В Dockerfile каждая написанная нами строка создает слой образа Docker.

Docker читает Dockerfile сверху вниз и не различает регистры, хотя команды принято писать в верхнем регистре. Docker читает команды в Dockerfile построчно, выполняет их, в результате чего для каждой строки в Dockerfile собирается образ.

Рассмотрим команды, которые обычно используются при создании Dockerfile.

FROM. Это первая команда, которую мы отправим в Dockerfile; она указывает, какую операционную систему мы будем использовать в качестве базового образа.

Следует учитывать, что перед **FROM** в Dockerfile может быть только инструкция **ARG**. **ARG** перед **FROM** фактически не входит в этап сборки, и никакие инструкции после **FROM** не будут использовать значение **ARG**. Если мы хотим объявить **ARG** перед **FROM**, чтобы использовать его на этапе сборки, нужно использовать **ARG** без значения внутри этапа сборки.

LABEL. Эта инструкция добавляет в образ метаданные. **LABEL** представляет собой пару «ключ — значение». Обычно с помощью **LABEL** мы предоставляем подробную информацию об образе.

ENV. С помощью этой команды мы задаем переменные среды (environment) при сборке образа Docker. Эти переменные будут использоваться при запуске контейнера.

RUN. Главная исполняющая команда в Dockerfile. Она создает новый слой поверх текущего и фиксирует результаты. Именно с помощью команды **RUN** мы собираем образ. Если мы используем **RUN** в форме оболочки, можно указать обратный слеш (`\`), чтобы продолжить инструкцию в следующей строке.

CMD. Команда **CMD** чем-то похожа на **RUN**, но выполняется не в процессе сборки, а при создании экземпляра контейнера на основе собранного образа. Это команда по умолчанию, которая выполняется при создании контейнера.

ENTRYPOINT. Данная команда выполняется при создании контейнера из образа и указывает точку входа. Например, если мы установили в образе приложение и хотим,

чтобы оно запускалось при создании контейнера, мы указываем это с помощью `ENTRYPOINT`. Может показаться, что инструкция `ENTRYPOINT` идентична `CMD`, но `ENTRYPOINT` считается родительской командой. С помощью `CMD` также можно передавать параметры.

ADD. Команда `ADD` просто копирует файлы из исходного расположения (на хосте) в целевое (файловую систему контейнера). Если источник представляет собой URL-адрес, в целевое расположение загружается содержимое, хранящееся по этому адресу.

EXPOSE. Указывает порты, которые контейнер будет прослушивать во время выполнения. По умолчанию используется протокол TCP, но можно указать UDP. Обратите внимание: `EXPOSE` не публикует порт, а просто сообщает о портах, которые можно опубликовать при запуске контейнера.

WORKDIR. Задаёт рабочий каталог для инструкций `RUN`, `CMD`, `ENTRYPOINT`, `COPY` и `ADD`. Если не указать `WORKDIR`, каталог все равно будет создан, даже если в конечном итоге он не будет использоваться в других инструкциях `Dockerfile`.

USER. Эта инструкция задает UID (имя пользователя) или GID (группу пользователей), которые будут использоваться при выполнении контейнера, созданного из образа, с инструкциями `RUN`, `CMD` и `ENTRYPOINT`.

VOLUME. Инструкция `VOLUME` просто создает точку монтирования с указанным именем. Значение тома можно записать как массив JSON или обычную строку. Создание нового тома инициализирует команда `docker run`.

Кеш сборки

Когда мы собираем образ, Docker выполняет инструкции в `Dockerfile` строка за строкой. Процесс отчасти оптимизирован — прежде чем выполнить инструкцию, Docker ищет существующий образ в кеше и, если находит, использует его, чтобы не создавать дубликат. Этот кеш, известный как *кеш сборки*, позволяет оптимизировать работу Docker.

Здесь стоит упомянуть, что локальная сборка образов на хосте Docker происходит немного иначе. Базовое содержимое локально собранного образа остается таким же, как в образе, извлеченном из реестра, но объект конфигурации, содержащий элементы конфигурации, включая упорядоченный список контрольных сумм (дайджестов) слоев, немного отличается.

Существует так называемый *промежуточный образ*, который создается, когда мы фиксируем слой во время сборки образа. Промежуточный образ имеет все внешние атрибуты *финального образа*. У него тоже есть объект конфигурации, который представляет собой список контрольных сумм, используемых Docker Engine для создания файловой системы. Эти промежуточные образы также включают идентификатор родительского образа, т. е. образа слоя, который находится сразу под ним.

Промежуточные образы и ссылки на родительские образы используются для сопоставления с существующими образами в кеше сборки. Если совпадение найдено, Docker будет использовать его, чтобы не создавать этот образ повторно.

Многоступенчатые сборки

Оптимизировать процесс сборки образов позволяют и *многоступенчатые сборки*, благодаря которым можно значительно сократить размер создаваемого образа. В многоступенчатых сборках используется несколько инструкций `FROM` в `Dockerfile`. Каждая инструкция `FROM` задает отдельную базу и начинает новый этап сборки. Мы можем копировать все необходимое из одного этапа на другой и исключать ненужное из финального образа. Многоступенчатая сборка позволяет существенно уменьшить размер финального образа, так что нам не придется стараться сократить количество слоев в образе.

Теперь, вооружившись необходимыми знаниями, давайте напишем `Dockerfile` и соберем образ.

```
vi Dockerfile
FROM ubuntu:latest
ENV HOME /root
LABEL ubuntu=myubuntu
ENTRYPOINT ["sleep"]
CMD ["50"]
RUN useradd -m -G root testuser
USER root
RUN apt-get update && apt-get install net-tools -y
RUN apt-get install iputils-ping -y
```

Далее сохраним файл, закроем редактор и запустим команду сборки:

```
docker build --tag myubuntu_image .
```

Это команда для сборки образа из текущего каталога (не пропустите точку в конце команды). Вы могли заметить, что мы намеренно не оптимизировали образ, используя несколько отдельных команд `RUN`, в результате чего каждой из них был собран и сохранен отдельный слой образа.

Чтобы оптимизировать сборку, достаточно запустить одну команду `RUN` и перечислить все элементы, разделив их двойным амперсандом (`&&`). Тогда образ будет меньше. Вот как это выглядит (рис. 2.39).



```
root@my-docker:~# vi Dockerfile
FROM ubuntu:latest
ENV HOME /root
LABEL ubuntu=myubuntu
ENTRYPOINT ["sleep"]
CMD ["50"]
RUN useradd -m -G root testuser
USER root
RUN apt-get update && apt-get install net-tools -y
RUN apt-get install iputils-ping -y
root@my-docker:~# docker build --tag myubuntu_image .
```

Рис. 2.39


```

root@my-docker:~# docker build --tag myubuntu_image .
Sending build context to Docker daemon 16.38kB
Step 1/9 : FROM ubuntu:latest
latest: Pulling from library/ubuntu
d51af753c3d3: Already exists
fc878cd0a91c: Already exists
6154df8ff988: Already exists
fee5db0ff82f: Already exists
Digest: sha256:747d2dbbaaee995098c9792d99bd333c6783ce56150dlb11e333bb- ceed5c54d7
Status: Downloaded newer image for ubuntu:latest
---> 1d622ef86b13
Step 2/9 : ENV HOME /root
---> Running in 37112affa648
Removing intermediate container 37112affa648
---> 9646522325be
Step 3/9 : LABEL ubuntu=myubuntu
---> Running in 9801e9252462
Removing intermediate container 9801e9252462
---> bd68bd81ff42
Step 4/9 : ENTRYPOINT ["sleep"]
---> Running in 94d8fbefddc5
Removing intermediate container 94d8fbefddc5
---> d79ce88560e6
Step 5/9 : CMD ["50"]
---> Running in 30bcc2a0452e
Removing intermediate container 30bcc2a0452e
---> 0c47c79e948e
Step 6/9 : RUN useradd -m -G root testuser
---> Running in f0ee56d8ce0d
Removing intermediate container f0ee56d8ce0d
---> b5d03fc71fd8
Step 7/9 : USER root
---> Running in d4b5afccc880
Removing intermediate container d4b5afccc880
---> 86f88926749a
Step 8/9 : RUN apt-get update && apt-get install net-tools -y
---> Running in 8b5033e1a4c9
Get:1 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal-updates InRelease [107 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-backports InRelease [98.3 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security InRelease [107 kB]
Get:5 http://archive.ubuntu.com/ubuntu focal/restricted amd64 Packages [33.4 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal/main amd64 Packages [1275 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 Packages [177 kB]
Get:8 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages [11.3 MB]
Get:9 http://archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 Packages [1079 B]
Get:10 http://archive.ubuntu.com/ubuntu focal-updates/restricted amd64 Packages [10.6 kB]

```

```

Get:11 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [224 kB]
Get:12 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [126 kB]
Get:13 http://archive.ubuntu.com/ubuntu focal-backports/universe amd64 Packages [2903 B]
Get:14 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 Packages [10.6 kB]
Get:15 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [117 kB]
Get:16 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 Packages [1079 B]
Get:17 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [39.8 kB]
Fetched 13.9 MB in 2s (8010 kB/s)
Reading package lists...
Reading package lists...
Building dependency tree...
Reading state information...
The following NEW packages will be installed:
  net-tools
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 196 kB of archives.
After this operation, 864 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu focal/main amd64 net-tools amd64
1.60+git20180626.aebd88e-lubuntul [196 kB]
debconf: delaying package configuration, since apt-utils is not installed
Fetched 196 kB in 0s (3553 kB/s)
Selecting previously unselected package net-tools.
(Reading database ... 4122 files and directories currently installed.)
Preparing to unpack .../net-tools_1.60+git20180626.aebd88e-lubuntul_amd64.deb ...
Unpacking net-tools (1.60+git20180626.aebd88e-lubuntul) ...
Setting up net-tools (1.60+git20180626.aebd88e-lubuntul) ...
Removing intermediate container 8b5033e1a4c9
--> d986a915ba26
Step 9/9 : RUN apt-get install iputils-ping -y
--> Running in 6d4c5f56864d
Removing intermediate container 6d4c5f56864d
--> efda6ce9dfd0
Successfully built efda6ce9dfd0
Successfully tagged myubuntu_image:latest
root@my-docker:~#

```

Часть выходных данных опущена для краткости, но из того, что мы видим, вполне можно получить представление о том, как Dockerfile используется для создания образа. Давайте посмотрим на следующую команду и изучим выходные данные (рис. 2.40):

```
docker images ls --digests
```

```

root@my-docker:~# docker image ls --digests

```

REPOSITORY	TAG	DIGEST	IMAGE ID	CREATED
myubuntu_image	latest	<none>	efda6ce9dfd0	17 minutes ago
ubuntu	latest	sha256:747d2dbbaae995098c9792d99bd333c6783ce56150d1b11e333bbcecd5c54d7	1d622ef66b13	6 weeks ago

73.9kB

Рис. 2.40

В следующем разделе мы оптимизировали Dockerfile, объединив команды `RUN` и разделив их двойным амперсандом (рис. 2.41):

```
FROM ubuntu:latest
ENV HOME /root
LABEL ubuntu=myubuntu
ENTRYPOINT ["sleep"]
CMD ["50"]
USER root
RUN apt-get update && apt-get install net-tools -y && apt-get install iputils-ping
-y && useradd -m -G root testuser
```



```
FROM ubuntu:latest
ENV HOME /root
LABEL ubuntu=myubuntu
ENTRYPOINT ["sleep"]
CMD ["50"]
USER root
RUN apt-get update && apt-get install net-tools -y && apt-get install iputils-ping -y && useradd -m -G root testuser
```

Рис. 2.41

Как вы, очевидно, заметили, объединив инструкции `RUN`, мы уменьшили общий размер образа, как видно на рис. 2.42.



REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myubuntu_image1	latest	215679da58b1	11 minutes ago	98.2MB
myubuntu_image	latest	efda6ce9dfd0	35 minutes ago	99MB
ubuntu	latest	1d622ef86b13	6 weeks ago	73.9MB

Рис. 2.42

Чтобы лучше понять, как уменьшить размер образа благодаря объединению инструкций `RUN` в одну строку, рассмотрим еще два примера. В первом примере мы создадим файл размером 100 МБ, а в следующей инструкции `RUN` удалим его. Во втором примере создадим файл размером 100 МБ и в этой же строке (слое) удалим его.

Итак, прежде всего очистим систему командой `docker image rm <id_образа>`.

Запустим следующий код, а затем рассмотрим соответствующий скриншот (рис. 2.43).

```
vi Dockerfile
FROM ubuntu:latest
RUN truncate -s 100M mytext.txt1
RUN rm -rf mytext.txt
```

Сохраним файл, закроем редактор и запустим следующую команду:

```
docker build -t test_size .
```

На рис. 2.44 видно, что Dockerfile успешно собран.

¹ Создается файл `mytext.txt` размером 100 Мбайт.


```
FROM ubuntu:latest
RUN truncate -s 100M mytext.txt
RUN rm -rf mytext.txt
```

Рис. 2.43

```
root@my-docker:~# docker build -t test_size .
Sending build context to Docker daemon 32.77kB
Step 1/3 : FROM ubuntu:latest
----> 1d622ef86b13
Step 2/3 : RUN truncate -s 100M mytext.txt
----> Using cache
----> 71a8549242d5
Step 3/3 : RUN rm -rf mytext.txt
----> Running in 44efbd59624f
Removing intermediate container 44efbd59624f
----> 29e38c7c15f9
Successfully built 29e38c7c15f9
Successfully tagged test_size:latest
root@my-docker:~#
```

Рис. 2.44

Теперь отредактируем Dockerfile и сохраним его под другим именем (см. рис 2.45).

```
vi Dockerfile
FROM ubuntu:latest
RUN truncate -s 100M mytext.txt && rm -rf mytext.txt
```

```
FROM ubuntu:latest
RUN truncate -s 100M mytext.txt && rm -rf mytext.txt
```

Рис. 2.45

Сохранив файл и закрыв редактор, запустим следующую команду:

```
docker build -t test_size1 .
```

Как видно на рис. 2.46, Dockerfile успешно собран.

```
root@my-docker:~# docker build -t test_size1 .
Sending build context to Docker daemon 32.77kB
Step 1/2 : FROM ubuntu:latest
----> 1d622ef86b13
Step 2/2 : RUN truncate -s 100M mytext.txt && rm -rf mytext.txt
----> Running in 0da1781865ac
Removing intermediate container 0da1781865ac
----> 5d8c4dd1858e
Successfully built 5d8c4dd1858e
Successfully tagged test_size1:latest
root@my-docker:~#
```

Рис. 2.46

Итак, образы успешно созданы; теперь давайте сравним их размеры (рис. 2.47).

```
docker images
```

```
root@my-docker:~# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
test_size1	latest	5d8c4dd1858e	About a minute ago	73.9MB
test_size	latest	29e38c7c15f9	9 minutes ago	179MB
ubuntu	latest	1d622ef86b13	6 weeks ago	73.9MB

```
root@my-docker:~#
```

Рис. 2.47

Как видите, размеры образов `test_size` и `test_size1` существенно различаются. Очевидно, что если мы создаем и удаляем файл в пределах одного слоя, размер образа не увеличивается. Если удалить файл на другом слое, размер образа будет больше, потому что слои неизменяемы. Небольшое задание: подумайте, почему размеры `ubuntu` и `test_size1` совпадают?

Заключение

Эта глава была посвящена контейнерам и образам. Вы узнали, как создаются и выполняются контейнеры и в каких состояниях они могут находиться; как войти в контейнер и запускать оттуда команды. Затем мы рассмотрели архитектуру контейнеров, процессы их журналирования и исследования; проанализировали структуру образов, принципы их создания и связь между контейнерами и образами; изучили способы создания образов, узнали, что такое кеш сборки и `Dockerfile` и как исследовать образ. Иными словами, в этой главе вы получили общее представление об экосистеме контейнеров.

Следующая глава посвящена драйверам хранилища и томам.

Основные тезисы

- ◆ Образы — это шаблоны для создания контейнеров.
- ◆ Из образа можно создать любое количество контейнеров.
- ◆ Контейнер создается путем добавления к образу тонкого слоя чтения и записи.
- ◆ Контейнеры можно определить как «эфемерные», они очень быстро запускаются.
- ◆ У контейнеров могут быть различные состояния.
- ◆ В контейнер можно заключить базу данных, свою операционную систему, собственный веб-сервер; практически всё, что можно выполнять на хосте.
- ◆ Очень важно надлежащим образом управлять журналами контейнера, чтобы быстро находить и устранять неисправности.
- ◆ Образы — это шаблоны, из которых создаются контейнеры, при этом сами образы неизменяемы, т. е. после создания в них нельзя вносить изменения.
- ◆ Образы состоят из слоев.
- ◆ У каждого слоя имеется контрольная сумма (дайджест), которая вычисляется алгоритмом SHA.

- ◆ Идентификатор образа (image id) представляет собой контрольную сумму (дайджест), которая содержит хеш SHA-256 объекта конфигурации JSON.
- ◆ Docker создает промежуточные образы, что позволяет пользоваться преимуществами кеша сборки.
- ◆ Образы можно сохранять в виде tar-файлов.
- ◆ Контейнеры можно сохранять как образы.
- ◆ Образы можно извлекать из реестра или создавать в Dockerfile.
- ◆ Dockerfile — это текстовый файл без расширения, в который помещаются команды для сборки образа.
- ◆ Образы можно оптимизировать, располагая изменчивые слои сверху, а более стабильные — в нижней части.
- ◆ В образе должно быть как можно меньше слоев, чтобы он занимал меньше места. Сократить размер создаваемого образа позволяют многоступенчатые сборки.

Контрольный тест

Выберите подходящие варианты ответов.

1. Dockerfile — это:
 - Файл образа.
 - Документ.
 - Текстовый файл.
 - Ничего из вышеперечисленного.
2. Контейнер создает...
 - Тонкий слой только для чтения поверх образа.
 - Тонкий слой для чтения и записи под образом.
 - Тонкий слой только для чтения в любом месте образа.
 - Тонкий слой для чтения и записи поверх образа.
3. Образы хранятся в каталоге:
 - /var/lib/docker/<драйвер_хранилища>
 - /var/lib/docker/
 - /var/lib/docker/netns
 - Ничего из вышеперечисленного.
4. Журналы Docker хранятся как файлы JSON в каталоге:
 - /var/lib/docker/<id_контейнера>
 - /var/lib/docker/containers/<id_контейнера>

- `/var/lib/docker/containers`
 - Ничего из вышеперечисленного.
5. Каждый слой в образе содержит:
- Идентификатор образа (image id).
 - Имя образа.
 - Контрольную сумму (дайджест) SHA.
 - Идентификатор образа и контрольную сумму SHA.
6. Сервер Docker — это ключевой компонент Docker Engine, который включает:
- Containerd.
 - Dockerd.
 - runc.
 - Ничего из вышеперечисленного.
7. Копирование при записи — это механизм, который...
- Позволяет копировать образ.
 - Создает копию образа, только если образ был изменен.
 - Создает копию образа, только если образ не был изменен.
 - Создает копию образа, только если образ был удален.
8. Контрольная сумма (или дайджест) слоя, упорядоченная определенным образом для сборки образа, — это:
- Объект конфигурации.
 - Контрольная сумма SHA.
 - Файловая система.
 - Ничего из вышеперечисленного.
9. Многоступенчатая сборка позволяет использовать:
- Несколько команд RUN в одном Dockerfile.
 - Несколько команд CMD в одном Dockerfile.
 - Несколько файлов Dockerfile вместе.
 - Несколько команд FROM в одном Dockerfile.
10. Контейнер Docker имеет статус Dead, если не удалось...
- Загрузить контейнер.
 - Запустить контейнер.
 - Корректно удалить контейнер, потому что ресурсы были недоступны.
 - Корректно создать контейнер.

Ответы

1	c
2	d
3	a
4	b
5	c

6	b
7	b
8	a
9	d
10	c

Вопросы

1. Что такое контейнер?
2. Что такое образы и как они связаны с контейнерами?
3. В каких состояниях может находиться контейнер?
4. Что означает «неизменяемость» применительно к образам?
5. Как наиболее эффективно управлять журналами?
6. Что такое Dockerfile?
7. Что такое объект конфигурации в образе?
8. Что такое кеш сборки?
9. Что такое многоступенчатая сборка?
10. Что такое промежуточный образ?

Ключевые термины

- ◆ Контейнеры
- ◆ Образы
- ◆ Копирование при записи
- ◆ Тонкий слой чтения и записи
- ◆ Журналы контейнеров
- ◆ Исследование контейнера и получение информации
- ◆ Неизменяемость (image id)
- ◆ Идентификатор образа
- ◆ Контрольная сумма (дайджест) SHA
- ◆ Объект конфигурации
- ◆ Dockerfile
- ◆ Кеш сборки
- ◆ Многоступенчатая сборка
- ◆ Промежуточный образ

ГЛАВА 3

Драйверы хранилища и тома

В этой главе мы изучим драйверы хранилища и тома. Большие объемы данных обычно не записываются на слой чтения и записи в контейнере, — в таких случаях используются *тома*, о которых мы и поговорим далее. При относительно небольших рабочей нагрузке и объемах данных, для которых достаточно слоя чтения и записи контейнера, используются *драйверы хранилища*. Однако существует множество тонкостей и нюансов, связанных с драйверами хранилища и томами, в которых нужно хорошо ориентироваться.

Итак, в этой главе вы узнаете, что такое драйверы хранилища и тома и как они работают; научитесь выбирать драйверы хранилища под актуальные рабочие нагрузки, если ядро поддерживает использование нескольких драйверов, а также узнаете, почему тома зачастую являются предпочтительным механизмом сохранения данных в Docker и каковы наиболее эффективные способы их применения.

Темы главы следующие.

- ◆ Драйверы хранилища Docker
- ◆ Поддерживаемые драйверы хранилища
- ◆ Поддержка backing filesystem
- ◆ Драйверы хранилища overlay и overlay2
- ◆ Тома Docker

Драйверы хранилища Docker

Для записи больших объемов данных используются тома Docker, а для небольших объемов на слое чтения и записи — *драйверы хранилища*. Драйверы хранилища — это программное обеспечение для записи данных на слой чтения и записи в контейнере Docker.

Благодаря модульной архитектуре Docker поддерживает различные драйверы хранилища. От выбранного драйвера хранилища зависят способы хранения образов и контейнеров на хосте Docker, а также управление ими.

Если ядро позволяет использовать несколько драйверов хранилища, нужно выбрать вариант с оптимальным сочетанием стабильности и производительности. Docker предлагает список рекомендованных драйверов, но при выборе нужно руководствоваться своими потребностями.

Поддерживаемые драйверы хранилища

Существует несколько драйверов хранилища, поддерживаемых в Docker.

- ◆ **Overlay2.** Предпочтительный вариант для всех дистрибутивов Linux. Не требует дополнительной конфигурации и может использоваться напрямую.
- ◆ **Aufs.** Для версий Docker до 18.06 на Ubuntu рекомендуется использовать aufs, но только если установлена Ubuntu 14.04 на ядре 3.13, и, следовательно, overlay2 не поддерживается.
- ◆ **Device mapper.** Раньше Device mapper рекомендовали для дистрибутивов CentOS и RHEL (Red Hat Enterprise Linux), поскольку их драйверы ядра не поддерживали overlay2, но сейчас лучше использовать вместо него overlay2.
- ◆ Для Device mapper требуется direct-lvm, а значит, нужна дополнительная конфигурация для создания тонкого пула с использованием блочных устройств. Можно использовать loopback-lvm, поскольку для него не требуется дополнительная конфигурация, но у него очень низкая производительность, поэтому такой вариант не рекомендуется для рабочей среды.
- ◆ **Btrfs и zfs.** Драйверы хранилища btrfs и zfs можно использовать в том случае, если в файловой системе, на которой установлен хост (backing filesystem), тоже установлены btrfs и zfs. В таком случае их можно использовать для выполнения сложных задач, например для создания моментальных снимков и т. д. Требуется дополнительная конфигурация.
- ◆ **Vfs.** Этот драйвер хранилища не подходит для рабочей среды из-за чрезвычайно низкой производительности и отсутствия поддержки механизма копирования при записи.

Таким образом, предпочтительный вариант — overlay2. Драйвер overlay объявлен устаревшим с версии Docker Engine Enterprise 18.09, а Device mapper — с версии Docker Engine 18.09.

При желании можно выбрать другой драйвер хранилища, и позже вы узнаете, как это сделать. Но помните, что для некоторых драйверов требуется дополнительная конфигурация, в том числе на хосте Docker (на уровне физического или логического диска).

Поддержка *backing filesystem*

В табл. 3.1 указана поддержка backing filesystem, доступная для разных драйверов хранилища. Что такое *backing filesystem*? Для хранения данных в Docker нужна файловая система. Обычно данные хранятся в каталоге `/var/lib/docker/` — это и есть backing filesystem.

Таблица 3.1. Поддержка backing filesystem для разных драйверов хранилища

Драйвер хранилища	Поддержка файловой системы	Примечания
Overlay, overlay2	Xfs, ext4	Требования для xfs: ftype=1
Aufs	Xfs, ext4	Aufs не может использоваться в качестве backing filesystem aufs, btrfs или ecryptfs
Device mapper	Direct-lvm	Поддержка Device mapper должна быть включена в ядро; требуется специальная конфигурация для работы с Docker
Btrfs	Btrfs	Btrfs
Zfs	Zfs	Zfs
Vfs	Любая файловая система	Vfs совместим с любой файловой системой, но этот драйвер хранилища используется редко, потому что работает медленно и не поддерживает механизм копирования при записи

Драйверы хранилища *overlay* и *overlay2*

Файловая система *OverlayFS* — это современная версия UnionFS, похожая на aufs, но более быстрая в работе и простая в реализации. По умолчанию Docker предоставляет для этой файловой системы два драйвера хранилища: *overlay* и *overlay2*. В *overlay* были проблемы с нехваткой inode, а в *overlay2* они решены. *Overlay2* считается более стабильным и производительным драйвером хранилища, поэтому давайте рассмотрим его поближе.

Overlay2 поддерживается Docker Community Edition и Docker Enterprise Edition 17.06.02-ee5 и выше. *Overlay2* поддерживается даже на файловой системе xfs, если установить флаг `-n ftype` со значением 1.

Прежде всего давайте проверим, доступна ли *OverlayFS* в драйвере ядра Linux, запустив следующую команду и изучив результаты на рис. 3.1.

```
lsmod | grep overlay
```

```
root@my-docker:~# lsmod | grep overlay
overlay                77824  0
root@my-docker:~# █
```

Рис. 3.1

Да, файловая система доступна. Теперь проверим текущий драйвер хранилища. Снова запустим команду и посмотрим результат на рис. 3.2.

```
docker info | grep Storage
```

```
root@my-docker:~# docker info | grep Storage
Storage Driver: overlay2
```

Рис. 3.2

Мы обнаружили, что используется драйвер хранилища `overlay2`, т. е. вариант по умолчанию в большинстве случаев, и он нам подходит. Чтобы лучше понять, как всё это работает, попробуем изменить `overlay2` на `overlay`.

Шаг 1. Останавливаем Docker:

```
systemctl stop docker
```

Шаг 2. Копируем содержимое каталога `/var/lib/docker` в другое место:

```
cp -au /var/lib/docker /var/docker.bkp
```

Шаг 3. Вносим изменения в файл `daemon.json` в каталоге `/etc/docker/`. Если файла там нет, нужно создать его и передать следующую запись:

```
{
  "storage-driver": "overlay"
}
```

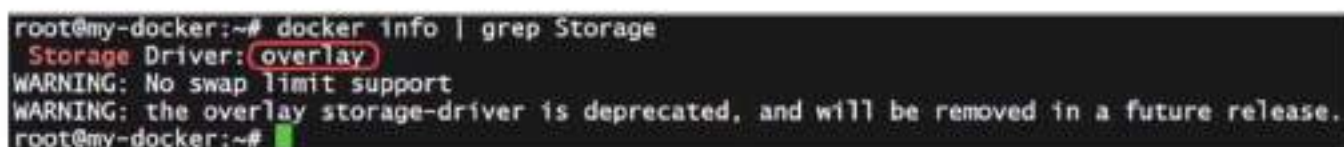
После того как запись передана или файл отредактирован, его нужно сохранить и закрыть.

Шаг 4. Запускаем Docker:

```
systemctl start docker
```

Шаг 5. Проверим драйвер хранилища с помощью следующей команды и получим выходные данные (рис. 3.3).

```
docker info | grep Storage
```



```
root@my-docker:~# docker info | grep Storage
Storage Driver: overlay
WARNING: No swap limit support
WARNING: the overlay storage-driver is deprecated, and will be removed in a future release.
root@my-docker:~#
```

Рис. 3.3

Все получилось. Драйвер хранилища изменен с `overlay2` на `overlay`. Точно так же можно вернуться к `overlay2`. Тут все просто, но если бы мы перешли на `device mapper` или `btrfs`, потребовалась бы дополнительная конфигурация, также нужно было бы убедиться, что выполнены все предварительные требования.

Характеристика и принцип работы драйвера хранилища `overlay2`

В OverlayFS каталоги и/или файловые системы структурированы таким образом, что изменения в каталоге или файловой системе записываются на верхнем слое, а не на нижнем. Нижний слой называется *lowerdir*, а верхний — *upperdir*. Процесс объединения и унификации этих слоев известен как *union mount* (каскадно-объединенное монтирование). Драйвер `overlay2` поддерживает до 128 слоев в *lowerdir*!

Давайте на примере посмотрим, как это работает. Сначала извлечем образ `redis`, затем изучим выходные данные на рис. 3.4.

```
docker image pull redis
```



```

root@my-docker:~# docker image pull redis
Using default tag: latest
latest: Pulling from library/redis
8559a31e96f4: Pull complete
85a6a5c53ff0: Pull complete
b69876b7abed: Pull complete
a72d84b9df6a: Pull complete
5ce7b314b19c: Pull complete
04c4bfb0b023: Pull complete
Digest: sha256:800f2587bf3376cb01e6307afe599ddce9439deafbd4fb8562829da96085c9c5
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
root@my-docker:~#

```

Рис. 3.4

Далее зайдем в каталог overlay2 и посмотрим на его содержимое (см. выходные данные на рис 3.5).

```

cd /var/lib/docker/overlay2
ls -ltr

```

```

root@my-docker:~# cd /var/lib/docker/overlay2
root@my-docker:/var/lib/docker/overlay2# ls -ltr
total 28
drwx----- 3 root root 4096 Jun 16 04:41 3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36
drwx----- 4 root root 4096 Jun 16 04:41 fb0c0b0d38e5f989b88b1aa218b017ede4771b223191d8b9d0a903bfc5559dfc
drwx----- 4 root root 4096 Jun 16 04:41 b8c33ada91b771d66b1e3bf778f1c82629157cc8ad4c723b5a2256f7df7afdd
drwx----- 4 root root 4096 Jun 16 04:41 0dbee01d98ce731682067f4a258fed94b50be76d503682c0223c8f6d0f068027
drwx----- 2 root root 4096 Jun 16 04:41 1
drwx----- 4 root root 4096 Jun 16 04:41 a265c17bf781b64e925f753dbbbf702469df7e9395518e98015bc435885238f2
drwx----- 4 root root 4096 Jun 16 04:41 679336a4517e46c0bdf4f2c0cd8620de6f43f6375a06cd1f4587c7ad9d8879cd
root@my-docker:/var/lib/docker/overlay2#

```

Рис. 3.5

Вы могли заметить, что образ `redis` состоит из шести слоев, а в `/var/lib/docker/overlay2` содержатся семь каталогов. Один из каталогов называется `1` (L в нижнем регистре). В нем содержатся сокращенные идентификаторы слоев в виде символических ссылок. Такой подход позволяет обойти ограничения на размер страниц при использовании команды монтирования (`mount`) с аргументами.

Чтобы понять, в чем тут суть, внимательно рассмотрим рис. 3.6.

```

cd 1
ls -ltr

```

```

root@my-docker:/var/lib/docker/overlay2# cd 1
root@my-docker:/var/lib/docker/overlay2/1# ls -ltr
total 24
lrwxrwxrwx 1 root root 72 Jun 16 04:41 /KIC21W3FTVMA201EPY68LE2ON -> ../3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36
lrwxrwxrwx 1 root root 72 Jun 16 04:41 QLV5FC4MCLMSS50WUKFVNM8F -> ../fb0c0b0d38e5f989b88b1aa218b017ede4771b223191d8b9d0a903bfc5559dfc
lrwxrwxrwx 1 root root 72 Jun 16 04:41 ZML4260W7TNNW702P2AK3U36 -> ../b8c33ada91b771d66b1e3bf778f1c82629157cc8ad4c723b5a2256f7df7afdd
lrwxrwxrwx 1 root root 72 Jun 16 04:41 3E3MWCN6JIC1F0HTNQB0KXT1I -> ../0dbee01d98ce731682067f4a258fed94b50be76d503682c0223c8f6d0f068027
lrwxrwxrwx 1 root root 72 Jun 16 04:41 3KXHBEB4UQP5ASPVSCCTVMGR12 -> ../a265c17bf781b64e925f753dbbbf702469df7e9395518e98015bc435885238f2
lrwxrwxrwx 1 root root 72 Jun 16 04:41 WQ0T5AMQAGBQUSBOUAYQCKEV -> ../679336a4517e46c0bdf4f2c0cd8620de6f43f6375a06cd1f4587c7ad9d8879cd

```

Рис. 3.6

В самом нижнем слое (он выделен красным) находится файл `link` с сокращенным идентификатором и каталог `diff`, включающий все содержимое слоя (см. рис. 3.7–3.9).

```
ls -ltr 3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36
```

```
root@my-docker:/var/lib/docker/overlay2# ls -ltr 3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36
total 8
-rw-r--r-- 1 root root 26 Jun 16 04:41 link
drwxr-xr-x 21 root root 4096 Jun 16 04:41 diff
-rw----- 1 root root 0 Jun 16 04:41 committed
root@my-docker:/var/lib/docker/overlay2#
```

Рис. 3.7

```
cd 3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36
```

```
ls -ltr
cat link
```

```
root@my-docker:/var/lib/docker/overlay2# cd 3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36
root@my-docker:/var/lib/docker/overlay2/3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36# ls -ltr
total 8
-rw-r--r-- 1 root root 26 Jun 16 04:41 link
drwxr-xr-x 21 root root 4096 Jun 16 04:41 diff
-rw----- 1 root root 0 Jun 16 04:41 committed
root@my-docker:/var/lib/docker/overlay2/3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36# cat link
HJC2IV3EIVWA20IIPf66BLEZQnroot@my-docker:/var/lib/docker/overlay2/3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36#
```

Рис. 3.8

```
cd diff
ls
```

```
root@my-docker:/var/lib/docker/overlay2/3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36# cd diff
root@my-docker:/var/lib/docker/overlay2/3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36/diff# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys usr var
root@my-docker:/var/lib/docker/overlay2/3d8a8ab913c1d9e383c7b69717677d6add36d7afaa4209c9f3af31406d9b7e36/diff#
```

Рис. 3.9

Слои, которые находятся над самым нижним, будут содержать файл с именем `lower`, где указан родительский слой, каталог `diff` с содержимым, а также каталог `work`, который используется OverlayFS. Наконец, там будет объединенный слой, где объединено все содержимое.

Тома Docker

Если записывать данные только на слой чтения и записи в контейнере, может возникнуть несколько проблем. Во-первых, и это основная проблема, данные не сохраняются после удаления контейнера. Во-вторых, поскольку слой записи использует драйвер хранилища для реализации объединенной файловой системы, работоспособность контейнера может снижаться. В-третьих, слой записи так тесно связан с самим контейнером, что управлять данными будет весьма непросто. Наконец, контейнеры быстро потеряют свою легкость и гибкость, если записывать в них все данные.

Тома предназначены для постоянного хранения данных. Для записи данных большого объема лучше использовать тома, а также механизмы *bind mount* (монтирование каталогов с хоста) и *tmpfs* (temporary file system — временная файловая система). Схема хранения данных представлена на рис. 3.10.

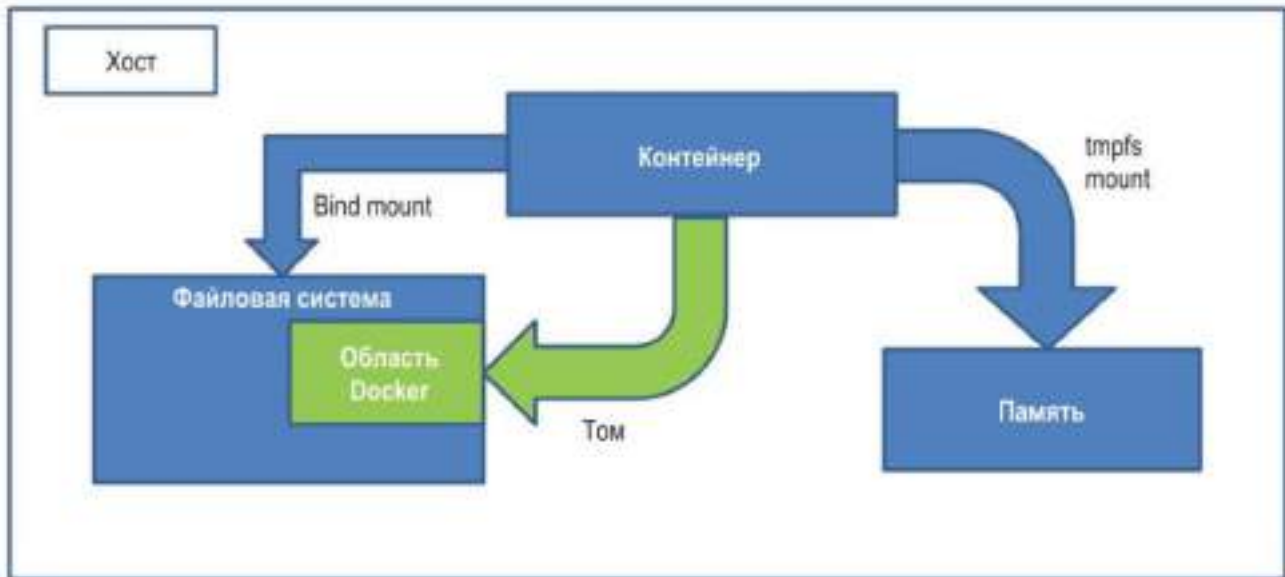


Рис. 3.10. Хранение данных в Docker

Итак, что такое том?

Том — это каталог в той части хост-системы, где находится Docker. Он подключен к каталогу в контейнере, чтобы можно было записывать и считывать данные. Данные контейнера хранятся в хост-системе, а значит, даже при удалении контейнера все данные остаются. Также можно использовать *bind mount* и сопоставить с каталогом в контейнере любой каталог из файловой системы хоста (не связанный с Docker, в отличие от тома). При использовании *tmpfs* каталог контейнера подключается к памяти хоста.

Тома решают проблему хранения и передачи данных. Более того, их производительность гораздо выше, чем у слоя контейнера для записи. Можно смонтировать том для нескольких контейнеров одновременно, это особенно удобно, если контейнерам нужно обмениваться данными между собой.

Тома также поддерживают использование драйверов томов, что позволяет выполнять дополнительные операции с данными, например шифровать их или перемещать в облако или на удаленные серверы.

Резюмируем информацию о томах, прежде чем двигаться дальше.

Тома — это просто физическая файловая система на хосте, подключенная к виртуальной файловой системе в Docker, так что любые изменения в контейнерах записываются в хост-систему. И наоборот: изменения на уровне хост-системы отражаются на уровне контейнера.

Рассмотрим несколько примеров, чтобы лучше понять, как это работает.

Прежде всего попробуем применить *bind mount*. В хост-системе создадим каталог с именем *testing* в каталоге *root*. Используя механизм *bind mount* (монтирование каталога с хоста), мы не затрагиваем *область Docker*. Затем подключаем новый каталог к каталогу *test1* в контейнере. Каталог *test1* будет создан «на лету», одновременно с контейнером.

В хост-системе сделаем следующее (рис. 3.11):

```
mkdir -p /root/testing
ls
```

```
root@mydocker:~# mkdir -p /root/testing
root@mydocker:~# ls
testing
root@mydocker:~# █
```

Рис. 3.11

Теперь создадим контейнер (рис. 3.12):

```
docker run -it --name myalpine -v /root/testing:/test1 alpine:latest
ls -ltr
```

```
root@mydocker:~# docker run -it --name myalpine -v /root/testing:/test1 alpine:latest
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
df20fa9351a1: Pull complete
Digest: sha256:185518070891758909c9f839cf4ca393ee977ac378609f700f60a771a2dfe321
Status: Downloaded newer image for alpine:latest
/ # ls -ltr
total 60
drwxr-xr-x 12 root root      4096 May 29 14:20 var
drwxr-xr-x  7 root root      4096 May 29 14:20 usr
drwxrwxrwt  2 root root      4096 May 29 14:20 tmp
drwxr-xr-x  2 root root      4096 May 29 14:20 srv
drwxr-xr-x  2 root root      4096 May 29 14:20 sbin
drwxr-xr-x  2 root root      4096 May 29 14:20 run
drwxr-xr-x  2 root root      4096 May 29 14:20 opt
drwxr-xr-x  2 root root      4096 May 29 14:20 mnt
drwxr-xr-x  5 root root      4096 May 29 14:20 media
drwxr-xr-x  7 root root      4096 May 29 14:20 lib
drwxr-xr-x  2 root root      4096 May 29 14:20 home
drwxr-xr-x  2 root root      4096 May 29 14:20 bin
drwxr-xr-x  2 root root      4096 Jul 11 04:53 test1
drwxr-xr-x  1 root root      4096 Jul 11 04:54 etc
dr-xr-xr-x 13 root root         0 Jul 11 04:54 sys
dr-xr-xr-x 121 root root         0 Jul 11 04:54 proc
drwxr-xr-x  5 root root      360 Jul 11 04:54 dev
drwx----- 1 root root      4096 Jul 11 04:54 root
/ # █
```

Рис. 3.12

Команда, приведенная выше, создает контейнер с именем `myalpine` в интерактивном режиме (`-i`) с подключенным терминалом (`-t`). Мы не указываем `-d`, потому что хотим войти в контейнер напрямую, а не запускать его в фоновом режиме. Мы также указываем флаг `-v` (или `--volume`, т. е. том), чтобы подключить каталог `/root/testing/` в хост-системе к каталогу `test1` в контейнере; этот каталог, как мы отмечали ранее, будет создан «на лету», вместе с контейнером (см. рис. 3.12).

Как и ожидалось, в контейнере содержится каталог `test1`. Создадим внутри этого каталога файл и что-нибудь в него запишем. Эти изменения должны отразиться в каталоге в хост-системе. Затем мы удалим контейнер и проверим, сохранились ли данные.

```
cd test1
vi myfile
```

Напишем в файле: **This is a demo to check for data persistence in a container** (Проверка сохранения данных в контейнере), затем сохраним и закроем его.

Проверив файл в контейнере, мы получим примерно следующие результаты (рис. 3.13).

```
/ # cd test1
/test1 # vi myfile
/test1 # cat myfile
This is a demo to check for data persistence in a container
/test1 #
```

Рис. 3.13

Теперь проверим, отражена ли эта запись в каталоге `/root/testing/` в хост-системе.

```
exit
cd /root/testing/
ls -ltr
cat myfile
```

Естественно, данные в хост-системе сохранились (рис. 3.14).

```
root@mydocker:~/testing# cd /root/testing
root@mydocker:~/testing# ls -ltr
total 4
-rw-r--r-- 1 root root 60 Jul 11 05:04 myfile
root@mydocker:~/testing# cat myfile
This is a demo to check for data persistence in a container
root@mydocker:~/testing#
```

Рис. 3.14

Прежде чем двигаться дальше, проверим, как исходный каталог в хост-системе (`/root/testing`) сопоставляется с целевым каталогом в контейнере (`/test1`). На рис. 3.15 можно видеть, что указан тип (**Type**) `bind`, потому что мы используем `bind mount`.

```
cd ~
docker container ls -a
docker inspect -f '{{ json .Mounts }}' 07484e3aaff7
```

```
root@mydocker:~/testing# cd ~
root@mydocker:~# docker container ls -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS   NAMES
07484e3aaff7   alpine:latest  "/bin/sh"               14 minutes ago Up 14 minutes           myalpine
root@mydocker:~# docker inspect -f '{{ json .Mounts }}' 07484e3aaff7
[{"Type": "bind", "Source": "/root/testing", "Destination": "/test1", "Mode": "", "RW": true, "Propagation": "rprivate"}]
```

Рис. 3.15

Проверим, сохранятся ли данные после удаления контейнера:

```
docker ps
docker container rm myalpine -f
cd /root/testing
ls -ltr
at myfile
```

Взгляните на рис. 3.16.



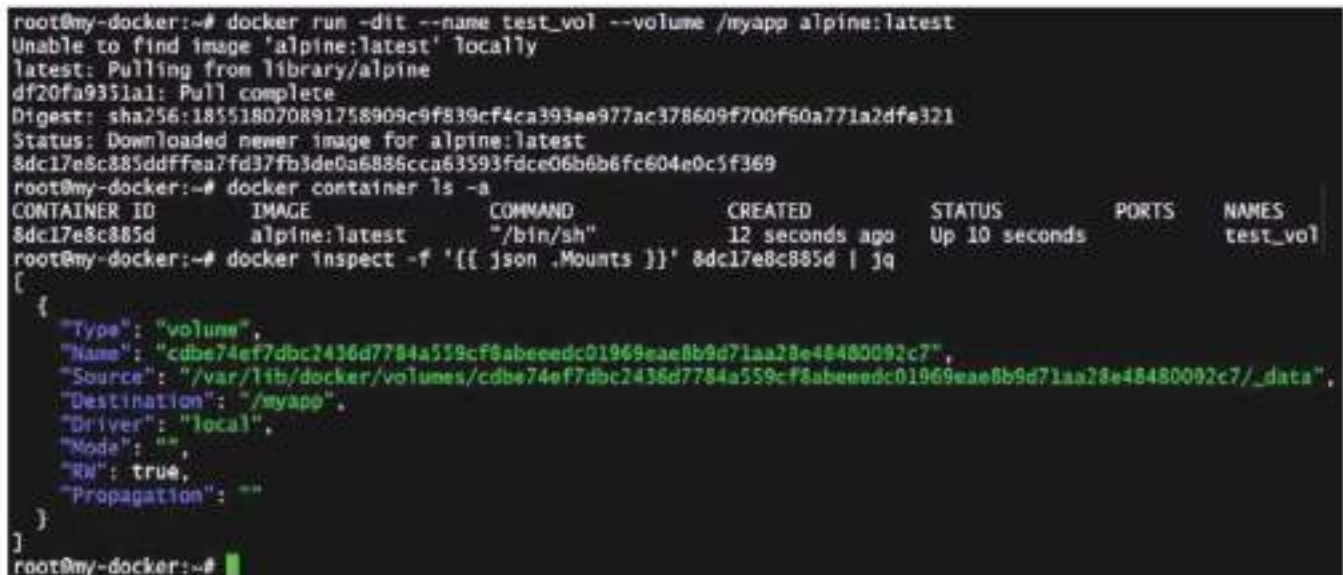
```
root@mydocker:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES
07484e3aaff7   alpine:latest   "/bin/sh"               23 minutes ago   Up 23 minutes           myalpine
root@mydocker:~# docker container rm myalpine -f
myalpine
root@mydocker:~# cd /root/testing
root@mydocker:~/testing# ls -ltr
total 4
-rw-r--r-- 1 root root 60 Jul 11 05:04 myfile
root@mydocker:~/testing# cat myfile
This is a demo to check for data persistence in a container
root@mydocker:~/testing#
```

Рис. 3.16

Как видим, данные не исчезли, что говорит об их сохраняемости.

Давайте рассмотрим пример сохранения данных в части хост-системы, определяемой как *область Docker*. Здесь мы укажем только целевой каталог, т. е. каталог в контейнере, а исходный каталог нам предоставит Docker (см. рис. 3.17). Фактически мы создадим *анонимный том*.

```
docker run -dit --name test_vol --volume /myapp alpine:latest
docker container ls -a
apt update && apt install jq -y1
docker inspect -f '{{ json .Mounts }}' 8dc17e8c885d | jq
```



```
root@my-docker:~# docker run -dit --name test_vol --volume /myapp alpine:latest
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
df20fa9351a1: Pull complete
Digest: sha256:185518070891758909c9f839cf4ca393ae977ac378609f700f60a771a2dfe321
Status: Downloaded newer image for alpine:latest
8dc17e8c885ddffea7fd37fb3de0a6886cca63593fdce06b6b6fc604e0c5f369
root@my-docker:~# docker container ls -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES
8dc17e8c885d   alpine:latest   "/bin/sh"               12 seconds ago   Up 10 seconds           test_vol
root@my-docker:~# docker inspect -f '{{ json .Mounts }}' 8dc17e8c885d | jq
[
  {
    "Type": "volume",
    "Name": "cdbe74ef7dbc2436d7784a559cf8abecedc01969eae8b9d71aa28e48480092c7",
    "Source": "/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abecedc01969eae8b9d71aa28e48480092c7/_data",
    "Destination": "/myapp",
    "Driver": "local",
    "Mode": "",
    "RW": true,
    "Propagation": ""
  }
]
```

Рис. 3.17

¹ Для этой команды необходимо установить `jq` — инструмент для форматирования выходных данных JSON. Если он уже установлен (как в данном случае), выполнять эту команду снова не нужно.

На рис. 3.17 виден тип (**Type**) `volume`, т. е. том; исходный каталог (**Source**) — каталог, который должен находиться в области *Docker* (см. рис. 3.10); целевой каталог (**Destination**) — это каталог, который мы указали при создании контейнера.

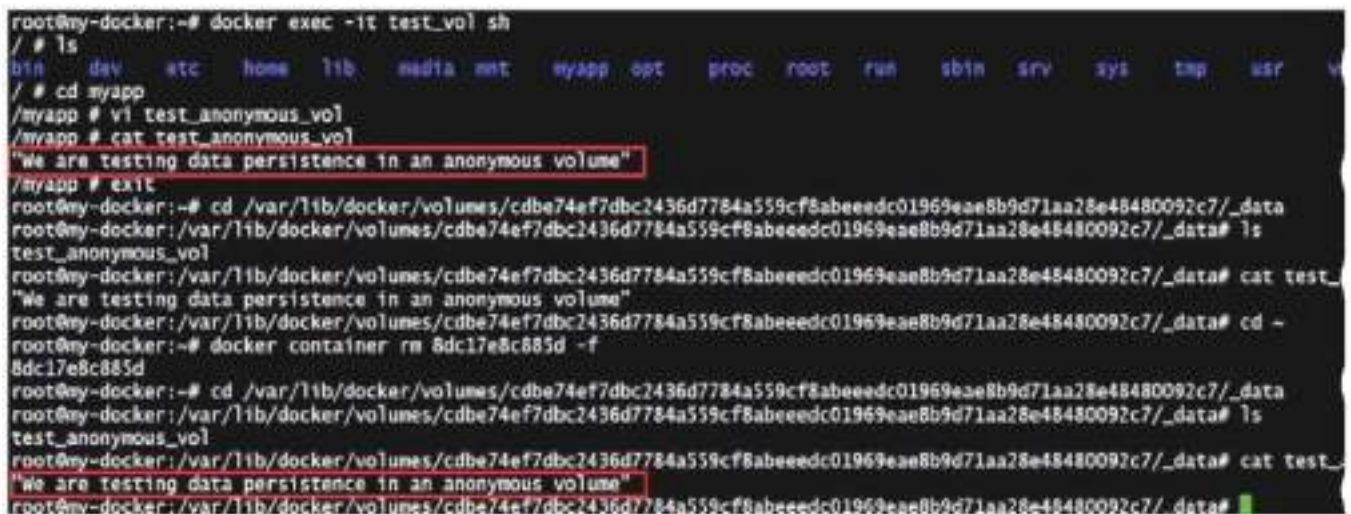
Давайте еще раз проверим сохраняемость данных. Создадим в контейнере файл, что-нибудь в него запишем, а затем посмотрим, сохранится ли файл, когда мы удалим контейнер.

```
docker exec -it test_vol sh
cd myapp
vi test_anonymous_vol
```

Напишем в файле: **We are testing data persistence in an anonymous volume** (Мы тестируем сохраняемость данных в анонимном томе), затем сохраним и закроем файл.

```
cat test_anonymous_vol
exit
cd /var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969e-
ae8b9d71aa28e48480092c7/_data
ls
cat test_anonymous_vol
cd -
docker container rm 8dc17e8c885d -f
cd /var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969e-
ae8b9d71aa28e48480092c7/_data
ls
cat test_anonymous_vol
```

Выходные данные показаны на рис. 3.18.



```
root@my-docker:~# docker exec -it test_vol sh
/ # ls
bin  dev  etc  home  lib  media  mnt  myapp  opt  proc  root  run  sbin  srv  sys  tmp  usr  v
/ # cd myapp
/myapp # vi test_anonymous_vol
/myapp # cat test_anonymous_vol
"We are testing data persistence in an anonymous volume"
/myapp # exit
root@my-docker:~# cd /var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data
root@my-docker:/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data# ls
test_anonymous_vol
root@my-docker:/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data# cat test_
"We are testing data persistence in an anonymous volume"
root@my-docker:/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data# cd -
root@my-docker:~# docker container rm 8dc17e8c885d -f
8dc17e8c885d
root@my-docker:~# cd /var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data
root@my-docker:/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data# ls
test_anonymous_vol
root@my-docker:/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data# cat test_
"We are testing data persistence in an anonymous volume"
root@my-docker:/var/lib/docker/volumes/cdbe74ef7dbc2436d7784a559cf8abeeedc01969eae8b9d71aa28e48480092c7/_data#
```

Рис. 3.18

Все работает! Однако можно выполнить эту задачу более эффективно. Мы можем указать именованный том, создав его независимо от контейнера. Главное преимущество *именованного тома* перед анонимным заключается в том, что нет необходимости иметь дело с длинными именами каталогов. Так значительно проще, осо-

бенно в рабочей среде, где, возможно, придется оперировать сотнями контейнеров, соблюдая при этом соглашения об именовании и прочие условия.

Итак, приступим к созданию тома.

```
docker volume create myvolume
```

Получаем выходные данные (рис. 3.19).

```
root@my-docker:~# docker volume create myvolume
myvolume
root@my-docker:~# docker volume inspect myvolume
[
  {
    "CreatedAt": "2020-06-20T09:32:35Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/myvolume/_data",
    "Name": "myvolume",
    "Options": {},
    "Scope": "local"
  }
]
```

Рис. 3.19

Как видите, том `myvolume` указывает на каталог `/var/lib/docker/volumes/myvolume/_data` — наш будущий исходный каталог, который мы сопоставим с контейнером. Давайте сделаем это (рис. 3.20):

```
docker container run -dit --name mycentos --volume myvolume:/test
centos:latest
```

```
root@my-docker:~# docker container run -dit --name mycentos --volume myvolume:/test centos:latest
Unable to find image 'centos:latest' locally
latest: Pulling from library/centos
6910e5a164f7: Pull complete
Digest: sha256:4062bbdd1bb0801b0aa38e0f83dece70fb7a5e9bce223423a68de2d8b784b43b
Status: Downloaded newer image for centos:latest
b2334b637cdbfa19236b77ea72b2261da3eb88f9d332abb8b6e4adaa8616adb0
root@my-docker:~# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS   NAMES
b2334b637cdb   centos:latest   "/bin/bash"             36 seconds ago   Up 34 seconds           mycentos
root@my-docker:~# docker inspect -f '{{ json .Mounts }}' b2334b637cdb | jq
[
  {
    "Type": "volume",
    "Name": "myvolume",
    "Source": "/var/lib/docker/volumes/myvolume/_data",
    "Destination": "/test",
    "Driver": "local",
    "Mode": "z",
    "RW": true,
    "Propagation": ""
  }
]
```

Рис. 3.20

Мы использовали ранее созданный том, чтобы сопоставить его с контейнером `mycentos`. Интересный момент: мы не только сохраним данные после удаления контейнера, но и можем подключить к одному тому несколько контейнеров. Это особенно удобно, когда несколько контейнеров используют одни и те же данные.

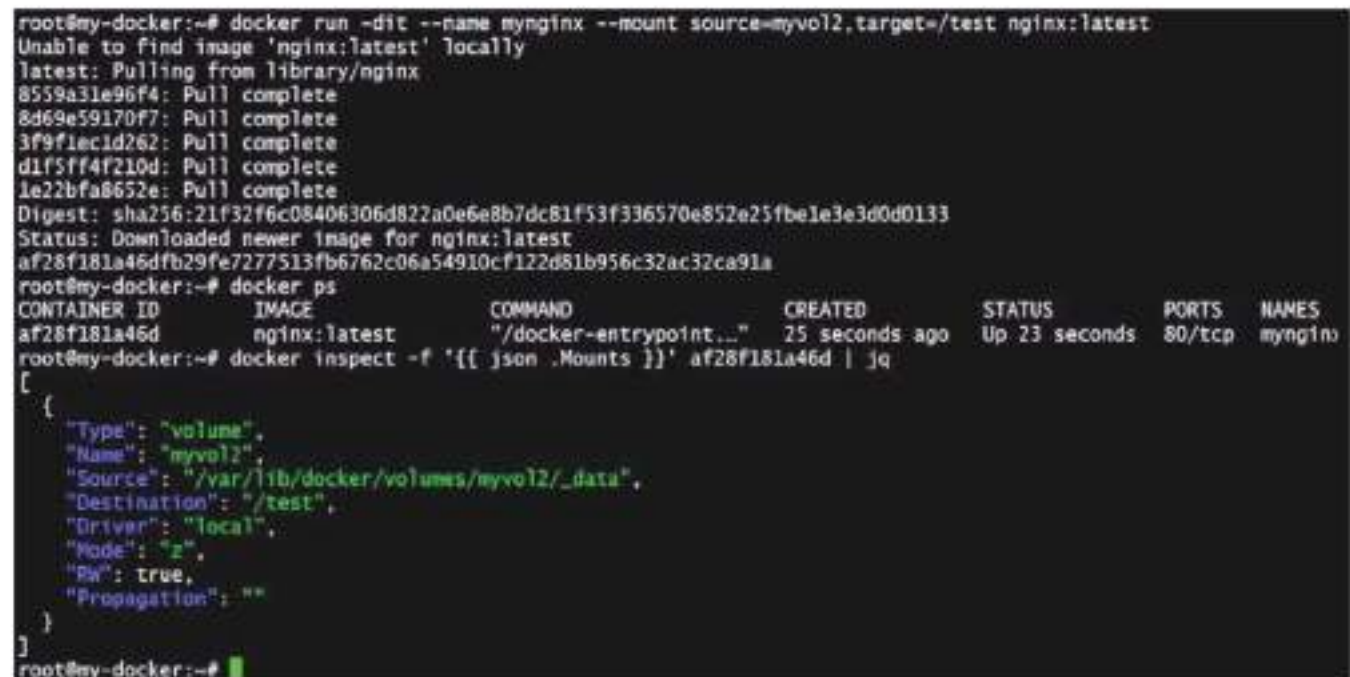
Стоит отметить, что флаг `-v` или `--volume` использовался только для сервисов Swarm до Docker 17.06. С версии 17.06 можно использовать также флаг `--mount` для отдельных контейнеров. Поскольку флаг `--mount` проще понимать и применять, я рекомендую именно его.

Рассмотрим пример использования `--mount`.

На этот раз не будем создавать том заранее, пусть Docker сделает это «на лету», — у него есть такая возможность.

Запустим следующую команду и изучим выходные данные на рис. 3.21.

```
docker run -dit --name mynginx --mount source=myvol2,target=/test nginx:latest
docker ps
docker inspect -f '{{ json .Mounts }}' af28f181a46d | jq
```



```
root@my-docker:~# docker run -dit --name mynginx --mount source=myvol2,target=/test nginx:latest
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
8559a31e96f4: Pull complete
8d69e59170f7: Pull complete
3f9f1ec1d262: Pull complete
d1f5ff4f210d: Pull complete
1e22bfa8652e: Pull complete
Digest: sha256:21f32f6c08406306d822a0e6e8b7dc81f53f336570e852e25fbc1e3e3d0d0133
Status: Downloaded newer image for nginx:latest
af28f181a46dfb29fe7277513fb6762c06a54910cf122d81b956c32ac32ca91a
root@my-docker:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS      NAMES
af28f181a46d   nginx:latest  "/docker-entrypoint..."  25 seconds ago  Up 23 seconds  80/tcp     mynginx
root@my-docker:~# docker inspect -f '{{ json .Mounts }}' af28f181a46d | jq
[
  {
    "Type": "volume",
    "Name": "myvol2",
    "Source": "/var/lib/docker/volumes/myvol2/_data",
    "Destination": "/test",
    "Driver": "local",
    "Mode": "z",
    "RW": true,
    "Propagation": ""
  }
]
```

Рис. 3.21

Существует еще один механизм сохранения файлов за пределами слоя чтения и записи контейнера — `tmpfs`. Файлы создаются в памяти хост-системы, а не на слое записи контейнера. В отличие от `bind mount` и томов, `tmpfs` монтируется временно и существует, только пока контейнер запущен. Когда контейнер останавливается, `tmpfs` удаляется, следовательно, сохраняемости файлов нет.

Рассмотрим пример использования `tmpfs`: выполним следующие команды и изучим выходные данные на рис. 3.22.

```
docker run -it --name test_tmpfs --mount type=tmpfs,destination=/testing centos:latest
ls
```

Нужно явно указать тип `tmpfs`, но в остальном команда должна быть вам знакомой. Итак, `-tmpfs` предоставляет временное хранилище за пределами слоя записи и чтения контейнера. Этот вариант имеет ограниченные возможности, но может пригодиться в некоторых случаях.


```

root@my-docker:~# docker run -it --name test_tmpfs --mount type=tmpfs,destination=/testing centos:latest
Unable to find image 'centos:latest' locally
latest: Pulling from library/centos
6910e5a164f7: Pull complete
Digest: sha256:4062bbdd1bb0801b0aa38e0f83dece70fb7a5e9bce223423a68de2d8b784b43b
Status: Downloaded newer image for centos:latest
[root@0d461e05e3bd6 /]# ls
bin dev etc home lib lib64 lost+found media mnt opt proc root run sbin srv sys testing tmp usr var
[root@0d461e05e3bd6 /]# exit

```

Рис. 3.22

Заключение

В этой главе мы рассмотрели слой чтения и записи контейнера и драйверы хранилища, используемые для хранения данных, записанных на этот слой; ознакомились с доступными драйверами хранилища и подробно обсудили рекомендуемый вариант для Docker — overlay2. Затем изучили концепцию сохраняемости данных и узнали, как тома и bind mount обеспечивают сохраняемость данных после удаления контейнера. Тома имеют более высокую производительность и эффективность по сравнению со слоем чтения и записи контейнера, и использование томов в рабочей среде, как вы могли убедиться, чрезвычайно важно. В целом, изучив эту главу, вы должны получить относительно полное представление о том, как реализуется хранение данных в контейнерах. В следующей главе мы подробно рассмотрим сети Docker.

Основные тезисы

- ◆ Данные всегда записываются на тонкий слой чтения и записи, который создается вместе с контейнером.
- ◆ Как правило, этот слой чтения и записи используется для небольших объемов данных, поскольку не предназначен для рабочих нагрузок с интенсивной записью.
- ◆ Драйверы хранилища — это программное обеспечение для записи данных на слой чтения и записи.
- ◆ Если ядро позволяет использовать несколько драйверов хранилища, нужно выбрать вариант с оптимальным сочетанием стабильности и производительности.
- ◆ Docker предлагает несколько драйверов хранилища, но обычно достаточно использовать overlay2.
- ◆ Драйвер хранилища overlay2 объединяет каталоги и/или файловые системы таким образом, что на самый нижний слой данные не записываются. Все изменения записываются на слои над ним. Нижний слой называется *lowerdir*, а верхний — *upperdir*. Слои объединяются в процессе, называемом *каскадно-объединенным монтированием* (union mount).
- ◆ На верхние слои записываются только *отличия* (*diff*), чтобы образ не увеличивался в размерах.

- ◆ Для постоянного хранения данных в системе используются тома и механизм `bind mount`, поскольку контейнеры временны, можно сказать, «эфемерны» по своей природе и могут сменять друг друга в любое время. При этом мы, естественно, не хотим терять данные, сохраненные в хост-системе.
- ◆ Механизм `tmpfs` позволяет записывать данные за пределами контейнера, в памяти хоста.
- ◆ При использовании `tmpfs` данные не хранятся постоянно. Когда контейнер останавливается, `tmpfs` удаляется, и файлы, записанные там, становятся недоступными.

Контрольный тест

Выберите подходящий вариант ответа.

1. Для каких рабочих нагрузок подходит слой чтения и записи в контейнере:
 - С интенсивной записью.
 - С интенсивным чтением.
 - С небольшим объемом данных.
 - Ничего из вышеперечисленного.
2. Драйвер хранилища — это программное обеспечение, которое...
 - Создает образы.
 - Создает контейнеры.
 - Записывает данные на слой контейнера только для чтения.
 - Записывает данные на слой контейнера для чтения и записи.
3. Механизм `tmpfs` позволяет...
 - Хранить данные в контейнере.
 - Хранить данные в томе.
 - Хранить данные в памяти системы.
 - Хранить данные в хранилище системы.
4. При использовании механизма `bind mount`...
 - Файл или каталог в хост-системе монтируется в контейнер.
 - Файл или каталог из области `Docker` монтируется в контейнер.
 - Файл или каталог из хост-системы сопоставляется с памятью контейнера.
 - Файловая система хоста и контейнер никак не связаны.
5. Тома поддерживают использование драйверов томов, которые позволяют...
 - Хранить данные на диске хост-компьютера.
 - Хранить данные в файловой системе контейнера.

- Шифровать данные и переносить их в другое расположение.
 - Сжимать данные.
6. Команда `docker run -it --name myalpine -v /root/test/:/test\ alpine:latest...`
- Создает контейнер и подключает каталог `/root/test/` в контейнере к каталогу `/test` на хост-компьютере.
 - Создает контейнер и подключает каталог `/root/test/` на хост-компьютере к каталогу `/test` в контейнере.
 - Не создает контейнер, но подключает каталог `/root/test/` на хост-компьютере к каталогу `/test` в контейнере.
 - Не создает контейнер, но подключает каталог `/root/test/` в контейнере к каталогу `/test` на хост-компьютере.
7. Данные, записанные на слой контейнера для чтения и записи...
- Не исчезают при остановке и запуске контейнера.
 - Теряются при остановке и запуске контейнера.
 - Могут потеряться частично.
 - Ничего из вышеперечисленного.
8. Драйверы хранилища `btrfs` и `zfs` можно использовать, если...
- Backing filesystem поддерживает `btrfs` и `zfs`.
 - Backing filesystem поддерживает `direct-lvm`.
 - Backing filesystem поддерживает `loopback-lvm`.
 - В backing filesystem тоже установлены `btrfs` и `zfs`.
9. Драйвер хранилища `overlay2` предпочтительнее, чем `overlay`, потому что...
- `overlay2` превосходит `overlay` по производительности.
 - RHEL и CentOS теперь поддерживают `overlay2`.
 - В `overlay2` нет проблем с нехваткой `inode`.
 - `overlay2` проще установить.

Ответы

1	c
2	d
3	c
4	a
5	c

6	b
7	a
8	d
9	c

Вопросы

1. Что такое драйвер хранилища?
2. Какие драйверы хранилища доступны в Docker?
3. На какой файловой системе Linux основаны драйверы хранилища группы overlay?
4. Что означает концепция отличий (diff) в контексте слоев образа?
5. Как работает драйвер хранилища overlay2?
6. Что такое том?
7. Что такое bind mount?
8. Чем том отличается от bind mount?
9. В каких случаях лучше использовать один том для нескольких контейнеров?
10. Что такое tmpfs?

Ключевые термины

- ◆ Драйвер хранилища
- ◆ Слой чтения и записи
- ◆ Overlay и overlay2
- ◆ aufs
- ◆ Многослойная файловая система
- ◆ Backing filesystem
- ◆ Device mapper
- ◆ Каскадно-объединенное монтирование (union mount)
- ◆ Lowerdir, upperdir
- ◆ Объединенный каталог
- ◆ Сохраняемость данных
- ◆ Хранилище
- ◆ Тома
- ◆ Драйвер тома
- ◆ Bind mount
- ◆ tmpfs

ГЛАВА 4

Модель *Container Network* и сетевой мост Docker

В этой главе рассматриваются концепция сетей в Docker и принципы реализации сетей в реальных развертываниях. Вы получите представление о сетевом стеке Docker и о том, как создать надежную и высокопроизводительную сетевую инфраструктуру.

Сети Docker ориентированы на приложения. С одной стороны, они обеспечивают все необходимое для нормального функционирования, с другой — предоставляют достаточный уровень абстракции, чтобы не усложнять жизнь разработчикам.

Глава включает следующие темы.

- ◆ Модель Container Network (Container Network Model, CNM)
- ◆ Интерфейсы драйвера CNM
- ◆ Libnetwork
- ◆ Драйверы Docker
- ◆ Сетевой мост Docker

Модель *Container Network*

Сеть Docker построена на архитектуре *Container Network Model* (CNM). CNM — это архитектура, предоставляющая простую и эффективную сетевую модель, которая позволяет запускать приложения в защищенных портативных контейнерах. Модель Container Network (CNM) представлена на рис. 4.1.

Рассмотрим элементы модели CNM подробнее.

- ◆ *Песочница* содержит полную конфигурацию сетевого стека контейнера, т. е. интерфейсы, таблицы маршрутизации, настройки DNS и т. д. Песочница полностью изолирована от внешнего мира, за исключением *конечных точек*.
- ◆ *Конечные точки* соединяют песочницу с внешним миром через сети. Можно представить конечные точки как двери наружу.

- ♦ Сеть может представлять собой сложную инфраструктуру, созданную пользователем, мост Linux или VLAN. Конечные точки, подключенные к одной сети, могут взаимодействовать друг с другом.

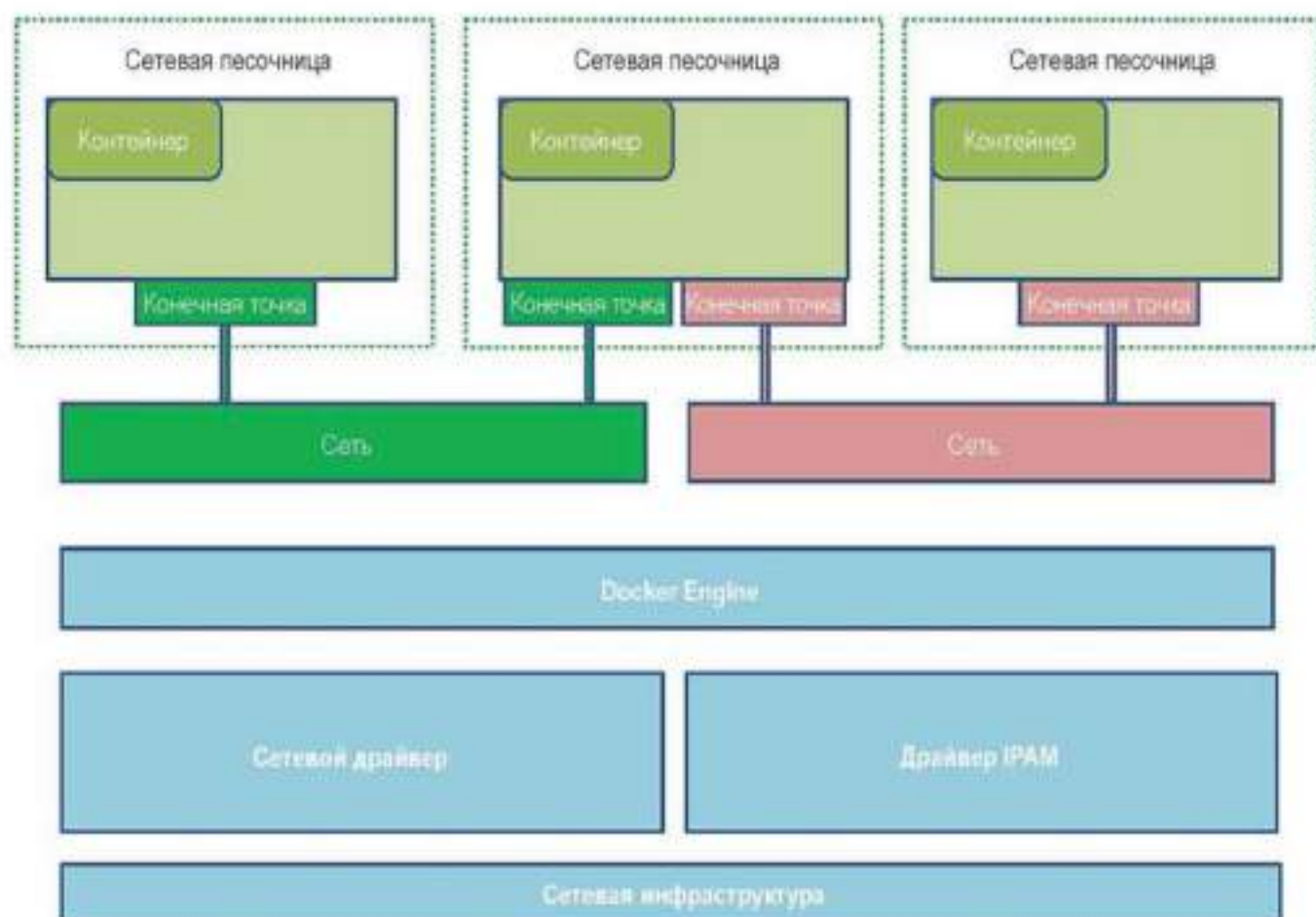


Рис. 4.1. Модель Container Network

Интерфейсы драйвера CNM

Драйверы — это часть программного обеспечения, которое поддерживает корректную работу сетей Docker. Преимущество драйверов в том, что они не требуют сложной настройки, по принципу *plug and play* («подключи и играй»), — их можно подключать в зависимости от своих потребностей и сразу использовать сеть. В одной сети может быть только один драйвер.

В Docker есть достаточно встроенных драйверов, но можно использовать и сторонние.

Существует еще один класс драйверов — *IP Address Management, IPAM* (управление IP-адресами), как показано на рис. 4.1. Эти драйверы предоставляют подсети или IP-адреса по умолчанию, если они не предоставлены или не назначены извне.

Libnetwork

Libnetwork — это реализация Container Network Model, библиотека для создания сетевых стеков и управления ими. *Libnetwork* можно назвать попыткой стандартизировать интерфейсы для подключения демона Docker к сетевым драйверам.

Драйверы Docker

Драйверы Docker могут быть двух категорий:

- ◆ собственные (встроенные) сетевые драйверы;
- ◆ сторонние (внешние) сетевые драйверы.

Собственные (встроенные) сетевые драйверы Docker являются частью Docker Engine. Ими можно управлять с помощью стандартных команд Docker; дополнительная конфигурация не требуется.

В табл. 4.1 приведены названия и краткие описания драйверов

Сторонние (внешние) драйверы Docker совместимы с CNM и могут подключаться к хосту Docker как плагины. Самые популярные сторонние драйверы — *Contiv* и *Weave*.

Информация о драйверах обобщена на рис. 4.2.

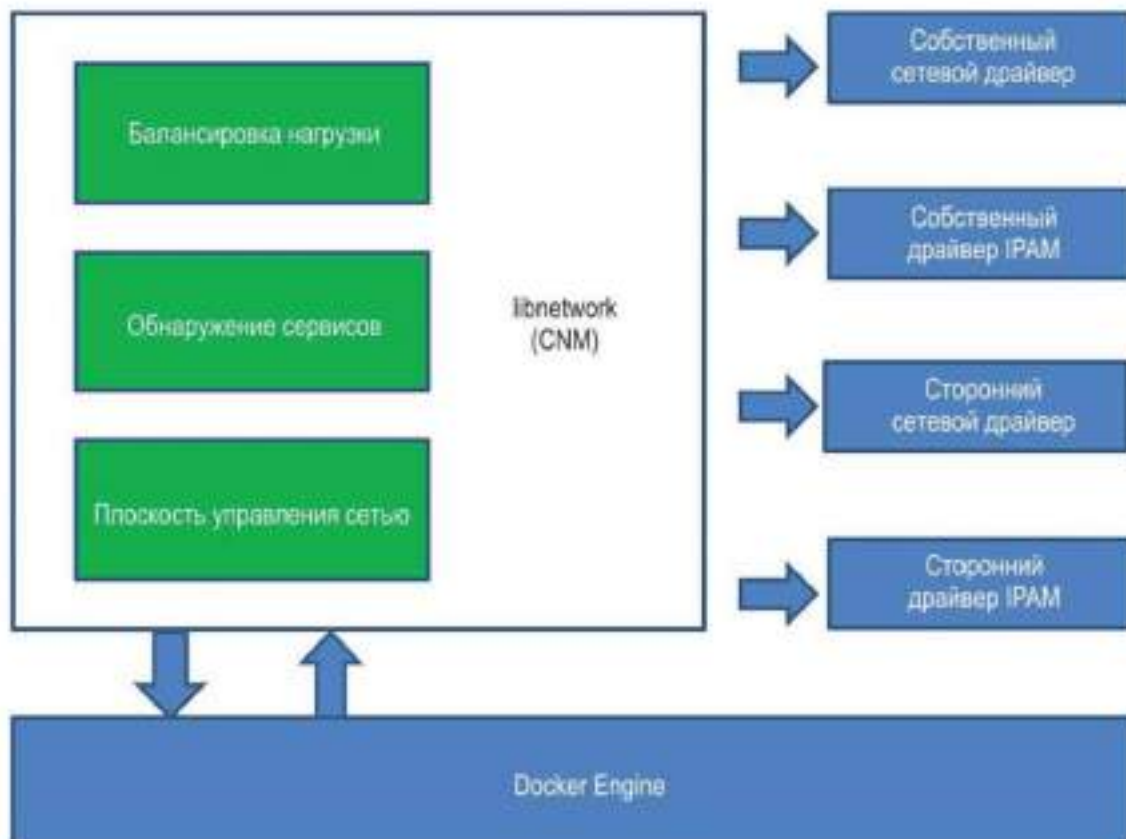


Рис. 4.2. Драйверы в сети Docker

Таблица 4.1. Собственные сетевые драйверы Docker

Название	Описание
Bridge	Сетевой мост Docker создается на хосте с использованием сетевого моста Linux. Контейнеры, соединенные мостом Docker, могут обмениваться данными друг с другом
Host	Поскольку у контейнера нет отдельного пространства имен, он просто использует сеть хоста, в котором находится
None	Драйвер none предоставляет своего рода сетевой стек внутри контейнера, но не предоставляет интерфейс для подключения к «внешнему миру»; таким образом, фактически контейнер полностью изолирован
Overlay	Сеть overlay (оверлейная сеть) объединяет несколько хостов без дополнительной конфигурации. Она использует традиционный сетевой мост Linux и VXLAN, чтобы обеспечить взаимодействие между контейнерами
MACVLAN	Драйвер MACVLAN использует мост MACVLAN в Linux для установки соединения между интерфейсами

Сетевой мост Docker

Сетевой мост в общих чертах можно представить как устройство канального уровня, которое используется как механизм передачи трафика между сегментами сети. При установке Docker автоматически применяет правила, которые запрещают контейнерам, подключенным к сетевому мосту, взаимодействовать друг с другом напрямую. Все коммуникации должны осуществляться через мост.

Концепция пространства имен Linux

Docker в значительной степени опирается на Linux в своих сетевых конструкциях. Например, Docker использует пространства имен, сетевые мосты Linux и iptables (утилита межсетевого экрана) при создании сетевой топологии. Драйвер сетевого моста Docker Bridge можно считать высокоуровневой реализацией моста Linux. *Сетевой мост Linux* — это устройство второго уровня, своего рода виртуальная реализация физического коммутатора в ядре Linux.

Сетевая изоляция контейнеров реализуется с помощью *пространств имен*. Если речь о персональном компьютере с одним пользователем, не стоит тратить усилия на изоляцию. Если задействовано несколько сервисов, в целях безопасности и стабильности следует изолировать эти сервисы друг от друга с помощью пространства имен.

Пространство имен представляет собой независимую единицу сетевого стека со всеми компонентами — маршрутами, правилами межсетевого экрана (брандмауэра) и сетевыми устройствами.

Прежде чем перейти к пространствам имен и сетевому мосту применительно к контейнерам, следует разобраться, что означает пространство имен в Linux.

Итак, пространства имен в Linux — это отдельные экземпляры сетевого стека. Чтобы выяснить, определены ли в системе Linux пространства имен, используем следующую команду (рис. 4.3):

```
ip netns list
```



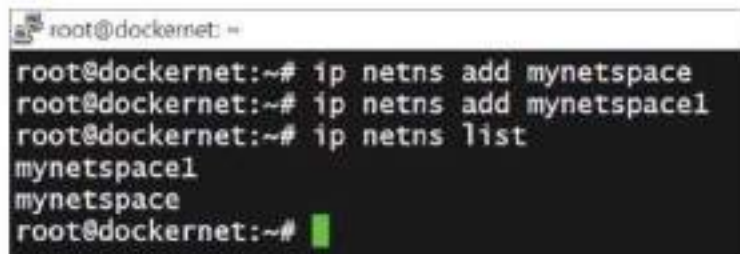
```
root@dockernet: ~
root@dockernet:~# ip netns list
root@dockernet:~#
```

Рис. 4.3

Как видите, пространства имен в сети пока не определены, и это нормально.

Давайте создадим пространство имен с именем `mynetspace`. Запустим несколько команд и посмотрим, что получится (рис. 4.4).

```
ip netns add mynetspace
ip netns add mynetspace1
ip netns list
```



```
root@dockernet: ~
root@dockernet:~# ip netns add mynetspace
root@dockernet:~# ip netns add mynetspace1
root@dockernet:~# ip netns list
mynetspace1
mynetspace
root@dockernet:~#
```

Рис. 4.4

Итак, пространство имен создано. Чуть позже мы создадим пару устройств *veth* (virtual ethernet).

В общих чертах, *veth* — это «туннель», соединяющий два пространства имен (см. рис. 4.5).

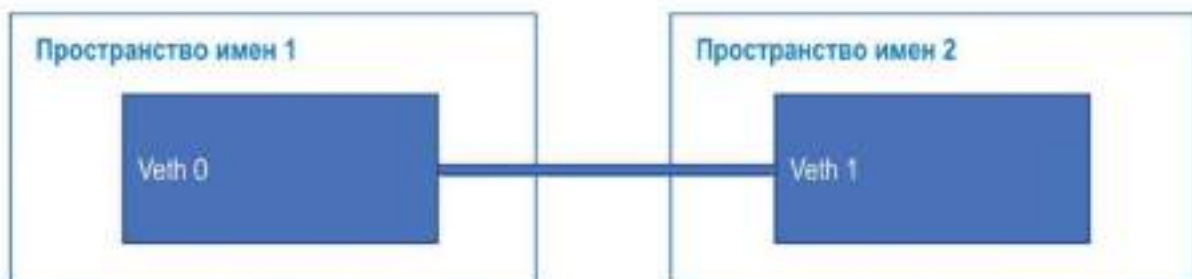


Рис. 4.5. Veth и пространства имен

О *veth* часто говорят как о *паре*, потому что это полнодуплексный канал связи, который имеет интерфейсы в каждом из *пространств имен*. Данные проходят через пару *veth* с одного конца на другой (см. рис. 4.6).

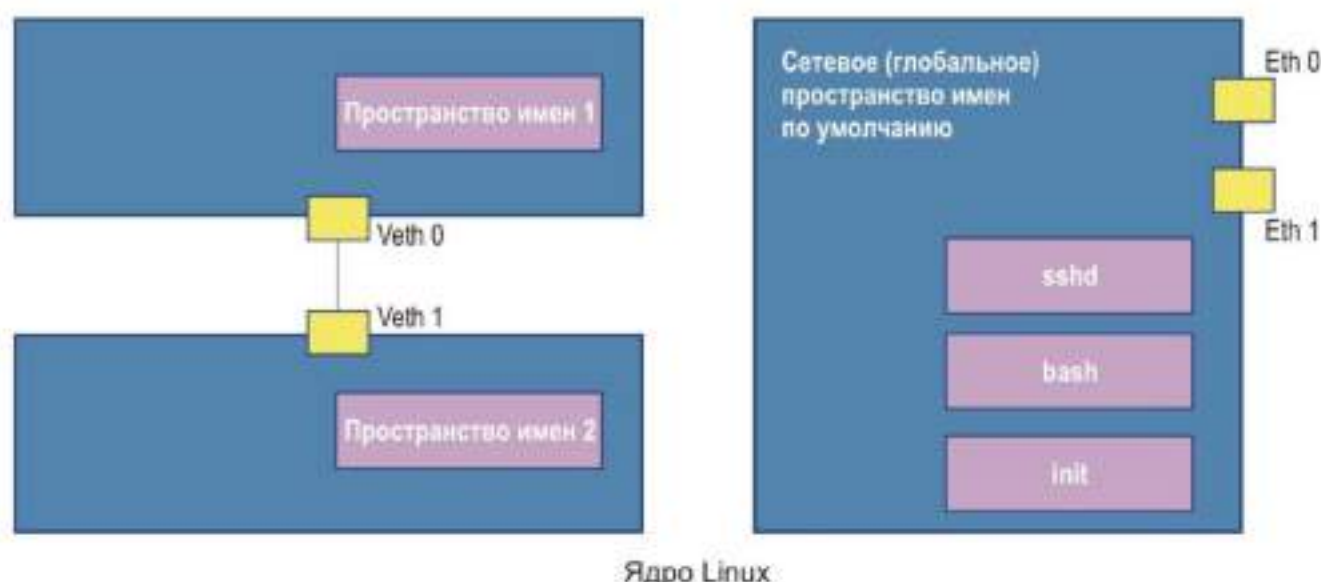


Рис. 4.6. Передача данных через пару veth

Теперь создадим пару veth с помощью следующих команд:

```
ip link add veth0 type veth peer name veth1
ip link list | grep veth
```

И получим выходные данные (рис. 4.7).

```
root@dockernet:~# ip link add veth0 type veth peer name veth1
root@dockernet:~# ip link list | grep veth
7: veth1@veth0: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
8: veth0@veth1: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
root@dockernet:~#
```

Рис. 4.7

Как видите, пара veth создана, но пока не прикреплена к созданным ранее пространствам имен. Сделаем это следующим образом (рис. 4.8):

```
ip link set veth0 netns mynetnspace
ip link set veth1 netns mynetnspace1
```

```
root@dockernet:~# ip link set veth0 netns mynetnspace
root@dockernet:~# ip link set veth1 netns mynetnspace1
```

Рис. 4.8

Далее задаем инструкцию `grep` для veth, как показано ниже (рис. 4.9):

```
ip link | grep veth
```

```
root@dockernet:~# ip link | grep veth
root@dockernet:~#
```

Рис. 4.9

Сейчас, мы ничего не видим, потому что переместили veth из пространства имен хоста в пространства имен, созданные нами. Проверим наличие пары veth в наших пространствах имен с помощью следующей команды и посмотрим на результат (рис. 4.10):

```
ip netns exec mynetnspace ip link | grep veth
ip netns exec mynetspacel ip link | grep veth
```

```
root@dockernet:~# ip netns exec mynetnspace ip link | grep veth
10: veth0@if9: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
root@dockernet:~# ip netns exec mynetspacel ip link | grep veth
9: veth1@if10: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
root@dockernet:~#
```

Рис. 4.10

Теперь, подключив каждый конец пары veth к созданным пространствам имен, мы добавим IP-адреса и запустим наши пространства имен, т. е. сделаем их работоспособными. Команды и выходные данные ниже (рис. 4.11).

```
ip netns exec mynetnspace ip addr add 10.0.0.15/24 dev veth0
ip netns exec mynetspacel ip addr add 10.0.0.16/24 dev veth1
ip netns exec mynetnspace ip link set veth0 up
ip netns exec mynetspacel ip link set veth1 up
```

```
root@dockernet:~# ip netns exec mynetnspace ip addr add 10.0.0.15/24 dev veth0
root@dockernet:~# ip netns exec mynetspacel ip addr add 10.0.0.16/24 dev veth1
root@dockernet:~# ip netns exec mynetnspace ip link set veth0 up
root@dockernet:~# ip netns exec mynetspacel ip link set veth1 up
```

Рис. 4.11

Таким образом, мы получили работающие пространства имен сети с полным сетевым стеком и IP-адресом. Давайте посмотрим на них: запустим следующие команды и изучим рис. 4.12.

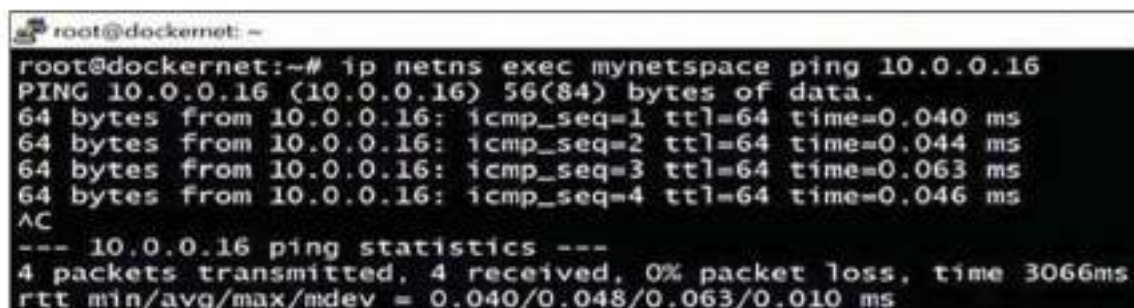
```
ip netns exec mynetnspace ip addr show
ip netns exec mynetspacel ip addr show
```

```
root@dockernet:~# ip netns exec mynetnspace ip addr show
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
10: veth0@if9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   link/ether ea:7c:c1:fa:eb:92 brd ff:ff:ff:ff:ff:ff link-netnsid 1
   inet 10.0.0.15/24 scope global veth0
       valid_lft forever preferred_lft forever
   inet6 fe80::e87c:c1ff:fe7a:eb92/64 scope link
       valid_lft forever preferred_lft forever
root@dockernet:~# ip netns exec mynetspacel ip addr show
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
9: veth1@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   link/ether 3e:a3:fb:70:83:fd brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet 10.0.0.16/24 scope global veth1
       valid_lft forever preferred_lft forever
   inet6 fe80::3a3:fbff:fe70:83fd/64 scope link
       valid_lft forever preferred_lft forever
root@dockernet:~#
```

Рис. 4.12

Чтобы быстро проверить, работают ли интерфейсы, можно отправить ping из одного пространства имен в другое. Запустим соответствующие команды и посмотрим на рис. 4.13 и 4.14:

```
ip netns exec mynetspace ping 10.0.0.16
```

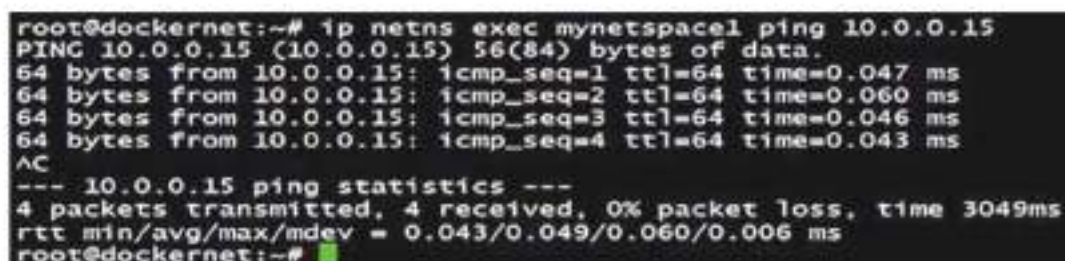


```
root@dockernet: ~
root@dockernet:~# ip netns exec mynetspace ping 10.0.0.16
PING 10.0.0.16 (10.0.0.16) 56(84) bytes of data:
64 bytes from 10.0.0.16: icmp_seq=1 ttl=64 time=0.040 ms
64 bytes from 10.0.0.16: icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from 10.0.0.16: icmp_seq=3 ttl=64 time=0.063 ms
64 bytes from 10.0.0.16: icmp_seq=4 ttl=64 time=0.046 ms
^C
--- 10.0.0.16 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3066ms
rtt min/avg/max/mdev = 0.040/0.048/0.063/0.010 ms
```

Рис. 4.13

Отправляем ping на другой сервер и смотрим выходные данные на рис. 4.14.

```
ip netns exec mynetspace1 ping 10.0.0.15
```



```
root@dockernet:~# ip netns exec mynetspace1 ping 10.0.0.15
PING 10.0.0.15 (10.0.0.15) 56(84) bytes of data:
64 bytes from 10.0.0.15: icmp_seq=1 ttl=64 time=0.047 ms
64 bytes from 10.0.0.15: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 10.0.0.15: icmp_seq=3 ttl=64 time=0.046 ms
64 bytes from 10.0.0.15: icmp_seq=4 ttl=64 time=0.043 ms
^C
--- 10.0.0.15 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3049ms
rtt min/avg/max/mdev = 0.043/0.049/0.060/0.006 ms
root@dockernet:~#
```

Рис. 4.14

Все работает отлично! Это довольно удобный способ реализовать изоляцию сети, хотя при таком подходе подключения сложно масштабировать. Представьте, что каждый раз при создании и соединении пространств имен придется создавать и пары veth, а их администрирование и обслуживание требует определенных усилий. Если пространств имен много, значит, много изолированных сетей, и если каждая сеть должна взаимодействовать с какой-либо другой, мы просто не справимся с таким количеством пар veth.

К счастью, на этот случай есть проверенный механизм — *сетевые мосты Linux*. Вместо того чтобы создавать отдельные каналы связи (veth) между всеми пространствами имен, можно просто подключить свои пространства имен к мосту Linux с помощью пары veth. И этого достаточно. Каждое пространство имен (ПИ), подключенное к мосту Linux, сможет общаться с другим пространством имен, как показано на рис. 4.15.

Как видно на рис. 4.15, контейнеры Docker взаимодействуют друг с другом через сетевой мост. На хосте Linux с Docker Engine сетевой мост создается автоматически. Сетевой мост адаптирует мост Linux и по умолчанию называется *docker0*.

Для отдельных сетей Docker по умолчанию использует сетевой мост. Один конец пары veth соединен с контейнером и называется *eth0*, а другой — с мостом *docker0*,

и его имя начинается с *veth*. Для контейнера мы не указываем внешний IP-адрес, поскольку адрес ему присвоит встроенный в Docker драйвер IPAM.

Прежде чем продолжить, очистим созданные пространства имен следующей командой:

```
ip netns del mynetspace
ip netns del mynetspace1
```

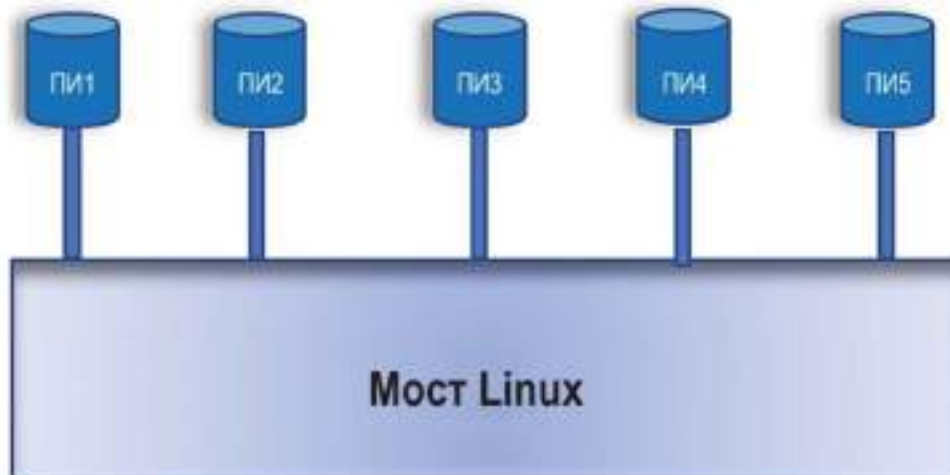


Рис. 4.15. Сетевой мост Linux

Мост Docker

Сейчас мы проверим, какие интерфейсы доступны в системе Linux, где установлен Docker (см. команду ниже и рис. 4.16).

```
ip addr
```

И мы действительно видим здесь интерфейс *docker0*.

```
root@dockernet:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 76:e5:36:33:e3:63 brd ff:ff:ff:ff:ff:ff
    inet 159.65.146.235/20 brd 159.65.159.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet 10.47.0.5/16 brd 10.47.255.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::74e5:36ff:fe33:e363/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether a2:d0:99:ad:84:61 brd ff:ff:ff:ff:ff:ff
    inet 10.139.232.20/16 brd 10.139.255.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a0d0:99ff:fead:8461/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:14:71:42:37 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
root@dockernet:~#
```

Рис. 4.16

Заключение

Итак, мы рассмотрели сетевую модель для контейнеров Docker; узнали, что такое драйверы и пространства имен, а также ознакомились с концепцией пространств имен Linux. Кроме того, мы обсудили сетевой мост Linux и узнали, как *docker0* использует мост Linux, чтобы обеспечить обмен данными между контейнерами с помощью пары veth, соединяющей мост *docker0* и отдельные контейнеры.

В следующей главе мы поговорим о практической реализации сетей в Docker.

Основные тезисы

- ◆ Сетевая архитектура Docker базируется на модели Container Network (CNM).
- ◆ Libnetwork — это реализация модели Container Network.
- ◆ Драйверы Docker — это часть программного обеспечения, которая поддерживает корректную работу сетей Docker.
- ◆ Пространства имен обеспечивают изоляцию контейнеров, чтобы они могли функционировать изолированно, не затрагивая сети друг друга.
- ◆ Мост *docker0* базируется на мосте Linux.
- ◆ Пара veth соединяет два контейнера или контейнер и мост *docker0*.
- ◆ Контейнеры соединенные через мост *docker0*, могут взаимодействовать между собой.

Контрольный тест

Выберите подходящий вариант ответа.

1. Сетевой мост Linux — это:
 - Устройство уровня 1.
 - Устройство уровня 3.
 - Устройство уровня 2.
 - Сетевой мост Linux не имеет отношения к модели OSI (Open Systems Interconnection).
2. Сетевая изоляция контейнеров реализуется при помощи...
 - Модели Container Network.
 - Пары veth.
 - Пространства имен.
 - Сетевых драйверов.
3. Если на хосте Linux запущен Docker Engine, то автоматически...
 - Создается сетевой мост.
 - Создается пара veth.

- Создаются пространства имен.
 - Устанавливаются собственные (встроенные) сетевые драйверы.
4. Сетевая песочница полностью изолирована, за исключением...
- Сетевого стека.
 - Конечных точек.
 - Драйверов IPAM.
 - Контейнеров.
5. Libnetwork — это:
- Сетевой интерфейс.
 - Репозиторий встроенных драйверов Docker.
 - Реализация IPAM.
 - Реализация CNM.
6. Какой сетевой драйвер позволяет подключаться к нескольким хостам без дополнительной конфигурации?
- MACVLAN.
 - Overlay.
 - Bridge.
 - Ничего из вышеперечисленного.

Ответы

1	с
2	с
3	а

4	б
5	д
6	б

Вопросы

1. Что такое сетевое пространство имен?
2. Как работает пара veth?
3. Нарисуйте и объясните модель Container Network.
4. Что представляет собой Libnetwork и каковы ее функции?
5. В чем заключается концепция сетевого моста Linux.
6. Что такое сетевые драйверы?

Ключевые термины

- ◆ Модель Container Network
- ◆ Libnetwork
- ◆ Veth
- ◆ Песочница
- ◆ Конечные точки
- ◆ Сетевой стек
- ◆ Изоляция контейнеров
- ◆ Сетевое пространство имен
- ◆ Сетевой мост Linux
- ◆ Docker0

ГЛАВА 5

Docker Swarm

Введение

При большом количестве контейнеров необходим инструмент, который обеспечит их гармоничное взаимодействие. *Docker Swarm* — именно такой инструмент, который предоставляет для Docker нативную технологию кластеризации. Docker Swarm, иначе называемый *режимом Swarm*, — это встроенная в Docker поддержка для оркестрации кластеров Docker Engine, которая позволяет управлять всеми контейнерами как единым целым.

Из этой главы вы узнаете больше об оркестрации контейнеров, осуществляемой с помощью встроенного в Docker механизма кластеризации Docker Swarm. Мы рассмотрим, как настроить Docker Swarm и обеспечить с его помощью высокую доступность и безопасное, надежное и оптимизированное функционирование ваших приложений.

Темы для изучения следующие.

- ◆ Что такое Docker Swarm
- ◆ Преимущества Docker Swarm
- ◆ Создание и настройка кластера Swarm
- ◆ Концепция сервиса в Swarm
- ◆ Блокировка и разблокировка кластера Swarm
- ◆ Сети в Docker Swarm
- ◆ Устранение неполадок в Docker Swarm

Что такое Docker Swarm

Docker Swarm — это платформа для развертывания приложений на нескольких хостах с помощью многофункционального API, который упрощает развертывание сложных приложений. Обычно приложения записываются в файл манифеста, который можно легко развернуть в кластер с помощью собственных команд Docker.

Весь процесс автоматизирован; Swarm может перенаправлять запрос от клиента в кластер Swarm. Когда кластер Swarm получает запрос, он берет на себя управление приложением. Начиная с версии 1.12, в Docker есть встроенный режим Swarm для управления кластером Docker Engine. Для создания кластера Swarm, развертывания в нем сервисов приложения и управления его поведением используется Docker CLI.

Преимущества Docker Swarm

Docker Swarm имеет ряд весьма интересных преимуществ.

1. *Объявление желаемого состояния.* Мы можем объявить, что именно хотим получить. Например, Docker Engine позволяет определить количество требуемых реплик приложения, его структуру и пр. Можно указать, что нам нужен фронтенд, база данных бэкенд и т. д.
2. *Масштабирование.* Docker Swarm позволяет легко менять масштаб встроенными средствами. Мы можем увеличить число реплик приложения, если вырос спрос, а потом так же легко уменьшить масштаб при необходимости.
3. *Отсутствие отдельного оркестратора.* При использовании Docker Swarm не требуется стороннее программное обеспечение для оркестрации. Управление кластерами уже интегрировано с Docker Engine.
4. *Поддержание желаемого состояния.* Управляющий узел Swarm непрерывно отслеживает кластер и вносит исправления, если фактическое состояние отличается от желаемого. Допустим, мы указали, что нам требуются пять реплик, но из-за сбоя оборудования одна из реплик становится недоступной. В этом случае Docker Swarm создаст недостающую реплику на рабочем узле.
5. *Сеть с несколькими хостами.* Docker Swarm предоставляет для наших сервисов готовую оверлейную сеть. Адреса автоматически назначаются контейнерам в оверлейной сети при запуске, изменении или обновлении приложения.
6. *Обнаружение сервисов.* В Swarm есть DNS-сервер, который используется для предоставления идентификатора каждому сервису, запущенному в Swarm. Управляющий узел Swarm также отвечает за балансировку нагрузки для контейнеров, защищая их от перегрузки.
7. *Встроенные средства безопасности.* Docker Swarm использует взаимную аутентификацию TLS (Transport Layer Security — протокол защиты транспортного уровня) и шифрование для безопасного обмена данными между узлами.
8. *Последовательные обновления.* Очень интересная особенность Docker Swarm. Сервисные обновления можно применять не ко всем узлам сразу, а постепенно, по группам узлов. Если что-то пойдет не так, можно легко откатить обновление без масштабных последствий.

Создание и настройка кластера Swarm

Настроить Swarm очень просто. Достаточно команды в одну строку. Узлы в Swarm бывают *управляющими* (manager) и *рабочими* (worker). Управляющие узлы отвечают за оркестрацию кластера, распределяют задачи между рабочими узлами и в целом контролируют кластеры.

Все метаданные о кластере хранятся в распределенной базе данных *etcd*, расположенной на всех управляющих узлах. База данных *etcd* работает сама по себе и не требует дополнительной настройки.

Для оркестрации приложений используются так называемые *сервисы* — отдельные задачи, которые будут выполняться в Swarm. Сервис можно масштабировать, обновлять, откатывать и т. д. Если мы назначаем для Swarm задачу, ее выполнение перенаправляется сервису. Создавая сервис, мы указываем нужный образ контейнера, команды, которые должны выполняться в запущенном контейнере, и т. д. Также можно указать порт, сеть, центральный процессор и прочее.

Когда сервис назначается для Swarm, мы называем его *задачей*, или *репликой*. Таким образом, сервис можно определить как описание желаемого состояния и задача, которую нужно выполнить. При создании сервиса на рабочих узлах планируется последовательность действий.

1. Сначала на управляющий узел Docker поступает запрос.
2. Затем управляющий узел назначает сервис, который будет выполняться на группе узлов.
3. Сервис выполняется на узлах в виде отдельных задач. Задачи выполняются однократно. Когда задача останавливается, она не выполняется снова, но может выполняться другая задача. Каждая задача находится в одном из следующих состояний жизненного цикла: **NEW** (новая), **PENDING** (ожидание), **RUNNING** (выполняется), **COMPLETE** (завершена) и т. д.

Вооружившись теорией, перейдем к практике и посмотрим, как это работает. Создадим простейший кластер Swarm, включающий один управляющий узел и два рабочих. Но сначала нужно выполнить предварительные условия и убедиться, что открыты следующие порты:

TCP 2377 (для соединения клиента и демона);

TCP 7946, UDP 7946 (для управления группами);

UDP 4789 (для оверлейных сетей на базе VXLAN).

Теперь выберем управляющий узел и получим его IP-адрес. Это можно сделать простой командой:

```
ifconfig -a eth0
```

Получаем выходные данные (рис. 5.1).

Нужный нам IP-адрес — 64.225.13.35. Инициализируем этот узел как управляющий, выполнив следующую команду, а затем изучим выходные данные на рис. 5.2:

```
docker swarm init --advertise-addr 64.225.13.35:2377 --listen-addr
64.225.13.35:2377
```

```

root@mgr:~# ifconfig -a eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 64.225.13.35 netmask 255.255.240.0 broadcast 64.225.15.255
    inet6 fe80::c00b:ebff:fee3:8a22 prefixlen 64 scopeid 0x20<link>
    ether c2:0b:eb:e3:8a:22 txqueuelen 1000 (Ethernet)
    RX packets 7061 bytes 105497709 (105.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3425 bytes 308526 (308.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Рис. 5.1

```

root@mgr:~# docker swarm init --advertise-addr 64.225.13.35:2377 --listen-addr 64.225.13.35:2377
Swarm initialized: current node (v1shajnl8wslv218zeopm4kd6) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-4ud9mjca2dz2vt75mq21sjpows4b4d106g7dr3zbhstfx37cmc-b51123p64wufw1v1e09w2nyf 64.225.13.35:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

```

Рис. 5.2

Проанализируем команду, которую мы только что применили.

`docker swarm init` велит Docker инициализировать Swarm.

`--advertise-addr` относится к IP-адресу и порту, через который другие узлы будут подключаться к управляющему узлу. Эту часть можно пропустить, потому что по умолчанию Docker выберет IP-адрес, доступный на `eth0`.

`--listen-addr` — это IP-адрес и порт, который мы хотим назначить для прослушивания трафика Swarm. Обычно он совпадает с `--advertise-addr`, кроме случаев, когда мы предполагаем использовать разные IP-адреса для соединения и прослушивания.

Обратите внимание: на рис. 5.2 можно видеть *ключ токена* для добавления рабочего узла в Swarm. Он также предоставляет команду для добавления управляющего узла и дает указание следовать инструкциям. Получить токен для управляющего или рабочего узла можно следующими командами:

```

docker swarm join-token manager
docker swarm join-token worker

```

Давайте присоединим к этому Swarm пару рабочих узлов. Сначала войдем на рабочий узел 1 и присоединим его к Swarm, а затем сделаем то же самое с рабочим узлом 2. Как видите, все довольно просто (рис. 5.3).

Добавляем второй рабочий узел (рис. 5.4).

```

root@wkr01:~# docker swarm join --token SWMTKN-1-4ud9mjca2dz2vt75mq21sjpows4b4d106g7dr3zbhstfx37cmc-b51123p64wufw1v1e09w2nyf 64.225.13.35:2377
This node joined a swarm as a worker.
root@wkr01:~#

```

Рис. 5.3

```

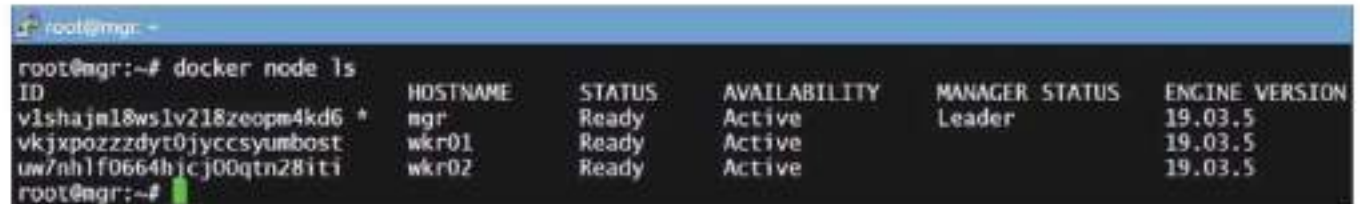
root@wkr02:~# docker swarm join --token SWMTKN-1-4ud9mjca2dz2vt75mq21sjpows4b4d106g7dr3zbhstfx37cmc-b51123p64wufw1v1e09w2nyf 64.225.13.35:2377
This node joined a swarm as a worker.
root@wkr02:~#

```

Рис. 5.4

Итак, мы создали кластер Swarm с одним управляющим и двумя рабочими узлами. Вернемся к управляющему узлу и посмотрим, что мы имеем (рис. 5.5):

```
docker node ls
docker node inspect self
docker node inspect --pretty self
docker node inspect --pretty wkr01
```



ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
v1shajml8wslv218zeopm4kd6 *	mgr	Ready	Active	Leader	19.03.5
vkjxpozzydyt0jyccsyumbost	wkr01	Ready	Active		19.03.5
uw7nhlf0664hjcj00qtn28iti	wkr02	Ready	Active		19.03.5

Рис. 5.5

`docker node ls` предоставляет список узлов в кластере Swarm. Эта команда не может быть выполнена с рабочих узлов. Узел, с которого она была выполнена, отмечен звездочкой (*) рядом с ID. В столбце **MANAGER STATUS** (статус управляющего узла) указано **LEADER** (лидер), — это означает, что управляющий узел сейчас активен. Если в Swarm несколько управляющих узлов, активным в тот или иной момент времени может быть только один из них. Для рабочих узлов в столбце **MANAGER STATUS** ничего не указано, потому что это не управляющие узлы. Запустим еще одну команду и подробно изучим управляющий узел:

```
root@mgr:~# docker node inspect self
[
  {
    "ID": "v1shajml8wslv218zeopm4kd5",
    "Version": {
      "Index": 9
    },
    "CreatedAt": "2019-12-15T05:46:69.372129773Z",
    "UpdatedAt": "2019-12-15T05:46:69.979553725Z",
    "Spec": {
      "Labels": {},
      "Role": "manager",
      "Availability": "active"
    },
    "Description": {
      "Hostname": "mgr",
      "Platform": {
        "Architecture": "x86_64",
        "OS": "linux"
      }
    },
    "Resources": {
      "NanoCPUs": 4000000000,
      "MemoryBytes": 8364035096
    }
  }
]
```


состоянии, оставшиеся управляющие узлы не смогут продолжать выполнение задачи и восстановить стабильное состояние в случае выхода лидера из строя.

Нечетное число управляющих узлов помогает избежать так называемой ситуации *split-brain* (букв.: расщепленный мозг), когда нарушается структура кластера, если управляющие узлы не достигли консенсуса.

Docker Swarm использует *алгоритм консенсуса Raft*. Raft допускает сбой до $(N-1)/2$ узлов. Если у нас, допустим, *пять* управляющих узлов, алгоритм Raft допускает потерю $(5-1)/2$, т. е. двух узлов. Если управляющих узла *три*, алгоритм соответственно позволяет потерять не больше одного $((3-1)/2=1)$.

Таким образом, если, например, мы потеряли один управляющий узел в кластере из пяти узлов, проблем не возникнет. Но теперь для достижения консенсуса по любому решению в кластере нам требуется $(N/2) + 1$, т. е. $(4/2) + 1 = 3$ узла.

В ситуации *split-brain* существующие задачи продолжают выполняться, но планировщик не сможет перераспределить нагрузку, чтобы справиться со сбоями.

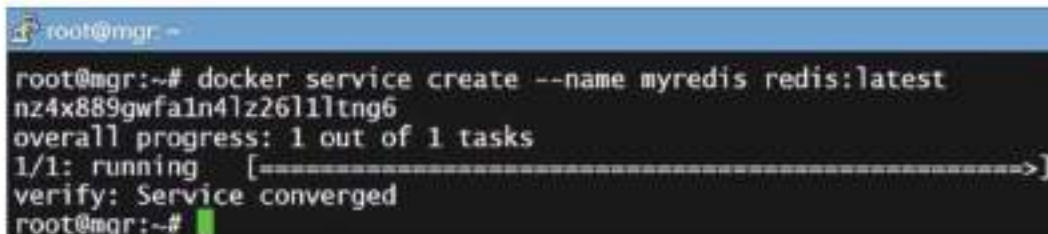
Если управляющих узлов слишком много, производительность будет снижаться, потому что для достижения консенсуса потребуется дополнительное время и ресурсы, следовательно, возможные преимущества не покроют потерю производительности. Поэтому рекомендуется настроить 3 или 5 управляющих узлов.

Концепция сервиса в Swarm

Ранее мы уже говорили о сервисах: когда мы разворачиваем приложение и Docker Engine находится в режиме Swarm, мы создаем *сервис*. При создании сервиса можно, помимо прочего, указать образ контейнера и сеть, которые нужно использовать, и какие команды выполнять внутри запущенных контейнеров. Когда мы планируем и создаем сервис, оркестратор в ответ создает задачи, или *реплики*.

Теперь перейдем к практике и создадим сервис, а затем изучим выходные данные на рис. 5.7.

```
docker service create --name myredis redis:latest
```



```
root@mgr: ~# docker service create --name myredis redis:latest
nz4x889gwfa1n4lz26l1ltng6
overall progress: 1 out of 1 tasks
1/1: running [----->]
verify: Service converged
root@mgr: ~#
```

Рис. 5.7

Выполним несколько проверок и посмотрим, что получилось (рис. 5.8).

```
docker service ls
docker service ps myredis
docker service inspect --pretty myredis
```



```

root@mgr:~# docker service ls
ID                NAME          MODE          REPLICAS  IMAGE          PORTS
nz4x889gwfa1     myredis       replicated    1/1        redis:latest
root@mgr:~# docker service ps myredis
ID                NAME          IMAGE          NODE       DESIRED STATE  CURRENT STATE          ERROR          PORTS
qjb1i85p38wr     myredis.1     redis:latest   wkr02      Running        Running 8 minutes ago
root@mgr:~# docker service inspect --pretty myredis

ID:                nz4x889gwfa1n41z26111tng6
Name:              myredis
Service Mode:      Replicated
  Replicas:         1
Placement:
UpdateConfig:
  Parallelism:      1
  On failure:       pause
  Monitoring Period: 3s
  Max failure ratio: 0
  Update order:     stop-first
RollbackConfig:
  Parallelism:      1
  On failure:       pause
  Monitoring Period: 3s
  Max failure ratio: 0
  Rollback order:   stop-first
ContainerSpec:
  Image:            redis:latest@sha256:1eedfc017b0cd3e232878ce38bd9328518219802a8ef37fe34f58dcf591688ef
  Init:             false
Resources:
Endpoint Mode:     vip

```

Рис. 5.8

`docker service ls` показывает, что созданный сервис один раз реплицирован и создан на основе образа `redis:latest` image.

`docker service ps myredis` предоставляет дополнительную информацию, например, о том, на каком узле выполняется сервис, каково его желаемое и текущее состояние и т. д.

`docker service inspect --pretty myredis` дает информацию о сервисе в удобочитаемом виде.

Создание реплик

Надеюсь, пока все понятно. Давайте копнем чуть глубже и посмотрим, как создаются реплики и масштабируются сервисы. Запустим следующие команды и изучим выходные данные на рис. 5.9:

```

docker service create --name myredis_replica --replicas=5 redis:latest
docker service ps myredis_replica

```

Интересный момент: реплики распределены по узлам, включая управляющий узел. Каждую реплику можно считать задачей, и каждая задача сопоставлена с контейнером. На рис. 5.9 видно, что на управляющем узле выполняются две задачи (контейнера), на рабочем узле `wkr01` выполняются еще две задачи, а на рабочем узле `wkr02` — одна.

Давайте остановим узел `wkr02` и посмотрим, что будет. Запустим команду и изучим выходные данные на рис. 5.10.

```

docker container ls -a
systemctl stop docker

```



```

root@mgr:~# docker service create --name myredis_replica --replicas=5 redis:latest
gik13rqkfsahfx8d3v1halefh
overall progress: 5 out of 5 tasks
1/5: running [----->]
2/5: running [----->]
3/5: running [----->]
4/5: running [----->]
5/5: running [----->]
verify: Service converged
root@mgr:~# docker service ps myredis_replica

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
zyjeq7sbh8vq	myredis_replica.1	redis:latest	wkr01	Running	Running 23 seconds ago		
3rjan2xg4177	myredis_replica.2	redis:latest	mgr	Running	Running 23 seconds ago		
rxganhpvagtn	myredis_replica.3	redis:latest	wkr01	Running	Running 23 seconds ago		
xbz0wrg9qr7g	myredis_replica.4	redis:latest	mgr	Running	Running 23 seconds ago		
pj59wggkqapr	myredis_replica.5	redis:latest	wkr02	Running	Running 23 seconds ago		

Рис. 5.9

```

root@wkr02:~# docker container ls -a

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b7c2a2783388	redis:latest	"docker-entrypoint.s..."	22 minutes ago	Up 22 minutes	6379/tcp	myredis_replica.5.pj59wggkqaprcs
jnad49vtpuv	redis:latest	"docker-entrypoint.s..."	57 minutes ago	Up 57 minutes	6379/tcp	myredis.1.gjb1i85p38wraant8pz21xfy8o

```

root@wkr02:~# systemctl stop docker
root@wkr02:~#

```

Рис. 5.10

Итак, мы отключили узел `wkr02`. Теперь посмотрим, перенесена ли реплика с `wkr02` на оставшиеся узлы. Для этого воспользуемся командой `docker service ps myredis_replica` на управляющем узле (рис. 5.11).

```

root@mgr:~# docker service ps myredis_replica

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
zyjeq7sbh8vq	myredis_replica.1	redis:latest	wkr01	Running	Running 33 minutes ago		
3rjan2xg4177	myredis_replica.2	redis:latest	mgr	Running	Running 33 minutes ago		
rxganhpvagtn	myredis_replica.3	redis:latest	wkr01	Running	Running 33 minutes ago		
xbz0wrg9qr7g	myredis_replica.4	redis:latest	mgr	Running	Running 33 minutes ago		
m43q19rt6n16	myredis_replica.5	redis:latest	mgr	Running	Running 10 minutes ago		
pj59wggkqapr	myredis_replica.5	redis:latest	wkr02	Shutdown	Running 10 minutes ago		

Рис. 5.11

Как видим, рабочий узел `wkr02` остановлен, и теперь на узле `wkr01` выполняются две задачи, а на управляющем узле — три. Таким образом, после остановки `wkr02` задача с этого узла была перенесена на узел `mgr`. Если снова запустить `wkr02`, задача на него не вернется.

Масштабирование сервиса

Изменять масштаб сервиса в Docker Swarm можно одной командой. Создадим сервис с четырьмя репликами, затем увеличим их количество до семи, а потом уменьшим до пяти. Запустим следующие команды и изучим выходные данные на рис. 5.12 и 5.13:

```

docker service create --name mynginx --replicas=4 nginx:latest
docker service ps mynginx
docker service scale mynginx=7
docker service ps mynginx

```

```

root@mgr:~# docker service create --name mynginx --replicas=4 nginx:latest
i6t4xx28erjawi56lj26y64zu
overall progress: 4 out of 4 tasks
1/4: running [----->]
2/4: running [----->]
3/4: running [----->]
4/4: running [----->]
verify: Service converged
root@mgr:~# docker service ps mynginx

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
ev6gbvv3jvbx	mynginx.1	nginx:latest	wkr02	Running	Running 25 seconds ago		
z3a007j2v8o7	mynginx.2	nginx:latest	wkr01	Running	Running 25 seconds ago		
eyupru9mz6t6	mynginx.3	nginx:latest	mgr	Running	Running 25 seconds ago		
r20pmwy4txvv	mynginx.4	nginx:latest	mgr	Running	Running 25 seconds ago		

```

root@mgr:~# docker service scale mynginx=7
mynginx scaled to 7
overall progress: 7 out of 7 tasks
1/7: running [----->]
2/7: running [----->]
3/7: running [----->]
4/7: running [----->]
5/7: running [----->]
6/7: running [----->]
7/7: running [----->]
verify: Service converged
root@mgr:~# docker service ps mynginx

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
ev6gbvv3jvbx	mynginx.1	nginx:latest	wkr02	Running	Running 2 minutes ago		
z3a007j2v8o7	mynginx.2	nginx:latest	wkr01	Running	Running 2 minutes ago		
eyupru9mz6t6	mynginx.3	nginx:latest	mgr	Running	Running 2 minutes ago		
r20pmwy4txvv	mynginx.4	nginx:latest	mgr	Running	Running 2 minutes ago		
e4vkvyrbuya0	mynginx.5	nginx:latest	mgr	Running	Running 15 seconds ago		
c8oh6g79eff3	mynginx.6	nginx:latest	wkr02	Running	Running 15 seconds ago		
nxqkj7rb1w4x	mynginx.7	nginx:latest	wkr01	Running	Running 15 seconds ago		

```

root@mgr:~#

```

Рис. 5.12

```
docker service scale mynginx=5
```

```

root@mgr:~# docker service scale mynginx=5
mynginx scaled to 5
overall progress: 5 out of 5 tasks
1/5: running [----->]
2/5: running [----->]
3/5: running [----->]
4/5: running [----->]
5/5: running [----->]
verify: Service converged
root@mgr:~# docker service ps mynginx

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
ev6gbvv3jvbx	mynginx.1	nginx:latest	wkr02	Running	Running 9 minutes ago		
z3a007j2v8o7	mynginx.2	nginx:latest	wkr01	Running	Running 9 minutes ago		
eyupru9mz6t6	mynginx.3	nginx:latest	mgr	Running	Running 9 minutes ago		
r20pmwy4txvv	mynginx.4	nginx:latest	mgr	Running	Running 9 minutes ago		
c8oh6g79eff3	mynginx.6	nginx:latest	wkr02	Running	Running 6 minutes ago		

```

root@mgr:~#

```

Рис. 5.13

Мы создали сервис с четырьмя репликами, чтобы увидеть, как реплики распределяются между узлами. Затем увеличили число реплик до семи, проверили, как они распределились между узлами, потом уменьшили число реплик до пяти и убедились, как легко и просто уменьшать и увеличивать масштаб в Docker Swarm.

Реплицированные и глобальные сервисы

В Docker Swarm есть два типа сервисов: *реплицированные* (replicated) и *глобальные* (global). Для реплицированных сервисов нужно указать число реплик, и управляющий узел Swarm сам распределит реплики по другим узлам. Если это глобальный сервис, управляющий узел Swarm размещает по одной задаче на каждом доступном узле, учитывая требования к ресурсам и другие ограничения. Если в кластер добавляется узел и работает глобальный сервис, на новом узле создается реплика этого сервиса.

Запустим несколько команд и посмотрим, что получилось (рис. 5.14).

```
docker service create --name replicated --replicas 3 --mode replicated
redis:latest
docker service ps replicated
```

```
root@mgr:~# docker service create --name replicated --replicas 3 --mode replicated redis:latest
o2zo5xafgvk15g8yr4pz60ahm
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service converged
root@mgr:~# docker service ps replicated
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
mpwe71d136sg	replicated.1	redis:latest	wkr02	Running	Running 27 seconds ago	
uh5pb3te33x0	replicated.2	redis:latest	mgr	Running	Running 26 seconds ago	
w96pkg6st9y0	replicated.3	redis:latest	wkr01	Running	Running 27 seconds ago	

Рис. 5.14

Итак, мы создали сервис с тремя репликами и явно указали реплицированный (replicated) режим по умолчанию. В итоге мы получили по одной реплике на каждом узле. Теперь давайте создадим глобальный (global) сервис и изучим результаты на рис. 5.15.

```
docker service create --name global --mode global nginx:latest
docker service ps global
```

Тут также все очевидно. На каждом узле создано по сервису, и пока не видно разницы между реплицированным и глобальным сервисами. Давайте попробуем доба-

```
root@mgr:~# docker service create --name global --mode global nginx:latest
w49nu4aeulmo6vhpwrribb6j8
overall progress: 3 out of 3 tasks
14codc8w1fe4: running [=====>]
pkp510b36i9v: running [=====>]
ou16s34ucx78: running [=====>]
verify: Service converged
root@mgr:~# docker service ps global
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
qehm1t4o5hs6	global.pkp510b36i9vpavewf19plzjt	nginx:latest	wkr02	Running	Running 21 seconds ago	
0t1f1eybw17t	global.ou16s34ucx78u97u2ntv43o2	nginx:latest	wkr01	Running	Running 21 seconds ago	
851tavvunrvt	global.14codc8w1fe4aobi7jco2xpqj	nginx:latest	mgr	Running	Running 18 seconds ago	

Рис. 5.15


```

root@wkr03:~# docker swarm join --token SWMTKN-1-16n8nyoipgwordrsopztl5wiybz9ya8gjjw9ormj06jphza7r-cv6sarvai1nkstx19yh33iwix 209
This node joined a swarm as a worker.
root@wkr03:~# docker container ls -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
85dc6a18b76c        nginx:latest        "nginx -g 'daemon of..." 9 minutes ago       Up 8 minutes        80/tcp             global.wlg8f5uyn9jj4kxjpev68vwgn
.n4k5uuusc6xv8t1otd7r25gfz
root@wkr03:~#

```

Рис. 5.16

вить в кластер Swarm еще один узел. На рис. 5.16 видно, что в кластер добавлен рабочий узел 3 — `wkr03`.

Как видим, сразу после добавления узла создается и запускается контейнер, т. е. реплика. В этом можно убедиться, запустив на управляющем узле следующую команду (рис. 5.17).

```
docker service ps global
```

```

root@mgr:~# docker service ps global
ID            PORTS          NAME                                IMAGE           NODE    DESIRED STATE  CURRENT STATE      ERROR
n4k5uuusc6kv  global.wlg8f5uyn9jj4kxjpev68vwgn  nginx:latest    wkr03          Running        Running 18 minutes ago
qehmlt4o5hs6  global.pkp510b36i9vpavewf19plzjt  nginx:latest    wkr02          Running        Running 40 minutes ago
011f1aybw17t  global.ou16s34ucx788u97u2ntv43o2  nginx:latest    wkr01          Running        Running 40 minutes ago
851tavvunrvt  global.14codc8w1fe4aobi7jco2xpcj  nginx:latest    mgr            Running        Running 40 minutes ago

```

Рис. 5.17

Теперь на каждом узле выполняется сервис, включая последний добавленный узел — `wkr03`. Следовательно, если установить для сервиса глобальный режим, при каждом добавлении узла в кластер планировщик размещает на нем задачу глобального сервиса.

Очистка кластера Swarm

По умолчанию Swarm выполняется в режиме **ACTIVE** (активен): узел получает и выполняет задачи, назначаемые управляющим узлом.

В процессе планового обслуживания можно перевести узел в режим **DRAIN** (очистка), чтобы он не получал новые задачи от управляющего узла Swarm. При этом все выполняющиеся задачи останавливаются и создаются на другом узле со статусом **ACTIVE**.

Рассмотрим эту концепцию на конкретном примере. Создадим сервис с двумя репликами, затем переведем узел с репликой выполняющегося сервиса в режим **DRAIN** и проверим, будет ли реплика перенесена на доступный узел. Применим несколько команд и посмотрим выходные данные на рис. 5.18.

Режим **DRAIN** не приводит к удалению отдельных контейнеров, созданных ранее с помощью команд `docker run`, `docker compose` или Docker Engine API. Переходя в режим **DRAIN**, узел сразу прекращает принимать новые задачи, а уже имеющиеся переносятся на другие доступные узлы.

```
docker service create --name myservice --replicas 2 nginx:latest
docker service ps myservice
```

```
root@mgr:~# docker service create --name myservice --replicas 2 nginx:latest
tqgebb7sv2357kzphifyfuedwt
overall progress: 2 out of 2 tasks
1/2: running [----->]
2/2: running [----->]
verify: Service converged
root@mgr:~# docker service ps myservice
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
400v0ce66xtd	myservice.1	nginx:latest	mgr	Running	Running 16 seconds ago		
d41gvcdni5q6	myservice.2	nginx:latest	wkr03	Running	Running 16 seconds ago		

Рис. 5.18

Здесь мы видим две реплики сервиса, созданные на узлах `mgr` и `wkr03`.

Переведем один из узлов в режим `DRAIN` и убедимся, что реплика была перенесена на другой доступный узел. Запустим команду для узла `wkr03` (см. выходные данные на рис. 5.19).

```
docker node update --availability drain wkr03
docker service ps myservice
```

```
root@mgr:~# docker node update --availability drain wkr03
wkr03
root@mgr:~# docker service ps myservice
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
400v0ce66xtd	myservice.1	nginx:latest	mgr	Running	Running 19 minutes ago	
qjaa6tntgoja	myservice.2	nginx:latest	wkr01	Running	Running 22 seconds ago	
d41gvcdni5q6	myservice.2	nginx:latest	wkr03	Shutdown	Shutdown 23 seconds ago	

Рис. 5.19

Реплика действительно появилась на узле `wkr01`, а узел `wkr03` остановлен (`Shutdown`). Давайте вернем узел `wkr03` и посмотрим, что изменится (рис. 5.20).

```
docker node update --availability active wkr03
docker node ps myservice
```

```
root@mgr:~# docker node update --availability active wkr03
wkr03
root@mgr:~# docker service ps myservice
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
400v0ce66xtd	myservice.1	nginx:latest	mgr	Running	Running 24 minutes ago	
qjaa6tntgoja	myservice.2	nginx:latest	wkr01	Running	Running 5 minutes ago	
d41gvcdni5q6	myservice.2	nginx:latest	wkr03	Shutdown	Shutdown 5 minutes ago	

Рис. 5.20

Узел `wkr03` по-прежнему остановлен, хотя на самом деле он активен, как можно увидеть на рис. 5.21, где приводятся выходные данные команды `docker node inspect --pretty wkr03`.

Когда узел переводится в режим `ACTIVE`, он готов получать и исполнять задачи в следующих случаях:

- ◆ изменение масштаба сервиса во время обновления сервиса;
- ◆ последовательное обновление;
- ◆ перевод другого узла в режим DRAIN;
- ◆ сбой другого узла (обеспечение высокой доступности).

```

root@mgr:~# docker node inspect --pretty wkr03
ID: wlg8f5uyn9jj4kxjpmv68vwgn
Hostname: wkr03
Joined at: 2019-12-17 06:31:03.644833261 +0000 utc
Status:
State: Ready
Availability: Active
Address: 178.128.247.56
Platform:
Operating System: linux
Architecture: x86_64
Resources:

```

Рис. 5.21

Блокировка и разблокировка кластера Swarm

Начиная с Docker 1.13, где было представлено *управление секретами*, журналы Raft хранятся на диске в зашифрованном виде. Может возникнуть вопрос: почему эти журналы так важны? Дело в том, что у всех управляющих узлов должен быть доступ к одной и той же версии журналов, чтобы в случае сбоя лидера любой из доступных управляющих узлов мог взять на себя его роль.

С появлением функции секретов в Docker можно обеспечивать шифрование журналов Raft в месте хранения, чтобы защитить их от злоумышленников.

Пара ключей, которая используется для шифрования, хранится вместе с зашифрованными журналами. Для шифрования журналов Raft и защиты взаимодействия между узлами используется закрытый (private) ключ.

При перезапуске Docker ключи для шифрования и дешифрования журналов Raft на диске и все, что касается обеспечения безопасности коммуникаций (Transport Layer Security, TLS) между узлами, кешируется в памяти каждого управляющего узла.

Начиная с версии Docker 1.13, эти ключи можно использовать, чтобы блокировать и разблокировать Swarm. Данная функция называется *автоблокировкой (autolock)*. При наличии автоблокировки управляющий узел после перезапуска должен предъявить ключ разблокировки кластера, чтобы вернуться в этот кластер.

При добавлении нового узла в кластер Swarm нет необходимости его разблокировать, поскольку ключ распространяется через взаимный TLS.

При перезапуске Docker нужно сначала разблокировать Swarm с помощью ключа шифрования, который Docker создал при блокировке Swarm. Этот ключ шифрования можно сменить в любое время.

Посмотрим, как это работает.

Прежде всего проверим, установлена у нас автоблокировка (True) или нет (False). По умолчанию она отсутствует (False).

```
docker info | grep Autolock
```

Сейчас мы ее установим. При установке автоблокировки автоматически будет создан ключ, который мы используем для разблокировки Swarm. Ключ должен храниться в надежном месте — если мы его потеряем, то не сможем разблокировать Swarm.

```
docker swarm update --autolock=true
```

Посмотрите на рис. 5.22, чтобы все стало понятно.

```

root@mgr:~# docker info | grep Autolock
Autolock Managers: false
WARNING: No swap limit support
root@mgr:~# docker swarm update --autolock=true
Swarm updated.
To unlock a swarm manager after it restarts, run the 'docker swarm unlock'
command and provide the following key:

    SWMKEY-1-nxTSgy+LNrKZzyQbb0Eu2qSKkMGC0KmpaF1sL5wXEFs

Please remember to store this key in a password manager, since without it you
will not be able to restart the manager.
root@mgr:~#
    
```

Рис. 5.22

Изменения не вступят в силу, пока Docker не будет перезапущен, хотя после команды `docker info | grep Autolock` значение изменится на True. При этом, если мы попытаемся запустить команду `docker node ls`, ничто нам не мешает, т. е. Swarm не заблокирован. Давайте перезапустим Docker, чтобы заблокировать кластер, и посмотрим на результат (рис. 5.23).

```
docker node ls
systemctl restart docker
```

```

root@mgr:~# docker node ls
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
nm1d9nhenqjryko8r3rx01a8u *      mgr         Ready     Active           Leader             19.03.5
t50zzq3ytmy65tnn93r8bq2f6        wkr01       Ready     Active           -                  19.03.5
twevtgnelqlu7fkh4zr8bfj1         wkr02       Ready     Active           -                  19.03.5
root@mgr:~# systemctl restart docker
root@mgr:~#
    
```

Рис. 5.23

После перезапуска Docker при попытке запустить команду `docker node ls` мы получаем сообщение (рис. 5.24): **Swarm is encrypted and needs to be unlocked before it can be used. Please use «docker swarm unlock» to unlock it** (Swarm зашифрован, и его нужно разблокировать. Пожалуйста, используйте «docker swarm unlock» для разблокировки).

```

root@mgr:~# systemctl restart docker
root@mgr:~# docker node ls
Error response from daemon: Swarm is encrypted and needs to be unlocked before it can be used.
Please use "docker swarm unlock" to unlock it.
root@mgr:~# █

```

Рис. 5.24

Теперь мы знаем, что Swarm заблокирован. Давайте посмотрим, как это повлияет на нашу работу: выведем рабочий узел из Swarm, а затем попробуем вернуть его обратно. Войдем на узел `wkr02` и запустим команду:

```
docker swarm leave
```

Узел выходит из Swarm. Теперь используем ключ токена, который мы сохранили ранее, чтобы снова присоединить узел к Swarm. Разумеется, у нас это не получится, как видно на рис. 5.25.

```

root@wkr01:~# docker swarm leave
Node left the swarm.
root@wkr01:~# docker swarm join --token SWMTKN-1-16a8myoipgwordwrsopztn15wiyb29ya8qjw9ormj06jphza7r-cvmsarvailnkstx19yh33iwiX 20
Error response from daemon: rpc error: code = Unavailable desc = all SubConns are in TransientFailure, latest connection error:
connection error: desc = "transport: Error while dialing dial tcp 209.97.186.213:2377: connect: connection refused"
root@wkr01:~# █

```

Рис. 5.25

Однако мы можем запускать на управляющем и рабочих узлах любые команды, не относящиеся к Swarm, и они будут выполняться без проблем. Например, мы можем создать контейнер на управляющем или рабочем узле. Это очень важный момент: в заблокированном кластере Swarm нельзя применять команды, относящиеся к Swarm.

Давайте разблокируем Swarm с помощью ключа и снова попробуем вернуть рабочий узел в кластер.

Запустим команду `docker swarm unlock` на управляющем узле. Система запрашивает ключ разблокировки, мы его предоставляем и видим, что Swarm разблокирован. Снова входим на узел `wkr02` и возвращаем его в Swarm. Как видите, все прекрасно работает! Команды и результаты ниже (рис. 5.26 и 5.27).

Управляющий узел (рис. 5.26):

```

docker swarm unlock
docker node ls

```

```

root@mgr:~# docker swarm unlock
Please enter unlock key:
root@mgr:~# docker node ls

```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
nm1d9nhenqjryko8r3rx01a8u *	mgr	Ready	Active	Leader	19.03.5
t50zzq3ytmy65tnn93r8bq2f6	wkr01	Ready	Active		19.03.5
twevtgnelqlu7fkh4zr8bfj1	wkr02	Unknown	Active		19.03.5

```

root@mgr:~# █

```

Рис. 5.26

Рабочий узел (рис. 5.27):

```
docker swarm join token <token>
```

```
root@wkr02:~# docker swarm join --token SWMTKN-1-4cvu1v6711si962rln4aahyu8ujwqvm4iw6ztmiflexxs9ku1h-8agc5z6ybt
This node joined a swarm as a worker.
root@wkr02:~#
```

Рис. 5.27

Сети в Docker Swarm

Понимание принципов работы сетей в Docker Swarm крайне важно. Этой теме посвящена вся следующая глава, а здесь мы рассмотрим сети в контексте Swarm.

При создании кластера Swarm происходит следующее:

1. Создается *сетевой мост* с именем `docker_gwbridge`, который используется для соединения контейнеров в кластере Swarm с внешним миром.
2. Создается *оверлейная сеть* с именем `ingress`, которая отвечает за коммуникации между контейнерами.

Если оверлейная сеть не определена пользователем, создается сеть по умолчанию с именем `ingress`, которая используется для контроля и передачи данных в пределах кластера Swarm.

Мы можем создавать пользовательские оверлейные сети и сетевые мосты, причем сервисы и контейнеры могут быть подключены к нескольким сетям одновременно.

Правила межсетевого экрана для демонов Docker при использовании оверлейных сетей

Нужно открыть следующие порты для входящего и исходящего трафика каждого хоста Docker в составе оверлейной сети:

- TCP 2377 (для управления коммуникациями кластера);
- TCP и UDP 7946 (для коммуникаций между узлами);
- UDP 4789 (для трафика оверлейной сети).

Теперь давайте узнаем, какие сети доступны для управляющего узла Swarm. Команда и выходные данные ниже (рис. 5.28).

```
docker network ls
```

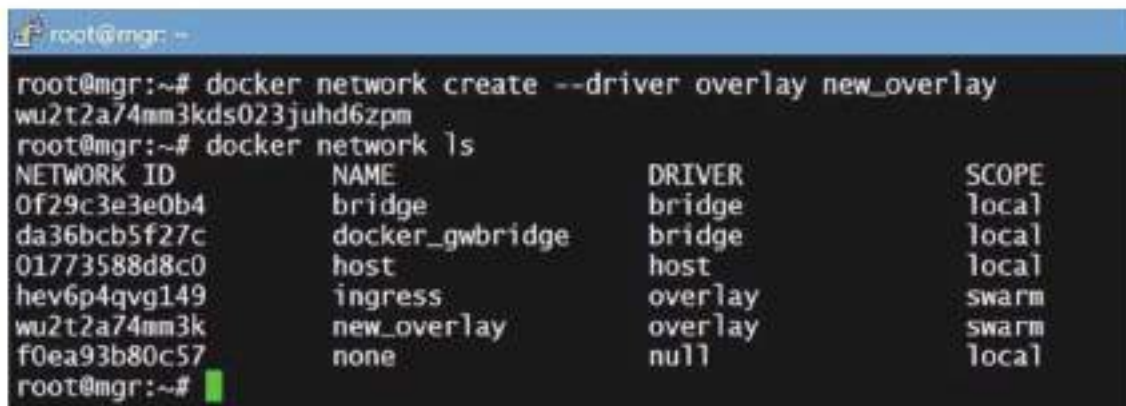
Как и ожидалось, мы видим сети `docker_gwbridge` и `ingress`.

```
root@mgr:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
0f29c3e3e0b4        bridge              bridge              local
da36bcb5f27c        docker_gwbridge     bridge              local
01773588d8c0        host                host                local
hev6p4qvg149        ingress             overlay             swarm
f0ea93b80c57        none                null                local
root@mgr:~#
```

Рис. 5.28

Создадим оверлейную сеть и посмотрим, что получится (рис. 5.29).

```
docker network create --driver overlay new_overlay
```

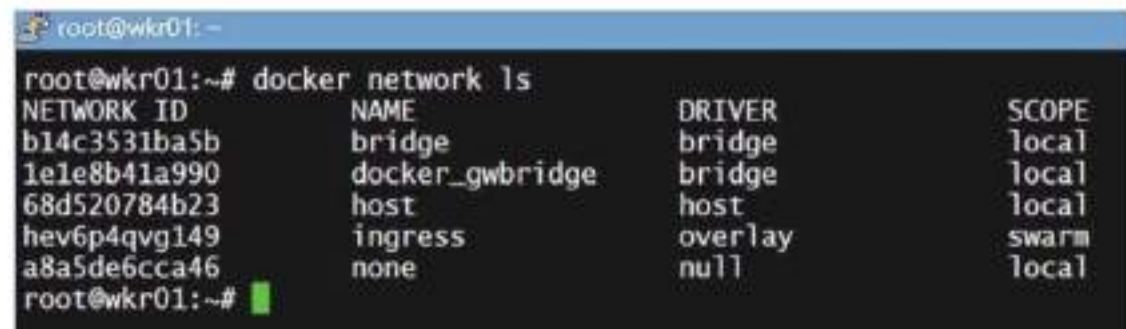


```
root@mgr:~# docker network create --driver overlay new_overlay
wu2t2a74mm3kds023juhd6zpm
root@mgr:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
0f29c3e3e0b4        bridge              bridge              local
da36bcb5f27c        docker_gwbridge     bridge              local
01773588d8c0        host                host                local
hev6p4qvg149        ingress             overlay             swarm
wu2t2a74mm3k        new_overlay         overlay             swarm
f0ea93b80c57        none                null                local
root@mgr:~#
```

Рис. 5.29

Итак, у нас есть новая оверлейная сеть с именем `new_overlay`. Нам не нужно отдельно создавать сеть на рабочих узлах, она будет распространена на них автоматически. Правда, она не существует, пока мы не создадим сервис и не свяжем его с новой сетью. Проверим сети на одном из рабочих узлов. См. команду ниже и рис. 5.30.

```
docker network ls
```



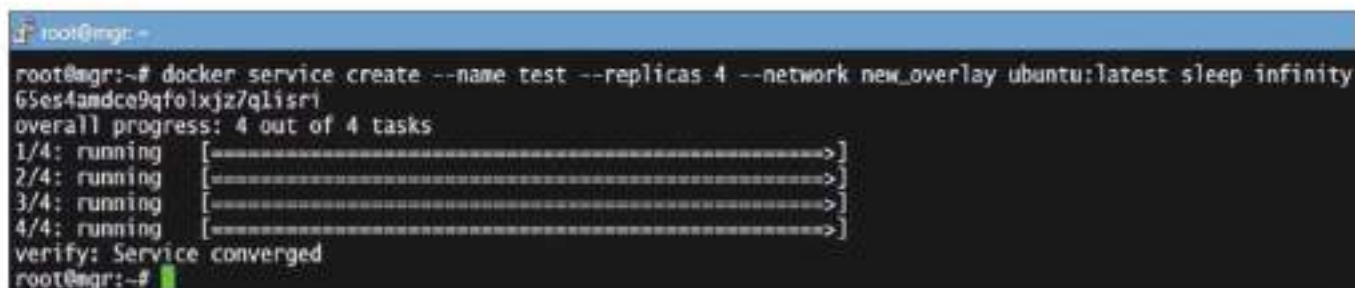
```
root@wkr01:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
b14c3531ba5b        bridge              bridge              local
1e1e8b41a990        docker_gwbridge     bridge              local
68d520784b23        host                host                local
hev6p4qvg149        ingress             overlay             swarm
a8a5de6cca46        none                null                local
root@wkr01:~#
```

Рис. 5.30

Естественно, мы не видим новую сеть `new_overlay`.

Создадим сервис и свяжем его с созданной оверлейной сетью. См. команду ниже и рис. 5.31.

```
docker service create --name test --replicas 4 --network new_overlay
ubuntu:latest sleep infinity
```



```
root@mgr:~# docker service create --name test --replicas 4 --network new_overlay ubuntu:latest sleep infinity
65es4amdce9qfolxjz7qlisri
overall progress: 4 out of 4 tasks
1/4: running [=====>]
2/4: running [=====>]
3/4: running [=====>]
4/4: running [=====>]
verify: Service converged
root@mgr:~#
```

Рис. 5.31

Итак, сервис создан. Теперь нужно вернуться на узел `wkr01`, чтобы проверить сеть. Соответствующая команда и выходные данные ниже (рис. 5.32).

```
docker network ls
```

```
root@wkr01:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
b14c3531ba5b        bridge              bridge              local
1e1e8b41a990        docker_gwbridge     bridge              local
68d520784b23        host                host                local
hev6p4qvg149        ingress             overlay             swarm
wu2t2a74mm3k        new_overlay         overlay             swarm
a8a5de6cca46        none                null                local
root@wkr01:~#
```

Рис. 5.32

Теперь сеть доступна на рабочем узле.

Пока все хорошо. Продвинемся дальше и проверим закрытый (private) IP-адрес, подсеть и шлюз. В целях тестирования мы запустим несколько команд на управляющем узле, а затем на одном из рабочих. Команда на управляющем узле:

```
docker network inspect new_overlay
```

И соответствующие выходные данные (рис. 5.33).

```
root@mgr:~# docker network inspect new_overlay
[
  {
    "Name": "new_overlay",
    "Id": "wu2t2a74mm3kds023juhd6zpm",
    "Created": "2019-12-18T09:17:24.668364224Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.1.0/24",
          "Gateway": "10.0.1.1"
        }
      ]
    }
  }
]
```

Рис. 5.33

Проверим IP-адрес при помощи следующей команды (результаты — на рис. 5.34):

```
docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
<container_id>
```

```
root@mgr:~# docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' 2203bed4ec53
10.0.1.6
root@mgr:~#
```

Рис. 5.34

Эту информацию можно получить и с рабочего узла (рис. 5.35).

Также мы можем легко узнать IP-адреса контейнеров (рис. 5.36).

```
root@wkr01:~# docker network inspect new_overlay
[
  {
    "Name": "new_overlay",
    "Id": "wu2t2a74mn3kds023juhd6zpm",
    "Created": "2019-12-18T09:17:24.669222546Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.1.0/24",
          "Gateway": "10.0.1.1"
        }
      ]
    }
  }
]
```

Рис. 5.35

```
root@wkr01:~# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
7b3921e9a977   ubuntu:latest  "sleep infinity"        39 minutes ago  Up 39 minutes          test.1.updh7wklqdlcz8lhwcg3ah40z
eafac71c5433   ubuntu:latest  "sleep infinity"        39 minutes ago  Up 39 minutes          test.2.j3tbnmagvfj6yj7a18z1vf17z
root@wkr01:~# docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' 7b3921e9a977
10.0.1.3
root@wkr01:~# docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' eafac71c5433
10.0.1.4
root@wkr01:~#
```

Рис. 5.36

Как видим, здесь есть ценная информация о сети. Теперь попробуем с управляющего узла отправить ping к рабочему и наоборот, чтобы проверить, можем ли мы получать доступ к узлам изнутри кластера Swarm.

Первой командой мы получаем `id_контейнера`, который потребуется во второй команде. С помощью второй команды мы входим в этот контейнер.

```
docker container ls -a
docker container exec -it <id_контейнера> sh
```

Следующая команда требуется для установки ping и traceroute, потому что они могут быть недоступны в контейнере.

```
apt-get update && apt-get install iputils-ping -y && apt-get install
traceroute
ping <ip_рабочего_узла>
```

Рис. 5.37 показывает, что команды ping и traceroute успешно выполнены с управляющего узла к одному из рабочих.

Теперь попробуем выполнить те же команды с одного из рабочих узлов (`wkr01`) к управляющему (рис. 5.38).


```

root@mgr: ~
# ping 10.0.1.3
PING 10.0.1.3 (10.0.1.3) 56(84) bytes of data.
64 bytes from 10.0.1.3: icmp_seq=1 ttl=64 time=1.79 ms
64 bytes from 10.0.1.3: icmp_seq=2 ttl=64 time=0.499 ms
64 bytes from 10.0.1.3: icmp_seq=3 ttl=64 time=0.587 ms
64 bytes from 10.0.1.3: icmp_seq=4 ttl=64 time=0.535 ms
^C
--- 10.0.1.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.499/0.853/1.794/0.544 ms
# ping 10.0.1.4
PING 10.0.1.4 (10.0.1.4) 56(84) bytes of data.
64 bytes from 10.0.1.4: icmp_seq=1 ttl=64 time=2.00 ms
64 bytes from 10.0.1.4: icmp_seq=2 ttl=64 time=0.703 ms
64 bytes from 10.0.1.4: icmp_seq=3 ttl=64 time=0.636 ms
64 bytes from 10.0.1.4: icmp_seq=4 ttl=64 time=0.668 ms
^C
--- 10.0.1.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.636/1.001/2.000/0.578 ms
# traceroute 10.0.1.3
traceroute to 10.0.1.3 (10.0.1.3), 30 hops max, 60 byte packets
 1 test.1.updh7wkbqdlcz8lhwcg3ah40z.new_overlay (10.0.1.3) 1.677 ms 1.573 ms 1.599 ms
# traceroute 10.0.1.4
traceroute to 10.0.1.4 (10.0.1.4), 30 hops max, 60 byte packets
 1 test.2.j3tbhmagvfj6yj7al8zlvfl7z.new_overlay (10.0.1.4) 1.934 ms * 1.784 ms
#
    
```

Рис. 5.37

```

root@wk01: ~
# ping 10.0.1.6
PING 10.0.1.6 (10.0.1.6) 56(84) bytes of data.
64 bytes from 10.0.1.6: icmp_seq=1 ttl=64 time=1.74 ms
64 bytes from 10.0.1.6: icmp_seq=2 ttl=64 time=0.636 ms
64 bytes from 10.0.1.6: icmp_seq=3 ttl=64 time=0.544 ms
^C
--- 10.0.1.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.544/0.975/1.747/0.547 ms
# traceroute 10.0.1.6
traceroute to 10.0.1.6 (10.0.1.6), 30 hops max, 60 byte packets
 1 test.4.xku3o7d327kf9gi99oemspob0.new_overlay (10.0.1.6) 2.022 ms 1.986 ms 1.950 ms
#
    
```

Рис. 5.38

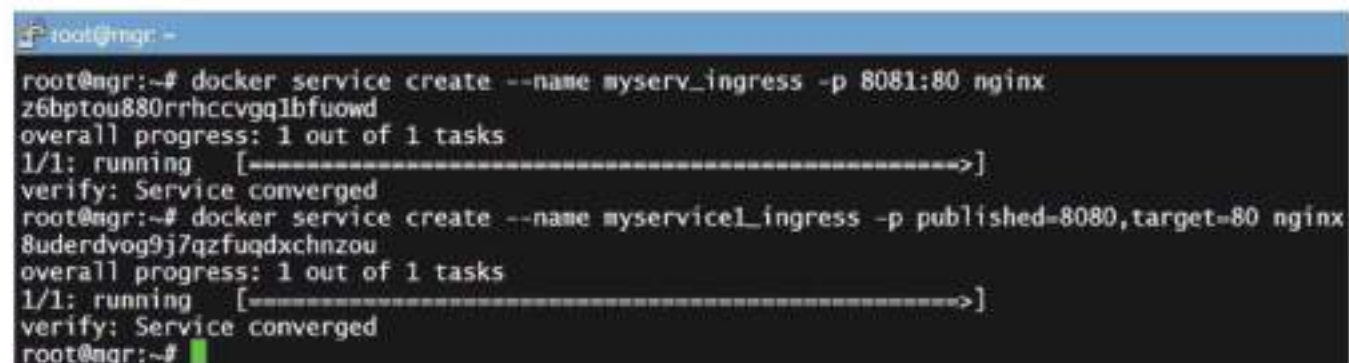
Мы видим, что можем свободно отправлять команду ping между узлами в кластере. Теперь узнаем, как опубликовать порты при создании сервиса.

Создание сервиса с опубликованным портом

У сервисов Swarm, соединенных оверлейной сетью, нет проблем с подключением, потому что они предоставляют друг другу свои порты. Клиент, подключающийся к кластеру Swarm извне, будет использовать доступные порты по умолчанию, если явно не указать, через какой порт следует подключаться. Для этого нужно опубликовать порты с помощью флага `-p` или `published`. Поддерживается как старый синтаксис с двоеточием в качестве разделителя, так и новый — с запятой. Более длинный синтаксис (полный вариант флага) предпочтительнее, поскольку это облегчает понимание.

Запустим несколько команд и посмотрим результат на рис. 5.39.

```
docker service create --name myserv_ingress -p 8081:80 nginx
docker service create --name myservice_ingress -p published=8080,target=80 nginx
```

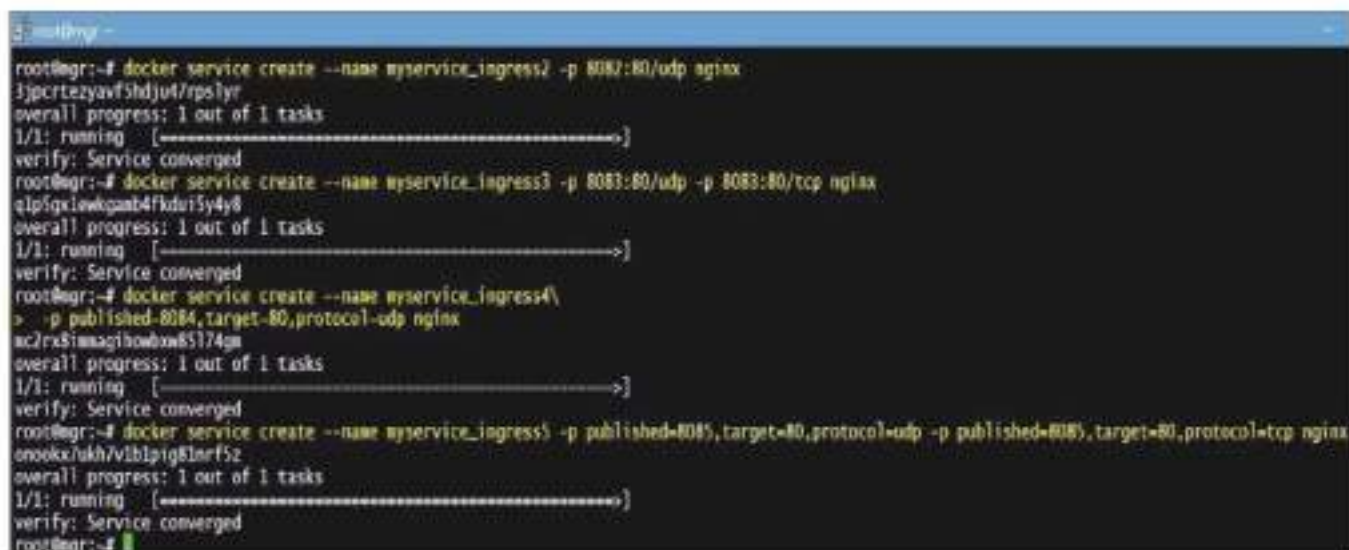


```
root@ngr:~# docker service create --name myserv_ingress -p 8081:80 nginx
z6bptou880rrhccvgqlbfuowd
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
root@ngr:~# docker service create --name myservice_ingress -p published=8080,target=80 nginx
8uderdvog9j7qzfuqdxchnzou
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
root@ngr:~#
```

Рис. 5.39

Можно даже указать протоколы. Чтобы лучше понять, как все работает, запустим еще несколько команд. Выходные данные см. на рис. 5.40.

```
docker service create --name myservice_ingress2 -p 8082:80/udp nginx
docker service create --name myservice_ingress3 -p 8083:80/udp -p 8083:80/tcp nginx
docker service create --name myservice_ingress4 \
  -p published=8084,target=80,protocol=udp nginx
docker service create --name myservice_ingress5 -p published=8085,target=80, \
  protocol=udp -p published=8085,target=80,protocol=tcp nginx
```



```
root@ngr:~# docker service create --name myservice_ingress2 -p 8082:80/udp nginx
3jpcrtezyavf5hdju4/rpslyr
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
root@ngr:~# docker service create --name myservice_ingress3 -p 8083:80/udp -p 8083:80/tcp nginx
qlp9xlewkqmb4fkdui5y4y8
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
root@ngr:~# docker service create --name myservice_ingress4 \
  -p published=8084,target=80,protocol=udp nginx
xc2rx8imaagihowbw85174gm
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
root@ngr:~# docker service create --name myservice_ingress5 -p published=8085,target=80,protocol=udp -p published=8085,target=80,protocol=tcp nginx
onookx/ukh7viblpig8lnrf5z
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
root@ngr:~#
```

Рис. 5.40

Обход маршрутизирующей *mesh*-сети в Swarm

Маршрутизирующая *mesh*-сеть использует внутренний алгоритм для маршрутизации запросов к рабочим узлам. Когда клиент отправляет запрос, неизвестно, какой узел его обслужит. В некотором смысле в Docker осуществляется балансировка

нагрузки. Даже если мы укажем флаг `--global`, чтобы разместить сервисы по доступным узлам, маршрутизирующая mesh-сеть все равно будет использоваться.

Возможно ли обойти эту маршрутизацию и в каких ситуациях это может понадобиться?

Если мы установим свой балансировщик нагрузки перед сервисом, необходимости в mesh-сети не будет. Для этого нужно запустить сервис *DNS Round Robin (DNSRR)*, установив флаг `--endpoint-mode` в `dnsrr`.

Шифрование трафика в оверлейной сети

Весь управляющий трафик в сервисе Swarm зашифрован по умолчанию. Для шифрования используется алгоритм *AES*. Ключи периодически меняются, например, управляющий узел в Swarm сменяет ключи каждые 12 часов.

При желании мы также можем зашифровать данные приложения. Для этого при создании и обновлении сети нужно указать флаг `--opt encrypted`, чтобы включить шифрование *IPSec (IP Security)* на уровне VXLAN. Docker создает туннели IPSec между всеми узлами, на которых запланированы задачи для сервисов, привязанных к оверлейной сети. Эти туннели тоже используют алгоритм AES в режиме GCM, а управляющие узлы автоматически меняют ключи каждые 12 часов. Однако шифрование влияет на производительность, и этот факт следует учитывать, если предполагается шифровать данные в рабочей среде.

Устранение неполадок в Docker Swarm

Если в Docker Swarm возникла проблема, для начала следует заглянуть в журналы сервисов, если драйвер поддерживает соответствующие команды. По умолчанию Docker использует драйвер журналирования *json-file*, который поддерживает эти команды. Журналы могут быть громоздкими, поэтому рекомендуется отслеживать потоки вывода журналов (`--follow`) или выводить только последние строки (`--tail`).

Заключение

Итак, мы изучили Docker Swarm и узнали, как он помогает обеспечивать *высокую доступность* посредством создания кластера; если происходит сбой одного узла в кластере, его задачи продолжают выполняться на оставшихся узлах. Затем мы рассмотрели алгоритм консенсуса в кластере и принципы оркестрации внутри Swarm, а также механизм балансировки нагрузки с помощью маршрутизирующей mesh-сети. Вы узнали, как в Docker Swarm легко увеличивать и уменьшать количество реплик, как работают эффективные механизмы обеспечения безопасности — автоблокировка и шифрование, как организованы сети и как использовать порт и протокол не по умолчанию при создании сервиса. В целом, изучив эту главу, вы получили вполне ясное представление о Docker Swarm.

Основные тезисы

- ◆ Docker Swarm — это нативное решение Docker для оркестрации и обеспечения высокой доступности.
- ◆ Docker Swarm интегрирован в экосистему Docker и использует собственный API.
- ◆ Принципы настройки и администрирования Docker Swarm интуитивно понятны.
- ◆ В Docker Swarm можно легко уменьшать и увеличивать количество реплик, так что это отличный вариант для контейнерных приложений.
- ◆ В Docker есть несколько встроенных средств безопасности, которые не требуют дополнительной настройки и сторонних инструментов.
- ◆ В Docker Swarm можно настраивать сеть под свои потребности, хотя конфигурация по умолчанию подходит для большинства ситуаций.
- ◆ Docker позволяет легко и быстро развертывать и масштабировать контейнеры даже в очень больших кластерах.

Контрольный тест

Выберите подходящий вариант ответа.

1. При помощи чего Docker Swarm позволяет быстро развертывать приложения на нескольких хостах?
 - Минимальный API.
 - Гипервизор.
 - Многофункциональный API.
 - Безопасные сети.
2. Docker Swarm — это:
 - Решение для резервного копирования.
 - Оркестратор.
 - Балансировщик нагрузки.
 - Оркестратор и балансировщик нагрузки.
3. Docker Swarm автоматически создает:
 - Оверлейную сеть.
 - Оверлейную сеть и сетевой мост.
 - Сетевой мост и сеть хоста.
 - Физическую сеть.

4. В каких режимах можно создавать реплики?
 - Универсальный и глобальный.
 - Универсальный и реплицированный.
 - Реплицированный и глобальный.
 - При создании реплик нельзя указать режим.
5. Как реализовать автоблокировку в Docker Swarm?
 - Изменить autolock на true.
 - Автоблокировка включена по умолчанию.
 - Изменить autolock на true и перезапустить Docker.
 - Изменить autolock на true и сменить ключи.
6. Что происходит, когда создается реплика в глобальном режиме?
 - При добавлении узла в кластер автоматически создается новая реплика.
 - При добавлении узла в кластер нужно вручную создать новую реплику.
 - На добавленных узлах нельзя создавать новые реплики.
 - В кластер нельзя добавлять узлы.
7. Когда Swarm заблокирован...
 - Узел в Swarm можно добавить только после разблокировки.
 - Можно добавить узел в Swarm сразу.
 - Можно добавить узел в Swarm, если доступен хотя бы один управляющий узел.
 - Можно добавить узел в Swarm только в качестве управляющего узла.
8. Если установлен режим DRAIN...
 - Существующие задачи продолжают выполняться, но новые приниматься не будут.
 - Существующие задачи будут сразу остановлены и не продолжают выполняться на других узлах.
 - Существующие задачи будут сразу остановлены и запущены снова на других узлах со статусом ACTIVE.
 - Существующие задачи продолжают выполняться, новые будут приниматься.

Ответы

1	c
2	d
3	b
4	c

5	c
6	a
7	a
8	c

Вопросы

1. Что такое Docker Swarm?
2. Что такое сервис в контексте Docker Swarm? Чем сервис отличается от задачи, или реплики?
3. Что такое маршрутизирующая mesh-сеть и как она работает?
4. Как можно заблокировать Docker Swarm?
5. Что такое консенсус в контексте Docker Swarm.
6. В чем разница между реплицированным и глобальным режимом?
7. Что такое оверлейные сети и как они работают в Docker Swarm?
8. Как опубликовать пользовательские порты и протоколы в сети Docker Swarm?

Ключевые термины

- ◆ Docker Swarm
- ◆ Управляющие и рабочие узлы
- ◆ Сервис
- ◆ Задача
- ◆ Реплика
- ◆ Токены и ключи
- ◆ Алгоритм консенсуса
- ◆ Оверлейная сеть
- ◆ Сетевой мост
- ◆ Маршрутизирующая mesh-сеть
- ◆ Автоблокировка
- ◆ Реплицированные и глобальные сервисы
- ◆ Режимы DRAIN и ACTIVE
- ◆ Опубликованные порты

ГЛАВА 6

Сети Docker

Введение

Эта глава посвящена сетям в Docker. Контейнеры Docker взаимодействуют друг с другом посредством сетей, создаваемых при установке программного обеспечения Docker. По большей части программное обеспечение Docker использует возможности сетей Linux для сетевой инфраструктуры, которую мы подробно рассмотрим здесь, в том числе принципы работы сетей, их различные варианты и потенциал.

Глава содержит несколько тем.

- ◆ Сети в Docker
- ◆ Сеть Bridge
- ◆ Сеть Host
- ◆ Сеть None
- ◆ Использование пространства имен существующего контейнера
- ◆ Сопоставление портов
- ◆ Сеть MACVLAN
- ◆ Сеть Overlay

Сети в Docker

Прежде всего узнаем, какие сети доступны на данный момент. Запустим следующую команду и проверим результат (рис. 6.1).

```
docker network ls
```

При любой установке Docker доступны три стандартные сети. Если установлен Docker Swarm, сетей будет больше, но об этом мы поговорим чуть позже.

```

root@dockernet:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
c9bdeacb94ec        bridge             bridge              local
cc4133211b65        host               host                local
fac19e93cfd5        none               null                local
root@dockernet:~#

```

Рис. 6.1

Сеть Bridge

Мы изучим сеть типа мост (bridge) при создании контейнера и посмотрим, как все работает с точки зрения сети. Но сначала разберемся в понятиях. *Сеть Bridge* можно представить как сетевое устройство канального уровня для перенаправления трафика в другие сети. При установке Docker автоматически устанавливается сеть Docker Bridge (мост) со своими правилами. Все контейнеры автоматически подключаются к мосту Docker, который предоставляет им канал для коммуникаций.

Теперь создадим контейнер `busybox` с именем `bbox`. Запустим следующую команду и изучим рис. 6.2.

```

docker run -dit --name bbox busybox:latest
docker container ls -a

```

```

root@dockernet:~# docker run -dit --name bbox busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
91f30d776fb2: Pull complete
Digest: sha256:9ddee63a712cea977267342e8750ecbc60d3aab25f04ceacfa795e6fce341793
Status: Downloaded newer image for busybox:latest
6a7e2313fb737914ff675deb3feeadd1e3da366e5b5c0df92ccc13799872164a0
root@dockernet:~# docker container ls -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
6a7e2313fb73       busybox:latest     "sh"                41 seconds ago     Up 40 seconds      0.0.0.0:80->0.0.0.0:80/tcp   bbox
root@dockernet:~#

```

Рис. 6.2

Контейнер создан. Давайте заглянем в него с помощью следующей команды и изучим выходные данные (рис. 6.3).

```

docker exec -it bbox sh
ip addr

```

```

root@my-docker:~# docker exec -it bbox sh
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
5: eth0@if6: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
/ #

```

Рис. 6.3

IP-адрес контейнера — 172.17.0.2. IP-адрес создается системой управления IP-адресами (IP Address Management, IPAM). Как правило, первый IP-адрес в диапазоне, т. е. 172.17.0.1, назначается мосту *docker0*. Широковещательный IP-адрес — 172.17.255.255. Все IP-адреса между этими двумя доступны для назначения контейнерам. Также мы видим, что сюда прикреплен один конец пары *veth* с именем *eth0@if6*. Другой конец подключен к хост-системе, как мы увидим чуть ниже.

Теперь выйдем из контейнера (команда *exit*) и вернемся на хост. Далее запустим команду *ip addr*; изучим выходные данные на рис. 6.4:

ip addr

```
root@dockernet:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 1a:03:65:ea:b3:32 brd ff:ff:ff:ff:ff:ff
    inet 165.22.228.221/20 brd 165.22.223.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet 10.47.0.5/16 brd 10.47.255.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::1803:65ff:feea:b332/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether ea:bd:0f:75:61:90 brd ff:ff:ff:ff:ff:ff
    inet 10.139.0.2/16 brd 10.139.255.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::e8bd:fff:fe75:6190/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:00:58:a7:65 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:ff:fe58:a765/64 scope link
        valid_lft forever preferred_lft forever
6: veth5adbd98@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 8e:32:7d:2a:34:87 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::8c32:7dff:fe2a:3487/64 scope link
        valid_lft forever preferred_lft forever
root@dockernet:~#
```

Рис. 6.4

Мы видим, что другой конец пары *veth* с именем *veth5adbd98@if5* подключен к системе. Общая схема представлена на рис. 6.5.

У пространства имен сети хоста есть интерфейс *docker0* и интерфейс *eth0*, который выступает в качестве конечной точки. Обычно *eth0* — это сетевая карта (Network Interface Card, NIC) или виртуальная сетевая карта (virtual NIC), если хост работает на виртуальной машине (ВМ). Хост и контейнер соединены парой *veth*, как показано на рис. 6.5.

Весь трафик проходит через интерфейс *eth0* и направляется к соответствующим контейнерам через *docker0*.

Пока, надеюсь, все понятно. Теперь давайте проверим, сможем ли мы с хоста увидеть пространство имен нашего контейнера. *Пространство имен сети* — это объект, который обычно находится в каталоге */var/run/netns/NAME*.

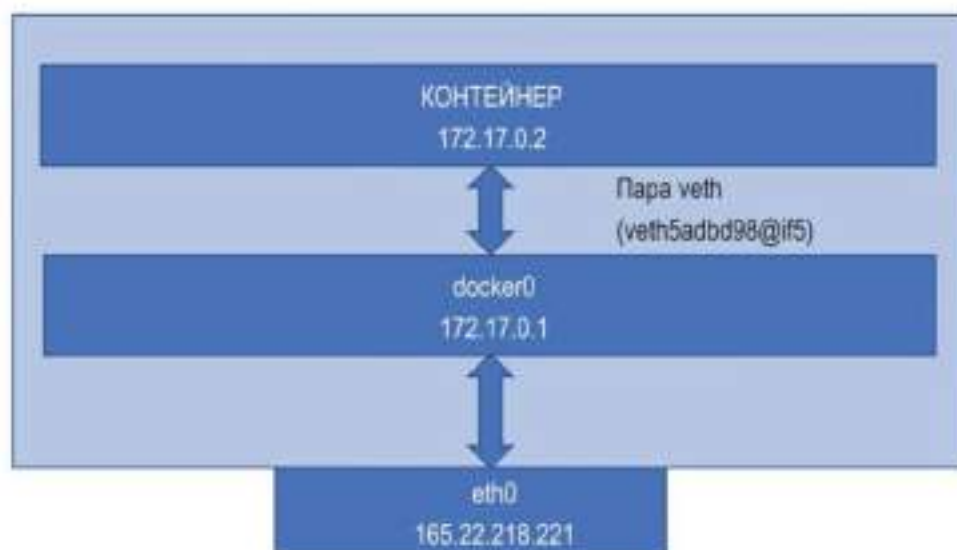


Рис. 6.5. Сетевая инфраструктура

К сожалению, Docker не создает пространство имен сети в расположении по умолчанию. Но мы можем без труда узнать, где оно находится, запустив следующую команду (рис. 6.6).

```
docker inspect bbox --format '{{ .NetworkSettings.SandboxKey }}'
```

```
root@dockernet:~# docker inspect bbox --format '{{ .NetworkSettings.SandboxKey }}'
/var/run/docker/netns/a85656544fd2
root@dockernet:~#
```

Рис. 6.6

Как видим, пространство имен находится в каталоге `/var/run/docker/netns`, а не `/var/run/netns/`. Это значит, что если мы попытаемся проверить пространство имен сети Docker командой `ip netns list`, у нас ничего не получится (рис. 6.7).

```
root@dockernet:~# ip netns list
root@dockernet:~#
```

Рис. 6.7

Обходным решением может быть создание символической ссылки с помощью нескольких команд.

Сначала установим переменные окружения:

```
container_id=bbox
container_netns=$(docker inspect ${container_id} --format\
'{{ .NetworkSettings.SandboxKey }}')
```

Следующей командой создаем символическую ссылку, затем посмотрим выходные данные на рис. 6.8.

```
ln -sv ${container_netns} /var/run/netns/${container_id}
```

```

root@dockernet:~# container_id=bbox
root@dockernet:~# container_netns=$(docker inspect ${container_id} --format '{{.NetworkSettings.SandboxKey }}')
root@dockernet:~# ln -sv ${container_netns} /var/run/netns/${container_id}
'/var/run/netns/bbox' -> '/var/run/docker/netns/a85656544fd2'
root@dockernet:~#

```

Рис. 6.8

Давайте проверим, видим ли мы теперь пространство: команда ниже и выходные данные на рис. 6.9.

```
ip netns list
```

```

root@dockernet:~# ip netns list
bbox (id: 2)
root@dockernet:~#

```

Рис. 6.9

Итак, мы видим пространство имен сети Docker на уровне Linux.

Теперь посмотрим, как контейнеры сообщаются друг с другом через сетевой мост. Сначала удалим наш контейнер (рис. 6.10):

```

docker container rm bbox -force
docker container ls

```

```

root@dockernet:~# docker container rm bbox --force
bbox
root@dockernet:~# docker container ls

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
root@dockernet:~#						

Рис. 6.10

Создадим пару контейнеров, которые, как мы знаем, по умолчанию будут использовать сетевой мост `docker0`.

```

docker run -dit --name mybox busybox:latest
docker run -dit --name mybox1 busybox:latest

```

Проверим выходные данные (рис. 6.11).

```

root@dockernet:~# docker run -dit --name mybox busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
0669b0daf1fb: Pull complete
Digest: sha256:b26cd013274a657b86e706210ddd5cc1f82f50155791199d29b9e86e935ce135
Status: Downloaded newer image for busybox:latest
a994fa431ff5c89179bf59a6394ee63647adc6811ccd1ea48c99471faddb4c3
root@dockernet:~# docker run -dit --name mybox1 busybox:latest
74ba8e7f1e0f39bd0163fb365d24beabdc911450ad5f55c75da189be37e65951
root@dockernet:~# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
74ba8e7f1e0f	busybox:latest	"sh"	12 seconds ago	Up 11 seconds		mybox1
a994fa431ff5	busybox:latest	"sh"	34 seconds ago	Up 33 seconds		mybox

```

root@dockernet:~#

```

Рис. 6.11

Зайдем в контейнеры и узнаем IP-адрес каждого из них, как показано на рис. 6.12.

```
docker exec -it mybox sh
ifconfig
```

```
root@dockernet:~# docker exec -it mybox sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:26 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2012 (1.9 KiB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/ # exit
root@dockernet:~# docker exec -it mybox1 sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:03
          inet addr:172.17.0.3  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:18 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:1356 (1.3 KiB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Рис. 6.12

Теперь попробуем отправить ping со второго контейнера на первый и наоборот, и посмотрим, что получится (рис. 6.13).

```
ping 172.17.0.2
exit
docker exec -it mybox sh
ping 172.17.0.3
```

Все работает, как и ожидалось. Но если мы попытаемся отправить ping к другому контейнеру, используя его имя, у нас ничего не получится (рис. 6.14).

Интересный момент: если мы создадим собственную сеть bridge и соединим ее с контейнерами, мы сможем отправлять ping от одного контейнера к другому, используя их имена. В этом можно убедиться на примере ниже (рис. 6.15).

```
docker network create -d bridge mybridge
docker network ls
docker run -dit --name mybox3 --net mybridge busybox:latest
docker run -dit --name mybox4 --net mybridge busybox:latest
docker exec -it mybox3 sh
ping mybox4
```



```

/ # ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.127 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.158 ms
64 bytes from 172.17.0.2: seq=2 ttl=64 time=0.103 ms
64 bytes from 172.17.0.2: seq=3 ttl=64 time=0.112 ms
64 bytes from 172.17.0.2: seq=4 ttl=64 time=0.085 ms
64 bytes from 172.17.0.2: seq=5 ttl=64 time=0.087 ms
^C
--- 172.17.0.2 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 0.085/0.112/0.158 ms
/ # exit
root@dockernet:~# docker exec -it mybox sh
/ # ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3): 56 data bytes
64 bytes from 172.17.0.3: seq=0 ttl=64 time=0.072 ms
64 bytes from 172.17.0.3: seq=1 ttl=64 time=0.091 ms
64 bytes from 172.17.0.3: seq=2 ttl=64 time=0.089 ms
64 bytes from 172.17.0.3: seq=3 ttl=64 time=0.105 ms
64 bytes from 172.17.0.3: seq=4 ttl=64 time=0.106 ms
64 bytes from 172.17.0.3: seq=5 ttl=64 time=0.090 ms
64 bytes from 172.17.0.3: seq=6 ttl=64 time=0.124 ms
^C
--- 172.17.0.3 ping statistics ---
7 packets transmitted, 7 packets received, 0% packet loss
round-trip min/avg/max = 0.072/0.096/0.124 ms
/ #

```

Рис. 6.13

```

/ # ping mybox1
ping: bad address 'mybox1'
/ # exit

```

Рис. 6.14

```

root@dockernet:~# docker network create -d bridge mybridge
bc7c08ebc3167f64fc7a5db02b375a4a752816bed45635a62c3e98c56d104b02
root@dockernet:~# docker network ls

```

NETWORK ID	NAME	DRIVER	SCOPE
49f3f92725fb	bridge	bridge	local
81f26227fa16	host	host	local
bc7c08ebc316	mybridge	bridge	local
9de55f916f1d	none	null	local

```

root@dockernet:~# docker run -dit --name mybox3 --net mybridge busybox:latest
8a0ea4b7e096e97731bd4b052f83d2405c8fbc15fa759a237a665efb523daca0
root@dockernet:~# docker run -dit --name mybox4 --net mybridge busybox:latest
5fa326f02b03aa2dfb8a6ccbc98ce03ff8a325253de45c3410b7b881984f3256
root@dockernet:~# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5fa326f02b03	busybox:latest	"sh"	4 seconds ago	Up 3 seconds		mybox4
8a0ea4b7e096	busybox:latest	"sh"	18 seconds ago	Up 16 seconds		mybox3
74ba8e7f1e0f	busybox:latest	"sh"	21 minutes ago	Up 21 minutes		mybox1
a994fa431ff5	busybox:latest	"sh"	21 minutes ago	Up 21 minutes		mybox

```

root@dockernet:~# docker exec -it mybox3 sh
/ # ping mybox4
PING mybox4 (172.18.0.3): 56 data bytes
64 bytes from 172.18.0.3: seq=0 ttl=64 time=0.110 ms
64 bytes from 172.18.0.3: seq=1 ttl=64 time=0.094 ms
64 bytes from 172.18.0.3: seq=2 ttl=64 time=0.107 ms
64 bytes from 172.18.0.3: seq=3 ttl=64 time=0.097 ms
64 bytes from 172.18.0.3: seq=4 ttl=64 time=0.184 ms
64 bytes from 172.18.0.3: seq=5 ttl=64 time=0.097 ms
^C64 bytes from 172.18.0.3: seq=6 ttl=64 time=0.088 ms
64 bytes from 172.18.0.3: seq=7 ttl=64 time=0.092 ms
64 bytes from 172.18.0.3: seq=8 ttl=64 time=0.089 ms
64 bytes from 172.18.0.3: seq=9 ttl=64 time=0.096 ms
^C
--- mybox4 ping statistics ---
10 packets transmitted, 10 packets received, 0% packet loss
round-trip min/avg/max = 0.088/0.105/0.184 ms
/ #

```

Рис. 6.15

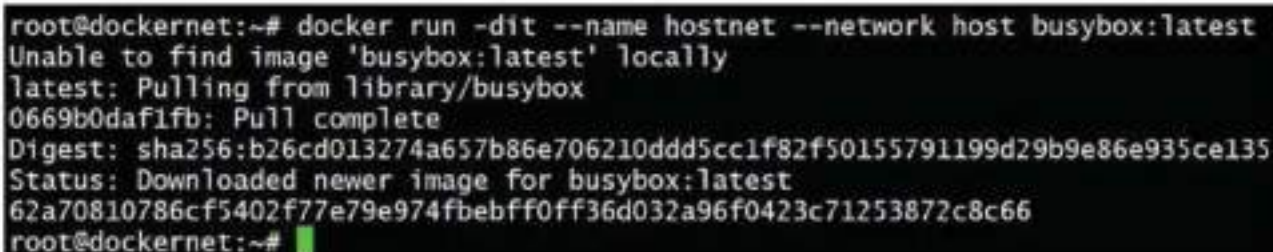
Сеть Host

Для создания контейнера также можно использовать сеть хоста (host). При этом контейнер создается без своего пространства имен сети и использует пространство имен сети хоста. Кроме того, контейнеру не назначается собственный IP-адрес. Зачем может понадобиться такая конфигурация? Обычно только в том случае, если требуется отладить или проанализировать трафик сети host.

У контейнера нет собственного сетевого стека, — он использует сетевой стек хоста, поэтому нет возможности сопоставить порты. Любые параметры для сопоставления портов будут игнорироваться, и мы получим предупреждение.

Запустить контейнер с использованием пространства имен хоста очень просто — достаточно подключить контейнер к сети хоста, как показано на рис. 6.16.

```
docker run -dit --name hostnet --network host busybox:latest
```



```
root@dockernet:~# docker run -dit --name hostnet --network host busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
0669b0daf1fb: Pull complete
Digest: sha256:b26cd013274a657b86e706210ddd5cc1f82f50155791199d29b9e86e935ce135
Status: Downloaded newer image for busybox:latest
62a70810786cf5402f77e79e974fbebff0ff36d032a96f0423c71253872c8c66
root@dockernet:~#
```

Рис. 6.16

Давайте узнаем IP-адрес хоста и сравним его с IP-адресом контейнера. IP-адреса выделены на рис. 6.17. Это ожидаемое поведение.

```
ifconfig -a eth0
docker exec -it hostnet sh
ip addr
```

Мы видим, что из контейнера доступен весь сетевой стек хоста, а IP-адреса совпадают. Как было сказано ранее, у контейнера нет собственного сетевого стека.

Сеть None

Если контейнеру не нужна связь с внешним миром, можно использовать сеть None. В этом случае контейнер будет полностью изолирован от внешнего мира, а выполняющееся в нем приложение будет существовать только в пределах контейнера. Давайте посмотрим, как это работает, запустив соответствующую команду и изучив рис. 6.18.

```
docker run -dit --name nonenet --network none busybox:latest
```

Как и ожидалось, у контейнера нет интерфейса для соединения с внешним миром.


```

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 178.128.203.125 netmask 255.255.240.0 broadcast 178.128.207.255
    inet6 fe80::2817:a5ff:fe86:84ca prefixlen 64 scopeid 0x20<link>
    inet6 2a03:b0c0:3:e0::3d:f001 prefixlen 64 scopeid 0x0<global>
    ether 2a:17:a5:86:84:ca txqueuelen 1000 (Ethernet)
    RX packets 6239 bytes 107197038 (107.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3771 bytes 478998 (478.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@dockernet:~# docker exec -it hostnet sh
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel qlen 1000
    link/ether 2a:17:a5:86:84:ca brd ff:ff:ff:ff:ff:ff
    inet 178.128.203.125/20 brd 178.128.207.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet 10.19.0.5/16 brd 10.19.255.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a03:b0c0:3:e0::3d:f001/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::2817:a5ff:fe86:84ca/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel qlen 1000
    link/ether 9e:82:84:99:db:34 brd ff:ff:ff:ff:ff:ff
    inet 10.135.7.32/16 brd 10.135.255.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::9c82:84ff:fe99:db34/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
    link/ether 02:42:a7:bd:a2:9f brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
/ #

```

Рис. 6.17

```

root@dockernet:~# docker run -dit --name nonenet --network none busybox:latest
d992914ed814f77859b97999ef2c6fefaf2cd6e5036582abf96c08fd221c61da
root@dockernet:~# docker exec -it nonenet sh
/ # ifconfig -a eth0
ifconfig: eth0: error fetching interface information: Device not found
/ #

```

Рис. 6.18

Использование пространства имен существующего контейнера

В Docker есть специальный механизм, который позволяет одному контейнеру использовать сетевой стек другого. У нового контейнера не будет собственного сетевого стека, и он будет использовать сетевой стек существующего контейнера. Новый контейнер будет иметь процессы и файловую систему, но IP-адрес и номера портов будут такими же, как у существующего контейнера, а друг с другом контейнеры будут взаимодействовать по интерфейсу обратной связи.

Сопоставление портов

Сопоставление портов чрезвычайно важно, т. к. это позволяет внешнему миру получать доступ к контейнеру через порт на хосте. Порт хоста должен сопоставляться с портом в контейнере, как на рис. 6.19.

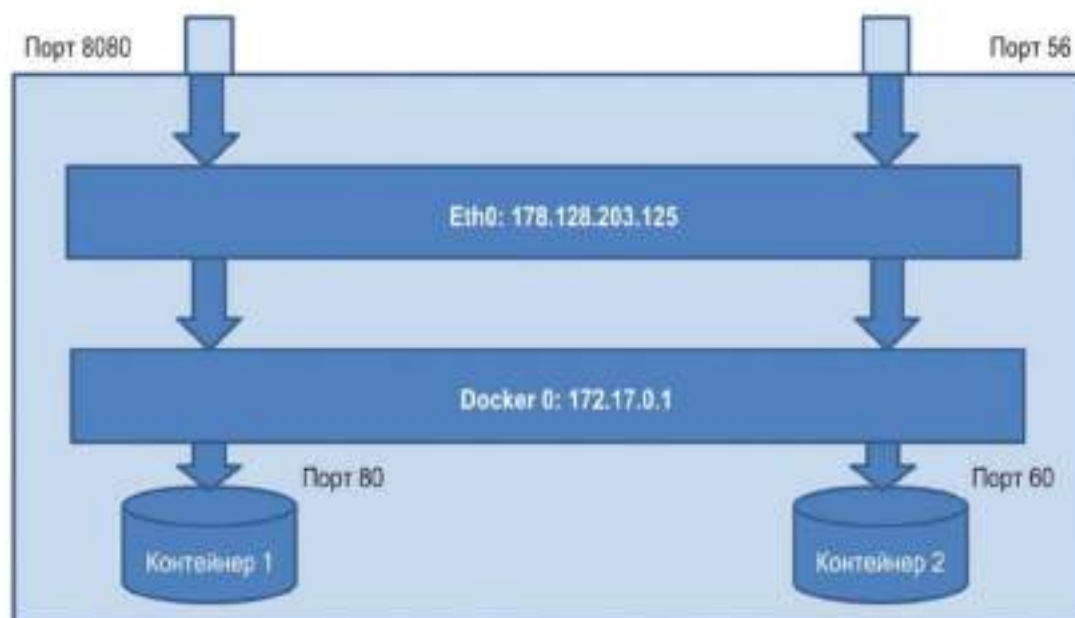


Рис. 6.19. Сопоставление портов

На схеме видно, что порт хоста 8080 сопоставляется с портом 80 контейнера, а порт хоста 56 сопоставляется с портом 60. Весь трафик извне будет проходить через порт на хосте к порту контейнера.

Но ничего из этого не требуется, если предполагается связь контейнера с внешним миром, потому что IP-адрес контейнера скрыт за IP-адресом хоста; получается, будто хост сам коммуницирует с другими приложениями. Для внешних соединений Docker использует технологию *NAT* (Network Address Translation — преобразование сетевых адресов) и фреймворк *netfilter* (межсетевой экран) Linux. Чтобы просмотреть действующие правила, используем инструмент командной строки *iptables*. Запустим команду `iptables -t nat -L` и изучим выходные данные на рис. 6.20.

```
iptables -t nat -L
```

Мы видим, что в цепочке (chain) *POSTROUTING* есть правило, которое маскирует, т. е. скрывает за интерфейсом хоста, все, что поступает с моста *docker0* с IP-адресом 172.17.0.0. Это позволяет контейнеру коммуницировать с внешним миром, как показано на рис. 6.21.

```
docker run -dit --name test_con busybox:latest
```

Итак, надеюсь, общий механизм вам понятен, а теперь посмотрим, какие флаги (параметры) используются для сопоставления портов с контейнерами. Команда ниже и выходные данные на рис. 6.22.

```
docker run -dit --name test1 -P nginx:latest
docker ps
```

```

root@dockernet:~# iptables -t nat -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source                destination              ADDRTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination              ADDRTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT)
target     prot opt source                destination              ADDRTYPE match dst-type LOCAL
MASQUERADE all  -- 172.17.0.0/16          anywhere

Chain DOCKER (2 references)
target     prot opt source                destination
RETURN     all  -- anywhere              anywhere
root@dockernet:~#

```

Рис. 6.20

```

root@dockernet:~# docker run -dit --name test_con busybox:latest
6f39296ea359ba92ff7d940ef415b57817dfb7754c4683d834139740d2b5532a
root@dockernet:~# docker exec -it test_con sh
/ # ping google.com
PING google.com (172.217.17.142): 56 data bytes
64 bytes from 172.217.17.142: seq=0 ttl=53 time=7.118 ms
64 bytes from 172.217.17.142: seq=1 ttl=53 time=6.713 ms
64 bytes from 172.217.17.142: seq=2 ttl=53 time=6.693 ms
64 bytes from 172.217.17.142: seq=3 ttl=53 time=6.717 ms
AC
--- google.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 6.693/6.810/7.118 ms
/ #

```

Рис. 6.21

```

root@dockernet:~# docker run -dit --name test1 -P nginx:latest
6398a045f7a7d0e1082eae15f4bcec488320cf8e9c423f8df2bfc47f5c1af253
root@dockernet:~# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
6398a045f7a7       nginx:latest       "nginx -g 'daemon of..." 4 seconds ago       Up 3 seconds       0.0.0.0:32778->80/tcp   test1
root@dockernet:~#

```

Рис. 6.22

Как видим, приложение *nginx* доступно через порт 80 в контейнере, и флагом *-P* этот порт был случайно сопоставлен с портом 32778 на хосте. Флаг *-P* случайным образом сопоставляет порт контейнера с портом на хосте. Давайте проверим, все ли работает, запустив следующую команду (выходные данные на рис. 6.23).

```
curl localhost:32778
```

Теперь сопоставим порт на хосте с портом в контейнере (рис. 6.24).

```
docker run -dit --name test2 -p 8080:80 nginx:latest
```

Порты в контейнерах можно легко сопоставить с доступными портами на хосте. Более того, можно указать протоколы. Например, порт 80 контейнера можно сопоставить с портом 8081 для протоколов TCP и UDP. Рассмотрим пример (рис. 6.25).

```
docker run -dit --name test3 -p 8081:80/tcp -p 8081:80/udp nginx:latest
```



```

root@dockernet:~# curl localhost:32778
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@dockernet:~# █

```

Рис. 6.23

```

root@dockernet:~# docker run -dit --name test2 -p 8080:80 nginx:latest
6cb70bec80c0aeb3bd27128f5a99db4e5f39164e4c078c7a10b2d8bb66d7be29
root@dockernet:~# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
6cb70bec80c0	nginx:latest	"nginx -g 'daemon of..."	5 seconds ago	Up 4 seconds	0.0.0.0:8080->80/tcp	test2
6398a045f7a7	nginx:latest	"nginx -g 'daemon of..."	18 minutes ago	Up 18 minutes	0.0.0.0:32778->80/tcp	test1

```

root@dockernet:~# █

```

Рис. 6.24

```

root@dockernet:~# docker run -dit --name test3 -p 8081:80/tcp -p 8081:80/udp nginx:latest
ed11c774132feacc1a68abf44ba0d8debf2bc74f20f701009b99416527951f49
root@dockernet:~# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ed11c774132f	nginx:latest	"nginx -g 'daemon of..."	26 seconds ago	Up 25 seconds	0.0.0.0:8081->80/tcp, 0.0.0.0:8081->80/udp	test3

```

root@dockernet:~# █

```

Рис. 6.25

Сеть MACVLAN

Иногда необходимо подключить приложение к физической сети. Для этого можно использовать сеть MACVLAN.

Интерфейс MACVLAN создается путем присоединения к физическому интерфейсу и имеет собственный MAC- и IP-адрес.

Чтобы лучше понять суть, внимательно прочтите следующие важные замечания о MACVLAN.

1. Интерфейс MACVLAN необходимо сопоставить с родительским интерфейсом. Для доступа к внешним сетям он будет использовать шлюз хоста.

2. Драйвер MACVLAN работает только на хосте Linux и не поддерживается в Docker Desktop для Mac и Windows или Docker Engine Enterprise (EE) для Windows Server.
3. Для использования MACVLAN требуется версия ядра Linux не ниже 3.9, а лучше 4.0 и выше.
4. Интерфейс MACVLAN должен быть назначен той же подсети, что и родительский интерфейс.
5. Сетевая карта (NIC) хоста должна находиться в режиме *promiscuous* (неразборчивый).
6. Для каждого родительского интерфейса в одной подсети можно определить только одну сеть типа MACVLAN.
7. Можно указать диапазон IP-адресов для контейнера, а также вспомогательный адрес, который система IPAM в Docker не должна назначать контейнерам.

Сеть MACVLAN может работать в режимах *bridge* и *trunk bridge* по стандарту 802.1Q. Разница в том, что в режиме *bridge* трафик проходит через физический интерфейс на хосте, а в режиме *trunk bridge* 802.1Q Docker создает подынтерфейс, который является частью родительского. Единственное преимущество последнего варианта заключается в том, что мы можем более детализированно контролировать процессы и параметры сети.

MACVLAN в режиме *bridge*

Давайте создадим сеть MACVLAN, используя родительский интерфейс `eth0`. Прежде всего узнаем IP-адрес и подсеть, связанные с интерфейсом `eth0`. Для этого используем команду `ifconfig -a eth0`, а затем изучим выходные данные на рис. 6.26.

```
root@dockernet:~# ifconfig -a eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 159.89.94.163 netmask 255.255.240.0 broadcast 159.89.95.255
    inet6 fe80::e037:c5ff:fe75:70af prefixlen 64 scopeid 0x20<link>
    ether e2:37:c5:75:70:af txqueuelen 1000 (Ethernet)
    RX packets 15513 bytes 109165518 (109.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6839 bytes 875207 (875.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Рис. 6.26

Из выходных данных на рис. 6.26 мы получили следующую информацию:

Адрес: 159.89.94.163

Маска подсети: 255.255.240.0 = 20

Сеть: 159.89.80.0/20

Широковещательный канал (broadcast): 159.89.95.255

HostMin: 159.89.80.1¹

HostMax: 159.89.95.254

Эти значения можно легко рассчитать, воспользовавшись калькулятором подсетей в Интернете, например <http://www.subnet-calculator.com>.

Далее мы создадим сеть MACVLAN, используя интерфейс eth0 и соответствующую подсеть. См. команды ниже и рис. 6.27 и 6.28.

```
docker network create --driver=macvlan --subnet=159.89.94.163/20 \
--gateway=159.89.80.1 --ip-range=158.89.80.2/24 \
-o parent=eth0 macvnet
```

```
docker network ls
```

В списке сетей на рис. 6.28 мы видим нашу сеть, созданную надлежащим образом.

```
root@dockernet:~# docker network create --driver=macvlan --subnet=159.89.94.163/20 \
> --gateway=159.89.80.1 --ip-range=159.89.80.2/24 \
> -o parent=eth0 macvnet
bbb3078e8e0739f693ef9c96b297835248d2a23c109e4587b44a72e1ccfd3113
root@dockernet:~#
```

Рис. 6.27

```
root@dockernet:~# docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
aa4f36c99123	bridge	bridge	local
f0b945b36ddc	host	host	local
bbb3078e8e07	macvnet	macvlan	local
5106289a3987	none	null	local

```
root@dockernet:~#
```

Рис. 6.28

Прежде чем двигаться дальше, остановимся на шагах, необходимых для создания сети.

1. Создаем сеть с помощью драйвера MACVLAN.
2. Предоставляем информацию о подсети, сопоставляя ее с интерфейсом eth0.
3. Указываем сведения о шлюзе.
4. Указываем диапазон IP-адресов, которые можно назначать контейнерам в этой сети. Наш диапазон — 159.89.80.1-254. Если не поставить этот флаг, Docker выберет произвольный IP-адрес в подсети интерфейса eth0.
5. Наконец подключаем интерфейс eth0 как родительский.

Давайте немного углубимся в детали и изучим информацию на рис. 6.29.

```
docker inspect macvnet
```

Как и ожидалось, мы видим драйвер MACVLAN, а также подсеть, диапазон IP-адресов и шлюз.

¹ Может использоваться как шлюз.

```

root@dockernet:~# docker inspect macvnet
[
  {
    "Name": "macvnet",
    "Id": "bbb3078e8e0739f693ef9c96b297835248d2a23c109e4587b44a72e1ccfd3113",
    "Created": "2020-04-14T06:26:09.238485086Z",
    "Scope": "local",
    "Driver": "macvlan",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "159.89.94.163/20",
          "IPRange": "159.89.80.2/24",
          "Gateway": "159.89.80.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {
      "parent": "eth0"
    },
    "Labels": {}
  }
]
root@dockernet:~#

```

Рис. 6.29

Пока все идет отлично. Теперь создадим контейнер и подключим его к нашей сети MACVLAN. См. команду ниже и рис. 6.30.

```

docker run -dit --name mymac --network macvnet busybox:latest
docker exec -it mymac sh
ip addr
ip route
exit

```

Здесь довольно много информации. Например, мы видим имя интерфейса, т. е. `eth0@if2`, а также видим, что Docker назначил первый доступный IP-адрес в указанном нами диапазоне (159.89.80.1 — IP шлюза). В разделе о маршруте указано, что мы используем сеть `eth0`.

Давайте заглянем в контейнер чуть глубже. Отправим `ping` на IP-адрес контейнера и на интерфейс `eth0` и посмотрим, что получилось (рис. 6.31).

```

ping 159.89.80.2
ping 159.89.94.163

```

Как видите, мы можем пинговать IP-адрес контейнера, но не хоста, — это поведение по умолчанию. Более того, мы не можем пинговать контейнер с хоста, хотя у них один интерфейс. Это видно на рис. 6.32.


```

root@dockernet:~# docker run -dit --name mymac --network macvnet busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
0669b0daf1fb: Pull complete
Digest: sha256:b26cd013274a657b86e706210ddd5cc1f82f50155791199d29b9e86e935ce135
Status: Downloaded newer image for busybox:latest
56ed860a4dd6d76946ba05dcc830ac74b0509286ee735885025f28b68a0f9bf1
root@dockernet:~# docker exec -it mymac sh
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
5: eth0@if2: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:9f:59:50:02 brd ff:ff:ff:ff:ff:ff
    inet 159.89.80.2/20 brd 159.89.95.255 scope global eth0
        valid_lft forever preferred_lft forever
/ # ip route
default via 159.89.80.1 dev eth0
159.89.80.0/20 dev eth0 scope link src 159.89.80.2
/ #

```

Рис. 6.30

```

    valid_lft forever preferred_lft forever
6: eth0@if2: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:9f:59:50:02 brd ff:ff:ff:ff:ff:ff
    inet 159.89.80.2/20 brd 159.89.95.255 scope global eth0
        valid_lft forever preferred_lft forever
/ # ping 159.89.80.2
PING 159.89.80.2 (159.89.80.2): 56 data bytes
64 bytes from 159.89.80.2: seq=0 ttl=64 time=0.062 ms
64 bytes from 159.89.80.2: seq=1 ttl=64 time=0.065 ms
64 bytes from 159.89.80.2: seq=2 ttl=64 time=0.080 ms
^C
--- 159.89.80.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.062/0.069/0.080 ms
/ # ping 159.89.94.163
PING 159.89.94.163 (159.89.94.163): 56 data bytes
^C
--- 159.89.94.163 ping statistics ---
3 packets transmitted, 0 packets received, 100% packet loss
/ #

```

Рис. 6.31

```

root@dockernet:~# ping 159.89.80.2
PING 159.89.80.2 (159.89.80.2) 56(84) bytes of data.
^C
--- 159.89.80.2 ping statistics ---
7 packets transmitted, 0 received, 100% packet loss, time 6149ms
root@dockernet:~#

```

Рис. 6.32

Итак, что мы имеем? У контейнера `mytac` есть свой MAC-адрес, IP-адрес и пространство имен. Контейнер подключен к интерфейсу `eth0`, который, в свою очередь, подключен к внешней сети, как показано на рис. 6.33.

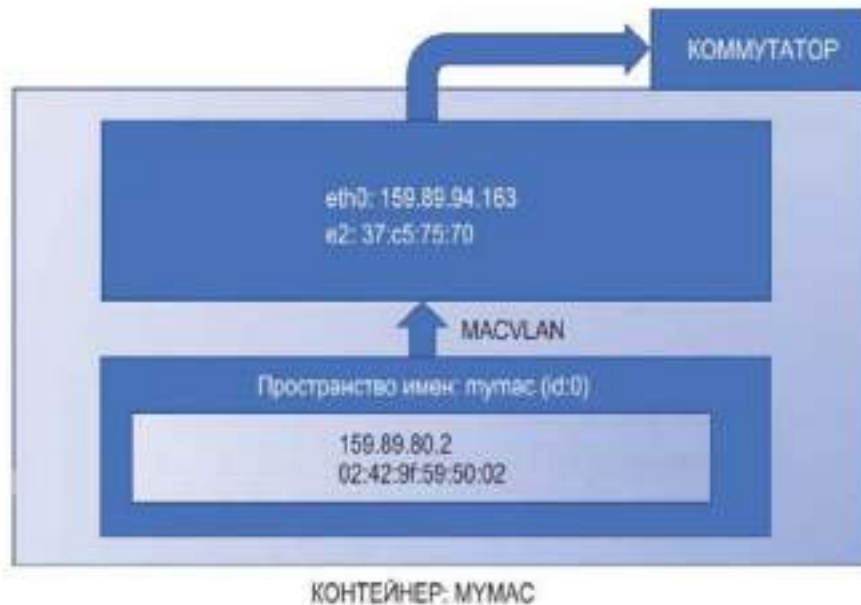


Рис. 6.33. Сеть MACVLAN

Прежде чем мы попробуем наладить связь между контейнером и хостом, давайте создадим еще один контейнер в той же сети MACVLAN и посмотрим, смогут ли контейнеры взаимодействовать друг с другом. См. команды ниже и рис. 6.34.

```
docker run -dit --name mymac --network macvnet busybox:latest
docker exec -it mymac sh
ip addr
ping 159.89.80.2
```

```
root@dockernet:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
e30720b23589   busybox:latest "sh"                    About an hour ago Up About an hour      mymac
root@dockernet:~# docker run -dit --name mymac1 --network macvnet busybox:latest
025addle0e0f61b638711f423eea47d54cafd4962e9499a28f666e2e9710e4d7
root@dockernet:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
025addle0e0f   busybox:latest "sh"                    5 seconds ago Up 3 seconds          mymac1
e30720b23589   busybox:latest "sh"                    About an hour ago Up About an hour      mymac
root@dockernet:~# docker exec -it mymac1 sh
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
7: eth0@if2: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:9f:59:50:03 brd ff:ff:ff:ff:ff:ff
    inet 159.89.80.3/20 brd 159.89.95.255 scope global eth0
        valid_lft forever preferred_lft forever
/ # ping 159.89.80.2
PING 159.89.80.2 (159.89.80.2): 56 data bytes
64 bytes from 159.89.80.2: seq=0 ttl=64 time=0.104 ms
64 bytes from 159.89.80.2: seq=1 ttl=64 time=0.082 ms
64 bytes from 159.89.80.2: seq=2 ttl=64 time=0.076 ms
^C
--- 159.89.80.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.076/0.087/0.104 ms
/ #
```

Рис. 6.34

Контейнеры могут обмениваться данными, потому что подключены к одной сети MACVLAN, которая находится в режиме bridge (мост), по аналогии с сетевым мостом Linux.

Теперь нас интересует, как успешно пинговать к контейнеру с хоста. Запустим несколько команд и посмотрим, что получится (рис. 6.35).

```
ip link add macnet0 link eth0 type macvlan mode bridge
ip addr add 159.89.80.2/20 dev macnet0
ip link set macnet0 up
```

```
root@dockernet:~# ip link add macnet0 link eth0 type macvlan mode bridge
root@dockernet:~# ip addr add 159.89.80.2/20 dev macnet0
root@dockernet:~# ip link set macnet0 up
root@dockernet:~# ip addr | grep mac
9: macnet0@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   inet 159.89.80.2/20 scope global macnet0
root@dockernet:~#
```

Рис. 6.35

С помощью вышеуказанных команд мы создали подключение с интерфейса eth0 к контейнеру, добавив IP-адрес контейнера. Другими словами, мы просто установили одностороннее подключение с хоста к контейнеру. Теперь попробуем отправить ping к контейнеру с хоста и убедимся, что все работает, взглянув на рис. 6.36.

```
root@dockernet:~# ping -c 5 159.89.80.2
PING 159.89.80.2 (159.89.80.2) 56(84) bytes of data:
64 bytes from 159.89.80.2: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 159.89.80.2: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from 159.89.80.2: icmp_seq=3 ttl=64 time=0.059 ms
64 bytes from 159.89.80.2: icmp_seq=4 ttl=64 time=0.048 ms
64 bytes from 159.89.80.2: icmp_seq=5 ttl=64 time=0.049 ms

--- 159.89.80.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4080ms
rtt min/avg/max/mdev = 0.034/0.049/0.059/0.012 ms
```

Рис. 6.36

MACVLAN в режиме *Trunk Bridge 802.1Q*

Создадим сеть с режимом trunk bridge по стандарту 802.1Q. На этот раз в качестве родительского интерфейса используем eth1 (см. рис. 6.37 и команды ниже).

```
docker network create --driver=macvlan --subnet=10.136.11.242/16 \
--gateway=10.136.0.1 --ip-range=10.136.0.2/24 \
-o parent=eth1.50 macvnet1
docker network ls
```

Адрес: 10.136.11.242

Маска подсети: 255.255.0.0 = 16

Сеть: 10.136.0.0/16

Широковещательный канал (broadcast): 10.136.255.255

HostMin: 10.136.0.1¹

HostMax: 10.136.255.254

```
root@dockernet:~# docker network create --driver=macvlan --subnet=10.136.11.242/16 \
> --gateway=10.136.0.1 --ip-range=10.136.0.2/24 \
> -o parent=eth1.50 macvnet1
849cb2a19aa3c22362a43fce52cb72130ad4a1434e029e8dc9a20281f9f4702f
root@dockernet:~# docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
aa4f36c99123	bridge	bridge	local
f0b945b36ddc	host	host	local
bbb3078e8e07	macvnet	macvlan	local
849cb2a19aa3	macvnet1	macvlan	local
5106289a3987	none	null	local

```
root@dockernet:~#
```

Рис. 6.37

Следующий логический шаг: проверить конфигурацию сети. Используем для этого команду `docker network inspect macvnet1` (см. рис. 6.38).

```
root@dockernet:~# docker network inspect macvnet1
[
  {
    "Name": "macvnet1",
    "Id": "849cb2a19aa3c22362a43fce52cb72130ad4a1434e029e8dc9a20281f9f4702f",
    "Created": "2020-04-14T12:00:24.104149274Z",
    "Scope": "local",
    "Driver": "macvlan",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "10.136.11.242/16",
          "IPRange": "10.136.0.2/24",
          "Gateway": "10.136.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {
      "parent": "eth1.50"
    },
    "Labels": {}
  }
]
```

Рис. 6.38

¹ Шлюз.

Мы видим на рис. 6.38, что имя родителя — подынтерфейс `eth1.50`. В остальном этот режим ничем не отличается от обычной сети `MACVLAN bridge`.

Сеть Overlay

Сетевой драйвер `overlay` соединяет несколько контейнеров по сети хоста. С помощью этого механизма можно построить изолированную сеть `overlay`, или оверлейную сеть, поверх существующей сети на хосте.

При установке Docker оверлейная сеть не создается по умолчанию. Для этого нужно инициализировать Swarm или присоединить узел к Docker Swarm. Создается не только оверлейная сеть с именем `ingress`, но и другая сеть — `docker_gwbridge`.

К оверлейной сети можно подключать отдельные контейнеры и сервисы Docker Swarm (о которых мы говорили в главе 5). Для подключения отдельных контейнеров нужно создать сеть с флагом `attachable`.

Перейдем к практике. Сначала создадим конфигурацию Docker Swarm с одним управляющим узлом и парой рабочих, после чего проверим конфигурацию сети на рис. 6.39.

```
root@docker-mgr:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d0b5012f297c        bridge              bridge              local
87af0abca567        docker_gwbridge     bridge              local
5c31ff9241d2        host                host                local
azoczw17t26u        ingress             overlay             swarm
d060f983a497        none                null                local
root@docker-mgr:~#
```

Рис. 6.39

Как и ожидалось, появились две новые сети: `-docker_gwbridge` типа `bridge` и `ingress` — типа `overlay`.

Если выполнить команду с рабочего узла, получится то же самое (рис. 6.40).

```
root@docker-wkr:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
7a22f7aa4bc5        bridge              bridge              local
c7a38a705c35        docker_gwbridge     bridge              local
de628e32b1fe        host                host                local
azoczw17t26u        ingress             overlay             swarm
3e6575d935a3        none                null                local
root@docker-wkr:~#
```

Рис. 6.40

Теперь давайте создадим контейнер и посмотрим, что будет, если мы попробуем подключить его к сети `ingress` (команда ниже и рис. 6.41).

```
docker run -dit --name test_container --network ingress busybox:latest
```

```
root@docker-manager:~# docker run -dit --name test_container --network ingress busybox:latest
021b49e1c747302c0db492c85c9eb30a4ad810951434716c1c30f1fbb3a14c0
docker: Error response from daemon: Could not attach to network ingress: rpc error: code = PermissionDenied desc = network ingress
not manually attachable.
root@docker-manager:~#
```

Рис. 6.41

Мы не можем создать контейнер и подключить его к сети `ingress`, потому что оверлейная сеть, по умолчанию ограниченная Swarm, не может использоваться для команд `docker run`. А что насчет команд `docker service`? Посмотрим на рис. 6.42.

```
root@docker-manager:~# docker service create --name myserv --network ingress redis:latest
Error response from daemon: rpc error: code = InvalidArgument desc = Service cannot be explicitly attached to the ingress network "ingress"
root@docker-manager:~#
```

Рис. 6.42

Привязать контейнер к пользовательской сети Swarm можно при условии, что мы создали подключаемую сеть (`attachable`).

Если углубиться в детали, мы увидим, что сеть `ingress` предоставляется только для Swarm (см. рис. 6.43).

```
docker inspect ingress
```

```
root@docker-manager:~# docker inspect ingress
[
  {
    "Name": "ingress",
    "Id": "rljaikz48gr7y88kfh2usuz4b",
    "Created": "2020-04-18T06:21:11.084295034Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.0.0/24",
          "Gateway": "10.0.0.1"
        }
      ]
    }
  }
]
```

Рис. 6.43

Что касается пространства имен оверлейной сети, то, как мы уже говорили, в Docker пространство имен сети находится не в том же каталоге, что стандартное пространство имен сети Linux, следовательно, мы не сможем использовать команды Linux вроде `ip netnslist` для просмотра пространств имен сети Docker. Поскольку чуть раньше мы создали символическую ссылку для указания пути, где хранится пространство имен сети Docker, мы сможем воспользоваться командой `ip netnslist` для просмотра пространств имен.

Давайте проверим пространства имен сети в Docker. См. команду ниже и рис. 6.44.

```
docker network ls
ls -ln /var/run/docker/netns
```

```
root@docker-mgr:~# docker network ls
NETWORK ID          NAME                DRIVER             SCOPE
d0b5012f297c        bridge             bridge             local
87af0abca567        docker_gwbridge    bridge             local
5c31ff9241d2        host               host               local
azoczwi7t26u        ingress            overlay            swarm
d060f983a497        none               null               local
root@docker-mgr:~# ls -ltr /var/run/docker/netns
total 0
-r--r--r-- 1 root root 0 Apr 24 09:49 ingress_sbox
-r--r--r-- 1 root root 0 Apr 24 09:49 1-azoczwi7t2
root@docker-mgr:~#
```

Рис. 6.44

Мы можем сопоставить имя пространства имен с id пространства имен оверлейной сети и увидим, что они похожи. Также мы видим здесь нечто с именем `ingress_sbox`. Это скрытый контейнер, один конец которого подключен к сети хоста, а другой — к оверлейной сети. Его основная функция связана с обнаружением сервисов и балансировкой нагрузки.

Давайте определим оверлейную сеть и используем флаг `attachable`, чтобы можно было запустить контейнер и подключить его к оверлейной сети.

```
docker network create --driver overlay --attachable my-overlay
```

Теперь проверим новую сеть и пространство имен, которое было создано после сети `my-overlay` (рис. 6.45).

```
docker network ls
```

```
root@docker-mgr:~# docker network ls
NETWORK ID          NAME                DRIVER             SCOPE
d0b5012f297c        bridge             bridge             local
87af0abca567        docker_gwbridge    bridge             local
5c31ff9241d2        host               host               local
azoczwi7t26u        ingress            overlay            swarm
bkbxlrjx95r9        my-overlay         overlay            swarm
d060f983a497        none               null               local
root@docker-mgr:~#
```

Рис. 6.45

Теперь проверим на новое пространство имен на рис. 6.46.

```
ls -ltr /var/run/docker/netns
```

```
root@docker-mgr:~# ls -ltr /var/run/docker/netns
total 0
-r--r--r-- 1 root root 0 Apr 24 09:49 ingress_sbox
-r--r--r-- 1 root root 0 Apr 24 09:49 1-azoczwi7t2
```

Рис. 6.46

Как ни странно, мы не видим новое пространство имен для созданной сети `my-overlay`. Это поведение по умолчанию — пространство имен создается только при создании контейнера в данной сети. Рассмотрим пример на рис. 6.47.

```
docker run -dit --name test_container --network my-overlay busybox:latest
ls -ltr /var/run/docker/netns
docker network ls
```

```
root@docker-mgr:~# docker run -dit --name test_container --network my-overlay busybox:latest
bccb4311355327bdef223b86bb1d3fe37d538dd8ea8152ad2872cc189d264f68
root@docker-mgr:~# ls -ltr /var/run/docker/netns
total 0
-r--r--r-- 1 root root 0 Apr 24 09:49 ingress_sbox
-r--r--r-- 1 root root 0 Apr 24 09:49 1-azoczwi7t2
-r--r--r-- 1 root root 0 Apr 25 07:30 1b-bkblx9
-r--r--r-- 1 root root 0 Apr 25 07:30 1-bkblx95
-r--r--r-- 1 root root 0 Apr 25 07:30 6ba280aebf58
root@docker-mgr:~# docker network ls


| NETWORK ID   | NAME            | DRIVER  | SCOPE |
|--------------|-----------------|---------|-------|
| d0b5012f297c | bridge          | bridge  | local |
| 87af0abca567 | docker_gwbridge | bridge  | local |
| 5c31ff9241d2 | host            | host    | local |
| azoczwi7t26u | ingress         | overlay | swarm |
| bkblx95r9    | my-overlay      | overlay | swarm |
| d060f983a497 | none            | null    | local |


root@docker-mgr:~#
```

Рис. 6.47

Итак, мы видим новое пространство имен, которое было создано после контейнера и подключения его к новой оверлейной сети `my-overlay`. Давайте заглянем в этот контейнер. Выходные данные показаны на рис. 6.48.

```
docker exec -it test_container sh
```

```
root@docker-mgr:~# docker exec -it test_container sh
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
24: eth0@if25: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue
    link/ether 02:42:0a:00:03:02 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.2/24 brd 10.0.3.255 scope global eth0
        valid_lft forever preferred_lft forever
26: eth1@if27: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
/ #
```

Рис. 6.48

Обратите внимание: к контейнеру подключены два интерфейса: `eth0@if25` и `eth1@if27`.

Интерфейс 27 находится на хосте, как показано на рис. 6.49. Однако интерфейс 25 мы нигде не видим. Но не беспокойтесь — мы изучим этот момент чуть позже.

Зато мы видим соединение `veth` между контейнером и `docker_gwbridge`, который работает так же, как мост Linux.

```

valid_lft forever preferred_lft forever
27 veth645f9038:f26: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker_gwbridge state UP group default
link/ether fa:ae:cd:8f:cc:a5 brd ff:ff:ff:ff:ff:ff link-netnsid 4
inet6 fe80::f8ae:cdff:fe8f:cca5/64 scope link
    valid_lft forever preferred_lft forever
root@docker-mgr:~#

```

Рис. 6.49

Прежде чем двигаться дальше, узнаем больше о `docker_gwbridge`.

Если мы сопоставим *интерфейс 27* (`eth1@if27`) внутри контейнера, то обнаружим, что он подключен к мосту с именем `docker_gwbridge`. Запустим следующую команду, чтобы исследовать сеть `docker_gwbridge`, и рассмотрим выходные данные ниже.

```
docker inspect docker_gwbridge
```

```

root@docker-mgr:~# docker inspect docker_gwbridge
[
  {
    "Name": "docker_gwbridge",

    "Id": "87af0abca56707273fef6eecd10ba4e38a3a43a75f815f671d6937ca-73b196a1",
    "Created": "2020-04-24T09:49:43.034481166Z",
    "Scope": "local",

    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "bccb4311355327bdef223b86bbld3fe37d538dd8ea8152ad2872c-c189d264f68": {
        "Name": "gateway_6ba280aebfb8",
        "EndpointID": "a31e09d34cc43afc82c494f9553c2e37327058cb-4f7bfdeec888ae0c3eab7ae6",
        "MacAddress": "02:42:ac:12:00:03",

```



```

    "IPv4Address": "172.18.0.3/16",
    "IPv6Address": ""
  },
  "ingress-sbox": {
    "Name": "gateway_ingress-sbox",
    "EndpointID": "436d9ace432c4be9e8ed8f6e73ecf40c-4194c160081013c7def6a49162be3052",
    "MacAddress": "02:42:ac:12:00:02",
    "IPv4Address": "172.18.0.2/16",
    "IPv6Address": ""
  }
},
"Options": {
  "com.docker.network.bridge.enable_icc": "false",
  "com.docker.network.bridge.enable_ip_masquerade": "true",
  "com.docker.network.bridge.name": "docker_gwbridge"
},
"Labels": {}
}
]

```

При ближайшем рассмотрении можно увидеть, что у сети `docker_gwbridge` имеется некоторая информация о нашем контейнере (IP-адрес, MAC-адрес и т. д.), а для `com.docker.network.bridge.enable_icc` установлено значение `false`.

Этот параметр не позволяет контейнерам на одном мосте взаимодействовать друг с другом. Дело в том, что, поскольку контейнеры уже подключены к оверлейной сети, они используют ее для обмена данными друг с другом.

В таком случае зачем нужна `docker_gwbridge`?

Сеть `docker_gwbridge` имеет особое назначение. Контейнеры в оверлейной сети могут прекрасно взаимодействовать друг с другом, но у них нет никакой связи с внешним миром. Для этого и нужна сеть `docker_gwbridge`. На рис. 6.50 показана архитектура сети.

Два контейнера общаются между собой по оверлейной сети, как показано на рис. 6.50. Каждый контейнер имеет два интерфейса: один для оверлейной сети, другой — для подключения к `docker_gwbridge`. Оба контейнера подключены к `docker_gwbridge`, но каждый использует *свой отрезок* сети `docker_gwbridge`, и единственная цель `docker_gwbridge` — обеспечить взаимодействие контейнеров с внешним миром. Если внимательно изучить выходные данные исследования сети `docker_gwbridge` (см. выше), можно увидеть, что для `com.docker.network.bridge.enable_ip_masquerade` установлено значение `true`; это значит, что IP-адреса могут коммуницировать с внешним миром, «маскируясь» под хост, на котором находится контейнер.

Теперь разберемся в том, как функционирует оверлейная сеть.

Прежде всего проверим, может ли контейнер в хост-системе 1 (управляющий узел Swarm) взаимодействовать с контейнером в хост-системе 2 (рабочий узел) и наоборот.

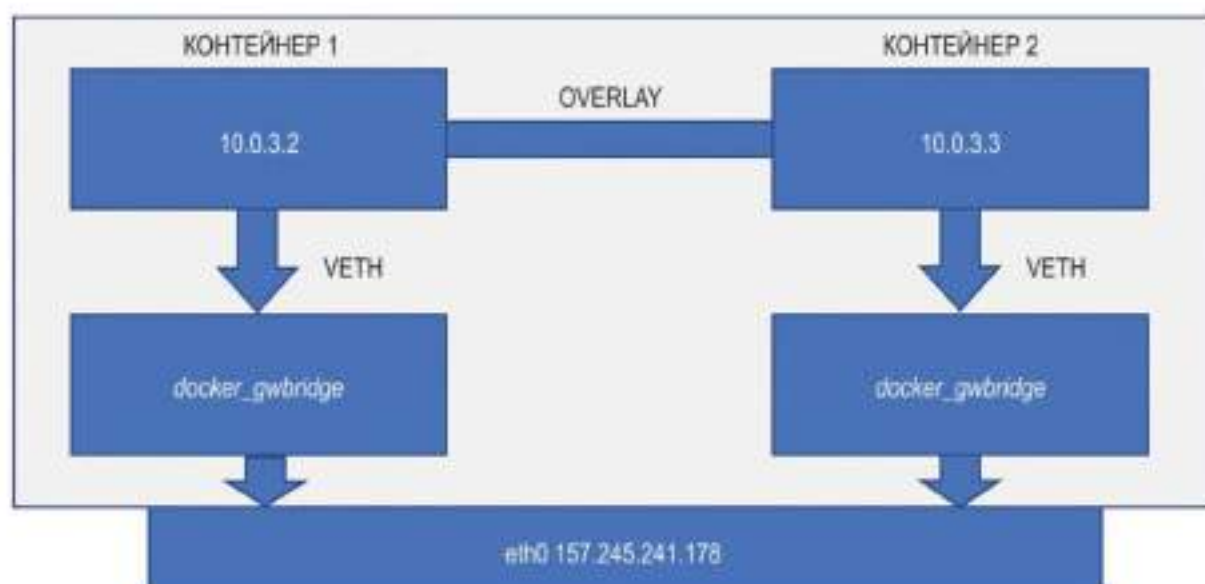


Рис. 6.50. Архитектура оверлейной сети

На рис. 6.51 представлены сведения о хост-системе 1.

На рис. 6.52 — сведения о хост-системе 2.

```

root@docker-ngr:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
bccb43113553   busybox:latest "sh"                    28 minutes ago Up 28 minutes      test_container

root@docker-ngr:~# docker exec -it bccb43113553 sh
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
24: eth0@if25: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue
    link/ether 02:42:0a:00:03:02 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.2/24 brd 10.0.3.255 scope global eth0
        valid_lft forever preferred_lft forever
26: eth1@if27: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
/ #
  
```

Рис. 6.51

```

root@docker-wkr:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
d8fde347877d   centos:latest  "/bin/bash"             7 seconds ago Up 4 seconds      test_container1

root@docker-wkr:~# docker exec -it test_container1 sh
sh-4.4# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:03:04 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.3.4/24 brd 10.0.3.255 scope global eth0
        valid_lft forever preferred_lft forever
28: eth1@if29: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
sh-4.4#
  
```

Рис. 6.52

Попробуем отправить ping с хост-системы 1 на хост-систему 2. IP-адрес контейнера в хост-системе 1 — 10.0.3.2, а контейнера в хост-системе 2 — 10.0.3.4. Изучим выходные данные на рис. 6.53 и 6.54. Соответствующие команды также приведены ниже.

```
docker exec -it test_container sh
ping 10.0.3.4
```

```
root@docker-mgr:~# docker exec -it test_container sh
/ # ping 10.0.3.4
PING 10.0.3.4 (10.0.3.4): 56 data bytes
64 bytes from 10.0.3.4: seq=0 ttl=64 time=1.888 ms
64 bytes from 10.0.3.4: seq=1 ttl=64 time=0.695 ms
64 bytes from 10.0.3.4: seq=2 ttl=64 time=0.672 ms
^C
--- 10.0.3.4 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.672/1.085/1.888 ms
/ #
```

Рис. 6.53

```
docker exec -it test_container1
sh ping 10.0.3.2
```

```
root@docker-wkr:~# docker exec -it test_container1 sh
sh-4.4# ping 10.0.3.2
PING 10.0.3.2 (10.0.3.2) 56(84) bytes of data:
64 bytes from 10.0.3.2: icmp_seq=1 ttl=64 time=1.81 ms
64 bytes from 10.0.3.2: icmp_seq=2 ttl=64 time=0.636 ms
64 bytes from 10.0.3.2: icmp_seq=3 ttl=64 time=0.625 ms
^C
--- 10.0.3.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 16ms
rtt min/avg/max/mdev = 0.625/1.024/1.812/0.557 ms
sh-4.4#
```

Рис. 6.54

Ping свободно проходит между контейнерами в обе стороны. Отлично! Но как все это работает? Как контейнеры взаимодействуют по оверлейной сети?

Чтобы понять, давайте вспомним, что нам уже известно.

Внутри контейнера мы видели два интерфейса помимо loorback. Рассмотрим для примера хост-систему 1.

1. eth0 с IP-адресом 10.0.3.2/24, подключенный к оверлейной сети.
2. eth1 с IP-адресом 172.18.0.3/16, подключенный к docker_gwbridge.

Как мы уже знаем, один конец пары veth в eth1 подключен к контейнеру, а другой — к docker_gwbridge.

Рассмотрим интерфейс eth0. Один конец пары veth находится в контейнере, как видно на рис. 6.55.


```

root@docker-mgr:~# docker exec -it test_container sh
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
24: eth0@if25: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue
    link/ether 02:42:0a:00:03:02 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.2/24 brd 10.0.3.255 scope global eth0
        valid_lft forever preferred_lft forever
26: eth1@if27: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
/ #

```

Рис. 6.55

Но где другой конец? Ищем интерфейс 25, заглянем в хост-систему (рис. 6.56). Там мы его тоже не находим. Где же он?

```

root@docker-mgr:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d6:36:18:79:8b:11 brd ff:ff:ff:ff:ff:ff
    inet 167.71.113.156/20 brd 167.71.127.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet 10.46.0.6/16 brd 10.46.255.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::d436:18ff:fe79:8b11/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 2e:9b:57:c6:61:92 brd ff:ff:ff:ff:ff:ff
    inet 10.138.218.179/16 brd 10.138.255.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::2c9b:57ff:fec6:6192/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:5f:a5:6d:e3 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:5fff:fea5:6de3/64 scope link
        valid_lft forever preferred_lft forever
9: docker_gwbridge: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:62:07:e9:62 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global docker_gwbridge
        valid_lft forever preferred_lft forever
    inet6 fe80::42:62ff:fe07:e962/64 scope link
        valid_lft forever preferred_lft forever
11: vethdd9ff2@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker_gwbridge state UP group default
    link/ether 22:87:98:22:16:b1 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::2087:98ff:fe22:16b1/64 scope link
        valid_lft forever preferred_lft forever
27: veth645f903@if26: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker_gwbridge state UP group default
    link/ether fa:ae:cd:8f:cc:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 4
    inet6 fe80::f8ae:cdff:fe8f:cca6/64 scope link
        valid_lft forever preferred_lft forever
root@docker-mgr:~#

```

Рис. 6.56

Другой конец пары veth находится в пространстве имен оверлейной сети, а не в хост-системе или где-то еще. Ранее мы исследовали пространство имен сети, посмотрим еще раз (рис. 6.57).

```

root@docker-mgr:~# ls -ln /var/run/docker/netns
total 0
-r--r--r-- 1 0 0 0 Apr 24 09:49 1-azoczwi7t2
-r--r--r-- 1 0 0 0 Apr 25 07:30 1-bkbxlrjx95
-r--r--r-- 1 0 0 0 Apr 25 07:30 6ba280aebfb8
-r--r--r-- 1 0 0 0 Apr 24 09:49 ingress_sbox
-r--r--r-- 1 0 0 0 Apr 25 07:30 1b_bkbxlrjx9
root@docker-mgr:~# docker network ls

```

NETWORK ID	NAME	DRIVER	SCOPE
d0b5012f297c	bridge	bridge	local
87af0abca567	docker_gwbridge	bridge	local
5c31ff9241d2	host	host	local
azoczwi7t26u	ingress	overlay	swarm
bkbxlrjx95r9	my-overlay	overlay	swarm
d060f983a497	none	null	local

```

root@docker-mgr:~#

```

Рис. 6.57

Легко заметить соответствие между нашей сетью `my-overlay` и связанным с ней пространством имен. (Помните: пространство имен сети имеет формат `1-xxxxxx`.)

Запустим следующие две команды и проверим: находится ли второй конец пары `veth` интерфейса `eth0` в пространстве имен оверлейной сети (см. рис. 6.58). Первая команда связывает `overlay_namespace` с пространством имен интересующей нас оверлейной сети. Вторая команда, `nsenter`, проверяет интерфейсы в этом пространстве имен.

```

overlay_namespace=/var/run/docker/netns/1-bkbxlrjx95
nsenter --net=$overlay_namespace ip link show

```

```

root@docker-mgr:~# overlay_namespace=/var/run/docker/netns/1-bkbxlrjx95
root@docker-mgr:~# nsenter --net=$overlay_namespace ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP mode DEFAULT group default
    link/ether 3a:cb:86:4b:89:02 brd ff:ff:ff:ff:ff:ff
21: vxlan0@if21: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue master br0 state UNKNOWN mode DEFAULT group default
    link/ether 52:82:3c:d0:70:53 brd ff:ff:ff:ff:ff:ff link-netnsid 0
23: veth0@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue master br0 state UP mode DEFAULT group default
    link/ether 3a:cb:86:4b:89:02 brd ff:ff:ff:ff:ff:ff link-netnsid 1
25: veth1@if24: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue master br0 state UP mode DEFAULT group default
    link/ether ca:c3:51:b0:6f:fc brd ff:ff:ff:ff:ff:ff link-netnsid 2
root@docker-mgr:~#

```

Рис. 6.58

Как и ожидалось, в данном пространстве имен мы видим *интерфейс 25*. Давайте визуализируем то, что мы сейчас обсудили, — это позволит лучше понять, как все происходит.

Интерфейс `eth1` подключен к `gwbridge` через соединение `veth`; `gwbridge` направляет трафик через интерфейс `eth0` на хост и обеспечивает соединение с внешним миром. Интерфейс `eth0` одним концом подключен к контейнеру, а другим — к пространству имен, связанному с этим контейнером, как видно на рис. 6.59.

Теперь вернемся немного назад, к выходным данным на рис. 6.56. Если посмотреть внимательно, там можно заметить интерфейс `VXLAN`, который обеспечивает взаимодействие между контейнерами с технической стороны.

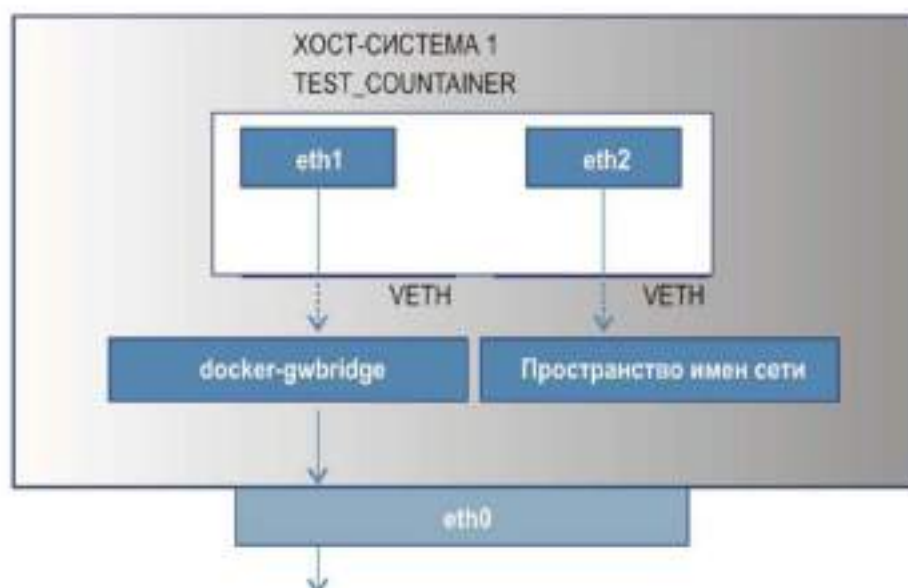


Рис. 6.59. Функционирование оверлейной сети

Что именно представляет собой VXLAN?

VXLAN (Virtual eXtensible Local Area Network — виртуальная расширяемая локальная сеть) — это технология виртуализации, которая поддерживает масштабируемость и оптимизирует облачные развертывания. VXLAN использует технологию туннелирования для инкапсуляции пакетов Ethernet уровня 2 в пакеты UDP (User Datagram Protocol) уровня 3. В результате мы получаем механизм для создания виртуализированных подсетей уровня 2 поверх физических сетей уровня 3.

Каждая подсеть уровня 2 уникально определяется виртуальным сетевым идентификатором VXLAN VNI (Virtual Network Identifier), который сегментирует трафик. Инкапсуляция и последующая декапсуляция осуществляются конечными устройствами туннеля VXLAN, или VTEP (Virtual Tunnel End Point).

VXLAN использует режим инкапсуляции пакетов MAC-in-UDP. К исходному кадру данных последовательно добавляются заголовок VXLAN, внешний заголовок UDP, внешний заголовок IP и внешний заголовок MAC.

Пакет VXLAN схематически изображен на рис. 6.60. Обратите внимание: фактическая полезная нагрузка только в *исходном кадре уровня 2*.

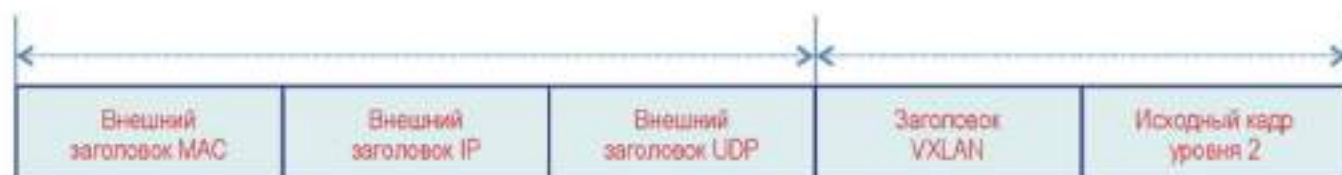


Рис. 6.60. Пакет VXLAN

Внешний заголовок *MAC* и внешний заголовок *IP* используются для обмена данными между хостами. Кадр уровня 2 инкапсулирован во внешний заголовок *UDP*, а заголовок *VXLAN* содержит ключевые метаданные о порте *VXLAN*, длине *UDP* и т. д.

Теперь, имея представление о пакете VXLAN, рассмотрим общую схему сети (рис. 6.61).

У каждого узла свое пространство имен, а интерфейс `eth0` контейнера подключен к пространству имен сети, точнее, к интерфейсу `Br0` внутри пространства имен сети. Интерфейс `Br0` можно считать виртуальным коммутатором. VTEP одним концом подключен к `Br0`, а другим — к туннелю VXLAN. Туннель VXLAN использует физические сети на уровне 3 и привязан к IP-адресам узлов и *UDP-порту 4789*.

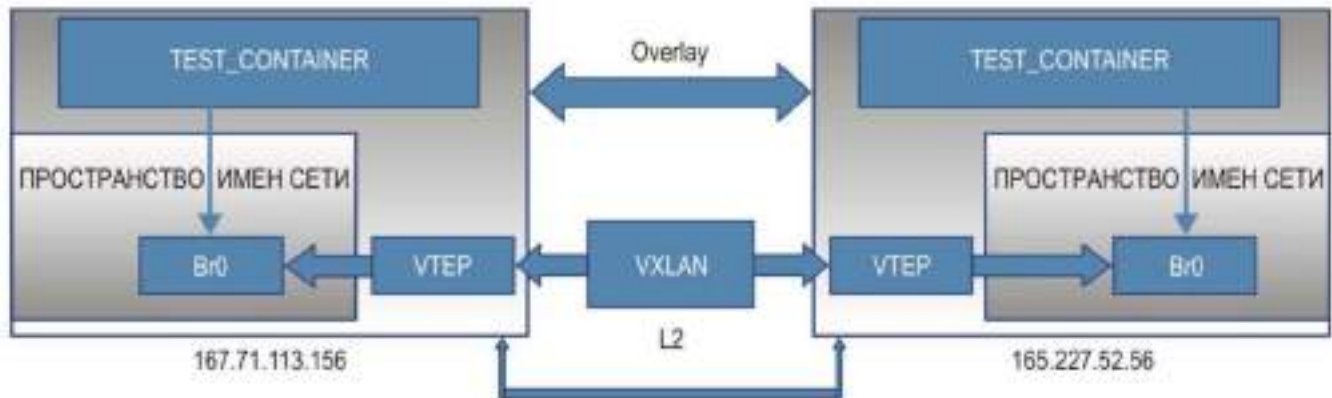


Рис. 6.61. Оверлейная сеть VXLAN

Чтобы проверить, используется ли VXLAN, отправим `ping` к контейнеру `test_container1` в хост-системе 2 с хост-системы 1 и на хост-системе 2 соберем данные `tcpdump` для анализа. Как мы знаем, Docker Swarm использует *UDP-порт 4789* для трафика в оверлейной сети. Запишем `tcpdump` для интерфейса `eth0` и *порта 4789*; результаты представлены на рис. 6.62 и 6.63.

From Host System 1

`ping 10.0.3.4`

From Host System 2

`tcpdump -i eth0 port 4789`

Host System 1 } — См. рис. 6.62

Host System 2 } — См. рис. 6.63

```
root@docker-mgr:~# docker exec -it test_container sh
/ # ping 10.0.3.4
PING 10.0.3.4 (10.0.3.4): 56 data bytes
64 bytes from 10.0.3.4: seq=0 ttl=64 time=1.962 ms
64 bytes from 10.0.3.4: seq=1 ttl=64 time=0.838 ms
64 bytes from 10.0.3.4: seq=2 ttl=64 time=0.754 ms
64 bytes from 10.0.3.4: seq=3 ttl=64 time=0.824 ms
64 bytes from 10.0.3.4: seq=4 ttl=64 time=0.756 ms
```

Рис. 6.62

Мы видим, что для маршрутизации трафика используется VXLAN. Также мы видим, что трафик с IP-адреса 167.71.113.156 перенаправляется к хосту `docker-mgr` через порт 4789, позволяя контейнеру 10.0.3.2 взаимодействовать с контейнером 10.0.3.4.

```

root@docker-wkr:~# tcpdump -i eth0 port 4789
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
11:26:43.421984 IP 167.71.113.156.35093 > docker-wkr.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.2 > 10.0.3.4: ICMP echo request, id 13056, seq 106, length 64
11:26:43.422130 IP docker-wkr.33581 > 167.71.113.156.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.4 > 10.0.3.2: ICMP echo reply, id 13056, seq 106, length 64
11:26:44.422193 IP 167.71.113.156.35093 > docker-wkr.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.2 > 10.0.3.4: ICMP echo request, id 13056, seq 107, length 64
11:26:44.422290 IP docker-wkr.33581 > 167.71.113.156.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.4 > 10.0.3.2: ICMP echo reply, id 13056, seq 107, length 64
11:26:45.422327 IP 167.71.113.156.35093 > docker-wkr.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.2 > 10.0.3.4: ICMP echo request, id 13056, seq 108, length 64
11:26:45.422453 IP docker-wkr.33581 > 167.71.113.156.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.4 > 10.0.3.2: ICMP echo reply, id 13056, seq 108, length 64
11:26:46.422478 IP 167.71.113.156.35093 > docker-wkr.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.2 > 10.0.3.4: ICMP echo request, id 13056, seq 109, length 64
11:26:46.422559 IP docker-wkr.33581 > 167.71.113.156.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.4 > 10.0.3.2: ICMP echo reply, id 13056, seq 109, length 64
11:26:47.422718 IP 167.71.113.156.35093 > docker-wkr.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.2 > 10.0.3.4: ICMP echo request, id 13056, seq 110, length 64
11:26:47.422816 IP docker-wkr.33581 > 167.71.113.156.4789: VXLAN, flags [I] (0x08), vni 4099
IP 10.0.3.4 > 10.0.3.2: ICMP echo reply, id 13056, seq 110, length 64

```

Рис. 6.63

Пока мы не знаем, как контейнеры на каждом хосте сопоставляют IP-адреса с MAC-адресами, чтобы данные направлялись на нужный сервер. Давайте посмотрим, как осуществляется это сопоставление. Запустим команду `nsenter` и используем подходящие флаги для `tcpdump`. Пингуем хост-систему 2 с хост-системы 1, а затем получаем выходные данные с хост-системы 2 (см. рис. 6.64).

```
nsenter --net=$overlay_namespace tcpdump -vv -epni any arp
```

```

root@docker-wkr:~# nsenter --net=$overlay_namespace tcpdump -vv -epni any arp
tcpdump: listening on any, link-type LINUX_SLL (Linux cooked), capture size 262144 bytes
12:25:00.232312 P 02:42:0a:00:03:04 ethertype ARP (0x0806), length 44: Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.2 tell 10.0.3.4, length 28
12:25:00.232409 Out 02:42:0a:00:03:04 ethertype ARP (0x0806), length 44: Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.2 tell 10.0.3.4, length 28
12:25:00.232413 In 02:42:0a:00:03:02 ethertype ARP (0x0806), length 44: Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.2 is-at 02:42:0a:00:03:02, length 28
12:25:00.232417 Out 02:42:0a:00:03:02 ethertype ARP (0x0806), length 44: Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.2 is-at 02:42:0a:00:03:02, length 28

```

Рис. 6.64

Чтобы не запутаться во всех этих данных, выделим два MAC-адреса и сосредоточимся на них. Если мы проверим наши контейнеры, мы найдем там эти MAC-адреса. Теперь мы видим, что MAC-адреса добавляются в пространство имен оверлейной сети, а VXLAN действует в качестве прокси для ответа на запросы ARP (Address Resolution Protocol — протокол определения адреса). См. команды ниже и рис. 6.65 и 6.66.

```
docker exec -it test_container sh } — См. рис. 6.65
```

```
docker exec -it test_container1 sh } — См. рис. 6.66
```



```

root@docker-mgr:~# docker exec -it test_container sh
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
24: eth0@if25: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue
    link/ether 02:42:0a:00:03:02 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.2/24 brd 10.0.3.255 scope global eth0
        valid_lft forever preferred_lft forever
26: eth1@if27: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
/ # █

```

Рис. 6.65

```

root@docker-wkr:~# docker exec -it test_container1 sh
sh-4.4# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:03:04 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.3.4/24 brd 10.0.3.255 scope global eth0
        valid_lft forever preferred_lft forever
28: eth1@if29: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:12:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
        valid_lft forever preferred_lft forever
sh-4.4# █

```

Рис. 6.66

В заключение рассмотрим еще один важный момент: узнаем, жестко ли закодирован MAC-адрес в пространстве имен оверлейной сети при создании контейнера. Запустим соответствующую команду и посмотрим выходные данные на рис. 6.67.

```
nsenter --net=$overlay_namespace ip neighbor show
```

```

root@docker-mgr:~# overlay_namespace=/var/run/docker/netns/1-bkx1rjx95
root@docker-mgr:~# nsenter --net=$overlay_namespace ip neighbor show
10.0.3.5 dev vxlan0 lladdr 02:42:0a:00:03:05 PERMANENT
10.0.3.4 dev vxlan0 lladdr 02:42:0a:00:03:04 PERMANENT
root@docker-mgr:~# █

```

Рис. 6.67

Здесь мы видим две записи. Теперь создадим контейнер и посмотрим, сможем ли мы найти запись, добавленную в пространство имен оверлейной сети. На хост-системе 2 запустим команду для создания контейнера (рис. 6.68).

```
docker run -dit --name test_container2 --net my-overlay redis:latest
```

Снова проверим хост-систему 1. Как и ожидалось, мы видим новую запись (рис. 6.69).

```
nsenter --net=$overlay_namespace ip neighbor show
```



```

root@docker-wkr:~# docker run -dit --name test_container2 --net my-overlay redis:latest
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
54fec2fa59d0: Pull complete
9c94e11103d9: Pull complete
04ab1bfc453f: Pull complete
5f71e6b94d83: Pull complete
2729a8234dd5: Pull complete
2683d7f17745: Pull complete
Digest: sha256:157a95b41b0dca8c308a33489dfdb28019e033110320414b4b16fad7d28c0f9f
Status: Downloaded newer image for redis:latest
61bc4a67fb92618e40b23db5757f1c2ee9204fcec86313cfa3baf341bc84b411
root@docker-wkr:~#

```

Рис. 6.68

```

root@docker-mgr:~# nsenter --net=$overlay_namespace ip neighbor show
10.0.3.4 dev vxlan0 lladdr 02:42:0a:00:03:04 PERMANENT
10.0.3.6 dev vxlan0 lladdr 02:42:0a:00:03:06 PERMANENT
10.0.3.5 dev vxlan0 lladdr 02:42:0a:00:03:05 PERMANENT
root@docker-mgr:~#

```

Рис. 6.69

Добавилась следующая запись: IP-адрес 10.0.3.6 с MAC-адресом 02:42:0a:00:03:06. Чтобы посмотреть детали, зайдём в наш новый контейнер и запустим нужные команды, после чего изучим выходные данные на рис. 6.70.

```

docker exec -it test_container2 sh
ifconfig -a

```

```

root@docker-wkr:~# docker exec -it test_container2 sh
# ifconfig -a
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1450
    inet 10.0.3.6 netmask 255.255.255.0 broadcast 10.0.3.255
    ether 02:42:0a:00:03:06 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.18.0.4 netmask 255.255.0.0 broadcast 172.18.255.255
    ether 02:42:ac:12:00:04 txqueuelen 0 (Ethernet)
    RX packets 542 bytes 8785238 (8.3 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 367 bytes 26418 (25.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 707 (707.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 707 (707.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Рис. 6.70

Это подтверждает наше предположение о том, что для каждого созданного контейнера в пространстве имен оверлейной сети будет запись для сопоставления IP-адреса с MAC-адресом.

Основные тезисы

- ◆ В Docker есть три сети по умолчанию: Bridge, Host и None.
- ◆ При установке Docker Swarm мы получаем еще две сети: оверлейную сеть `ingress` и сетевой мост `docker_gwbridge`.
- ◆ Сетевой мост Docker по умолчанию основан на мосте Linux и называется `docker0`.
- ◆ Контейнер может коммуницировать с внешним миром напрямую, но получить доступ к Docker извне можно только через интерфейс хоста — обычно это `eth0`.
- ◆ Порт на сервере можно легко сопоставить с контейнером, но этот порт на хосте нельзя сопоставить с другим контейнером.
- ◆ Сопоставлять порты можно по протоколу TCP или UDP.
- ◆ Если мы используем сеть Host, контейнер сопоставляется с сетью хоста, а собственной сети у него нет.
- ◆ Для полной изоляции контейнера от внешнего мира используется сеть None.
- ◆ Для подключения приложения к физической сети используется сеть MACVLAN.
- ◆ Docker Swarm использует оверлейную сеть `ingress` для связи с другими контейнерами, тогда как обмен данными с внешним миром реализуется через `docker_gwbridge`.

Контрольный тест

Выберите подходящий вариант ответа.

1. К какому типу относится сеть `docker0`?
 - Bridge.
 - None.
 - Overlay.
 - Ничего из вышеперечисленного.
2. Выберите верное утверждение об VXLAN:
 - Подсети уровня 2 работают поверх физических сетей уровня 3.
 - Подсети уровня 3 работают поверх физических сетей уровня 2.
 - VXLAN — это конфигурация уровня 7.
 - Ничего из вышеперечисленного.

3. Как сетевой мост `docker0` осуществляет обмен данными между контейнерами?
 - По IP-адресам или именам контейнеров.
 - Только по именам контейнеров.
 - Только по IP-адресам контейнеров.
 - Ничего из вышеперечисленного.
4. На каком порте работает туннель VXLAN?
 - TCP 2377.
 - UDP 2377.
 - TCP 4789.
 - UDP 4789.
5. В каком каталоге Docker создает пространство имен сети?
 - `/var/run/docker/`.
 - `/var/run/netns`.
 - `/var/run/docker/netns`.
 - Ничего из вышеперечисленного.
6. В каком режиме должна быть сетевая карта (NIC) хоста для MACVLAN?
 - Non-promiscuous (не неразборчивый).
 - Promiscuous (неразборчивый).
 - Не имеет значения.
 - Ничего из вышеперечисленного.
7. При использовании пространства имен существующего контейнера...
 - У нового контейнера не будет собственной файловой системы.
 - У нового контейнера не будет ни собственной сети, ни собственной файловой системы.
 - Два контейнера могут иметь отдельные порты.
 - У нового контейнера будет своя файловая система, но общий сетевой стек с существующим контейнером.
8. Какова функция флага `-P`?
 - Случайное (рандомное) сопоставление порта контейнера с портом на хосте.
 - Сопоставление порта на хосте со случайным портом на контейнере.
 - Сопоставление случайных портов на контейнере и хосте друг с другом.
 - Ничего из вышеперечисленного.

9. Выберите верное утверждение:

- MAC-адрес жестко кодируется в хост-системе при создании или добавлении контейнера в Swarm.
- MAC-адрес жестко кодируется в пространстве имен оверлейной сети при создании или добавлении контейнера в Swarm.
- В пространстве имен оверлейной сети нет конфигурации MAC-адресов.
- MAC-адрес хранится как переменная сети в пространстве имен оверлейной сети.

10. Br0 можно считать:

- Физическим коммутатором.
- Физическим интерфейсом.
- Виртуальным пространством имен.
- Виртуальным коммутатором.

Ответы

1	a
2	a
3	c
4	d
5	c

6	b
7	d
8	a
9	b
10	d

Вопросы

1. Какие типы сетей создаются в Docker по умолчанию?
2. Что такое пространство имен сети в контексте контейнера?
3. Какие новые сети создаются, если контейнер помещается в Docker Swarm?
4. Что такое сетевой мост `docker0`?
5. Каким образом сопоставляются порты?
6. Что такое сеть `MACVLAN`? Какие два режима есть у сети `MACVLAN`?
7. Что такое `veth` в контексте пространства имен Linux?
8. Как работает оверлейная сеть?
9. Что такое `VXLAN`?
10. Как настроить сеть `MACVLAN`, чтобы ее можно было пинговать с хоста?

Ключевые термины

- ◆ Сеть Bridge
- ◆ Сеть Host
- ◆ Сеть None
- ◆ Сеть MACVLAN
- ◆ Сеть Overlay (оверлейная сеть)
- ◆ `docker_gwbridge`
- ◆ Интерфейс Ethernet
- ◆ Veth
- ◆ Docker Swarm
- ◆ Сопоставление портов
- ◆ Пространство имен сети
- ◆ Сетевой режим trunk bridge 802.1Q
- ◆ Виртуальный коммутатор Br0
- ◆ VXLAN
- ◆ VTEP

ГЛАВА 7

Безопасность в Docker.

Часть 1

Введение

Средства безопасности Docker можно разделить на две категории: функции безопасности Linux (в этой книге мы рассматриваем Docker на Linux, поэтому подробно обсудим безопасность на уровне этой операционной системы) и встроенные (собственные) средства безопасности для контейнеров Docker, предоставляемые Docker Enterprise Edition. Теме безопасности посвящены две главы; здесь мы рассмотрим соответствующие функции Linux.

Изучив эту главу, вы будете свободно ориентироваться в функциях безопасности Linux и научитесь использовать их для защиты экосистемы контейнеров Docker. Также вы научитесь принимать взвешенные решения относительно конфигураций безопасности и эффективно использовать встроенные средства безопасности ОС Linux.

Темы для изучения следующие.

- ◆ Пространства имен ядра
- ◆ Контрольные группы
- ◆ Capabilities
- ◆ Обязательный контроль доступа (SELinux и AppArmor)
- ◆ Seccomp

Пространства имен ядра

В предыдущих главах мы неоднократно сталкивались с понятием пространства имен, но не углублялись в его суть. Сделаем это сейчас. *Пространство имен* — это механизм для изоляции, который объединяет объекты или группы объектов по имени и снабжает их уникальными идентификаторами.

Пространства имен ядра — пожалуй, фундаментальный элемент безопасности в Docker, который можно визуализировать следующим образом: каждый контейнер

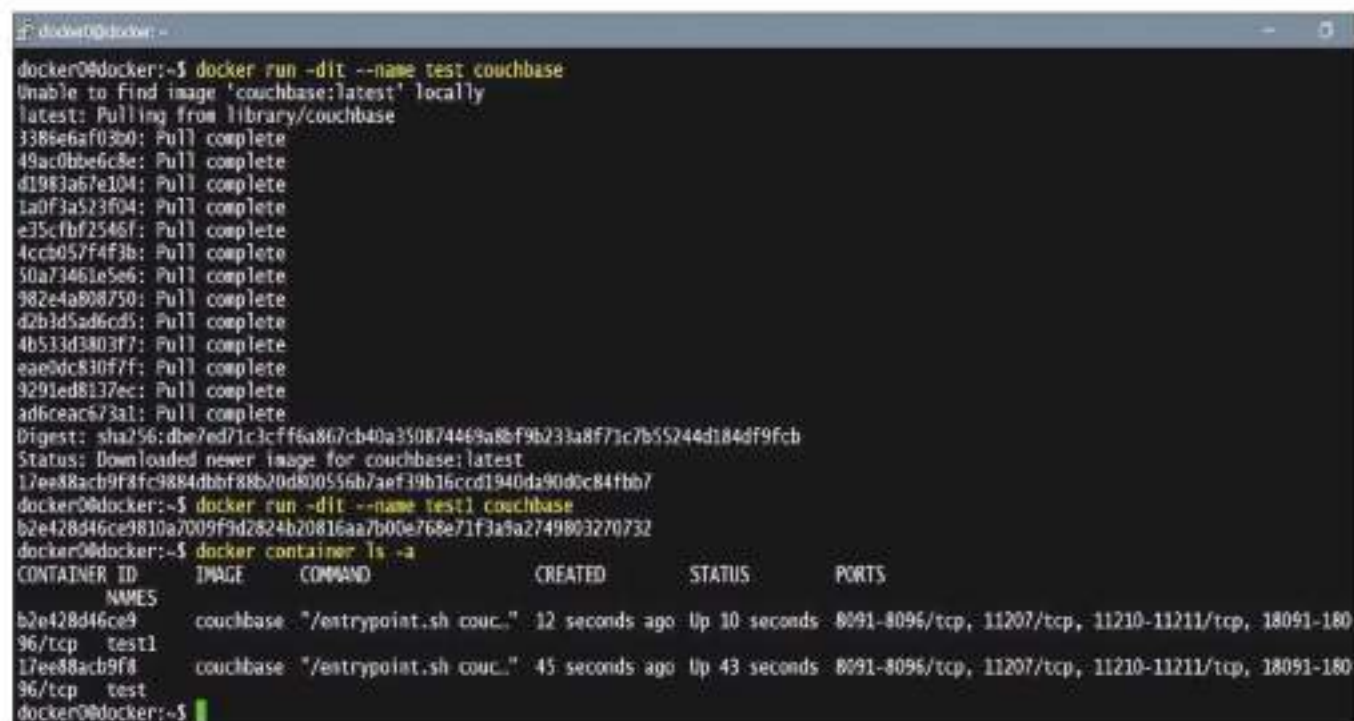
Docker имеет *свой фрагмент операционной системы*, что позволяет запускать сотни контейнеров на одном хосте без *конфликтов* на уровне ОС. Например, контейнеры могут использовать один порт или один набор файлов конфигурации без каких-либо проблем. Это возможно благодаря изоляции, которую предоставляют пространства имен. Docker на Linux использует следующий набор пространств имен ядра:

- ◆ PID (ID процессов);
- ◆ NET (сети);
- ◆ MNT (файловые системы / точки монтирования);
- ◆ IPC (межпроцессное взаимодействие);
- ◆ USR (пользователи);
- ◆ UTS (хосты).

PID, т. е. process id: идентификатор процесса. Уникальный идентификационный номер для процесса автоматически генерируется при запуске в системе Linux. Docker использует пространство имен PID для создания отдельного дерева процессов для каждого контейнера.

Давайте создадим пару контейнеров и посмотрим, как это работает. Это будут контейнеры для базы данных Couchbase (база данных класса NoSQL для критически важных бизнес-приложений). Затем заглянем внутрь и посмотрим, как выполняются процессы. Итак, создаем контейнеры и изучаем выходные данные на рис. 7.1.

```
docker run -dit --name test couchbase
docker run -dit --name test1 couchbase
docker container ls -a
```



```

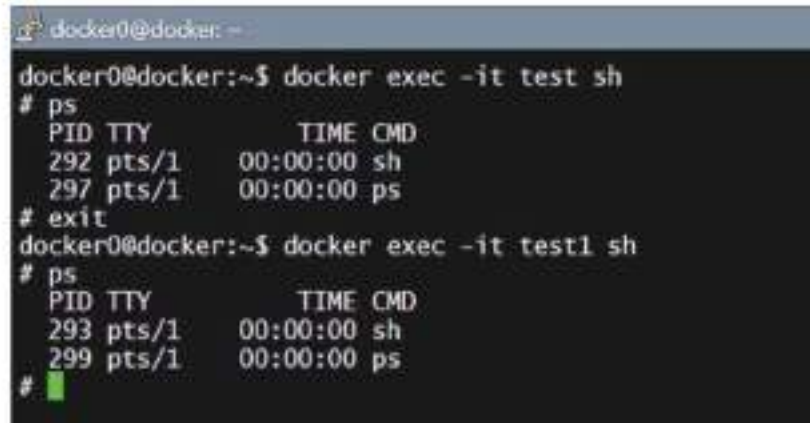
docker0@docker:~$ docker run -dit --name test couchbase
Unable to find image 'couchbase:latest' locally
latest: Pulling from library/couchbase
3386e6af03b0: Pull complete
49ac0bbe6c8e: Pull complete
d1983a67e104: Pull complete
1a0f3a523f04: Pull complete
e35c1bf2546f: Pull complete
4ccb057f4f3b: Pull complete
50a73461e5e6: Pull complete
982e4a808750: Pull complete
d2b3d5ad6cd5: Pull complete
4b533d3803f7: Pull complete
eae0dc830f7f: Pull complete
9291ed8137ec: Pull complete
ad6ceac673a1: Pull complete
Digest: sha256:db7ed71c3cffe6a867cb40a350874469a8bf9b233a8f71c7b55244d184df9fcb
Status: Downloaded newer image for couchbase:latest
17ee88acb9f8c9884dbbf88b20d80556b7aef39b16ccd1940da90dc84fbb7
docker0@docker:~$ docker run -dit --name test1 couchbase
b2e428d46ce9810a7009f9d2824b20816aa7b00e768e71f3a9a2749803270732
docker0@docker:~$ docker container ls -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b2e428d46ce9   couchbase     "/entrypoint.sh couc..." 12 seconds ago Up 10 seconds 8091-8096/tcp, 11207/tcp, 11210-11211/tcp, 18091-18096/tcp
17ee88acb9f8   couchbase     "/entrypoint.sh couc..." 45 seconds ago Up 43 seconds 8091-8096/tcp, 11207/tcp, 11210-11211/tcp, 18091-18096/tcp
docker0@docker:~$

```

Рис. 7.1

Далее запустим несколько команд внутри контейнеров; выходные данные — на рис. 7.2.

```
docker exec -it test sh
ps
docker exec -it test1 sh
ps
```



```

docker0@docker: ~$ docker exec -it test sh
# ps
  PID TTY          TIME CMD
   292 pts/1        00:00:00 sh
   297 pts/1        00:00:00 ps
# exit
docker0@docker:~$ docker exec -it test1 sh
# ps
  PID TTY          TIME CMD
   293 pts/1        00:00:00 sh
   299 pts/1        00:00:00 ps
#

```

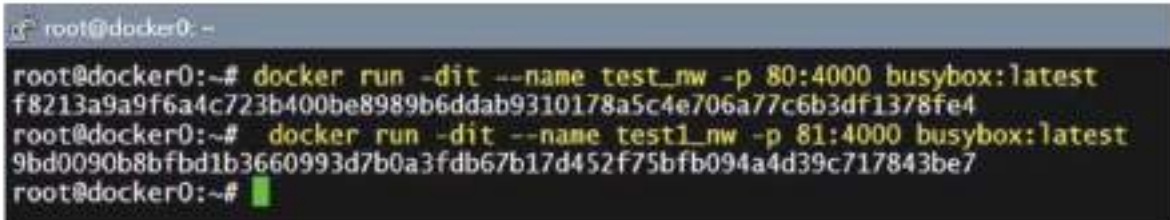
Рис. 7.2

Мы видим, что в контейнерах выполняются совершенно одинаковые процессы, но с разными PID. Благодаря изоляции процессы в одном контейнере никак не пересекаются с процессами в другом.

NET (сети): сетевой стек одного контейнера полностью изолирован от другого контейнера; у контейнера нет привилегированного доступа к сокетам или интерфейсам другого контейнера. При этом контейнеры могут взаимодействовать друг с другом на уровне сети посредством сетевых интерфейсов так же, как и с внешними хостами. При указании общедоступных портов для контейнеров или использовании ссылок между контейнерами может проходить IP-трафик.

Благодаря изоляции пространства имен в сети можно сопоставлять с одним и тем же портом несколько контейнеров на уровне контейнеров, если они используют разные порты на уровне хоста. Давайте создадим пару контейнеров и посмотрим, как это работает. На этот раз используем контейнеры `busybox`. `BusyBox` — это комплект программного обеспечения, который предоставляет несколько утилит Unix в одном исполняемом файле (см. команды ниже и рис. 7.3).

```
docker run -dit --name test_nw -p 80:4000 busybox:latest
docker run -dit --name test1_nw -p 81:4000 busybox:latest
```



```

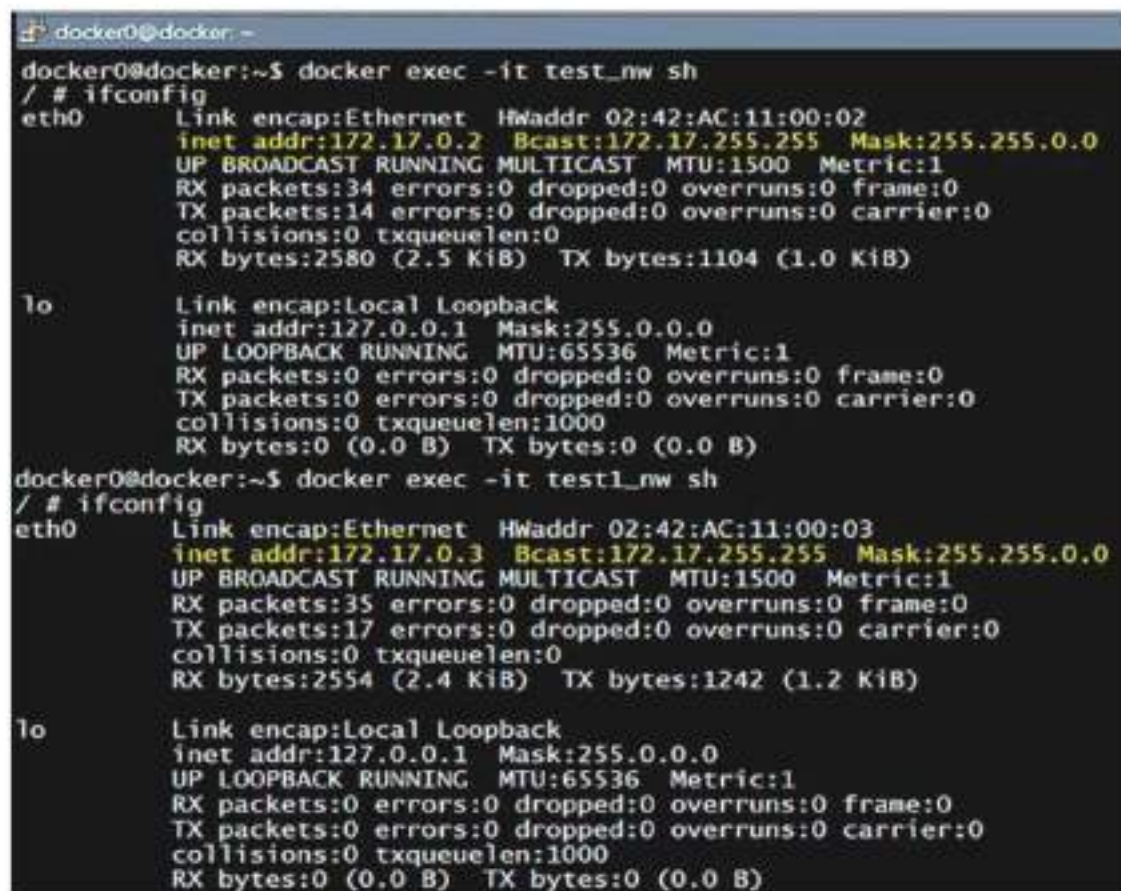
root@docker0: ~$ docker run -dit --name test_nw -p 80:4000 busybox:latest
f8213a9a9f6a4c723b400be8989b6ddab9310178a5c4e706a77c6b3df1378fe4
root@docker0:~$ docker run -dit --name test1_nw -p 81:4000 busybox:latest
9bd0090b8bfbdb1b3660993d7b0a3fdb67b17d452f75bfb094a4d39c717843be7
root@docker0:~$

```

Рис. 7.3

Мы видим, что порт 4000 каждого контейнера можно сопоставить с разными портами на хосте. Обратите внимание: каждый контейнер получает собственный интерфейс `eth0` со своим IP-адресом и диапазоном портов (рис. 7.4).

```
docker exec -it test_nw sh
ifconfig
docker exec -it test1_nw sh
ifconfig
```



```
docker0@docker:~$ docker exec -it test_nw sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:34 errors:0 dropped:0 overruns:0 frame:0
          TX packets:14 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2580 (2.5 KiB)  TX bytes:1104 (1.0 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

docker0@docker:~$ docker exec -it test1_nw sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:03
          inet addr:172.17.0.3  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:35 errors:0 dropped:0 overruns:0 frame:0
          TX packets:17 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2554 (2.4 KiB)  TX bytes:1242 (1.2 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Рис. 7.4

MNT (файловые системы / точки монтирования, от mount): у каждого контейнера есть собственная файловая система. У хоста, разумеется, есть своя полноценная операционная система. Контейнеры имеют доступ только к пространству имен своей файловой системы и не видят пространство имен файловой системы других контейнеров. Структура файловой системы контейнеров *busybox* показана на рис. 7.5.

```
docker exec -it test_nw sh
ls
docker exec -it test1_nw sh
ls
```

Как видим, у контейнеров схожие файловые системы, и контейнеры могут использовать их как угодно, никак не затрагивая файловые системы друг друга.

IPC (inter-process communication — межпроцессное взаимодействие): в Linux существует механизм совместного использования памяти несколькими процессами.


```

docker0@docker:~$ docker exec -it test_nw sh
/ # ls
bin  dev  etc  home  proc  root  sys  tmp  usr  var
/ # exit
docker0@docker:~$ docker exec -it test1_nw sh
/ # ls
bin  dev  etc  home  proc  root  sys  tmp  usr  var
/ # █

```

Рис. 7.5

Сегменты общей памяти используются для ускорения межпроцессных коммуникаций на скорости памяти и не ограничиваются каналами или сетевым стеком. Межпроцессное взаимодействие связывает процессы в крупномасштабных архитектурах ПО. У контейнеров есть собственное пространство имен IPC, которое можно настроить для общего доступа к памяти. По умолчанию один контейнер не имеет доступа к памяти другого контейнера или хоста. Ниже приводятся два примера создания контейнеров с общим доступом к памяти (см. команды ниже и рис. 7.6).

```

docker run -dit --name shared --ipc shareable centos
docker run -dit --name shared1 --ipc container:shared centos

```

```

docker0@docker:~$ docker run -dit --name shared --ipc shareable centos
71c0347a3321a4e927d49fcaedfd5784c02b5abb7c3bac23af3f766e35e9ad22
docker0@docker:~$ docker run -dit --name shared1 --ipc container:shared centos
e7d4c8ea6c8aca0488c6360690372e05688d91a04ba39cb347167cc088db785e
docker0@docker:~$ █

```

Рис. 7.6

Первая команда с флагом `ipc shareable` использует собственное приватное пространство имен IPC. Важно отметить, что существует возможность предоставить доступ к нему другим контейнерам. Во второй команде к пространству имен IPC предоставляется доступ для контейнера с именем `shared`, который мы создали первой командой, как показано на рис. 7.6.

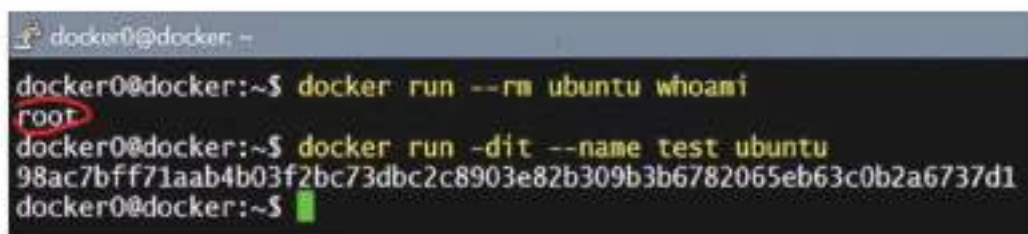
USR (пользователи, от *users*): контейнер Docker имеет пространство имен пользователей, изолированное от других пользователей в других контейнерах и на хосте. Пространство имен USR не активировано по умолчанию: если UID (*user id* — *id* пользователя) в контейнере совпадет с UID в хосте, у пользователя будет такое же разрешение, что и у пользователя хоста, а это может привести к катастрофическим последствиям, особенно если у контейнеров и хоста есть общие тома. В этом случае можно с помощью флага Docker `userns-remap` сопоставить идентификаторы пользователей в контейнере с идентификаторами пользователей в Linux, где у них гораздо меньше привилегий и разрешений. Например, пользователя `root` с UID 0 в контейнере можно сопоставить с хостом Linux и запустить с другим *id*, чтобы избежать случайного или намеренного злоупотребления привилегиями `root`.

Как правило, большинство контейнеров выполняются от имени `root` и имеют почти неограниченный доступ к состоянию контейнера. Возможно, это не лучший под-

ход, поскольку любой процесс, выполняющийся от имени пользователя, наследует все привилегии и разрешения, что может привести к уязвимости контейнера. Чтобы снизить риски, можно создать в контейнере пользователя с ограниченными привилегиями, чтобы он мог работать только внутри контейнера. Рассмотрим эту концепцию на примере.

Создадим контейнер на основе образа `ubuntu`. Но сначала убедимся, что `ubuntu` по умолчанию выполняется от имени `root` (см. команду ниже и рис. 7.7).

```
docker run --rm ubuntu whoami
docker run -dit --name test ubuntu
```



```
docker0@docker: ~$ docker run --rm ubuntu whoami
root
docker0@docker: ~$ docker run -dit --name test ubuntu
98ac7bff71aab4b03f2bc73dbc2c8903e82b309b3b6782065eb63c0b2a6737d1
docker0@docker: ~$
```

Рис. 7.7

Теперь создадим нового пользователя внутри контейнера (рис. 7.8).

```
docker exec -it test sh
ls
```



```
docker0@docker: ~$ docker exec -it test sh
# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
#
```

Рис. 7.8

Добавим пользователя `test_user`, как показано на рис. 7.9.

```
adduser test_user
cat /etc/passwd
```

Итак, мы создали пользователя, который будет работать в контейнере по принципу минимальных привилегий. Таким образом мы гарантированно *усилим* защиту контейнера.

UTS (Unix Time Sharing — разделение времени Unix): пространство имен UTS используется для изоляции имени хоста, что гарантирует изоляцию двух элементов, связанных с системным вызовом `uname`. Это означает, что процесс имеет копии имени хоста и доменного имени *NIS* (см. ниже), и их можно менять, не затрагивая остальную часть системы.

NIS (Network Information System) — это служба каталогов, созданная Sun Microsystems, которая не поддерживается с тех пор, как Sun Microsystems была поглощена корпорацией Oracle. В настоящее время NIS используется очень редко.


```

# adduser test_user
Adding user `test_user' ...
Adding new group `test_user' (1000) ...
Adding new user `test_user' (1000) with group `test_user' ...
Creating home directory `/home/test_user' ...
Copying files from `/etc/skel' ...
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Changing the user information for test_user
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
test_user:x:1000:1000:,,,:/home/test_user:/bin/bash
#

```

Рис. 7.9

Контрольные группы

Контрольные группы (cgroups) — это механизм объединения процессов на основе потребления ими ресурсов, приоритизации, стабильности и в некоторой степени безопасности. Представьте, что в систему проникла опасная ошибка или вредоносный код или пользователь намеренно внедрил бесконечный цикл, нарушив работу системы.

Для защиты от таких рисков в ядре Linux существуют контрольные группы, или cgroups (control groups), которые позволяют ограничить использование ресурсов в системе следующим образом.

1. *Ограничение потребления ресурсов.* Для групп можно установить максимальный лимит памяти, ЦП, сети и других ресурсов.
2. *Приоритизация потребления ресурсов.* Если ресурсов недостаточно, при распределении можно дать одной группе приоритет над другой.

3. *Учет.* Использование ресурсов каждой группой измеряется и отслеживается.
4. *Контроль потребления ресурсов.* Некоторые группы можно приостановить, а потом запустить снова.

По сути, `cgroups` — это просто набор процессов с общими ограничениями и/или приоритетами. `cgroups` можно организовать в иерархическую древовидную структуру, в которой подгруппы будут наследовать лимиты родительской группы.

В Docker не существует ограничений ресурсов по умолчанию, т. е. контейнер может использовать столько ресурсов, сколько позволит планировщик ядра на хосте. При этом Docker позволяет контролировать потребление памяти и ресурсов ЦП контейнерами, предоставляя флаги для команд Docker.

Память

Очевидно, что нехватка памяти будет иметь серьезные последствия. Если в хост-системе заканчивается память, ядро выдаст исключение *OOME* (Out Of Memory Exception — нехватка памяти) и начнет рандомно останавливать процессы, чтобы освободить память, а это может привести к сбою самого Docker. Очевидно, что таких ситуаций нужно избегать во что бы то ни стало!

Отчасти Docker решает эту проблему путем приоритизации OOME, чтобы сначала завершались другие процессы в хост-системе, а демон Docker продолжал работать. То есть, скорее всего, будут завершаться контейнеры, а не процесс демона Docker или другие системные процессы.

Чтобы смягчить последствия исключения OOME, можно задать жесткие ограничения памяти и запретить одному контейнеру использовать всю память. Существуют также механизмы для использования мягких лимитов, которые срабатывают при выполнении некоторых условий.

Рассмотрим самые важные лимиты, связанные с памятью.

Флаг `--memory`, или `-m`, устанавливает верхний лимит памяти, доступной контейнеру. Однако с этим флагом нужно устанавливать объем памяти не менее 4 Мбайт.

Флаг `--memory-reservation` устанавливает мягкий лимит; он всегда будет меньше, чем значение `--memory`. Флаг `--memory-reservation` означает, что в случае нехватки памяти объем выделенной памяти может опуститься до значения `--memory-reservation` — минимальный объем, необходимый для выполнения контейнера. Когда ситуация с памятью улучшится, контейнер получит объем памяти, установленный флагом `--memory`.

Давайте создадим пару контейнеров, используя эти флаги, и посмотрим, что получится (рис. 7.10).

```
docker run -dit -name test --memory 200m --memory-reservation 150m
busybox:latest
```

Мы можем создать контейнер без каких-либо проблем, но внизу есть предупреждение: **WARNING: Your kernel does not support swap limit capabilities or the cgroup**

is not mounted. Memory limited without swap (Внимание: ваше ядро не поддерживает возможности лимитов пространства подкачки или контрольная группа не смонтирована. Память ограничена без подкачки).

```
docker0@docker0:~$ docker run -dit --name test --memory 200m --memory-reservation 150m busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
bdbbaa22dec6: Pull complete
Digest: sha256:6915be4043561d64e0ab0f8f098dc2ac48e077fe23f488ac24b665166898115a
Status: Downloaded newer image for busybox:latest
WARNING: Your kernel does not support swap limit capabilities or the cgroup is not mounted. Memory limited without swap.
1fc654be40081a1f962f43124dda582ae7400ef324409260de2d3914c07e9ff8
```

Рис. 7.10

Как сказано в документации по Docker, эти сообщения можно получить в системах Ubuntu или Debian, но не в системах на основе RPM, где эти возможности включены по умолчанию. См. <https://docs.docker.com/install/linux/linux-postinstall/>.

В своей облачной системе Ubuntu я использовал определенный набор команд, чтобы включить эти возможности.

Я вошел на хост как root и проделал следующие шаги.

1. Отредактировал файл `/etc/default/grub.d/50-cloudimg-settings.cfg`, добавив строку `GRUB_CMDLINE_LINUX_DEFAULT="console=ttyS0 cgroup_enable=memory swapaccount=1"`.
2. Сохранил файл.
3. Обновил GRUB (GRand Unified Bootloader — загрузчик ОС): `update-grub`.
4. Перезапустил систему: `reboot`.

Теперь мы создадим еще один контейнер с лимитами памяти и больше не увидим это предупреждение (рис. 7.11).

```
docker run -dit --name test1 --memory 512m --memory-reservation 450m
busybox:latest
```

```
docker0@docker0:~$ docker run -dit --name test1 --memory 512m --memory-reservation 450m busybox:latest
5b8712e930c7aa81c3d1df28999b6f2f9f8572a070709497d9b2a3ca9a8b59f9
docker0@docker0:~$
```

Рис. 7.11

`--memory-swap` — еще один флаг для управления потреблением памяти; он очень полезен тем, что позволяет записывать излишки данных на диск. Конечно, это скажется на производительности, зато контейнер продолжит работу и не выдаст ошибку.

Настройка может показаться несколько запутанной, но я постараюсь максимально все упростить.

1. Если для флага `--memory-swap` установлено положительное целое число, использование флага `-memory` обязательно.

2. Значение `--memory-swap` должно быть больше значения `--memory`, потому что `--memory-swap` включает в себя `--memory`. Например, если для `--memory-swap` установить 1024 Мбайт, а для `--memory` — 768 Мбайт, контейнер сможет использовать 768 Мбайт памяти и 256 Мбайт пространства подкачки (swap), т. е. 1024–768 Мбайт.
3. Если для `--memory-swap` установлено значение 0, флаг `--memory-swap` будет игнорироваться.
4. Если для `--memory` и `--memory-swap` установлены одинаковые положительные целые числа (например, 512 Мбайт), флаг `--memory-swap` будет игнорироваться, и у контейнера не будет доступа к swap.
5. Если флаг `--memory` установлен, а `--memory-swap` — нет, контейнеру будет доступно столько же пространства подкачки (swap), сколько и памяти, установленной `--memory`. Например, если мы установили для `--memory` 512 Мбайт, по умолчанию контейнеру будет доступно 512 Мбайт swap, если таковой объем имеется в системе.
6. Если явно задать для флага `--memory-swap` значение -1, у контейнера будет доступ к неограниченному объему swap в хост-системе.

Если использовать внутри контейнера такие инструменты, как `free`, это даст нам сведения об объеме swap, доступном на хосте, а не в контейнере. `Free` и подобные инструменты не позволяют определить объем swap внутри контейнера.

Прежде чем перейти к следующему разделу, давайте создадим контейнер, используя рассмотренные параметры памяти. Внимательно посмотрите на рис. 7.12.

```

docker0@docker0:~$ docker run -dit --name test --memory 768m --memory-reservation 512m --memory-swap 1024m centos:latest
8a8f319212d656180aac0f57ee0c50905788e071d15c49327b36c967012bb25f
docker0@docker0:~$

```

Рис. 7.12

Как видите, мы можем создать контейнер с несколькими флагами для управления памятью.

Центральный процессор

По умолчанию контейнерам доступно сколько угодно циклов центрального процессора (ЦП) на хост-компьютере. Очевидно, что в реальном мире ресурсы ЦП ограничены. У контейнеров не должно быть возможности использовать все ресурсы ЦП, иначе, если они будут исчерпаны, мы столкнемся с закономерными последствиями.

Существует несколько флагов для среды выполнения, которые позволяют контролировать доступ контейнера к ресурсам ЦП. Если установить эти флаги, Docker соответственно изменит параметры `cgroups` на хост-компьютере.

Здесь мы рассмотрим три наиболее распространенных флага:

1. `--cpus` позволяет явно указать, сколько ресурсов ЦП будет доступно контейнеру; можно установить нецелое, десятичное число. Например, если у хост-компьютера 4 ЦП, можно указать 2,6 или 1,2 ЦП.
2. `--cpuset-cpus` позволяет указать детализированные параметры в виде списка, разделенного запятой, или диапазона с дефисом. Например, 0-2 будет означать ЦП номер 0, 1 и 2, а 0,4 — возможность использования контейнером первого и пятого ЦП.
3. `--cpu-shares`: флаг мягкого ограничения. Если для этого флага установить значение больше 1024 (значение по умолчанию), контейнер будет иметь приоритет на получение ресурсов ЦП, если в системе произойдет какой-то сбой. Если установить значение меньше 1024, соответственно доступ к ресурсам ЦП уменьшится пропорционально.

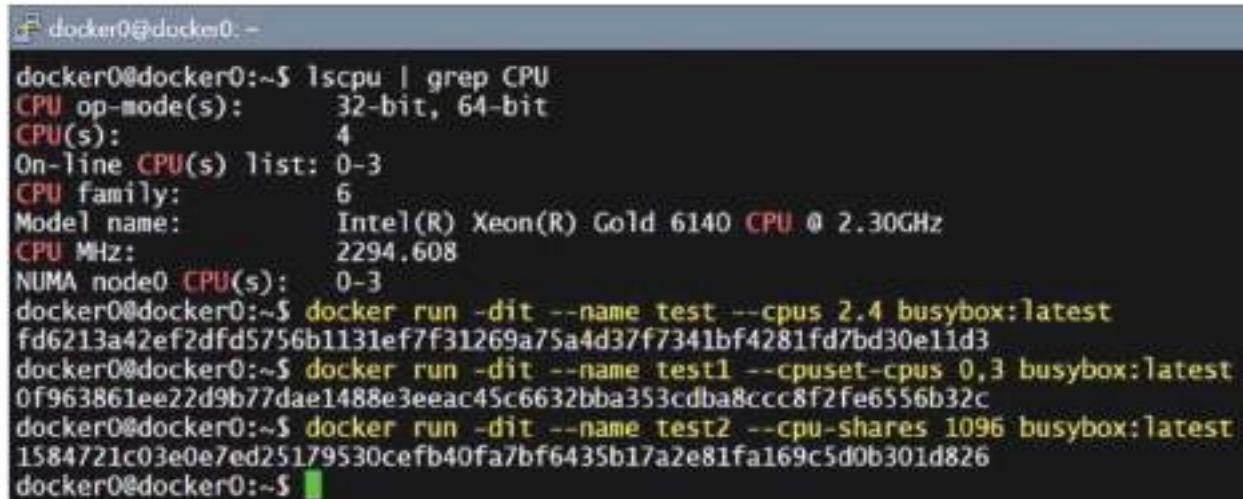
Рассмотрим несколько примеров.

Прежде всего узнаем, сколько ЦП у нас в системе:

```
lscpu | grep CPU
```

Затем запустим несколько команд и изучим выходные данные на рис. 7.13.

```
docker run -dit --name test --cpus 2.4 busybox:latest
docker run -dit --name test1 --cpuset-cpus 0,3 busybox:latest
docker run -dit --name test2 --cpu-shares 1096 busybox:latest
```



```
docker0@docker0:~$ lscpu | grep CPU
CPU op-mode(s): 32-bit, 64-bit
CPU(s): 4
On-line CPU(s) list: 0-3
CPU family: 6
Model name: Intel(R) Xeon(R) Gold 6140 CPU @ 2.30GHz
CPU MHz: 2294.608
NUMA node0 CPU(s): 0-3
docker0@docker0:~$ docker run -dit --name test --cpus 2.4 busybox:latest
fd6213a42ef2dfd5756b1131ef7f31269a75a4d37f7341bf4281fd7bd30e11d3
docker0@docker0:~$ docker run -dit --name test1 --cpuset-cpus 0,3 busybox:latest
0f963861ee22d9b77dae1488e3eeac45c6632bba353cdba8ccc8f2fe6556b32c
docker0@docker0:~$ docker run -dit --name test2 --cpu-shares 1096 busybox:latest
1584721c03e0e7ed25179530cefb40fa7bf6435b17a2e81fa169c5d0b301d826
docker0@docker0:~$
```

Рис. 7.13

Capabilities

В традиционных системах Unix есть две категории процессов.

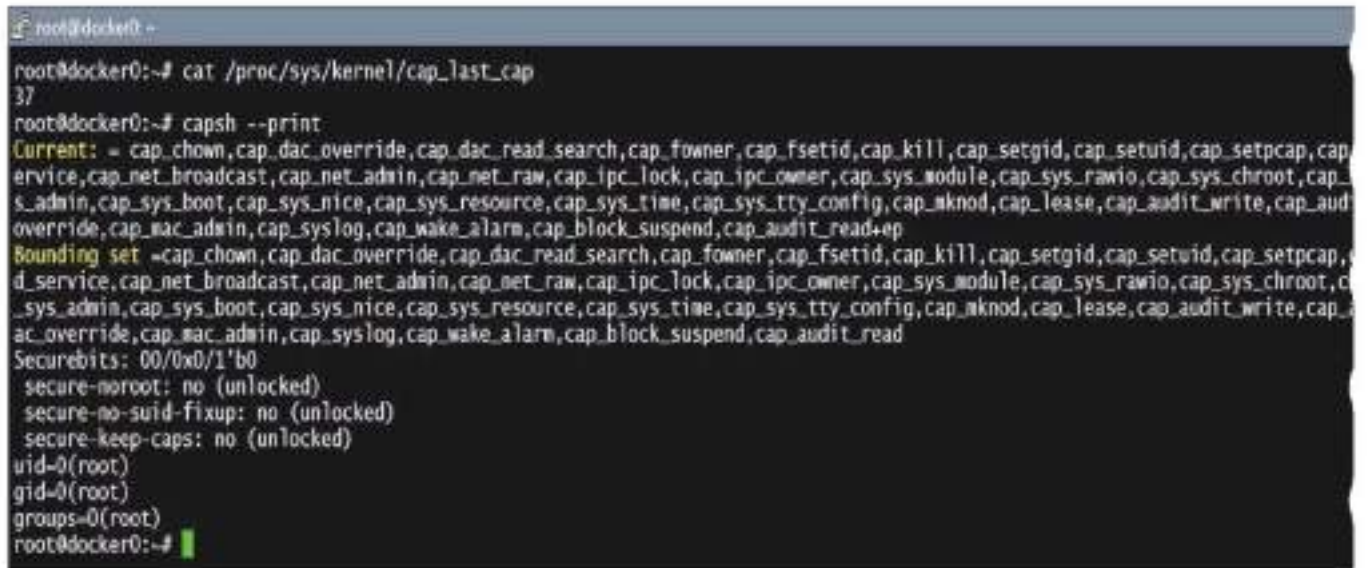
Привилегированные процессы выполняются без проверки разрешений. Обычно эти процессы соответствуют статусу суперпользователя, или root, и имеют `uid 0` (идентификатор пользователя).

Непривилегированные процессы не имеют `UID 0` и предполагают все необходимые проверки разрешений.

Начиная с версии ядра Linux 2.2, привилегии и разрешения, связанные с суперпользователем или `root`, выделены в отдельный набор *capabilities* (возможности), который можно включать и отключать независимо для каждого потока.

Запустим соответствующие команды в нашей системе Ubuntu 18.04.1 и посмотрим, что получится (рис. 7.14): команду `cat /proc/sys/kernel/cap_last_cap`, чтобы посмотреть число *capabilities* в системе; и команду `capsh --print`, чтобы вывести их список.

```
cat /proc/sys/kernel/cap_last_cap
capsh --print
```



```
root@dockero:~# cat /proc/sys/kernel/cap_last_cap
37
root@dockero:~# capsh --print
Current: = cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_service,cap_net_broadcast,cap_net_admin,cap_net_raw,cap_ipc_lock,cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_sys_time,cap_sys_tty_config,cap_mknod,cap_lease,cap_audit_write,cap_audit_override,cap_mac_admin,cap_syslog,cap_wake_alarm,cap_block_suspend,cap_audit_read+ep
Bounding set =cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_service,cap_net_broadcast,cap_net_admin,cap_net_raw,cap_ipc_lock,cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_sys_time,cap_sys_tty_config,cap_mknod,cap_lease,cap_audit_write,cap_audit_override,cap_mac_admin,cap_syslog,cap_wake_alarm,cap_block_suspend,cap_audit_read
Securebits: 00/0x0/1'b0
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
uid=0(root)
gid=0(root)
groups=0(root)
root@dockero:~#
```

Рис. 7.14

Итак, мы видим, что у нас 37 *capabilities*, а ниже приводится их список. На рис. 7.14 выделены два набора *capabilities*: `Current` и `Bounding set`. В набор `Current` входят текущие *capabilities*, которые могут быть предоставлены исполняемым файлом. `Bounding set` включает все *capabilities*, доступные в дереве процессов.

При создании контейнера доступны только *capabilities*, необходимые для выполнения приложения, а остальные по умолчанию отключены. Рассмотрим конкретный пример, чтобы понять этот механизм: создадим контейнер и посмотрим, какие *capabilities* будут доступны ему по умолчанию (рис. 7.15).

```
docker run --rm centos:latest sh -c "capsh --print"
```

Как видите, по умолчанию контейнеру Docker доступен очень ограниченный набор *capabilities*. Это встроенная мера безопасности. Но доступных *capabilities* достаточно для того, чтобы выполнять большинство приложений. Если в конкретном случае требуются дополнительные *capabilities*, их можно добавить (`add`) в контейнер. И наоборот: чтобы усилить безопасность, можно удалять (`delete`) любые *capabilities* из контейнера. Для этого используются флаги `--cap-add` и `--cap-delete`. Обратимся к примерам.


```

root@docker0:~# docker run --rm centos:latest sh -c "capsh --print"
Current: = cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_
nod,cap_audit_write,cap_setfcap+ep
Bounding set =cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_
_mknod,cap_audit_write,cap_setfcap
Securebits: 00/0x0/1'b0
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
uid=0(root)
gid=0(root)
groups=
root@docker0:~#

```

Рис. 7.15

В первом примере проверим, можем ли мы использовать команду `chown` для изменения UID файла внутри контейнера, а затем удалим сарability `cap_chown` и посмотрим, что получится. Команды и выходные данные — ниже (рис. 7.16).

```

run -dit --name test:latest
docker exec -it ubuntu sh
touch myfile
ls
ls -l myfile
chown 5000 myfile
ls -l myfile
exit
docker run -dit --name test1 --cap-drop cap_chown centos:latest
docker exec -it test1 sh
touch myfile
ls
ls -l myfile
chown 5000 myfile

```

```

docker0@docker0:~$ docker run -dit --name test centos:latest
04e34f554d060298e5371325aba4816a2a767f031fe72e99cd43d59c96ca20f8
docker0@docker0:~$ docker exec -it test sh
sh-4.4# touch myfile
sh-4.4# ls
bin dev etc home lib lib64 lost+found media mnt myfile opt proc root run sbin srv sys tmp
sh-4.4# ls -l myfile
-rw-r--r-- 1 root root 0 Jan 12 08:44 myfile
sh-4.4# chown 5000 myfile
sh-4.4# ls -l myfile
-rw-r--r-- 1 5000 root 0 Jan 12 08:44 myfile
sh-4.4# exit
exit
docker0@docker0:~$ docker run -dit --name test1 --cap-drop cap_chown centos:latest
b0d5e6d9f82a58130fba8956568f72c8179850b2103688603a018027594118cd
docker0@docker0:~$ docker exec -it test1 sh
sh-4.4# touch myfile
sh-4.4# ls
bin dev etc home lib lib64 lost+found media mnt myfile opt proc root run sbin srv sys tmp
sh-4.4# ls -l myfile
-rw-r--r-- 1 root root 0 Jan 12 08:50 myfile
sh-4.4# chown 5000 myfile
chown: changing ownership of 'myfile': Operation not permitted
sh-4.4#

```

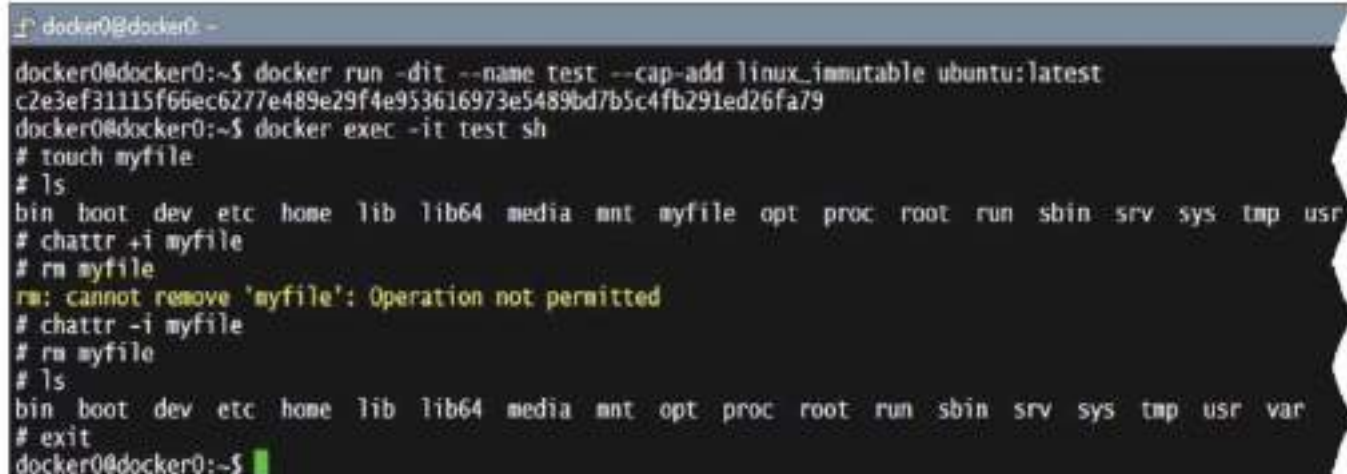
Рис. 7.16

Как и ожидалось, мы не можем выполнить операцию и сменить владельца.

Теперь добавим `seccomp`, не доступный в контейнере по умолчанию, и посмотрим, что будет. Выберем `seccomp` `linux_immutable`, который позволяет делать файл неизменяемым (`immutable`), так что даже пользователь `root` не сможет его удалить. Это хороший способ защитить файл от случайного удаления.

Создадим контейнер и добавим `seccomp` `linux_immutable`. Затем войдем в контейнер, создадим файл и изменим его атрибуты на `immutable` (неизменяемый) с помощью команды `chattr` (**change attribute** — изменить атрибут). После чего попытаемся удалить файл, но у нас ничего не выйдет, потому что файл *неизменяемый*. Удалим `test` `immutable` командой `chattr` и снова попробуем удалить файл. На этот раз все должно получиться (см. рис. 7.17).

```
docker run -dit --name test --cap-add linux_immutable ubuntu:latest
docker exec -it test sh
touch myfile
ls
chattr +i myfile
rm myfile
chattr -i myfile
rm myfile
ls
exit
```



```
docker0@docker0:~$ docker run -dit --name test --cap-add linux_immutable ubuntu:latest
c2e3ef31115f66ec6277e489e29f4e953616973e5489bd7b5c4fb291ed26fa79
docker0@docker0:~$ docker exec -it test sh
# touch myfile
# ls
bin boot dev etc home lib lib64 media mnt myfile opt proc root run sbin srv sys tmp usr
# chattr +i myfile
# rm myfile
rm: cannot remove 'myfile': Operation not permitted
# chattr -i myfile
# rm myfile
# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
# exit
docker0@docker0:~$
```

Рис. 7.17

Обязательный контроль доступа

*Обязательный контроль доступа*¹ (Mandatory Access Control, MAC, или Simplified Mandatory Access Control Kernel, SMACK) — это набор обязательных правил контроля доступа на уровне операционной системы, который защищает данные и про-

¹ Другие названия: «Мандатное управление доступом», «Принудительный контроль доступа». — *Ред.*

цессы от несанкционированного доступа или изменений пользователями и программами. С версии Linux 2.6.25 *обязательный контроль доступа* *встроен в операционную систему*.

В системах с поддержкой MAC системные администраторы могут реализовать по всей организации политику безопасности, которую невозможно изменить, в отличие от политик *дискреционного контроля доступа*¹ (Discretionary Access Policies, DAC).

В Linux для обеспечения обязательного контроля доступа обычно используются *AppArmor* или *SELinux*. Docker ожидает загрузку и применение политики AppArmor.

Политики применяются на уровне контейнера, а не демона. В Docker есть профиль *AppArmor* по умолчанию, известный как *default-profile*, который автоматически назначается всем создаваемым контейнерам. Профиль содержит набор умеренно строгих правил управления доступом. В версиях Docker до 1.13 этот профиль генерировался в каталоге `/etc/apparmor.d/docker`, но, начиная с версии 1.13, в последующих версиях исполняемый файл Docker генерирует профиль в `tmpfs`, а затем загружает его в ядро.

Прежде чем перейти к работе AppArmor и SELinux в контексте контейнеров, узнаем, как они работают в принципе.

Начнем с *AppArmor*. Это основной модуль безопасности ядра Linux, который позволяет ограничивать доступ пользователя или программы к ресурсам системы. Для этого создается набор профилей, который контролирует доступ пользователей и программ к ресурсам и возможности работы с ними. Это такие возможности (capabilities), как доступ к сети или сырым сокетам, а также разрешения на чтение, запись или выполнение файлов.

AppArmor предоставляется для дистрибутивов на основе Debian, таких как Ubuntu, Astra Linux, Kali Linux и т. д.; SELinux доступен на Fedora, RHEL, CentOS, Oracle Linux и других системах на основе RPM.

Давайте посмотрим, какие профили по умолчанию загружены в систему Ubuntu. Но сначала убедимся, что у нас установлены утилиты AppArmor:

```
apt install apparmor-utils
```

Далее запустим команду `apparmor_status`, чтобы увидеть профили, загруженные в систему по умолчанию. Как показано на рис. 7.18, загружено 16 профилей.

Обычно эти профили находятся в каталоге `/etc/apparmor.d`, как видно на рис. 7.19.

```
cd /etc/apparmor.d
ls
```

Профили по умолчанию предоставляют встроенный механизм безопасности, так что даже суперпользователи `root` должны проходить проверки обязательного

¹ Другие названия: «Избирательное управление доступом», «Разграничительное управление доступом». — *Ред.*

```

root@dockerubuntu: ~
root@dockerubuntu:~# apparmor_status
apparmor module is loaded.
16 profiles are loaded.
16 profiles are in enforce mode.
/sbin/dhclient
/usr/bin/lxc-start
/usr/bin/man
/usr/lib/NetworkManager/nm-dhcp-client.action
/usr/lib/NetworkManager/nm-dhcp-helper
/usr/lib/connman/scripts/dhclient-script
/usr/lib/snapd/snap-confine
/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/usr/sbin/tcpdump
docker-default
lxc-container-default
lxc-container-default-cgns
lxc-container-default-with-mounting
lxc-container-default-with-nesting
man_filter
man_groff
0 profiles are in complain mode.
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:~#

```

Рис. 7.18

```

root@dockerubuntu: /etc/apparmor.d
root@dockerubuntu:~# cd /etc/apparmor.d
root@dockerubuntu:/etc/apparmor.d# ls
abstractions  disable      local  lxc-containers  tunables      usr.bin.man      usr.sbin.rsyslogd
cache         force-complain  lxc    sbin.dhclient  usr.bin.lxc-start  usr.lib.snapd.snap-confine.real  usr.sbin.tcpdump
root@dockerubuntu:/etc/apparmor.d#

```

Рис. 7.19

контроля доступа к ресурсу. Если проверка не пройдена, ресурс остается недоступным.

Давайте проверим это. Возьмем простую команду Unix, например `useradd`, и создадим для нее профиль. Затем попытаемся создать пользователя с помощью команды `useradd` от имени `root` и посмотрим, что получится.

Но сначала рассмотрим несколько команд для AppArmor. Указав `aa-` и дважды нажав клавишу `<TAB>`, мы получим список доступных команд для AppArmor. С помощью этих команд можно создавать, включать, отключать и настраивать профиль AppArmor (см. рис. 7.20).

```

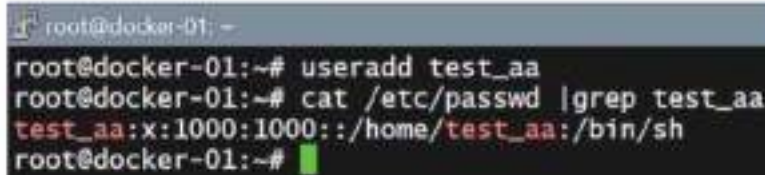
root@dockerubuntu: ~
root@dockerubuntu:~# aa-
aa-audit      aa-cleanprof  aa-decode    aa-enabled   aa-exec      aa-logprof      aa-remove-unknown  aa-unconfined
aa-autodep    aa-complain   aa-disable   aa-enforce   aa-genprof   aa-mergeprof    aa-status          aa-update-browser
root@dockerubuntu:~#

```

Рис. 7.20

Теперь создадим профиль для команды `useradd`. Но сначала попробуем от имени `root` создать пользователя командой `useradd`, и проверим, работает ли эта команда (см. рис. 7.21).

```
useradd test_aa
cat /etc/passwd |grep test_aa
```



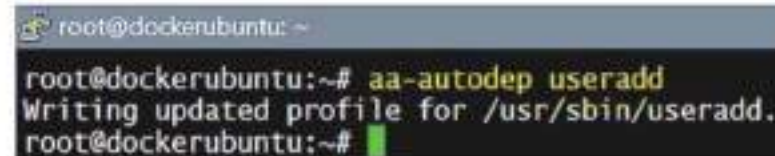
```
root@docker-01: ~
root@docker-01:~# useradd test_aa
root@docker-01:~# cat /etc/passwd |grep test_aa
test_aa:x:1000:1000:./home/test_aa:/bin/sh
root@docker-01:~#
```

Рис. 7.21

Как видим, команда `useradd` работает. Теперь используем `aa-autodep`, чтобы настроить профиль для команды `useradd`.

С помощью `aa-autodep` генерируется минимальный профиль AppArmor для набора исполняемых файлов (см. рис. 7.22).

```
aa-autodep useradd
```



```
root@dockerubuntu: ~
root@dockerubuntu:~# aa-autodep useradd
Writing updated profile for /usr/sbin/useradd.
root@dockerubuntu:~#
```

Рис. 7.22

Эта команда создает запись в каталоге `/etc/apparmor.d` для `-usr.sbin.useradd`, как показано на рис. 7.23. И к уже имеющимся 16 профилям добавляется еще один (см. рис. 7.24).

```
cd /etc/apparmor.d
ls
cd ~
aa-status
```

Мы видим, что теперь здесь действительно 17 профилей, причем один из них, тот, который мы только что создали, находится в режиме *complain* (жалоба). Режим *complain* означает, что доступ к ресурсам ограничен не будет, но система «пожалу-



```
root@dockerubuntu: /etc/apparmor.d
root@dockerubuntu:~# aa-autodep useradd
Writing updated profile for /usr/sbin/useradd.
root@dockerubuntu:~# clear
root@dockerubuntu:~# cd /etc/apparmor.d
root@dockerubuntu:/etc/apparmor.d# ls
abstractions disable local lxc-containers tunables usr.bin.man usr.sbin.rsyslogd usr.sbin.useradd
cache Force-complain lxc sbin.dhclient usr.bin.lxc-start usr.lib.snapd.snap-confine.real usr.sbin.tcpdump
root@dockerubuntu:/etc/apparmor.d#
```

Рис. 7.23

```

root@dockerubuntu:/etc/apparmor.d# cd ~
root@dockerubuntu:~# aa-status
apparmor module is loaded.
17 profiles are loaded.
16 profiles are in enforce mode.
  /sbin/dhclient
  /usr/bin/lxc-start
  /usr/bin/man
  /usr/lib/NetworkManager/nm-dhcp-client.action
  /usr/lib/NetworkManager/nm-dhcp-helper
  /usr/lib/connman/scripts/dhclient-script
  /usr/lib/snapd/snap-confine
  /usr/lib/snapd/snap-confine//mount-namespace-capture-helper
  /usr/sbin/tcpdump
docker-default
lxc-container-default
lxc-container-default-cgns
lxc-container-default-with-mounting
lxc-container-default-with-nesting
man_filter
man_groff
1 profiles are in complain mode.
  /usr/sbin/useradd
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:~#

```

Рис. 7.24

ется» и регистрирует в журнале *нарушение доступа*. Мы можем создавать пользователей, но при этом все наши действия будут зарегистрированы в журнале.

Попробуем создать еще одного пользователя и посмотрим, что получится (см. команду ниже и рис. 7.25).

```

useradd test_aal
cat /etc/passwd | grep test_aal

```

```

root@docker-01:~# useradd test_aal
root@docker-01:~# cat /etc/passwd |grep test_aal
test_aal:x:1001:1001::/home/test_aal:/bin/sh
root@docker-01:~#

```

Рис. 7.25

Итак, все получилось без проблем. Теперь используем `aa-enforce`, чтобы сделать профиль обязательным (см. рис. 7.26).

```

aa-enforce useradd
aa-status

```

Теперь все 17 профилей имеют *режим enforce* (принудительный), и нет ни одного профиля в режиме `complain`. Следовательно, как вы могли догадаться, сейчас мы не имеем возможности создать пользователя даже от имени `root`. Проверим это на практике (рис. 7.27).

```

useradd test-aa2

```

Так и есть: мы не смогли создать пользователя, даже как `root`.

```

root@dockerubuntu:/etc/apparmor.d# aa-enforce useradd
Setting /usr/sbin/useradd to enforce mode.
root@dockerubuntu:/etc/apparmor.d# aa-status
apparmor module is loaded.
17 profiles are loaded.
17 profiles are in enforce mode.
  /sbin/dhclient
  /usr/bin/lxc-start
  /usr/bin/man
  /usr/lib/NetworkManager/nm-dhcp-client.action
  /usr/lib/NetworkManager/nm-dhcp-helper
  /usr/lib/connman/scripts/dhclient-script
  /usr/lib/snapd/snap-confine
  /usr/lib/snapd/snap-confine//mount-namespace-capture-helper
  /usr/sbin/tcpdump
  /usr/sbin/useradd
  docker-default
  lxc-container-default
  lxc-container-default-cgns
  lxc-container-default-with-mounting
  lxc-container-default-with-nesting
  man_filter
  man_groff
0 profiles are in complain mode.
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:/etc/apparmor.d#

```

Рис. 7.26

```

root@dockerubuntu: ~
root@dockerubuntu:~# useradd test_aa2
Cannot open audit interface - aborting.
root@dockerubuntu:~#

```

Рис. 7.27

Можно использовать `aa-disable`, чтобы выгрузить профиль из ядра и предотвратить его загрузку при запуске AppArmor, или `aa-logprof` — чтобы просканировать профиль и разрешить создание новых пользователей только пользователям `root`. Если мы укажем `(A)llow` (разрешить) и `(S)ave` (сохранить изменения), пользователь `root` сможет создавать пользователей, как показано на рис. 7.28.

```

aa-logprof
(A)llow и (S)ave the changes

```

Теперь попробуем добавить пользователя и посмотрим выходные данные на рис. 7.29.

```

useradd test_aa2

```



```

root@dockerubuntu:~# aa-logprof
Reading log entries from /var/log/syslog.
Updating AppArmor profiles in /etc/apparmor.d.
Complain-mode changes:

Profile:   /usr/sbin/useradd
Capability: net_admin
Severity:  8

[1 - #include <abstractions/lxc/container-base>]
[2 - #include <abstractions/lxc/start-container>]
[3 - capability net_admin,
(A)llow / [(D)eny] / (I)gnore / Audi(t) / Abo(r)t / (F)inish
Adding #include <abstractions/lxc/container-base> to profile.
Deleted 2 previous matching profile entries.
Enforce-mode changes:

- Changed Local Profiles -

The following local profiles were changed. Would you like to save them?

[1 - /usr/sbin/useradd]
(S)ave Changes / Save Selec(t)ed Profile / [(V)iew Changes] / View Changes b/w (C)lean profiles / Abo(r)t
Writing updated profile for /usr/sbin/useradd.

```

Рис. 7.28

```

root@dockerubuntu:~# useradd test_aa2
root@dockerubuntu:~#

```

Рис. 7.29

Docker и AppArmor

Для каждого создаваемого контейнера Docker назначается профиль AppArmor по умолчанию. В версиях Docker до 1.13.0 профиль по умолчанию создавался в `/etc/apparmor.d/docker`. Начиная с версии 1.13.0, профиль создается в `tmpfs`, а затем загружается в ядро.

Профиль для Docker по умолчанию называется *docker-default*. Он содержит умеренно строгие правила и подходит для большинства приложений. Профиль *docker-default* загружается в ядро вместе с другими профилями. Рассмотрим профили, которые загружаются по умолчанию (см. рис. 7.30).

```
aa-status
```

Среди 16 профилей мы видим *docker-default*. Давайте создадим контейнер, выполним несколько команд и посмотрим, что получится (рис. 7.31).

```

docker run -dit --name test_profile busybox:latest
docker ps
aa-status

```

Мы видим, что для одного процесса (нашего контейнера) определен профиль, а также видим профиль *docker-default* с PID 4261. Давайте создадим еще один контейнер, явно указав профиль *docker-default* с помощью флага `--security-opt`.

```

docker run -dit --name test_profile1 --security-opt apparmor=docker-default ubuntu:latest

```

```

root@dockerubuntu:~# aa-status
apparmor module is loaded.
16 profiles are loaded.
16 profiles are in enforce mode.
/sbin/dhclient
/usr/bin/lxc-start
/usr/bin/man
/usr/lib/NetworkManager/nm-dhcp-client.action
/usr/lib/NetworkManager/nm-dhcp-helper
/usr/lib/connman/scripts/dhclient-script
/usr/lib/snapd/snap-confine
/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/usr/sbin/tcpdump
docker-default
lxc-container-default
lxc-container-default-cgns
lxc-container-default-with-mounting
lxc-container-default-with-nesting
man_filter
man_groff
0 profiles are in complain mode.
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:~#

```

Рис. 7.30

```

root@dockerubuntu:~# docker run -dit --name test_profile busybox:latest
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
b4bbbaa22dec6: Pull complete
Digest: sha256:6915be4043561d64e0ab0f8f098dc2ac48e077fe23f488ac24b665166898115a
Status: Downloaded newer image for busybox:latest
c74f04bc0bbf37feb409df0b7a8ec8920ac09248636316043c2aeddbe8cd2ae9
root@dockerubuntu:~# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
c74f04bc0bbf        busybox:latest     "sh"               13 seconds ago     Up 12 seconds      0.0.0.0:2375->0.0.0.0:2375   test_profile
root@dockerubuntu:~# aa-status
apparmor module is loaded.
16 profiles are loaded.
16 profiles are in enforce mode.
/sbin/dhclient
/usr/bin/lxc-start
/usr/bin/man
/usr/lib/NetworkManager/nm-dhcp-client.action
/usr/lib/NetworkManager/nm-dhcp-helper
/usr/lib/connman/scripts/dhclient-script
/usr/lib/snapd/snap-confine
/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/usr/sbin/tcpdump
docker-default
lxc-container-default
lxc-container-default-cgns
lxc-container-default-with-mounting
lxc-container-default-with-nesting
man_filter
man_groff
0 profiles are in complain mode.
1 processes have profiles defined.
1 processes are in enforce mode.
  docker-default (4261)
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:~#

```

Рис. 7.31

Теперь снова проверим статус профилей (рис. 7.32).

```
aa-status
```

Мы видим два активных профиля `docker-default` в режиме `enforce`.

```
root@dockerubuntu:~# docker run -dit --name test_profile1 --security-opt apparmor=docker-default ubuntu:latest
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
5c939e3a4d10: Pull complete
c63719cdbe7a: Pull complete
19a861ea6baf: Pull complete
651c9d2d6c4f: Pull complete
Digest: sha256:8d31dad0c58f552e890d68bbfb735588b6b820a46e459672d96e585871acc110
Status: Downloaded newer image for ubuntu:latest
8ea07f1194349c3276db5f4b2ce61044077c9894d548b79a45a20be86df6a161
root@dockerubuntu:~# aa-status
apparmor module is loaded.
16 profiles are loaded.
16 profiles are in enforce mode.
/sbin/dhclient
/usr/bin/lxc-start
/usr/bin/man
/usr/lib/NetworkManager/nm-dhcp-client.action
/usr/lib/NetworkManager/nm-dhcp-helper
/usr/lib/connman/scripts/dhclient-script
/usr/lib/snapd/snap-confine
/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/usr/sbin/tcpdump
docker-default
lxc-container-default
lxc-container-default-cgns
lxc-container-default-with-mounting
lxc-container-default-with-nesting
man_filter
man_groff
0 profiles are in complain mode.
2 processes have profiles defined.
2 processes are in enforce mode.
  docker-default (4261)
  docker-default (4482)
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:~#
```

Рис. 7.32

Можно создать собственный профиль, чтобы прикреплять его к создаваемым контейнерам. Для проверки давайте создадим профиль с именем `myprofile` в каталоге `/etc/apparmor.d`, который не позволяет чтение, запись и выполнение в каталоге `/etc`. Сохраним файл.

```
cd /etc/apparmor.d
vi myprofile
profile myprofile1 flags=(attach_disconnected,mediate_deleted) {
    #include <abstractions/base>
    file,
    umount,
deny /etc/** wrx,
}
```

Теперь загрузим профиль и проверим, получим ли мы 17 профилей (рис. 7.33).

```
apparmor_parser -r -W /etc/apparmor.d/myprofile
aa-status
```



```

root@docker-01:/etc/apparmor.d# apparmor_parser -r -W /etc/apparmor.d/myprofile
root@docker-01:/etc/apparmor.d# aa-status
apparmor module is loaded.
17 profiles are loaded.
17 profiles are in enforce mode.
  /sbin/dhclient
  /usr/bin/lxc-start
  /usr/bin/man
  /usr/lib/NetworkManager/nm-dhcp-client.action
  /usr/lib/NetworkManager/nm-dhcp-helper
  /usr/lib/connman/scripts/dhclient-script
  /usr/lib/snapd/snap-confine
  /usr/lib/snapd/snap-confine//mount-namespace-capture-helper
  /usr/sbin/tcpdump
  docker-default
  lxc-container-default
  lxc-container-default-cgns
  lxc-container-default-with-mounting
  lxc-container-default-with-nesting
  man_filter
  man_groff
  myprofile
0 profiles are in complain mode.
1 processes have profiles defined.
1 processes are in enforce mode.
  docker-default (4019)
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.

```

Рис. 7.33

Как видим, все 17 профилей загружены. Давайте создадим контейнер, к которому будет привязан наш профиль (см. команды ниже и рис. 7.34).

```

docker run -dit --name test_myprofile --security-opt "apparmor=myprofile"
alpine:latest

```

```

root@docker-01:/etc/apparmor.d# docker run -dit --name test_myprofile --security-opt "apparmor=myprofile" alpine:latest
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
c9b1b535fdd9: Pull complete
Digest: sha256:ab00606a42621fb68f2ed6ad3c88be54397f981a7b70a79db3d1172b11c4367d
Status: Downloaded newer image for alpine:latest
683b487da0e57c54e1dedbf53cc293701b41b7e7588e75fee21ab0f0d8069b4a
root@docker-01:/etc/apparmor.d#

```

Рис. 7.34

Если использовать `aa-status`, мы получим два контейнера с профилем `docker-default` и третий контейнер с созданным нами профилем — `myprofile`. На рис. 7.35 все довольно очевидно.

Пока все идет хорошо. Заглянем в контейнер и посмотрим, работает ли наш профиль `myprofile` (см. рис. 7.36).

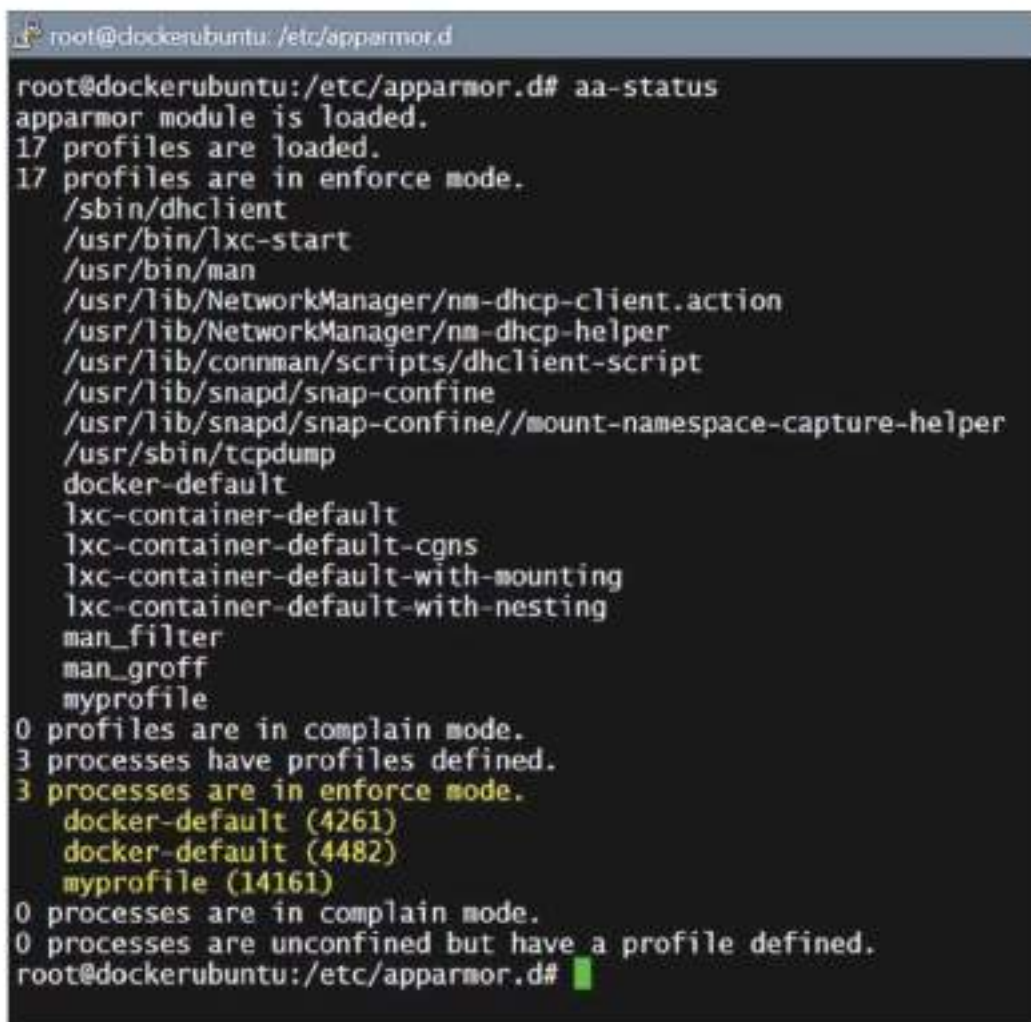
```

docker ps
docker exec -it test_myprofile sh
ls
cd /etc
mkdir test

```

```
cd ~
cd /dev
mkdir test
ls
```

Как видите, мы не можем создать каталог в каталоге `/etc` внутри контейнера. Выдано сообщение: **Permission denied** (В разрешении отказано), потому что мы создали



```
root@dockerubuntu: /etc/apparmor.d
root@dockerubuntu:/etc/apparmor.d# aa-status
apparmor module is loaded.
17 profiles are loaded.
17 profiles are in enforce mode.
/sbin/dhclient
/usr/bin/lxc-start
/usr/bin/man
/usr/lib/NetworkManager/nm-dhcp-client.action
/usr/lib/NetworkManager/nm-dhcp-helper
/usr/lib/connman/scripts/dhclient-script
/usr/lib/snapd/snap-confine
/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/usr/sbin/tcpdump
docker-default
lxc-container-default
lxc-container-default-cgns
lxc-container-default-with-mounting
lxc-container-default-with-nesting
man_filter
man_groff
myprofile
0 profiles are in complain mode.
3 processes have profiles defined.
3 processes are in enforce mode.
  docker-default (4261)
  docker-default (4482)
  myprofile (14161)
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
root@dockerubuntu:/etc/apparmor.d#
```

Рис. 7.35



```
root@docker-01:~# docker exec -it test_myprofile sh
/ # ls
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
/ # cd /etc
/etc # mkdir test
mkdir: can't create directory 'test': Permission denied
/etc # cd ~
/root # cd /dev
/dev # mkdir test
/dev # ls
console  fd          mqueue  ptmx       random    stderr     stdout     tty         zero
core     full        null     pts        shm        stdin      test       urandom
/dev #
```

Рис. 7.36

пользовательский профиль AppArmor, который не позволяет производить чтение, запись и выполнение в каталоге `/etc`. Но если мы попытаемся создать каталог внутри `/dev` или в любом другом каталоге в контейнере, у нас все получится. Попробуйте сами.

Docker и SELinux

SELinux — еще один механизм для защиты контейнеров Docker, предназначенный для RHEL, CentOS, Fedora, Oracle Linux и подобных систем.

SELinux помечает каждый файл, каталог, процесс, сеть, порт и т. д. Для управления доступом помеченного процесса к помеченному объекту, например к файлу, написан ряд правил. Мы называем это политикой безопасности. Соблюдение этих правил обеспечивает ядро.

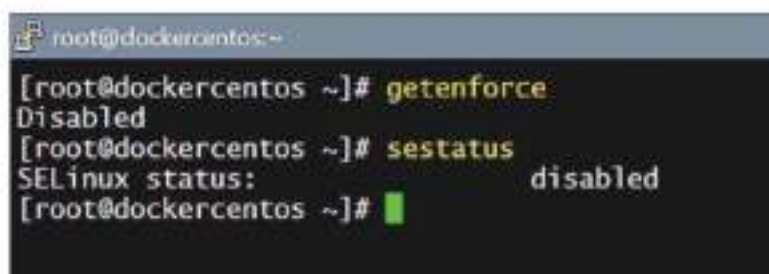
SELinux применяется на уровне демона. Ядро Linux — это центральный компонент операционных систем Linux. Оно отвечает за безопасность, управление ресурсами системы, взаимодействие между аппаратным и программным обеспечением. Ядро играет важную роль в поддержке безопасности на более высоких уровнях.

Система меток в SELinux управляет доступом, который ядро предоставляет отдельным процессам. Правила определяют разрешения на основе метки процесса и метки объекта.

SELinux кеширует решения о предоставлении или отклонении доступа, чтобы оптимизировать проверку правил и тем самым повысить производительность системы. Этот кеш называется *Access Vector Cache, AVC* (кеш векторного доступа).

SELinux не выполняет проверку, если доступ уже отклонен в соответствии с политикой дискреционного контроля доступа (*Discretionary Access Control, DAC*).

Теперь давайте войдем в систему Linux и проверим статус SELinux, используя команду `getenforce` или `sestatus` (см. рис. 7.37).



```

root@dockercentos:~# getenforce
Disabled
root@dockercentos:~# sestatus
SELinux status: disabled
root@dockercentos:~#
  
```

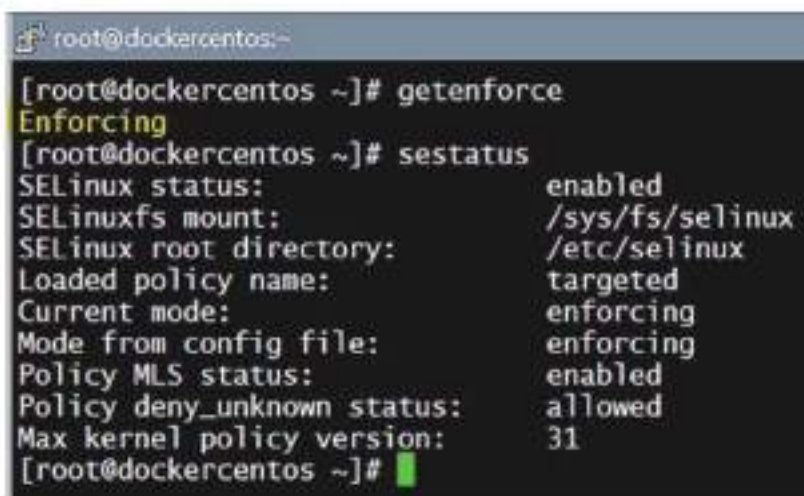
Рис. 7.37

Мы видим, что SELinux отключен (*disabled*). Давайте включим его. Внесем изменения в файл конфигурации в `/etc/selinux`, добавив слово *enforcing* (обязательное исполнение) в строку `SELinux=`. Есть еще два варианта: *permissive* (необязательное исполнение) и *disabled* (отключено).

```
vi /etc/selinux/config
```


Чтобы изменения вступили в силу, нужно перезапустить систему. После перезапуска мы видим, что SELinux включен (см. рис. 7.38).

```
reboot
sestatus
```

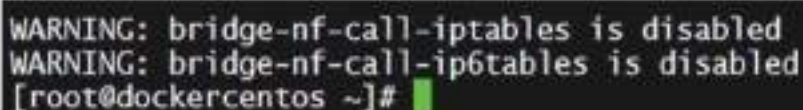


```
root@dockercentos:~#
[root@dockercentos ~]# getenforce
Enforcing
[root@dockercentos ~]# sestatus
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:       /etc/selinux
Loaded policy name:            targeted
Current mode:                  enforcing
Mode from config file:         enforcing
Policy MLS status:             enabled
Policy deny_unknown status:    allowed
Max kernel policy version:     31
[root@dockercentos ~]#
```

Рис. 7.38

После включения SELinux в системе нужно выполнить `systemctl restart docker` для перезапуска Docker, потому что это не произойдет автоматически.

Выполнив команду `docker info`, мы получаем предупреждение. Здесь не о чем волноваться — это можно легко исправить, как видно на рис. 7.39.



```
WARNING: bridge-nf-call-iptables is disabled
WARNING: bridge-nf-call-ip6tables is disabled
[root@dockercentos ~]#
```

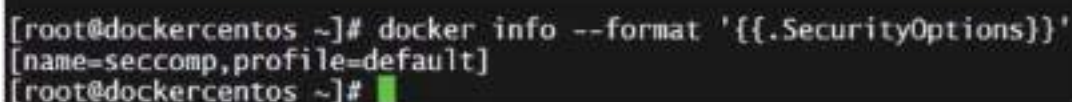
Рис. 7.39

Нужно просто внести изменения в файл `/etc/sysctl.conf`, добавив следующие строки:

```
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
```

Затем нужно запустить команду `sysctl -p`, чтобы изменения вступили в силу. Теперь `docker info` выдает выходные данные без каких-либо предупреждений. Однако *SELinux не включен в демоне Docker по умолчанию*. Это нужно сделать явно. Если мы проверим Docker, то увидим, что SELinux не включен; записи для SELinux отсутствуют (см. рис. 7.40).

```
docker info --format '{{.SecurityOptions}}'
```

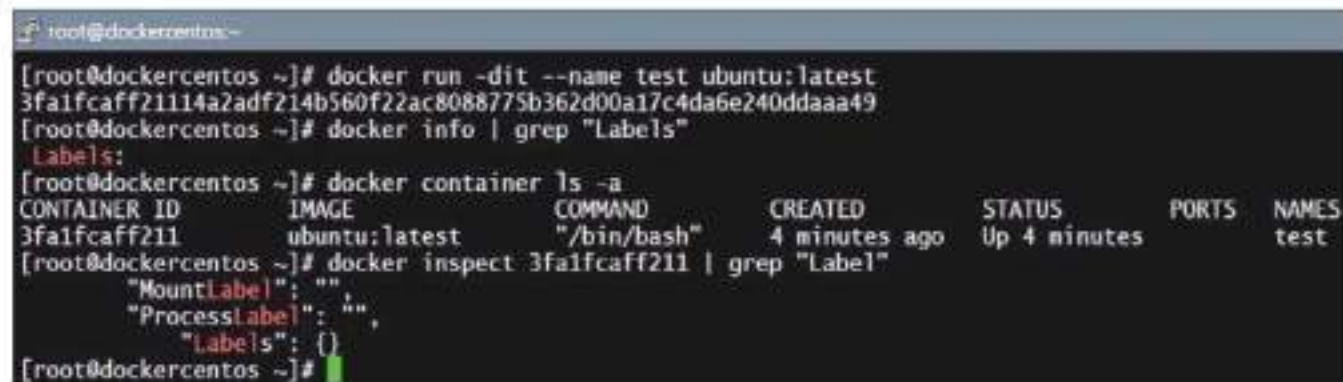


```
[root@dockercentos ~]# docker info --format '{{.SecurityOptions}}'
[name=seccomp,profile=default]
[root@dockercentos ~]#
```

Рис. 7.40

Если мы посмотрим раздел **Labels** (метки) в выходных данных команды `docker info`, то увидим, что никакие метки не были созданы. Это важно отметить сейчас, потому что мы выполним ту же команду после включения SELinux в Docker. Давайте, кроме того, создадим контейнер и проверим его метки. Внимательно посмотрим на рис. 7.41.

```
docker run -dit --name test ubuntu:latest
docker info | grep "Labels"
docker inspect <id_контейнера> | grep "Labels"
```



```
root@dockercentos:~# docker run -dit --name test ubuntu:latest
3fa1fcaff21114a2adf214b560f22ac8088775b362d00a17c4da6e240ddaaa49
root@dockercentos:~# docker info | grep "Labels"
Labels:
root@dockercentos:~# docker container ls -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
3fa1fcaff211        ubuntu:latest       "/bin/bash"        4 minutes ago       Up 4 minutes                test
root@dockercentos:~# docker inspect 3fa1fcaff211 | grep "Label"
  "MountLabel": "",
  "ProcessLabel": "",
  "Labels": {}
root@dockercentos:~#
```

Рис. 7.41

Мы видим, что у контейнера нет меток. А, как нам известно, *система меток* — это ключевой компонент SELinux.

Давайте включим SELinux в Docker: изменим файл `/etc/docker/daemon.json` и добавим следующие строки; или, если файл `daemon.json` по этому пути отсутствует, создадим его, введем следующее:

```
{
  "selinux-enabled": true
}
```

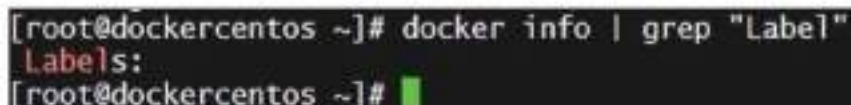
После внесения изменений нужно перезапустить Docker командой `systemctl restart docker`, чтобы изменения вступили в силу. Мы видим, что в параметры безопасности (**SecurityOptions**) добавлена запись `name=selinux` (рис. 7.42).



```
root@dockercentos:~# systemctl restart docker
root@dockercentos:~# docker info --format '{{.SecurityOptions}}'
name=seccomp,profile=default name=selinux
root@dockercentos:~#
```

Рис. 7.42

Однако если мы запусти команду `docker info | grep "Label"`, то не увидим никаких записей относительно меток (рис. 7.43).

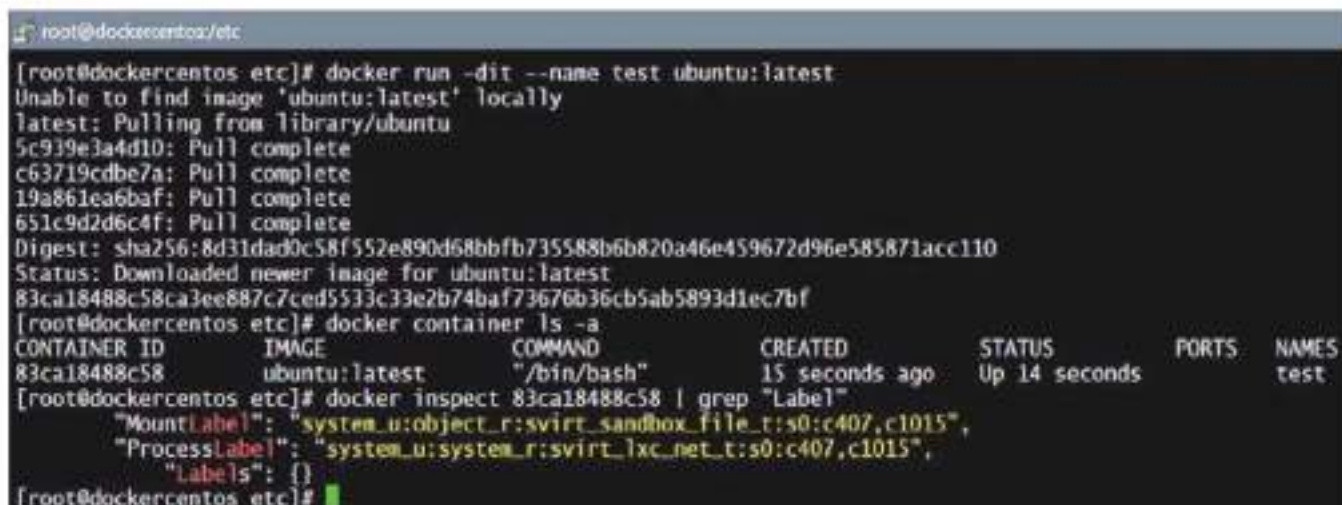


```
[root@dockercentos:~]# docker info | grep "Label"
Labels:
[root@dockercentos:~]#
```

Рис. 7.43

Теперь давайте создадим контейнер и посмотрим, что изменится после включения SELinux (см. команды ниже и рис. 7.44).

```
docker run -dit --name test ubuntu:latest
docker container ls -a
docker inspect <id_контейнера> | grep "Label"
```



```
root@dockercentos:/etc
[root@dockercentos etc]# docker run -dit --name test ubuntu:latest
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
5c939e3a4d10: Pull complete
c63719cdbe7a: Pull complete
19a861ea6baf: Pull complete
651c9d2d6c4f: Pull complete
Digest: sha256:8d31dad0c58f552e890d68bbfb735588b6b820a46e459672d96e585871acc110
Status: Downloaded newer image for ubuntu:latest
83ca18488c58ca3ee887c7ced5533c33e2b74baf73676b36cb5ab5893d1ec7bf
[root@dockercentos etc]# docker container ls -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
83ca18488c58        ubuntu:latest      "/bin/bash"         15 seconds ago     Up 14 seconds      0.0.0.0:22->22      test
[root@dockercentos etc]# docker inspect 83ca18488c58 | grep "Label"
  "MountLabel": "system_u:object_r:svirt_sandbox_file_t:s0:c407,c1015",
  "ProcessLabel": "system_u:system_r:svirt_lxc_net_t:s0:c407,c1015",
  "Labels": {}
[root@dockercentos etc]#
```

Рис. 7.44

Теперь в контейнере есть метки, потому что мы включили SELinux в системе, и Docker его использует. Мы видим, что метки представлены в виде списка с разделителем двоеточием в формате «*пользователь SELinux: роль SELinux: тип: уровень конфиденциальности: уникальная категория*».

Попробуем разобраться, что все это означает. Когда мы создаем контейнер с включенным SELinux, SELinux помечает его, указывая пользователя, роль, тип, уровень конфиденциальности и набор из двух случайных чисел от 0 до 1023, которые должны быть уникальными. Эти числа имеют префикс «с», что означает «категория» (category); SELinux использует уровень конфиденциальности s0. Этот механизм называется *мультикатегорийной безопасностью* (Multi-Category Security, MCS). На рис. 7.45 приводится таблица преобразования мультикатегорийной безопасности (MCS) для SELinux.

```
cat /etc/selinux/targeted/setrans.conf
```

Из этой таблицы мы можем почерпнуть полезную информацию об использовании механизма мультикатегорийной безопасности. Но в общем мы можем применять любой механизм разделения объектов на категории, как показано на рис. 7.46.

```
docker run -dit --name max --security-opt label=level:s0:c0,c3 busybox
docker container ls -a
docker inspect <id_контейнера> | grep "Label"
```

Такой подход помогает разделить контейнеры для обеспечения дополнительной безопасности. Разделение посредством SELinux позволяет, например, защитить процесс и содержимое одного контейнера от атак со стороны процесса другого контейнера.


```
# Multi-Category Security translation table for SELinux
#
# Uncomment the following to disable translation library
# disable=1
#
# Objects can be categorized with 0-1023 categories defined by the admin.
# Objects can be in more than one category at a time.
# Categories are stored in the system as c0-c1023. Users can use this
# table to translate the categories into a more meaningful output.
# Examples:
# s0:c0=CompanyConfidential
# s0:c1=PatientRecord
# s0:c2=Unclassified
# s0:c3=TopSecret
# s0:c1,c3=CompanyConfidentialRedHat
s0=SystemLow
s0-s0:c0.c1023=SystemLow-SystemHigh
```

Рис. 7.45

```
[root@dockercentos targeted]# docker run -dit --name max --security-opt label=level:s0:c0,c3 busybox
cd21d37722ec322185ba9d1cd2d6d8ea57434a69eb3bee8b5895dd807e3e09f5
[root@dockercentos targeted]# docker container ls -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
cd21d37722ec        busybox            "sh"               27 seconds ago     Up 26 seconds      0.0.0.0:80->80      max
[root@dockercentos targeted]# docker inspect cd21d37722ec | grep "Label"
  "MountLabel": "system_u:object_r:svirt_sandbox_file_t:s0:c0,c3",
  "ProcessLabel": "system_u:system_r:svirt_lxc_net_t:s0:c0,c3",
  "Labels": {}
[root@dockercentos targeted]#
```

Рис. 7.46

Typ определяет разрешения и привилегии пользователя. Например, для *ProcessLabel* определен тип *svirt_lxc_net_t*, с которым связан набор разрешений. Это разрешения, используемые Docker, потому что Docker связан с типом *svirt_lxc_net_t*.

В рамках набора привилегий Docker получает доступ к */usr/var/* и другим каталогам, а также полный доступ к объектам с меткой *svirt_sandbox_file_t*. Это значит, что *svirt_lxc_net_t* может записывать данные в файлы с меткой *svirt_sandbox_file_t*.

Итак, SELinux — это отличный механизм для применения *обязательного контроля доступа* (MAC), который изолирует среды выполнения контейнеров и обеспечивает их защиту. Если SELinux вызывает проблемы, его можно переключить в *режим необязательного исполнения* (permissive), при котором нарушения будут регистрироваться в журнале без блокирования действий.

Seccomp

Secure Computing Mode (безопасный режим вычислений), или *Seccomp* — это функция ядра, которая позволяет ограничить системные вызовы от контейнера. Эта функция доступна при выполнении двух условий.

1. В Docker встроен seccomp.
2. Конфигурация ядра включает CONFIG_SECCOMP.

Наличие `CONFIG_SECCOMP` легко проверить, запустив следующую команду (см. рис. 7.47).

```
grep CONFIG_SECCOMP= /boot/config-$(uname -r)
```



```
root@dockercentos:~#
[root@dockercentos ~]# grep CONFIG_SECCOMP= /boot/config-$(uname -r)
CONFIG_SECCOMP=y
[root@dockercentos ~]#
```

Рис. 7.47

Профиль по умолчанию для Docker блокирует 44 системных вызова из более чем 300. Он не строгий, зато совместим со множеством приложений. На странице <https://docs.docker.com/engine/security/seccomp/> приводится таблица вызовов, которые блокируются профилем по умолчанию.

Заключение

В этой главе мы рассмотрели различные функции безопасности ядра Linux, которые можно использовать для Docker; обсудили модули безопасности Linux — AppArmor и SELinux, обеспечивающие обязательный контроль доступа (MAC); получили представление о пространствах имен ядра, контрольных группах (cgroup), capabilities и seccomp и научились использовать эти функции для защиты и изоляции контейнеров.

Основные тезисы

- ◆ Linux предоставляет ряд функций для защиты контейнеров.
- ◆ Пространства имен изолируют сегмент операционной системы для каждого контейнера.
- ◆ Контрольные группы (cgroups) — это механизм объединения процессов на основе потребления ими ресурсов, приоритизации, стабильности и в некоторой степени безопасности.
- ◆ Начиная с версии ядра Linux 2.2 привилегии и разрешения для суперпользователя, или root, выделены в отдельный набор capabilities (возможностей), который можно включать и отключать независимо для каждого потока.
- ◆ Модули безопасности Linux, AppArmor и SELinux, служат в качестве промежуточного слоя между операционной системой и механизмами обеспечения безопасности, повышая защищенность контейнеров.
- ◆ Secure Computing Mode (Seccomp) — это функция ядра, которая позволяет ограничить системные вызовы от контейнера.

Контрольный тест

Выберите подходящий вариант ответа:

1. Сколько пространств имен ядра использует Docker на Linux?
 - 4.
 - 5.
 - 6.
 - 7.
2. Благодаря изоляции пространства имен мы можем сопоставлять с одним и тем же портом несколько контейнеров на уровне контейнеров, если они используют разные порты на уровне хоста. О каком пространстве имен здесь идет речь?
 - PID.
 - NET.
 - Ядро.
 - IPC.
3. Для каких операционных систем обычно используется AppArmor?
 - RHEL.
 - CentOS.
 - Fedora.
 - Ubuntu.
4. На каком уровне загружается SELinux?
 - Демон.
 - Контейнер.
 - Swarm.
 - Хост.
5. При создании контейнера большинство встроенных capabilities Linux...
 - Изменены.
 - Недоступны.
 - Доступны.
 - Ограничены.
6. Какую задачу выполняет SELinux?
 - Повышение производительность системы.
 - Ограничение системных вызовов.
 - Ограничение вызовов пользователей.
 - Метка объектов, файлов, сетей и т. д.

7. Отказ в предоставлении доступа, в соответствии с политикой дискреционного контроля доступа (DAC), имеет приоритет...
 - Над AppArmor.
 - Перед пользователем root.
 - Над SELinux.
 - Перед привилегированным пользователем.
8. Docker позволяет контролировать потребление ресурсов...
 - Сетей и ЦП.
 - Памяти и ЦП.
 - Сетей и памяти.
 - Памяти и пользователей.

Ответы

1	c
2	b
3	d
4	a

5	b
6	d
7	c
8	b

Вопросы

1. Какие варианты обеспечения безопасности предоставляются для Docker операционной системой?
2. Что такое пространства имен ядра?
3. В чем заключается концепция контрольных групп (cgroups)?
4. Что такое обязательный контроль доступа (MAC) и чем он отличается от дискреционного контроля доступа (DAC)?
5. Что такое `seccomp`?
6. В чем заключается концепция `capabilities`?
7. Как контейнеры Docker работают с `capabilities`?
8. Какие преимущества дает SELinux контейнерам Docker?
9. Как настроить профиль AppArmor для контейнера?
10. Какими способами контролируется потребление ресурсов памяти и ЦП контейнером?

Ключевые термины

- ◆ Пространства имен ядра
- ◆ Контрольные группы
- ◆ Обязательный контроль доступа
- ◆ Дискреционный контроль доступа доступом
- ◆ Capabilities
- ◆ Seccomp
- ◆ AppArmor
- ◆ SELinux
- ◆ Изоляция
- ◆ Разделение

ГЛАВА 8

Безопасность в Docker.

Часть 2

Введение

В этой главе мы рассмотрим функции безопасности, доступные в Docker Enterprise Edition (Docker EE), и способы их эффективного использования для защиты контейнеров.

Вы узнаете, как установить Docker Enterprise Edition с Universal Control Plane и Docker Trusted Registry и как использовать функции безопасности в Docker Enterprise Edition, в частности, сканирование на наличие уязвимостей, а также управление доступом на основе ролей.

Темы для изучения следующие.

- ◆ Docker Enterprise Edition
- ◆ Установка Docker Enterprise Edition
- ◆ Функции безопасности Docker Enterprise Edition
- ◆ Управление доступом на основе ролей

Docker Enterprise Edition

Это пакет для критически важных приложений и разработок корпоративного уровня. Он включает Docker Engine Enterprise, Docker Desktop Enterprise, защищенный реестр образов и расширенную панель управления.

13 ноября 2019 года компания Mirantis Inc. приобрела платформу Docker Enterprise вместе со всеми продуктами, клиентами и сотрудниками. См. <https://www.docker.com/faq-for-docker-enterprise-customers-and-partners>.

Docker Enterprise включает три основных компонента:

1. Docker Engine Enterprise.
2. Docker Trusted Registry.
3. Universal Control Plane.

Docker Engine Enterprise обеспечивает создание образов и выполнение контейнеров.

Universal Control Plane (UCP) управляет оркестраторами, такими как Docker Swarm и Kubernetes, и помогает в развертывании приложений.

Docker Trusted Registry (DTR) — это реестр корпоративного уровня для хранения образов.

Установка Docker Enterprise Edition

Для установки Docker Enterprise Edition требуется лицензия. На странице <https://hub.docker.com/editions/enterprise/docker-ee-trial/trial> можно оформить пробную лицензию. На момент написания книги (2019 год) пробная лицензия предоставлялась на месяц. В качестве операционной системы мы будем использовать Ubuntu. Прежде всего необходимо загрузить ключ лицензии и сохранить его в защищенном месте.

Поскольку Enterprise Edition работает в режиме Swarm, нам понадобятся минимум два сервера на Linux — один для управляющего и один для рабочего узла. Мы установим Docker Enterprise Edition на оба сервера, UCP на один, а DTR — на другой.

Нужно перейти на сайт hub.docker.com и скопировать URL-адрес для загрузки ПО, как показано на рис. 8.1.

Затем вставить скопированный URL-адрес (рис. 8.2).



Рис. 8.1

Далее выполняем следующие действия.

Если в системе запущены предыдущие версии Docker, например *docker.io* или *docker-engine*, сначала их нужно удалить:

```
apt-get remove docker docker-engine docker-ce docker-ce-cli docker.io
apt-get update
```

```

root@docker-01:~#
root@docker-01:~# wget https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a
--2020-02-17 08:50:14-- https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a
Resolving storebits.docker.com (storebits.docker.com)... 99.86.58.107, 99.86.58.20, 99.86.58.46, ...
Connecting to storebits.docker.com (storebits.docker.com)|99.86.58.107|:443... connected.
HTTP request sent, awaiting response... 307 Temporary Redirect
Location: /ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ [following]
--2020-02-17 08:50:14-- https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/
Reusing existing connection to storebits.docker.com:443.
HTTP request sent, awaiting response... 200 OK
Length: 1101 (1.1K) [text/html]
Saving to: 'sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a'

sub-1c10b54e-3021-46e6-a0c0-2ce2519b 100%[=====>] 1.08K --.-KB/s 1n 0s
2020-02-17 08:50:15 (79.7 MB/s) - 'sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a' saved [1101/1101]

root@docker-01:~#

```

Рис. 8.2

Теперь установим пакеты, чтобы утилиты APT (Advanced Packaging Tool — улучшенный инструмент для упаковки) могли использовать репозиторий по протоколу *HTTPS* (HyperText Transfer Protocol Secure — безопасный протокол передачи гипертекста).

```

apt-get install apt-transport-https ca-certificates\
curl software-properties-common -y

```

Настраиваем среду следующим образом:

```

DOCKER_EE_URL=https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-
46e6-a0c0-2ce2519b572a
DOCKER_EE_VERSION=19.03

```

Двигаемся дальше и добавляем официальный ключ GPG для Docker следующей командой (см. рис. 8.3).

```

curl -fsSL "${DOCKER_EE_URL}/ubuntu/gpg" | sudo apt-key add -

```

```

root@docker-01:~#
root@docker-01:~# DOCKER_EE_URL=https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a
root@docker-01:~# DOCKER_EE_VERSION=19.03
root@docker-01:~# curl -fsSL "${DOCKER_EE_URL}/ubuntu/gpg" | sudo apt-key add -
OK

```

Рис. 8.3

Убедимся, что используем нужный ключ, выполнив поиск по последним 8 символам отпечатка DD91 1E99 5A64 A202 E859 07D6 BC14 F10B 6D08 5F96:

```

apt-key fingerprint 6D085F96

```

Следующая команда добавляет стабильный репозиторий, необходимый для установки Docker Enterprise Edition.

```

add-apt-repository \
    "deb [arch=$(dpkg --print-architecture)] $DOCKER_EE_URL/ubuntu \
    $(lsb_release -cs) \
    stable-$DOCKER_EE_VERSION"

```

Смотрим, что получилось, на рис. 8.4.


```

root@docker-01:~# apt-key fingerprint 60085F36
pub  rsa4096 2017-02-22 [SCEA]
    0091 1E99 5A64 A202 E859 0706 BC14 F108 6008 5F36
uid      [ unknown] Docker Release (EE deb) <docker@docker.com>
sub  rsa4096 2017-02-22 [S]

root@docker-01:~# add-apt-repository \
> "deb [arch=$(dpkg --print-architecture)] $DOCKER_EE_URL/ubuntu \
> $(lsb_release -cs) \
> stable-$DOCKER_EE_VERSION"
Get:1 http://mirrors.digitalocean.com/ubuntu bionic InRelease [242 kB]
Hit:2 http://mirrors.digitalocean.com/ubuntu bionic-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu bionic-security InRelease
Hit:4 http://mirrors.digitalocean.com/ubuntu bionic-backports InRelease
Get:5 https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic InRelease [116 kB]
Get:6 https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-19.03 amd64 Packages [3919 B]
Fetchd 362 kB in 2s (230 kB/s)
Reading package lists... Done
root@docker-01:~#

```

Рис. 8.4

Далее обновляем индекс пакетов apt:

```
apt-get update
```

После этого устанавливаем последнюю версию Docker Engine Enterprise и containerd:

```
apt-get install docker-ee docker-ee-cli containerd.io -y
```

Теперь посмотрим список доступных версий в репозитории:

```
apt-cache madison docker-ee
```

Из списка выберем одну версию, как показано на рис. 8.5.

```
apt-get install docker-ee=5:19.03.5~3-0~ubuntu-bionic\
docker-ee-cli=5:19.03.5~3-0~ubuntu-bionic containerd.io
```

```

root@docker-01:~# apt-cache madison docker-ee
docker-ee | 5:19.03.5~3-0~ubuntu-bionic | https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-1
9.03 amd64 Packages
docker-ee | 5:19.03.4~3-0~ubuntu-bionic | https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-1
9.03 amd64 Packages
docker-ee | 5:19.03.3~3-0~ubuntu-bionic | https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-1
9.03 amd64 Packages
docker-ee | 5:19.03.2~3-0~ubuntu-bionic | https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-1
9.03 amd64 Packages
docker-ee | 5:19.03.1~3-0~ubuntu-bionic | https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-1
9.03 amd64 Packages
docker-ee | 5:19.03.0~3-0~ubuntu-bionic | https://storebits.docker.com/ee/ubuntu/sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a/ubuntu bionic/stable-1
9.03 amd64 Packages
root@docker-01:~# sudo apt-get install docker-ee=5:19.03.5~3-0~ubuntu-bionic\
> docker-ee-cli=5:19.03.5~3-0~ubuntu-bionic containerd.io
Reading package lists... Done
Building dependency tree
Reading state information... Done
containerd.io is already the newest version (1.2.6-3).
docker-ee-cli is already the newest version (5:19.03.5~3-0~ubuntu-bionic).
docker-ee is already the newest version (5:19.03.5~3-0~ubuntu-bionic).
The following package was automatically installed and is no longer required:
  grub-pc-bin
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 98 not upgraded.
root@docker-01:~#

```

Рис. 8.5

Наконец, убедимся, что все в порядке, запустив следующую команду и изучив выходные данные на рис. 8.6.

```
docker run hello-world
```

```
root@docker-01:~#
root@docker-01:~# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
1b930d010525: Pull complete
Digest: sha256:9572f7cdcee8591948c2963463447a53466950b3fc15a247fcad1917ca215a2f
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Рис. 8.6

```
root@docker-01:~# docker version
Client: Docker Engine - Enterprise
Version: 19.03.5
API version: 1.40
Go version: go1.12.12
Git commit: 2ee0c57608
Built: Wed Nov 13 07:45:31 2019
OS/Arch: linux/amd64
Experimental: false

Server: Docker Engine - Enterprise
Engine:
Version: 19.03.5
API version: 1.40 (minimum version 1.12)
Go version: go1.12.12
Git commit: 2ee0c57608
Built: Wed Nov 13 07:43:49 2019
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: 1.2.6
GitCommit: 894b81a4b802e4eb2a91d1ce216b8817763c29fb
runc:
Version: 1.0.0-rc8
GitCommit: 425e105d5a03fabd737a126ad93d62a9eeede87f
docker-init:
Version: 0.18.0
GitCommit: fec3683
root@docker-01:~#
```

Рис. 8.7

Также проверим версию Docker следующей командой (выходные данные — на рис. 8.7):

```
docker version
```

Отлично, мы сделали это! Установили Docker Enterprise Edition на первом сервере, который будет нашим управляющим узлом. Теперь установим Docker Enterprise Edition на другом узле, выполнив те же самые действия.

После установки Docker Enterprise Edition на обоих узлах мы установим UCP на первом узле, присоединим оба узла к кластеру Swarm и установим DTR на втором узле.

Установка Universal Control Plane

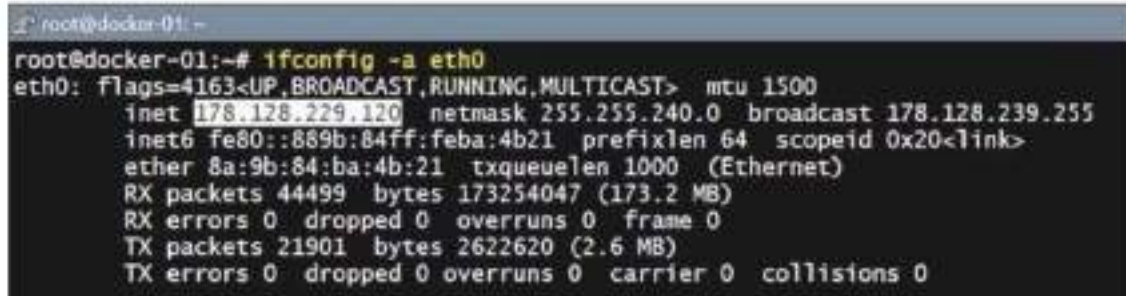
Docker *Universal Control Plane* (UCP) — это решение корпоративного уровня для централизованного управления кластерами и приложениями.

Для установки UCP потребуется 16 Гбайт оперативной памяти, 4 виртуальных ЦП и 25–100 Гбайт свободного пространства на диске.

Версия Docker Engine Enterprise должна быть 19.03, а версия ядра Linux — не ниже 3.10; узел должен иметь статический IP-адрес.

Проверим IP-адрес следующей командой и посмотрим результаты на рис. 8.8.

```
ifconfig -a eth0
```



```
root@docker-01:~# ifconfig -a eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 178.128.229.120 netmask 255.255.240.0 broadcast 178.128.239.255
    inet6 fe80::889b:84ff:feba:4b21 prefixlen 64 scopeid 0x20<link>
    ether 8a:9b:84:ba:4b:21 txqueuelen 1000 (Ethernet)
    RX packets 44499 bytes 173254047 (173.2 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 21901 bytes 2622620 (2.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Рис. 8.8

Теперь извлекаем последнюю версию UCP:

```
docker image pull docker/ucp:latest
```

И устанавливаем UCP:

```
docker container run --rm -it --name ucp \
    -v /var/run/docker.sock:/var/run/docker.sock \
    docker/ucp:latest install \
    --host-address 178.128.229.120 \
    --interactive
```

Во время установки нужно указать имя пользователя и пароль администратора. Мы укажем имя пользователя *admin*. Мы не вводили значение *SAN* (Storage Area Network — сеть хранения данных), а просто нажали <Enter>, чтобы двигаться дальше.

Установка продолжается несколько минут и состоит из 35 отдельных шагов. На рис. 8.9 показана вторая половина этого процесса.

UCP автоматически устанавливает Project Calico для взаимодействия между контейнерами посредством Kubernetes. Можно установить альтернативный плагин CNI (Container Network Interface — интерфейс контейнерной сети), например Weave или Flannel.

```
INFO[0026] Step 15 of 35: [Deploy Internal Client CA Server]
INFO[0028] Step 16 of 35: [Deploy UCP Controller Server]
INFO[0036] Step 17 of 35: [Deploy Kubernetes API Server]
INFO[0048] Step 18 of 35: [Deploy Kubernetes Controller Manager]
INFO[0054] Step 19 of 35: [Deploy Kubernetes Scheduler]
INFO[0059] Step 20 of 35: [Deploy Kubelet]
INFO[0060] Step 21 of 35: [Deploy Kubernetes Proxy]
INFO[0061] Step 22 of 35: [Wait for Healthy UCP Controller and Kubernetes API Server]
INFO[0061] Step 23 of 35: [Create Kubernetes Pod Security Policies]
INFO[0068] Step 24 of 35: [Install Kubernetes CNI Plugin]
INFO[0094] Step 25 of 35: [Install KubeDNS]
INFO[0098] Step 26 of 35: [Create UCP Controller Kubernetes Service Endpoints]
INFO[0101] Step 27 of 35: [Install Metrics Plugin]
INFO[0105] Step 28 of 35: [Install Kubernetes Compose Plugin]
INFO[0117] Step 29 of 35: [Deploy Manager Node Agent Service]
INFO[0117] Step 30 of 35: [Deploy Worker Node Agent Service]
INFO[0117] Step 31 of 35: [Deploy Windows Worker Node Agent Service]
INFO[0117] Step 32 of 35: [Deploy Cluster Agent Service]
INFO[0117] Step 33 of 35: [Set License]
INFO[0117] Step 34 of 35: [Set Registry CA Certificates]
INFO[0117] Step 35 of 35: [Wait for All Nodes to be Ready]
INFO[0122] All Installation Steps Completed
root@docker-01:~#
```

Рис. 8.9

Установив UCP, попробуем открыть его в браузере: **https://178.128.229.120**. Это общедоступный IP-адрес узла, на котором мы установили UCP. Затем мы введем учетные данные администратора и загрузим лицензию.

При попытке войти мы получаем предупреждение о том, что подключение небезопасно (рис. 8.10). Это нестрашно. Просто нажимаем на **Advanced** (Расширенные)

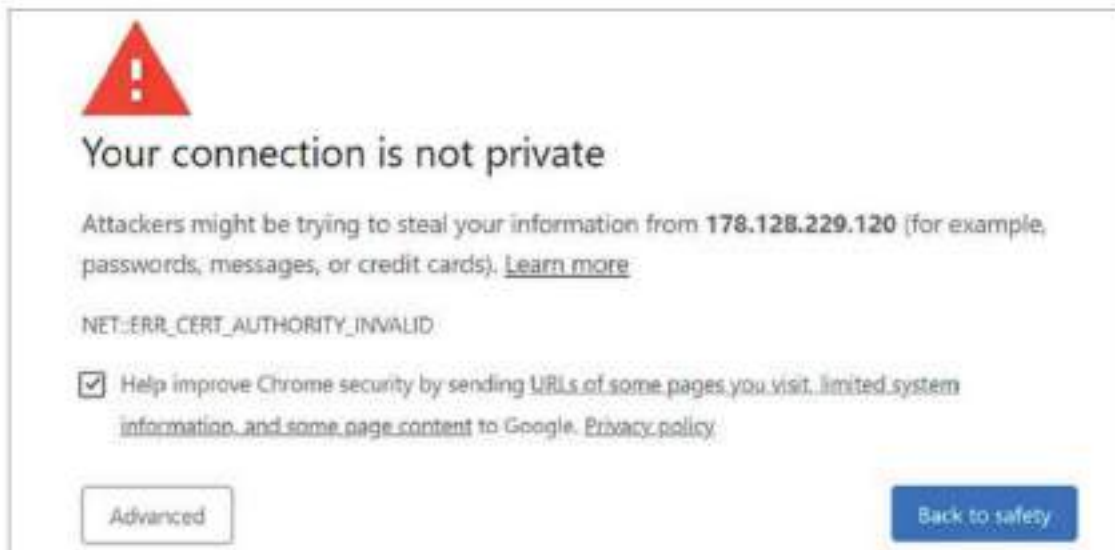


Рис. 8.10

и следуем инструкциям, чтобы предоставить учетные данные администратора и загрузить лицензию. После этого подключение будет установлено (рис. 8.11).

Это стартовая страница UCP. Следующая задача — подключить в интерфейсе UCP второй узел к кластеру Swarm в качестве рабочего узла.

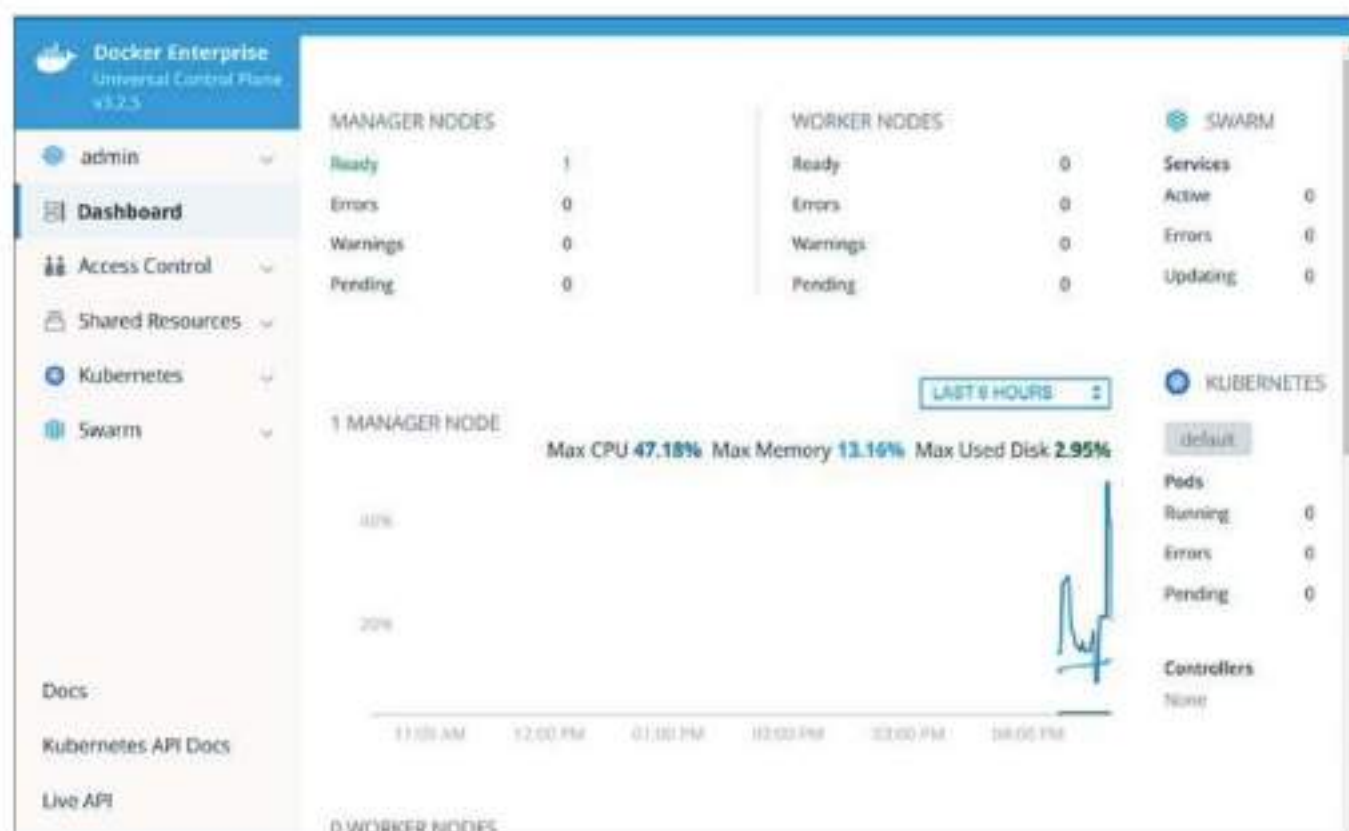


Рис. 8.11

Переходим в раздел **Shared Resources** (Общие ресурсы) | **Nodes** (Узлы) | **Add Nodes** (Добавить узлы). Откроется следующая страница (рис. 8.12).

Node Type*

☒ WINDOWS ☒ LINUX

Node Role* ⓘ

☒ MANAGER ☒ WORKER

☐ Use a custom listen address ⓘ

☐ Use a custom advertise address ⓘ

Nodes are added to the cluster by using Docker's 'swarm join' command. Use the following command on the engine you want to connect to the cluster.

```
docker swarm join --token SWMTKN-1-3gzcg989uh2uhna0i59yq685ot54i96zeas1rotz233qx37eop-7eye0m23knmkndh6c9tg4rpz
178.128.229.120:2377
```

Рис. 8.12

Копируем команду `swarm join` и запускаем ее на рабочем узле для присоединения к кластеру.

На рис. 8.13 мы видим, что второй узел присоединен к кластеру в качестве рабочего узла.

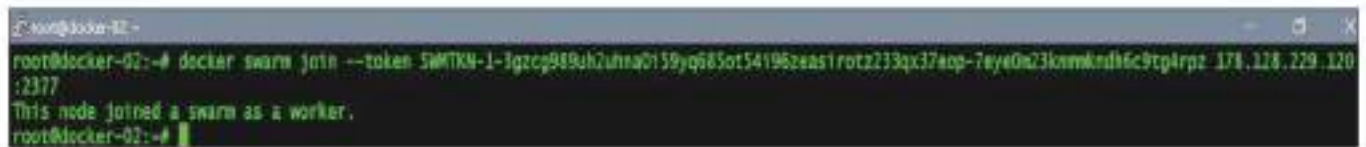


Рис. 8.13

Установка Docker Trusted Registry

Чтобы установить *Docker Trusted Registry* (DTR), в UCP переходим в **Admin Settings** (Параметры администратора) | **Docker Trusted Registry**. Здесь нужно указать определенную информацию. Чтобы не усложнять процесс, мы не будем настраивать балансировщик нагрузки, а просто вставим общедоступный IP-адрес второго узла в поле **DTR EXTERNAL URL** (Внешний URL DTR) и отключим проверку TLS-сертификатов, установив соответствующий флажок (см. рис. 8.14).

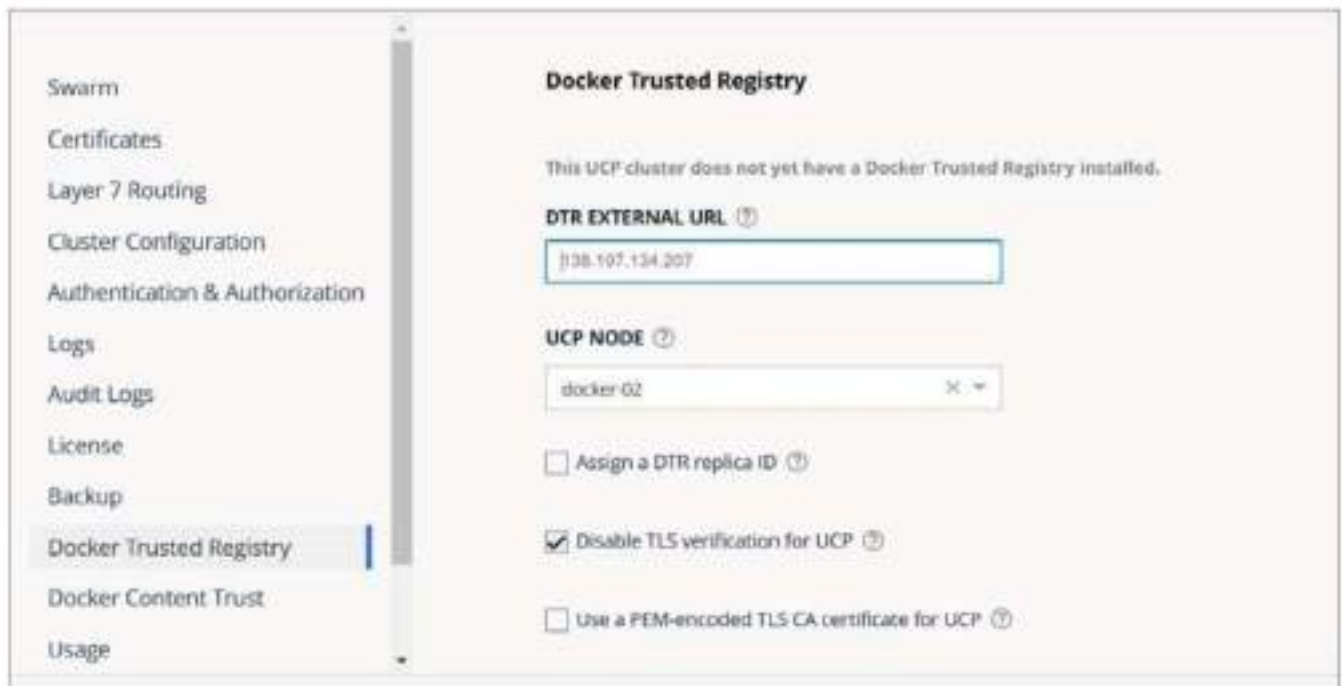


Рис. 8.14

Прокрутим вниз и увидим команду, сгенерированную для установки DTR (рис. 8.15).

Копируем команду и запускаем ее на первом узле, чтобы установить DTR и зарегистрировать UCP. Затем проверяем, что получилось.

В браузере указываем следующее: `https://<общедоступный_IP_второго_узла>`. В нашем случае это 138.197.134.207. Вводим соответственно `https:// 138.197.134.207`, чтобы войти в систему и перейти на следующую страницу (см. рис. 8.16).



Рис. 8.15



Рис. 8.16

Итак, мы установили UCP и DTR. Теперь создадим свой первый репозиторий для наших экспериментов.

Нажимаем **New Repository** (Создать репозиторий), указываем имя и нажимаем **Create** (Создать), как показано на рис. 8.17. Это всё, что требовалось сделать на данном этапе.

Загрузка и установка клиентского пакета

Docker UCP защищает кластер с помощью управления доступом на основе ролей (об этом мы поговорим чуть позже), так что только авторизованные пользователи могут вносить в него изменения.

При попытке выполнить команду Docker на узле UCP мы получим ошибку, если наш запрос не прошел проверку подлинности действующего клиентского сертификата. Поэтому нужно сгенерировать клиентский сертификат, загрузить его и установить в системе.

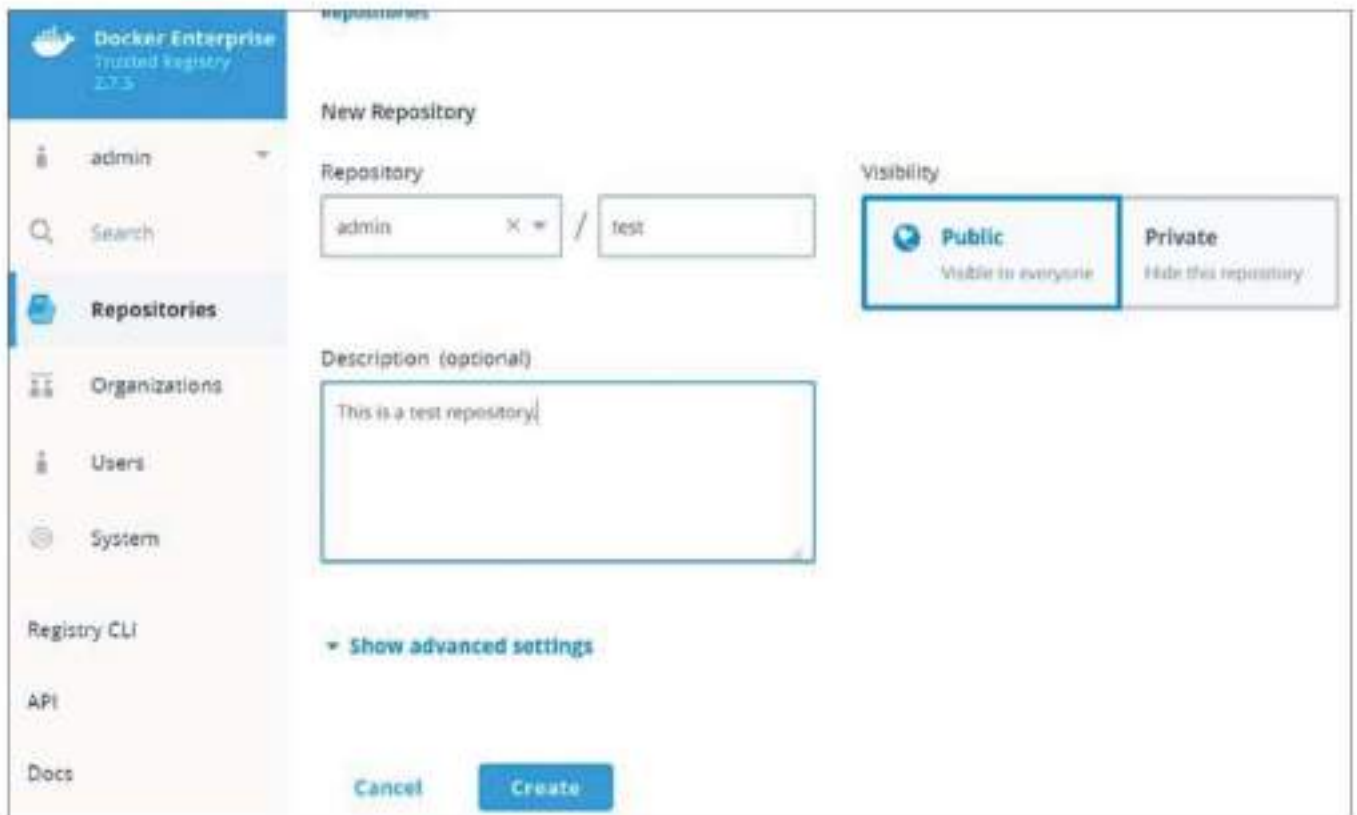


Рис. 8.17

Чтобы загрузить *пакет клиентских сертификатов* (client certificate bundle), мы входим в UCP и переходим на страницу **My Profile** (Мой профиль), как показано на рис. 8.18.



Рис. 8.18

Далее генерируем новый *клиентский пакет* (client bundle), копируем его на наш узел, распаковываем и используем скрипт `env.sh`, чтобы начать использовать сертификаты (см. рис. 8.19 и 8.20).

```
ls -ltr
```

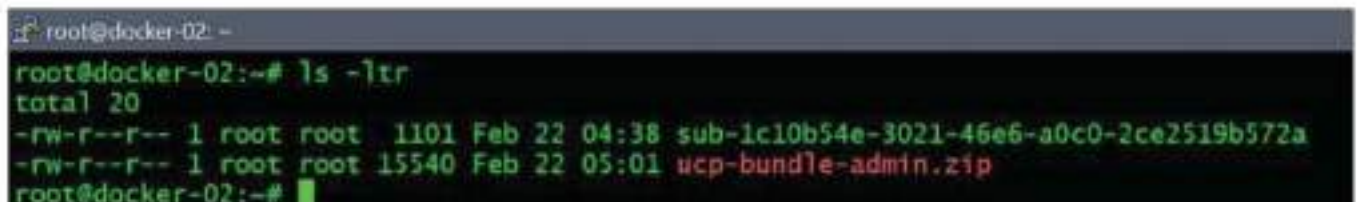
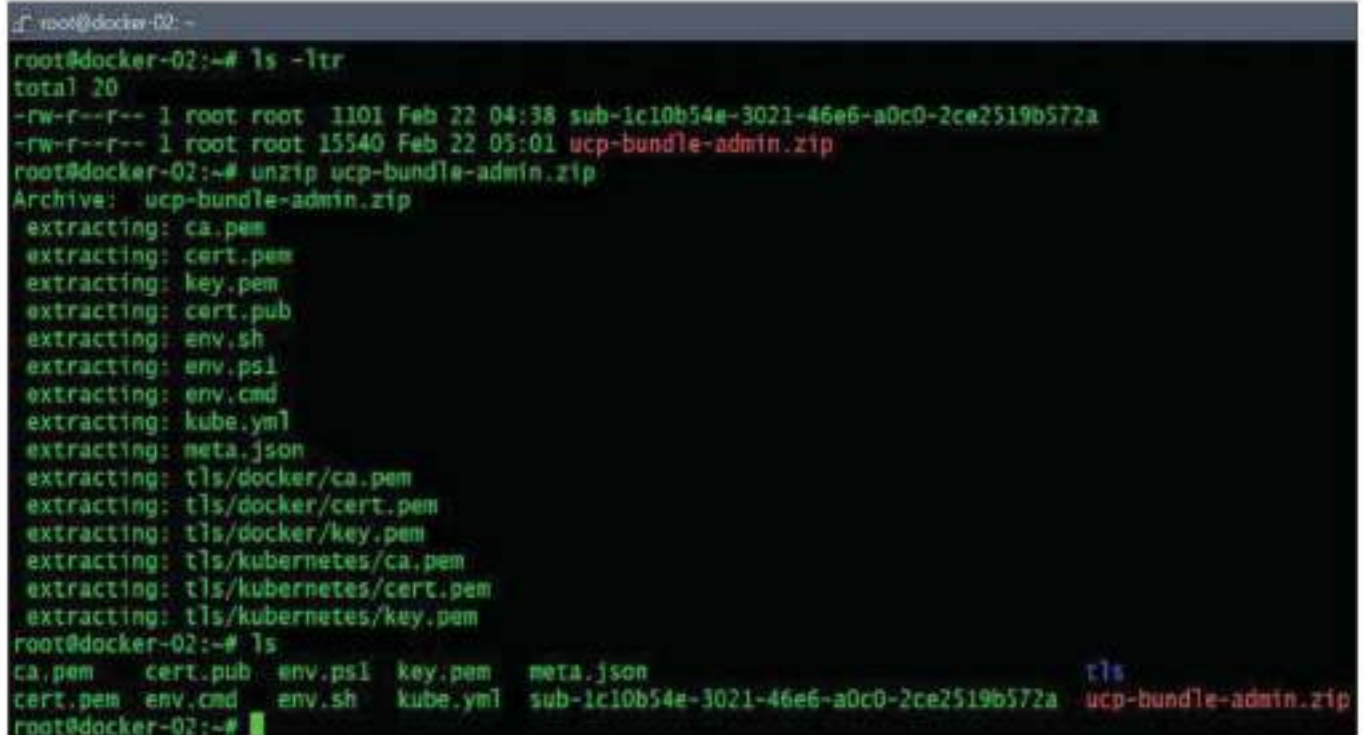


Рис. 8.19

```
unzip ucp-bundle-admin.zip
```



```

root@docker-02:~# ls -ltr
total 20
-rw-r--r-- 1 root root 1101 Feb 22 04:38 sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a
-rw-r--r-- 1 root root 15540 Feb 22 05:01 ucp-bundle-admin.zip
root@docker-02:~# unzip ucp-bundle-admin.zip
Archive: ucp-bundle-admin.zip
  extracting: ca.pem
  extracting: cert.pem
  extracting: key.pem
  extracting: cert.pub
  extracting: env.sh
  extracting: env.ps1
  extracting: env.cmd
  extracting: kube.yml
  extracting: meta.json
  extracting: tls/docker/ca.pem
  extracting: tls/docker/cert.pem
  extracting: tls/docker/key.pem
  extracting: tls/kubernetes/ca.pem
  extracting: tls/kubernetes/cert.pem
  extracting: tls/kubernetes/key.pem
root@docker-02:~# ls
ca.pem  cert.pub  env.ps1  key.pem  meta.json  tls
cert.pem  env.cmd  env.sh  kube.yml  sub-1c10b54e-3021-46e6-a0c0-2ce2519b572a  ucp-bundle-admin.zip
root@docker-02:~#

```

Рис. 8.20

Запускаем скрипт `env.sh` для успешной установки клиентского сертификата UCP и проверяем результат (рис. 8.21).

```
docker version --format '{{.Server.Version}}'
```



```

root@docker-02:~# eval "$(env.sh)"
root@docker-02:~# docker version --format '{{.Server.Version}}'
ucp/3.2.5
root@docker-02:~#

```

Рис. 8.21

Client Bundle (клиентский пакет) генерирует пару ключей — закрытый (приватный) ключ и открытый (публичный) ключ, который авторизует запросы к UCP. Имейте в виду, что теперь мы взаимодействуем с Docker Engine не напрямую, а только через UCP. Client Bundle также содержит набор скриптов, который настраивает клиентские утилиты Docker CLI и Kubectl для взаимодействия с UCP.

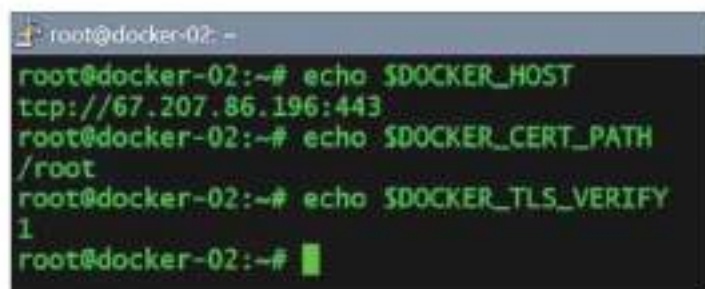
Утилита Client Bundle автоматически обновляет переменные среды `DOCKER_HOST`, `DOCKER_CERT_PATH` и `DOCKER_TLS_VERIFY`.

Переменная среды `DOCKER_HOST` устанавливает URL-адрес демона Docker, а переменная `DOCKER_CERT_PATH` указывает, где находятся файлы `ca.pem`, `cert.pem` и `key.pem`, используемые для проверки TLS. Переменная среды `DOCKER_TLS_VERIFY` запускает проверку TLS с помощью демона Docker, если указано любое значение, отличное от пустой

строки. Таким образом, при любом взаимодействии с демоном Docker через UCP будут использоваться сертификаты, доступные в клиентском пакете.

Чтобы проверить, как это работает, запустим несколько команд на узле, где мы установили клиентский пакет, и проверим выходные данные на рис. 8.22.

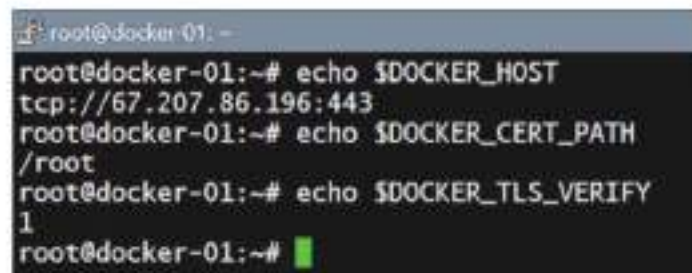
```
echo $DOCKER_HOST
echo $ DOCKER_CERT_PATH
echo $ DOCKER_TLS_VERIFY
```



```
root@docker-02:~# echo $DOCKER_HOST
tcp://67.207.86.196:443
root@docker-02:~# echo $DOCKER_CERT_PATH
/root
root@docker-02:~# echo $DOCKER_TLS_VERIFY
1
root@docker-02:~#
```

Рис. 8.22

То же самое можно сделать и для другого узла (рис. 8.23).



```
root@docker-01:~# echo $DOCKER_HOST
tcp://67.207.86.196:443
root@docker-01:~# echo $DOCKER_CERT_PATH
/root
root@docker-01:~# echo $DOCKER_TLS_VERIFY
1
root@docker-01:~#
```

Рис. 8.23

Функции безопасности Docker Enterprise Edition

Одно из главных преимуществ Docker EE — эффективные функции безопасности. Лучшая из них, на мой взгляд, *сканирование образов Docker на наличие уязвимостей*.

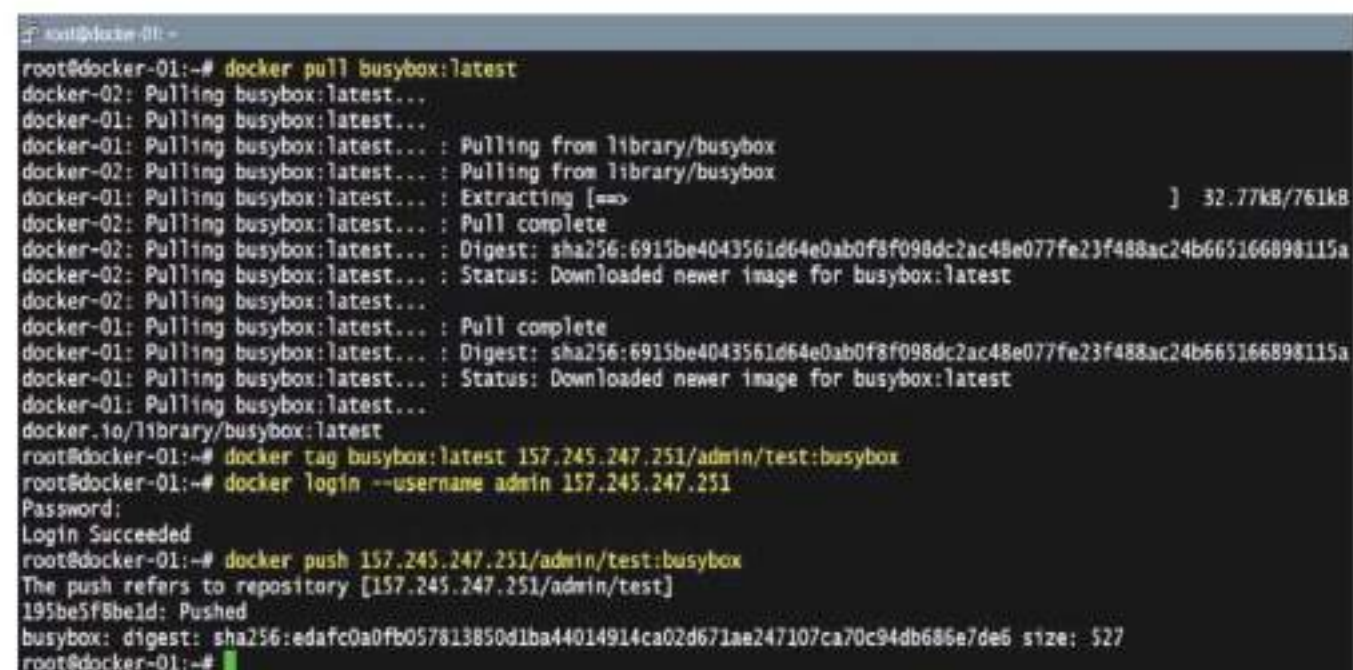
Давайте посмотрим, как это работает. В первом примере извлечем последний образ *busybox* из Docker Hub, переименуем его, отправим в Docker Trusted Registry (DTR), а затем просканируем этот образ на наличие уязвимостей. Там не должно быть никаких уязвимостей, потому что это новейший образ *busybox*, который был проверен, наверное, тысячи раз.

Во втором примере извлечем очень старый образ *busybox* (в нем должны быть какие-то уязвимости), переименуем его и тоже отправим в DTR. На этот раз может обнаружиться несколько уязвимостей.

Есть еще одна важная вещь, о которой следует узнать, прежде чем перейти к практике. Существует *база данных общеизвестных уязвимостей* информационной безопасности (Common Vulnerabilities and Exposure, CVE), которая время от времени обновляется. DTR проверяет наличие новых уязвимостей, загружает их, а затем применяет, не прерывая выполняющееся сканирование.

Если нет доступа к Интернету, для обновления базы данных CVE нужно загрузить и установить *tag-файл* с обновлениями. Рассмотрим эту концепцию на примере. Запустим следующие команды (см. рис. 8.24).

```
docker pull busybox:latest
docker tag busybox:latest <IP_АДРЕС_DTR>/<имя_пользователя>/<имя_репозитория>: busybox
docker login --username <имя_пользователя> <IP_АДРЕС_DTR>
docker push <IP_АДРЕС_DTR>/<имя_пользователя>/<имя_репозитория>: busybox
```



```
root@docker-01:~# docker pull busybox:latest
docker-02: Pulling busybox:latest...
docker-01: Pulling busybox:latest...
docker-01: Pulling busybox:latest... : Pulling from library/busybox
docker-02: Pulling busybox:latest... : Pulling from library/busybox
docker-01: Pulling busybox:latest... : Extracting [==>] 32.77kB/761kB
docker-02: Pulling busybox:latest... : Pull complete
docker-02: Pulling busybox:latest... : Digest: sha256:6915be4043561d64e0ab0f8f098dc2ac48e077fe23f488ac24b665166898115a
docker-02: Pulling busybox:latest... : Status: Downloaded newer image for busybox:latest
docker-02: Pulling busybox:latest... : Pull complete
docker-01: Pulling busybox:latest... : Digest: sha256:6915be4043561d64e0ab0f8f098dc2ac48e077fe23f488ac24b665166898115a
docker-01: Pulling busybox:latest... : Status: Downloaded newer image for busybox:latest
docker-01: Pulling busybox:latest...
docker.io/library/busybox:latest
root@docker-01:~# docker tag busybox:latest 157.245.247.251/admin/test:busybox
root@docker-01:~# docker login --username admin 157.245.247.251
Password:
Login Succeeded
root@docker-01:~# docker push 157.245.247.251/admin/test:busybox
The push refers to repository [157.245.247.251/admin/test]
195be5f8be1d: Pushed
busybox: digest: sha256:edafc0a0fb057813850d1ba44014914ca02d671ae247107ca70c94db686e7de6 size: 527
root@docker-01:~#
```

Рис. 8.24

На странице DTR, если мы зайдем в **Repositories** (Репозитории) | **admin/test** | **Tags** (Теги), мы увидим наш образ с тегом *busybox* (см. рис. 8.25).

Открыв **System** (Система) | **Security** (Безопасность), мы увидим, что сканирование пока не включено. Нужно включить его (**Enable Scanning**), а затем кликнуть **Enable Online Syncing** (Включить онлайн-синхронизацию), чтобы получать обновления от базы данных CVE (см. рис. 8.26).

Теперь можно начать сканирование, как показано на рис. 8.27.

Как и ожидалось, в процессе сканирования никаких уязвимостей в образе не обнаружилось (см. рис. 8.28).

Теперь, как мы планировали, извлечем старый образ *busybox* и попробуем найти в нем уязвимости (см. рис. 8.29).

Образ *busybox_old* извлечен и доступен в DTR (см. рис. 8.30).

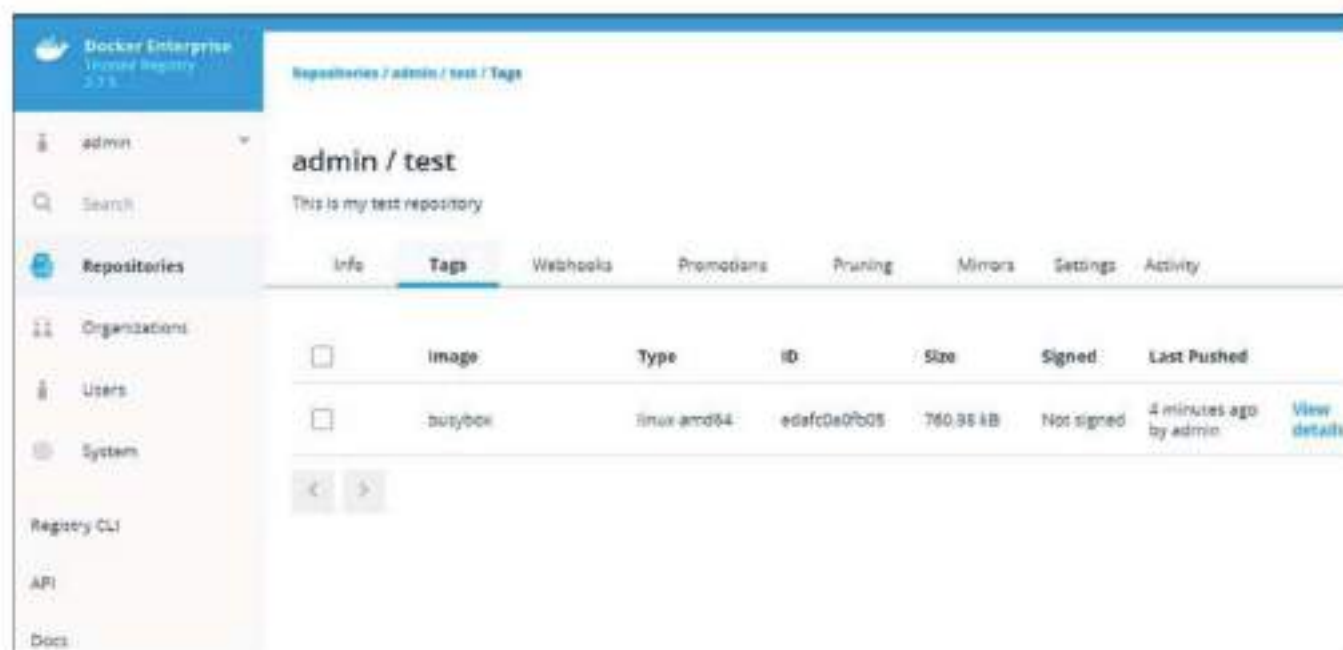


Рис. 8.25



Рис. 8.26

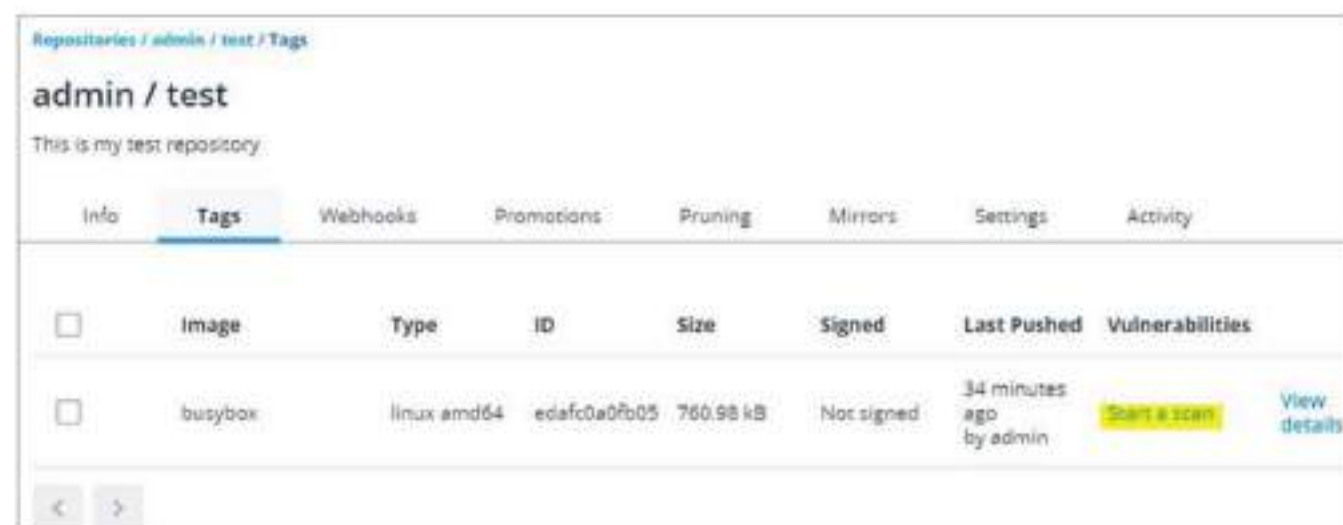


Рис. 8.27



Рис. 8.28

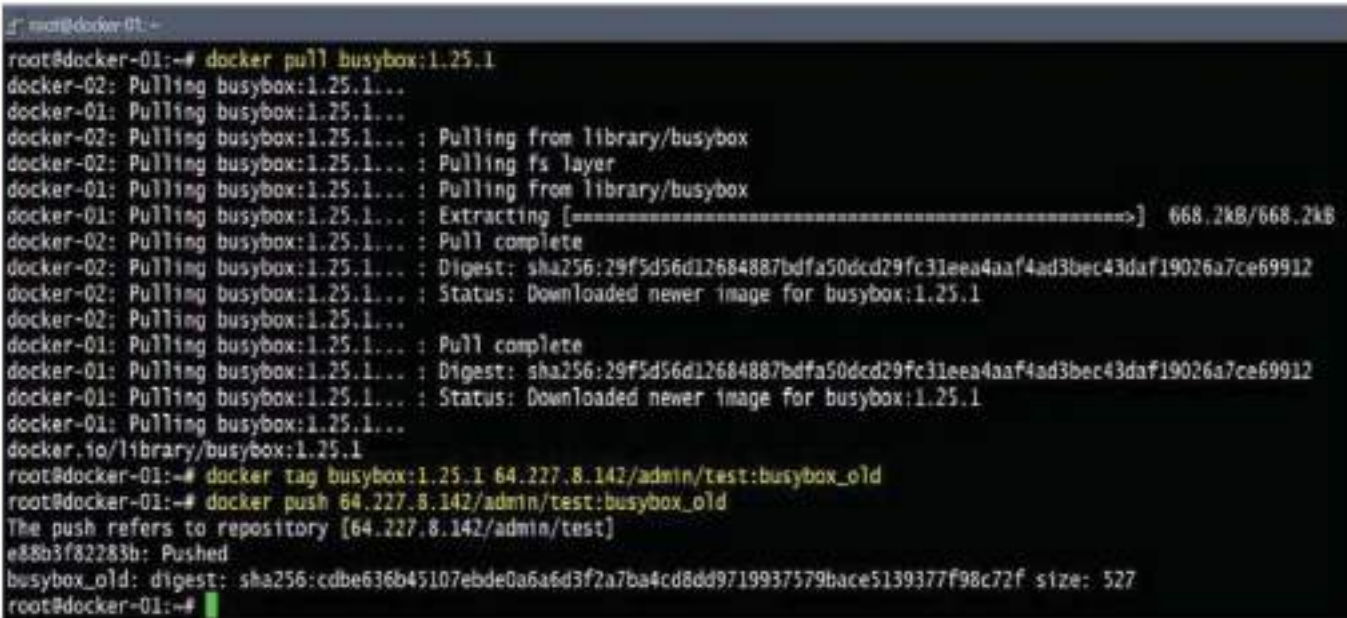


Рис. 8.29

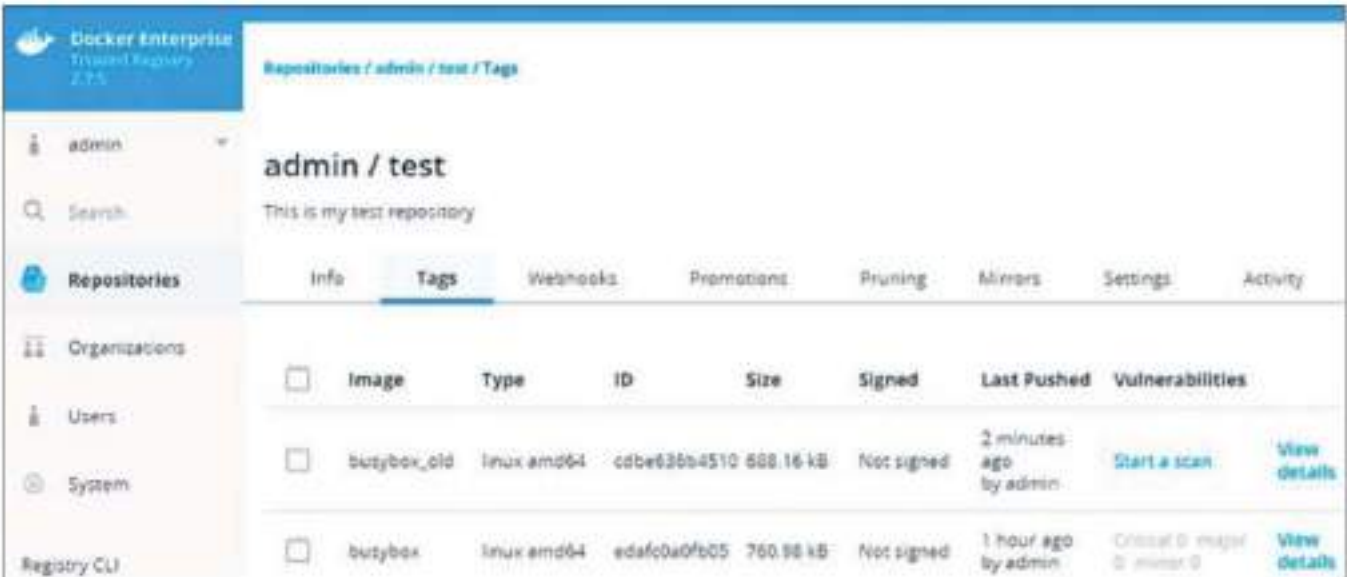


Рис. 8.30



Рис. 8.31

Теперь просканируем извлеченный образ и посмотрим результаты на рис. 8.31.

Мы видим две критичные и пять серьезных уязвимостей, которые следует изучить подробнее, поскольку, как известно, «дьявол кроется в деталях» (см. рис. 8.32).

На скриншоте видно, в каком слое обнаружены уязвимости.

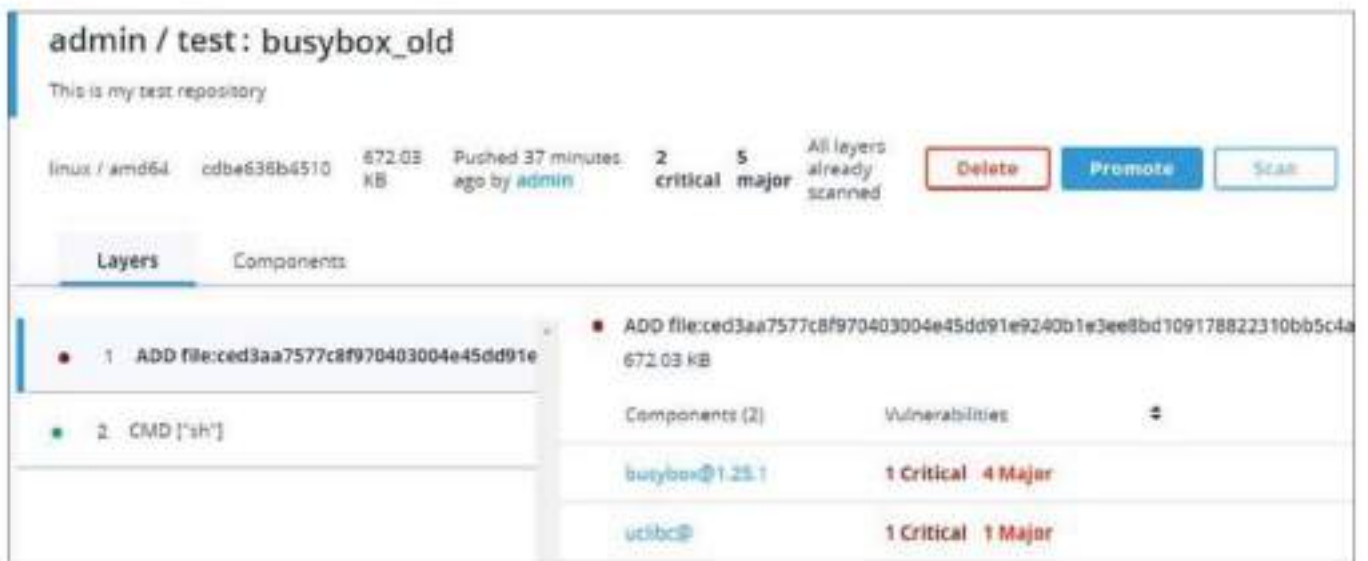


Рис. 8.32

Если углубиться в детали, можно понять, какие проблемы имеются с этим образом (см. рис. 8.33).

Итак, теперь вы знаете, как осуществляется сканирование образов на наличие уязвимостей в Docker.

Можно настроить автоматическое сканирование при каждой отправке образа. Для этого в **Repositories** (Репозитории) | <Имя_репозитория> | **Settings** (Настройки) нужно включить сканирование при отправке (**On push**), как показано на рис. 8.34.

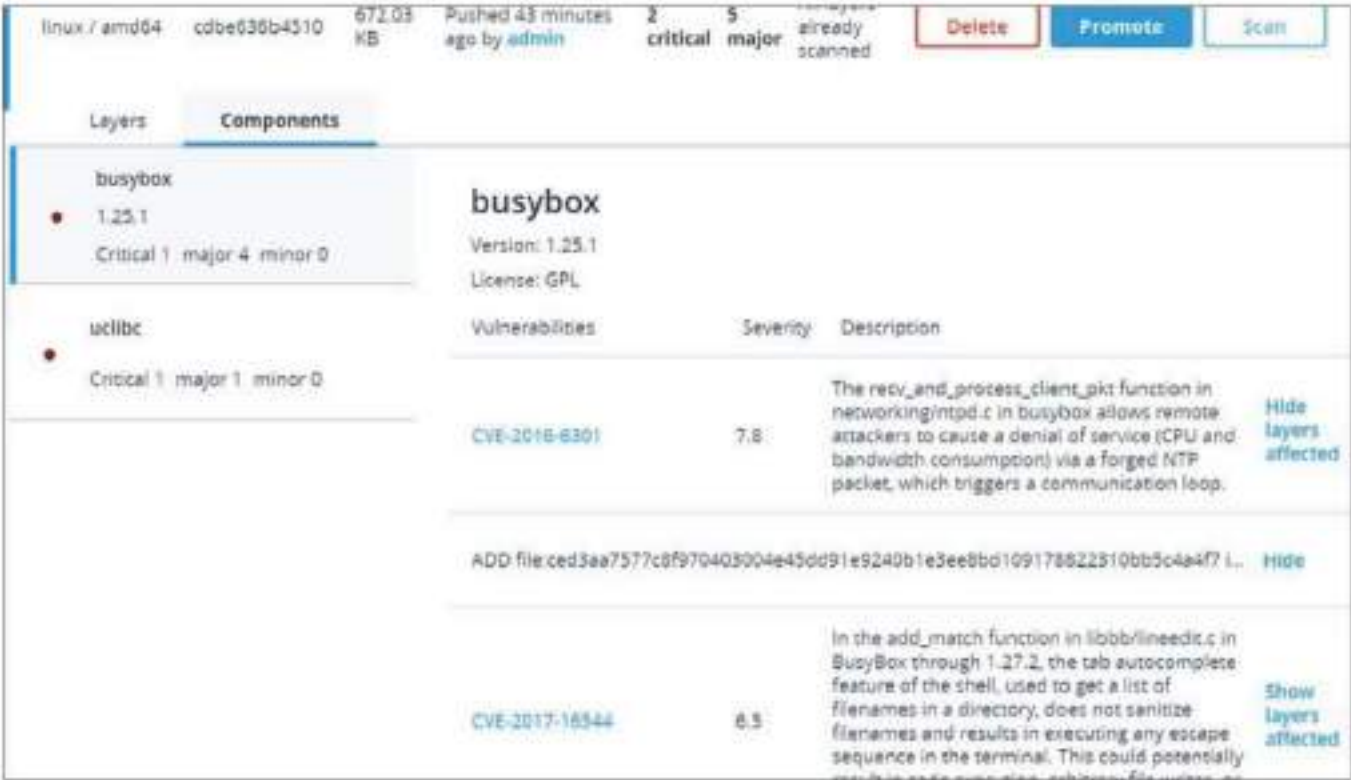


Рис. 8.33

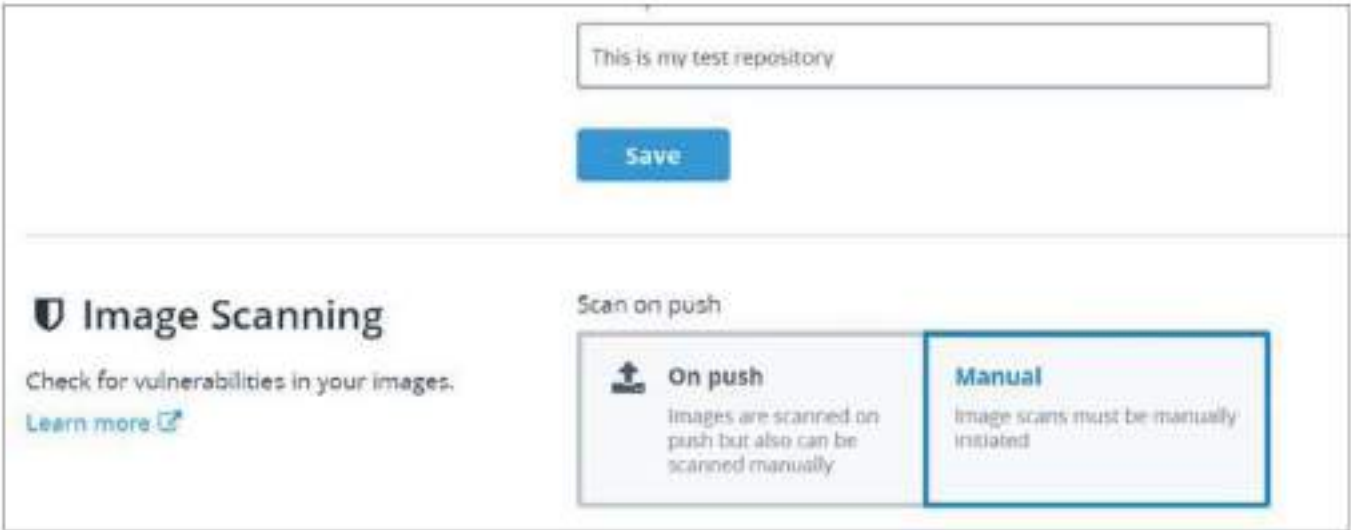


Рис. 8.34

Управление доступом на основе ролей

Docker Universal Control Plane (UCP) позволяет создавать пользователей с разным уровнем доступа для ресурсов — образов, сервисов, сетей, томов и т. д.

Можно контролировать кто (субъект), сколько и какого типа (роль) доступ имеет к набору ресурсов (коллекции): создавать отдельных пользователей или группы пользователей и предоставлять им различные наборы разрешений (роли), осуществляя таким образом детальный контроль доступа.

Обычно предоставлением разрешений занимается администратор. Субъектов можно разделить на несколько категорий:

- ◆ пользователи;
- ◆ организации;
- ◆ команды.

Пользователь — это тот, кто использует UCP и может быть членом команды в организации или нескольких организаций.

Организация — это группа пользователей с одинаковым набором разрешений и привилегий.

Команда — это тоже группа пользователей с одинаковым набором разрешений и привилегий, но команда, как правило, является частью организации. Организация может состоять из нескольких команд, а команда может, в свою очередь, входить в несколько организаций.

Каждый пользователь по умолчанию имеет разрешение для доступа к Swarm. Создать пользователя очень просто: в UCP открываем страницу **Users** (Пользователи) и нажимаем **Create** (Создать). В данном случае мы не хотим, чтобы создаваемый пользователь был администратором (см. рис. 8.35).

Рис. 8.35

Новый пользователь создан. Как видно на рис. 8.36, он имеет статус **Active** (активен).

Пока все хорошо.

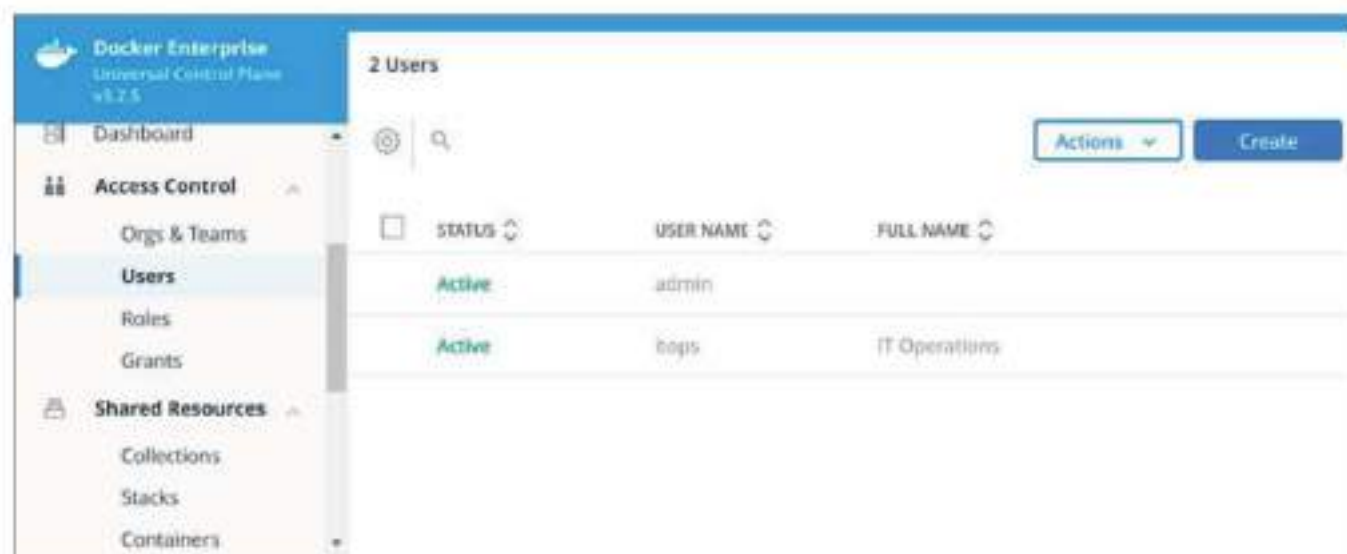


Рис. 8.36

Так же легко мы можем создать организацию и добавить в нее пользователей. Давайте создадим организацию с именем **operations** и добавим в нее пользователя (см. рис. 8.37).



Рис. 8.37

Но сначала для организации **operations** нужно создать команду (рис. 8.38).

Когда команда будет создана, мы добавим в нее пользователя. На рис. 8.39 показано добавление команды **operations_team**.

Далее мы добавляем пользователя **itops** (рис. 8.40).

Итак, что мы имеем?

Мы создали организацию **operations**, а внутри нее — команду **operations_team**. Затем мы добавили в команду пользователя **itops**.

Двигаемся дальше.

Как мы уже знаем, *роль* — это набор привилегий и разрешений для доступа к ресурсам. В Docker и Kubernetes доступно несколько предопределенных *ролей*. На рис. 8.41 показаны предопределенные роли для Docker Swarm.

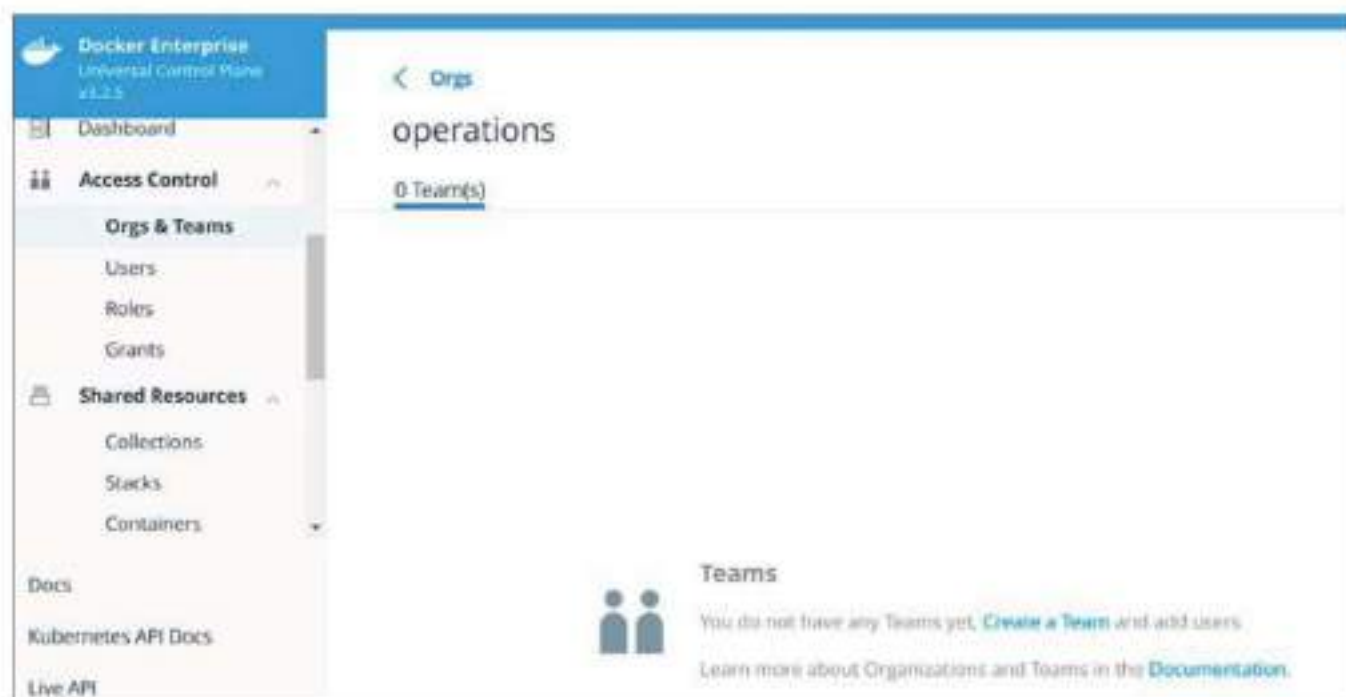


Рис. 8.38

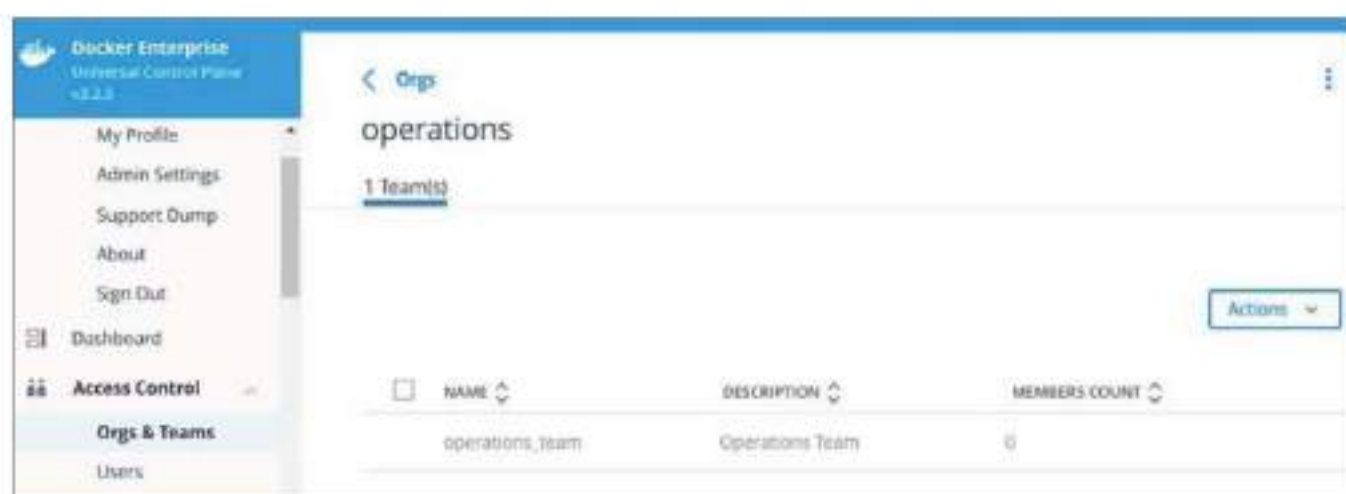


Рис. 8.39

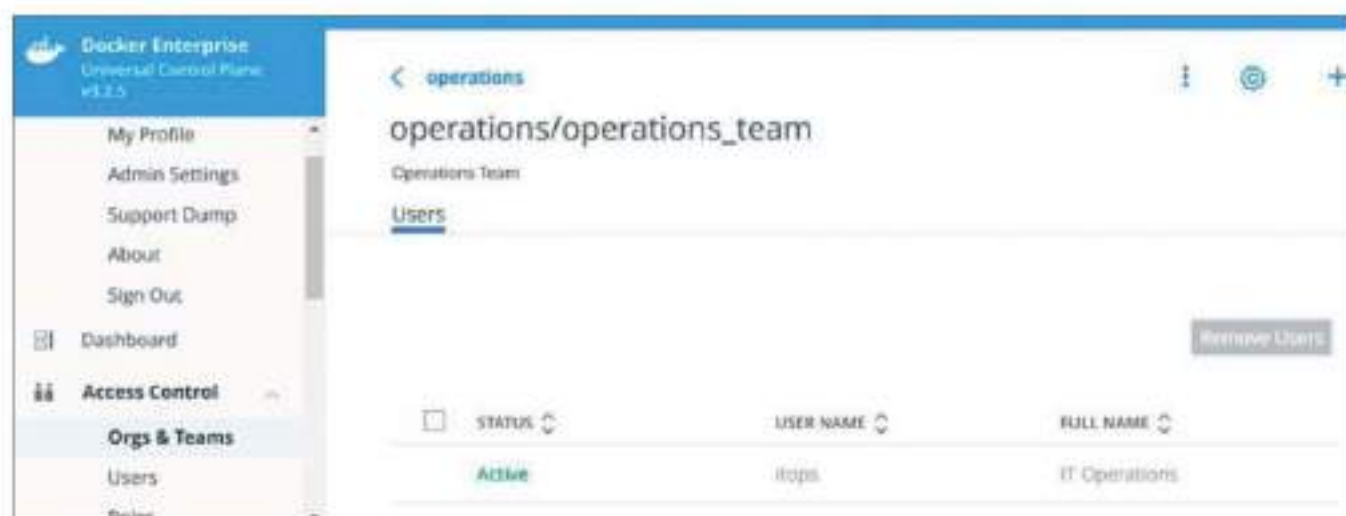


Рис. 8.40

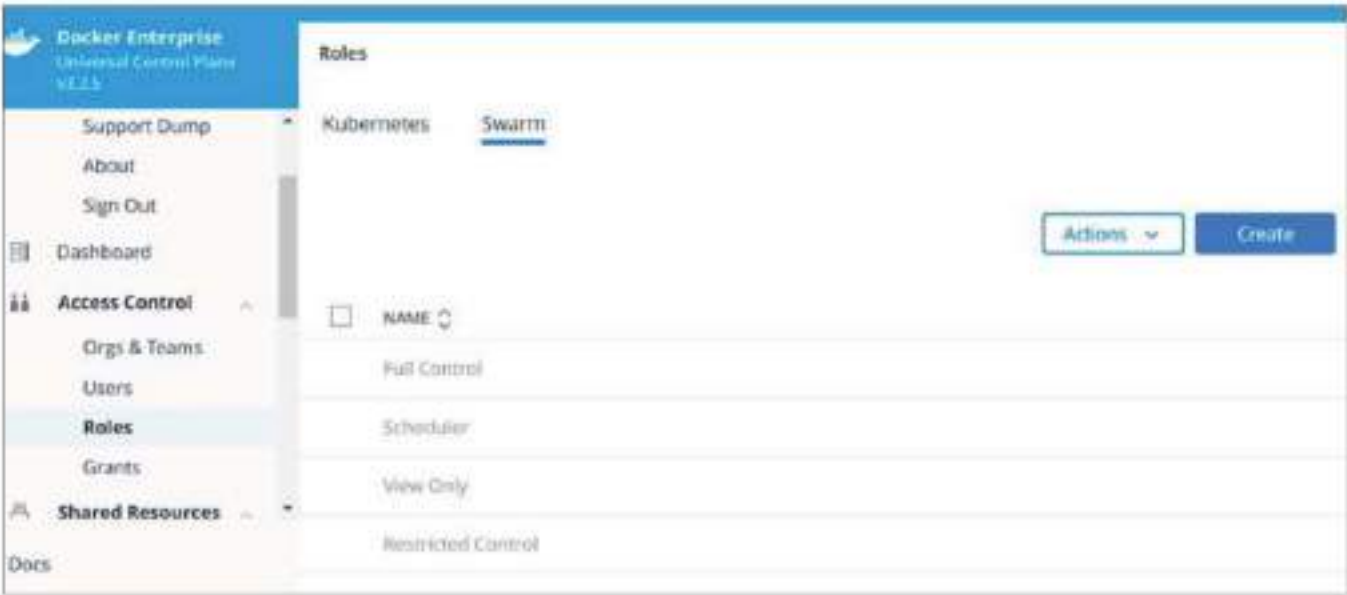


Рис. 8.41

В табл. 8.1 приводится описание этих ролей.

Таблица 8.1. Роли для Docker Swarm

Название роли	Описание	Примечания
View Only (только просмотр)	Пользователь может просматривать ресурсы, но не создавать	Очень ограниченная роль
Restricted Control (ограниченный контроль)	Пользователь может просматривать ресурсы и вносить изменения, но существуют ограничения на запуск сервиса или контейнера, если это повлияет на узел. Например, пользователь не может запускать контейнер в привилегированном режиме или с дополнительными capabilities	Роль дает гораздо больше возможностей по сравнению с View Only, но все равно имеет много ограничений. Например, пользователь не может запустить команду <code>exec</code> для входа в контейнер или присоединить к контейнеру каталог узла
Scheduler (планировщик)	Пользователь может планировать рабочие нагрузки и просматривать узлы	Пользователь не может просматривать рабочие нагрузки. Для просмотра требуется отдельное разрешение, в частности Container View
Full Control (полный контроль)	Роль с самыми широкими полномочиями. Пользователь может просматривать и изменять все предоставленные ресурсы	Пользователь может создавать контейнеры, но не просматривать контейнеры, созданные другими

Мы уже упоминали о коллекциях. Коллекции — это ресурсы, с которыми будет работать субъект (пользователь) в соответствии с его разрешениями для этой коллекции. Например, у нас могут быть три коллекции: для разработки, тестирования и рабочей среды, и у пользователя **itops** могут быть разные роли в этих коллекциях. Чтобы лучше понять этот механизм, создадим пользовательскую коллекцию и пользовательскую роль, а затем предоставим эту роль и коллекцию пользователю **itops**. Выполним несколько шагов.

Шаг 1. Создание пользовательской коллекции.

Нажимаем последовательно: **Collections** (Коллекции) | **Swarm** | **Shared** (Общие) | **Private** (Приватные) | **Create Collection** (Создать коллекцию). Присваиваем коллекции имя **My_collection**. На рис. 8.42 видно, что коллекция создана успешно.



Рис. 8.42

Шаг 2. Создание пользовательской роли.

Нажимаем последовательно: **Roles** (Роли) | **Swarm** | **Create** (Создать) | **Names** (Имена) | **Operations** (Операции), устанавливаем флажки, чтобы разрешить все доступные операции, а затем кликнем **Create** (Создать).

Теперь у нас есть роль **My_role** (см. рис. 8.43).

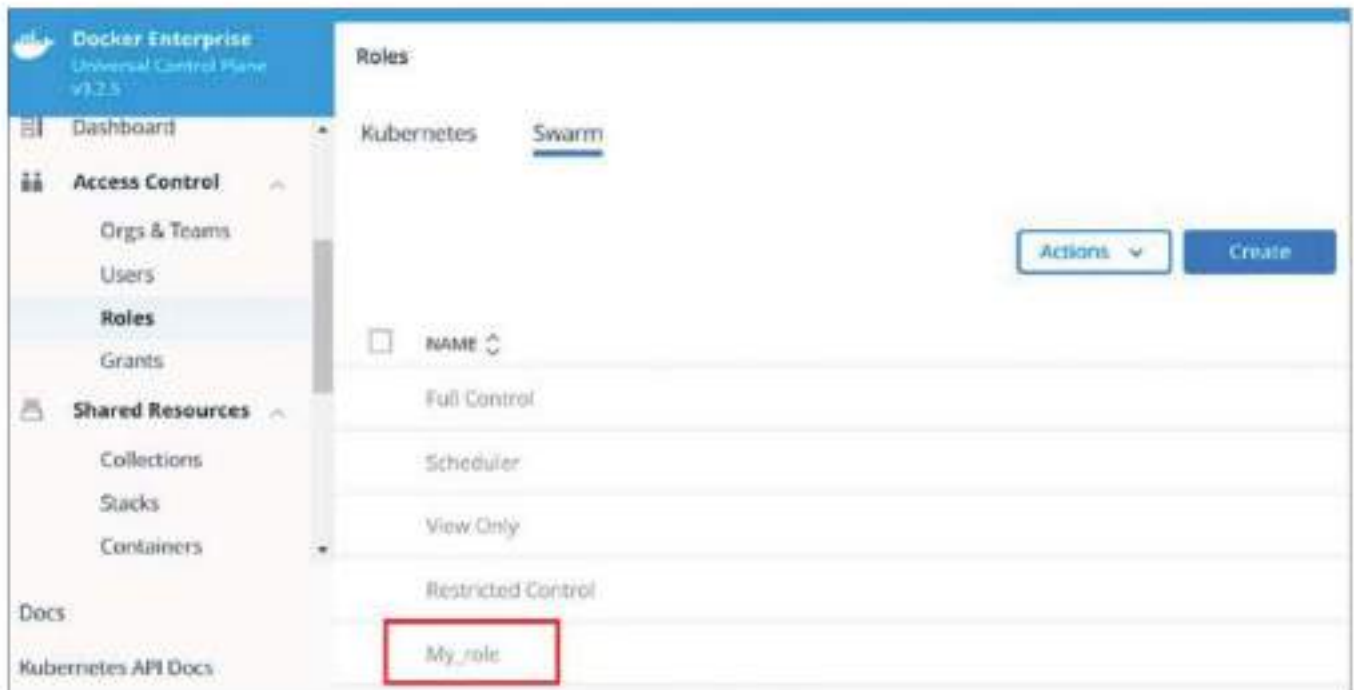


Рис. 8.43

Шаг 3. *Создание предоставляемых разрешений (grant).*

Нажимаем последовательно: **Grant** (Предоставление) | **Swarm** | **Subject** (Субъект), здесь выбираем **IT Operations** из раскрывающегося списка, **Next** (Далее) | **Resource set** (Набор ресурсов), переходим к **My_collection**, затем **Role** (Роль), здесь выбираем **My_role** из раскрывающегося списка и нажимаем **Create** (Создать).

На рис. 8.44 видно, что предоставляемое разрешение (grant) создано.

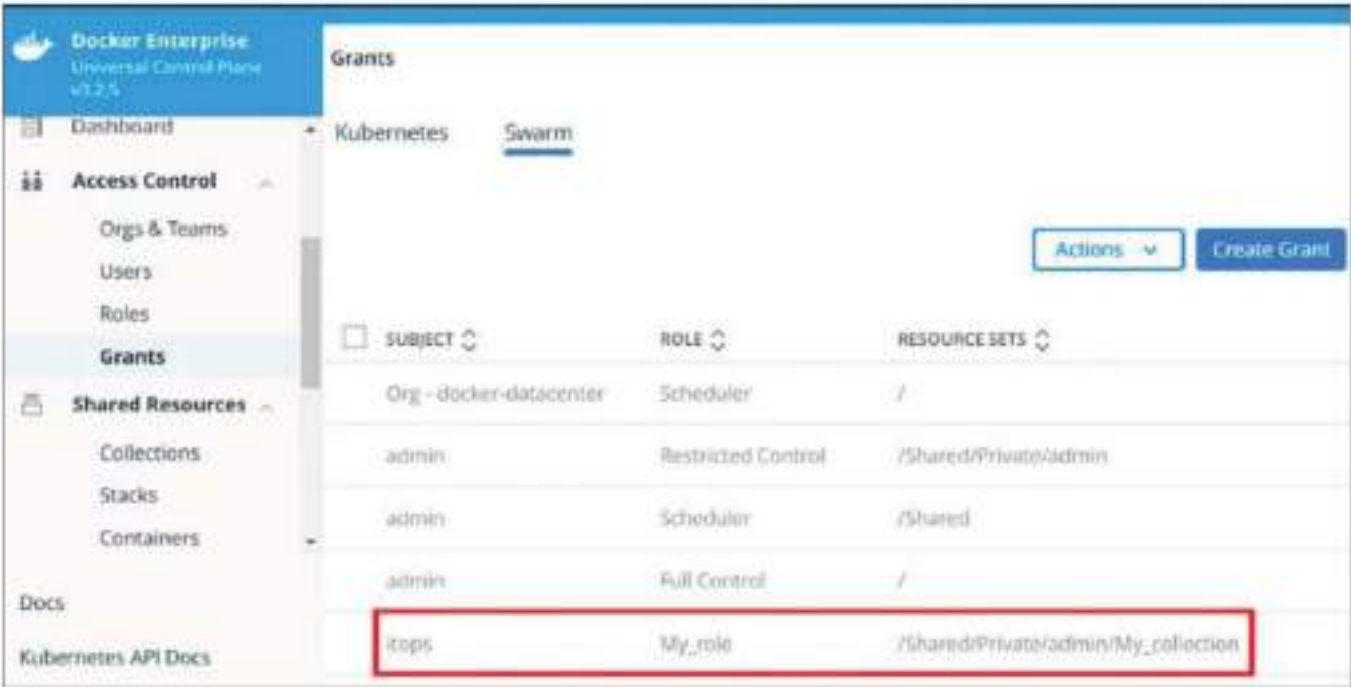


Рис. 8.44

Мы предоставили пользователю **itops** пользовательскую роль **My_role** для созданного нами набора ресурсов (пользовательской коллекции) **My_collection**.

Как видите, мы контролируем, кто имеет доступ к тем или иным ресурсам и что он может делать с ресурсами. В этом заключается суть *управления доступом на основе ролей* (Role Based Access Control, RBAC).

В Docker имеются и встроенные коллекции, описанные в табл. 8.2.

Таблица 8.2. Встроенные коллекции

Коллекция по умолчанию	Описание	Примечания
/	Это универсальный путь ко всем ресурсам в кластере Swarm. Сюда входят ресурсы, которых нет ни в одной другой коллекции	Плейсхолдер (placeholder) для ресурсов, не являющихся частью какой-либо другой коллекции
/System	Управляющие узлы UCP и системные сервисы DTR	По умолчанию доступ есть у администраторов
/Shared	Все рабочие узлы, доступные для планирования	Рабочие узлы можно перемещать и изолировать

Таблица 8.2 (окончание)

Коллекция по умолчанию	Описание	Примечания
/Shared/Private/	Приватные коллекции пользователя	Приватные коллекции инициализируются только после того, как пользователь впервые войдет в узел
/Shared/Legacy/	Коллекции устаревших версий	UCP 2.1 и ниже

Заключение

В этой главе мы подробно рассмотрели Docker Enterprise Edition и все шаги, необходимые для его установки в ОС Ubuntu. Затем мы установили Universal Control Plane (UCP) и Docker Trusted Registry (DTR). Кроме того, загрузили и установили в системе клиентский пакет (client bundle). Мы проверили на практике, как осуществляется сканирование на наличие уязвимостей, и узнали, как это помогает проверять образы, извлеченные из реестра. Также мы познакомились с еще одним механизмом безопасности — управлением доступом на основе ролей. В целом из этой главы вы узнали, как использовать Docker Enterprise Edition и каковы его функции для защиты ОС.

Основные тезисы

- ◆ Docker Enterprise Edition включает Docker Engine Enterprise, Docker Universal Control Plane и Docker Trusted Registry.
- ◆ Для использования Docker Enterprise Edition требуется лицензия, но можно установить пробную версию на один месяц (подробности можно уточнить на официальном сайте Docker).
- ◆ Docker Enterprise Edition работает в режиме Swarm, поэтому необходимы хотя бы два сервера.
- ◆ Если в системе работают предыдущие версии Docker, например docker.io или docker-engine, их нужно удалить перед установкой Docker Enterprise Edition.
- ◆ Docker Universal Control Plane — это решение корпоративного уровня для централизованного управления кластерами и приложениями.
- ◆ Docker Trusted Registry — это реестр для хранения образов корпоративного уровня.
- ◆ Необходимо создать и загрузить клиентский сертификат, а затем установить его в системе, потому что иначе мы не сможем выполнять команды на узле UCP.
- ◆ Утилита клиентского пакета автоматически обновляет переменные `DOCKER_HOST`, `DOCKER_CERT_PATH` и `DOCKER_TLS_VERIFY`.
- ◆ Извлекаемые из реестра образы проверяются функцией сканирования на наличие уязвимостей.

- ◆ Universal Control Plane позволяет создавать пользователей с разным уровнем доступа (роли) к ресурсам (коллекций): образам, сервисам, сетям, томам и т. д.
- ◆ Управление доступом на основе ролей (Role Based Access Control, RBAC) позволяет указать субъекта с определенной ролью, которая дает тот или иной уровень доступа к набору ресурсов (коллекции).
- ◆ Доступ можно детально контролировать, назначая наборы разрешений (роли) отдельным пользователям или группам пользователей.

Контрольный тест

Выберите подходящий вариант ответа.

1. База данных CVE содержит:
 - Все исправления уязвимостей, доступные для Docker.
 - Последние исправления уязвимостей, доступные для Docker.
 - Все исправления нарушений безопасности в Docker.
 - Все известные уязвимости с регулярными обновлениями.
2. Какие задачи выполняет Universal Control Plane?
 - Повышает производительность системы.
 - Динамически применяет исправления уязвимостей.
 - Обеспечивает централизованное управление.
 - Поддерживает стабильность в системе.
3. Выберите верное утверждение о Docker Trusted Registry:
 - Docker Trusted Registry входит в Docker Community Edition.
 - Docker Trusted Registry — это решение корпоративного уровня от Docker для хранения образов.
 - Docker Trusted Registry повышает производительность системы.
 - Docker Trusted Registry не предоставляет информацию о рисках для безопасности.
4. Выберите верное утверждение:
 - В Docker и Kubernetes нет predefined ролей.
 - В Docker есть predefined роли, а в Kubernetes — нет.
 - В Docker и Kubernetes есть predefined роли.
 - В Kubernetes есть predefined роли, а в Docker — нет.
5. Что такое организация?
 - Организация — это группа пользователей с одинаковым набором разрешений и привилегий.

- Организация — это группа пользователей с одинаковым набором разрешений и привилегий, которая должна входить в команду.
 - Организация — это группа пользователей, которые входят в одну команду и могут иметь разные разрешения и привилегии.
 - Ничего из вышеперечисленного.
6. Какие переменные среды автоматически обновляются утилитой клиентского пакета (Client Bundle)?
- `docker_host`, `docker_certification_path`, `docker_tls_verify`.
 - `docker_host_path`, `docker_cert_path`, `docker_tls_verify`.
 - `docker_host`, `docker_cert_path`, `docker_tls_verify`.
 - `docker_host`, `docker_cert_path`, `docker_tls_verification`.
7. Как обновить базу данных CVE при отсутствии Интернета?
- Загрузить и установить `jar`-файл с обновлениями.
 - Загрузить и установить `script`-файл с обновлениями.
 - Загрузить и установить `tar`-файл с обновлениями.
 - Загрузить и установить `zip`-файл с обновлениями.
8. Какой из вариантов *не* является коллекцией по умолчанию?
- `/Shared/Legacy/`.
 - `/Shared/Private/`.
 - `/System`.
 - `/System/Shared/`.
9. Какой из вариантов *не* является ролью по умолчанию?
- `Role Only`.
 - `View Only`.
 - `Full Control`.
 - `Restricted Control`.
10. Выберите верное утверждение:
- Клиентский пакет создает пару ключей — закрытый ключ и открытый ключ, который разрешает запросы к UCP.
 - Клиентский пакет создает пару ключей — закрытый ключ и вторичный ключ, который разрешает запросы к UCP.
 - Клиентский пакет создает пару ключей — закрытый ключ и открытый ключ, который разрешает запросы к DTR.
 - Клиентский пакет создает пару ключей — закрытый ключ и открытый ключ, который разрешает запросы к Docker Engine.

Ответы

1	d
2	c
3	b
4	c
5	a

6	c
7	c
8	d
9	a
10	a

Вопросы

1. Из каких компонентов состоит Docker Enterprise Edition?
2. Каковы основные функции Universal Control Plane?
3. Каковы основные функции Docker Trusted Registry?
4. Для чего нужен клиентский пакет (Client Bundle)?
5. Какие переменные среды обновляет клиентский пакет?
6. Что такое CVE?
7. Что такое Role Based Access Control (RBAC)?
8. Что означают понятия «пользователи», «организации» и «команды» в контексте RBAC?
9. Что такое «роль»?
10. Что такое «коллекция»?

Ключевые термины

- ◆ Docker Enterprise Edition
- ◆ Universal Control Plane
- ◆ Docker Trusted Registry
- ◆ Client Bundle
- ◆ Репозиторий
- ◆ Docker Swarm
- ◆ База данных CVE
- ◆ Сканирование на наличие уязвимостей
- ◆ Роли
- ◆ Команды
- ◆ Коллекции
- ◆ Организации

Предметный указатель

A

AppArmor 171
AVC 181

B

Backing filesystem 64
bind mount 68, 69
BusyBox 32, 159

C

capabilities 168, 171
cgroups 163
CI/CD 20
CLI 22
CNM (Container Network Model) 81
COMMIT 50
complain 173
Container Network Model (CNM) 81
Containerd 43
CVE 204

D

default-profile 171
detach 33
diff 45
Docker Engine 21, 43
Docker Engine Community 23
Docker Enterprise 23
Docker Hub 23
Docker Registry 23
Docker Swarm 21

Docker Trusted Registry, DTR 192, 199
Docker Universal Control Plane (UCP) 208
DOCKER_CERT_PATH 202
docker_gwbridge 142
DOCKER_HOST 202
docker0 88
docker-default 176
Dockerfile 52
♦ команды 52
DRAIN 104

E

enforce 174
etcd 95
eth0 121

F

Filebeat 43

I

ingress 109
interactive 33
IPAM 82
IPC 160

J

jq 49
json-file 115
♦ драйвер журналирования 39

L

Libnetwork 83
 Logstash 43
 lowerdir (нижний слой) 66
 LXC (Linux Container) 18

M

MAC-in-UDP 148
 MCS (Multi-Category Security) 184
 Mesh-сеть 114
 MNT 160
 Moby 21

N

NET 159
 NIS 162

O

OOME 164
 Overlay2 64, 65
 OverlayFS 65

P

PID 158

R

Raft, алгоритм консенсуса 99
 REST API 22

Role Based Access Control (RBAC) 214
 runC 43

S

Secure Computing Mode 185
 SELinux 181
 SLA 23
 split-brain 99
 Swarm 93
 ◇ консенсус 98
 ◇ настройка 95
 ◇ реплика 95

T

tmpfs 68, 75
 trunk bridge 131
 tty 34

U

Union mount 66
 Universal Control Plane, UCP 192, 196
 upperdir (верхний слой) 66
 USR 161
 UTS 162

V

veth 85
 VNI 148
 VTEP 148
 VXLAN 148

A

Автоблокировка 106
 Архитектура контейнера 43

B

Виртуальная машина (BM) 19

Г

Глобальные сервисы 103

Д

Дайджест 45
 Дискреционный контроль доступа 171
 Драйверы
 ◇ Docker
 ◦ собственные 83
 ◦ сторонние 83
 ◇ IPAM 82
 ◇ журналирования 39
 ◇ сетевые 82
 ◇ хранилища 63

Ж

Журнал Docker 39

К

Каскадно-объединенное монтирование 66

Кеш сборки 53

Коллекции 212

Конечные точки 81

Контейнер 31

◊ запуск 32

◊ состояния 34

◊ получение информации 38

Контейнеризация 17, 18

Контрольная сумма 45

Копирование при записи 47

М

Микросервисы 24

Многоступенчатые сборки 54

Мультикатегорийная безопасность 184

Н

Непривилегированные процессы 168

О

Образ Docker 44

Образы, получение информации 49

Объект конфигурации 46

Оверлейная сеть 109, 138

П

Песочница 81

Привилегированные процессы 167

Промежуточный образ 53

Пространство имен 84, 157

◊ сети 121

Р

Рабочий узел 95

Реестр 23

Реплицированные сервисы 103

Роли 212

С

Сервис 95, 99

Сетевой мост 84, 109

◊ Linux 84, 88

Сканирование образов Docker 203

Т

Токен 96

Том

◊ анонимный 72

◊ именованный 73

◊ создание 74

Тома 68

У

Управляющий узел 95